数学

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 viii

第一章 高精度计算 1
- 1.1 精确分数四则运算 . . . 1
- 1.2 高精度整数：有符号加减乘与小整数除法 . . . 6

第二章 快速幂 15
- 2.1 无符号 64 位模乘与模幂 . . . 15
- 2.2 模幂与几何级数同时求值 . . . 16

第三章 数论 18
- 3.1 数论基础 . . . 18
  - 3.1.1 幂的约数和 . . . 18
- 3.2 模算术简介 . . . 20
  - 3.2.1 动态模整数与合数模逆元 . . . 20
  - 3.2.2 静态模整数 . . . 24
  - 3.2.3 完整 64 位固定模整数 . . . 27
  - 3.2.4 Montgomery 固定模整数 . . . 31
  - 3.2.5 32 位 Barrett 快速模乘 . . . 35
  - 3.2.6 模运算的适用条件 . . . 37
    - 3.2.6.1 除法与解同余式不是一回事 . . . 37
    - 3.2.6.2 知道整数可整除时，先升模数再整除 . . . 38
    - 3.2.6.3 组合数的整数性不等于分母可逆 . . . 38
    - 3.2.6.4 大指数要同时保留余数和阈值信息 . . . 39
- 3.3 素数 . . . 40
  - 3.3.1 确定性 64 位 Miller–Rabin 判素 . . . 40
- 3.4 欧拉函数 . . . 41
  - 3.4.1 单个数的欧拉函数 . . . 41
- 3.5 筛法 . . . 42
  - 3.5.1 线性筛、欧拉函数与 Möbius 函数 . . . 42
  - 3.5.2 区间筛素数 . . . 43
- 3.6 分解质因数 . . . 45
  - 3.6.1 Pollard–Rho 因数分解 . . . 45
- 3.7 裴蜀定理 & 一次不定方程 . . . 47
  - 3.7.1 扩展欧几里得与 Bézout 系数 . . . 47
  - 3.7.2 二元一次不定方程 . . . 48
  - 3.7.3 一次不定方程：特解与正整数解范围 . . . 50
    - 3.7.3.1 有符号矩形范围：把两维约束化成一个参数区间 . . . 51
- 3.8 费马小定理 & 欧拉定理 . . . 52
  - 3.8.1 十进制大指数：扩展欧拉降幂 . . . 52
  - 3.8.2 Carmichael：模乘法群的公共指数 . . . 53
- 3.9 模逆元 . . . 54
  - 3.9.1 模整数类型的批量逆元 . . . 54

3.9.2 任意正模数的单个逆元 . . . 55
3.9.3 任意数组的批量逆元 . . . 56
3.9.4 连续整数的线性逆元表 . . . 58
3.10 线性同余方程 . . . 59
3.10.1 线性同余方程 . . . 59
3.11 中国剩余定理 . . . 60
3.11.1 广义 CRT：逐项合并同余 . . . 60
3.11.2 Garner：大周期 CRT 与混合进制 . . . 62
3.12 升幂引理 . . . 64
3.12.1 素因子估值与升幂：先约去公共因子 . . . 64
3.12.1.1 基本运算与公共因子 . . . 64
3.12.1.2 奇素数：差式与和式的条件不同 . . . 64
3.12.1.3 素数 2：先分指数奇偶 . . . 64
3.12.1.4 把大整数整除转成指数条件 . . . 65
3.13 阶乘取模 . . . 66
3.13.1 Legendre 赋值与 Kummer 进位判据 . . . 66
3.13.1.1 组合数：逐位相加时进了几次位 . . . 66
3.13.1.2 素数幂为零与 Lucas 的边界 . . . 67
3.14 卢卡斯定理 . . . 68
3.14.1 Lucas：小素数模的大下标组合数 . . . 68
3.14.2 合数模数组合数（扩展 Lucas） . . . 69
3.14.3 有界上标的任意模数组合数 . . . 72
3.14.4 素数模 Gaussian 二项式 . . . 76
3.15 二次剩余 . . . 80
3.15.1 素数模平方根（Cipolla） . . . 80
3.16 阶 & 原根 . . . 81
3.16.1 乘法阶与原根 . . . 81
3.17 离散对数 . . . 85
3.17.1 扩展 BSGS . . . 85
3.18 高次剩余 & 单位根 . . . 86
3.18.1 素数模高次剩余：全部根的紧凑表示 . . . 86
3.18.2 素数幂模高次剩余：含非互素被开方数 . . . 87
3.18.3 合数求根准备：分解模数与素数幂原根 . . . 90
3.18.4 一般合数模高次剩余：CRT 组合与按索引取根 . . . 91
3.19 数论分块 . . . 94
3.19.1 整除分块：前缀和来源决定算法 . . . 94
3.19.1.1 按数据与资源选前缀和 . . . 94
3.20 莫比乌斯反演 . . . 96
3.20.1 矩形 GCD 计数 . . . 96
3.20.2 整除与反演：先辨认求和方向 . . . 97
3.20.2.1 约数和与倍数和不能混用 . . . 97
3.20.2.2 GCD 权值先反演，再交换求和 . . . 98
3.20.3 GCD 计数：缩放、边界与对称性 . . . 98
3.20.3.1 恰好等于 k：先缩放整除对象，再做矩形容斥 . . . 98
3.20.3.2 去掉顺序之前，先检查交换是否仍在定义域内 . . . 98
3.21 狄利克雷双曲线法 & 杜教筛 . . . 99
3.21.1 杜教筛 . . . 99

3.22 素数计数 . . . 101
3.22.1 素数计数（整除分块筛） . . . 101
3.23 Min_25 筛 . . . 103
3.23.1 Min_25 积性函数求和 . . . 103
3.24 类欧几里德算法 . . . 106
3.24.1 有符号整除和：类欧几里德 . . . 106
3.24.2 整除和、带权和与平方和 . . . 108
3.24.3 类欧几里德：有符号整除求和 . . . 110
3.24.4 类欧几里德的三个求和量 . . . 111

第四章 多项式与生成函数 112
4.1 快速傅里叶变换 . . . 112
4.1.1 复数 FFT（可复用单位根） . . . 112
4.1.2 浮点 FFT 整数卷积 . . . 116
4.1.3 拆系数 FFT 任意模卷积 . . . 118
4.1.4 CDQ 分治卷积递推 . . . 120
4.2 快速数论变换 . . . 122
4.2.1 可指定素数与原根的 NTT 卷积 . . . 122
4.2.2 精确有符号整数卷积 . . . 125
4.2.3 二维 NTT 与二元多项式卷积 . . . 127
4.2.4 三模 NTT：精确任意模数卷积 . . . 130
4.3 快速沃尔什变换 . . . 136
4.3.1 按位 OR、AND、XOR 卷积 . . . 136
4.3.2 不相交子集卷积 . . . 138
4.4 Chirp Z 变换 . . . 140
4.4.1 Chirp Z：等比点求值 . . . 140
4.5 多项式牛顿迭代 . . . 142
4.5.1 形式幂级数求逆 . . . 142
4.5.2 多项式带余除法 . . . 143
4.5.3 二元形式幂级数求逆 . . . 146
4.6 多项式多点求值 | 快速插值 . . . 150
4.6.1 任意点多项式多点求值 . . . 150
4.6.2 任意点多项式快速插值 . . . 153
4.7 多项式初等函数 . . . 155
4.7.1 形式幂级数开根（含前导零与无解） . . . 155
4.7.2 形式幂级数微积分、对数与指数 . . . 160
4.7.3 形式幂级数快速幂（十进制大指数） . . . 163
4.8 常系数齐次线性递推 . . . 165
4.8.1 特征多项式求线性递推第 k 项 . . . 165
4.8.2 快速递推与有理生成函数单项 . . . 167
4.9 多项式平移 | 连续点值平移 . . . 169
4.9.1 多项式系数平移 . . . 169
4.10 符号化方法 . . . 170
4.10.1 生成函数建模到模板：三个可核验例子 . . . 170
4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积） . . . 170
4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆） . . . 171
4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数） . . . 171
4.10.1.4 抄代码前的四项检查与本地证据 . . . 172

4.11 Lagrange 反演 . . . 173
4.11.1 隐式生成函数与 Lagrange 反演 . . . 173
4.11.1.1 适用条件与取系数公式 . . . 173
4.11.1.2 形式留数证明 . . . 173
4.11.1.3 模型一按内部结点计数的满有序多叉树 . . . 173
4.11.1.4 模型二带标号有根树的 EGF . . . 174
4.11.1.5 落到现有接口及模数边界 . . . 174
4.12 指数生成函数 . . . 175
4.12.1 平移幂和多项式 . . . 175

第五章 组合数学 177
5.1 排列组合 . . . 177
5.1.1 阶乘组合数 . . . 177
5.1.2 计数建模：先确认哪些对象被区分 . . . 178
5.1.2.1 五个判别问题与两个最小反例 . . . 178
5.1.2.2 OGF、EGF 的选择来自乘法，而非题目用词 . . . 179
5.1.3 组合数选型：先看模数，再看下标与预算 . . . 179
5.2 容斥原理 . . . 180
5.2.1 容斥与二项式反演：恰好与至少 . . . 180
5.2.1.1 从交集和恢复恰好数量 . . . 180
5.2.1.2 至少数量与变换方向 . . . 180
5.2.1.3 例：有标号球放入有标号盒 . . . 181
5.2.1.4 直接使用现有组合数与模整数 . . . 181
5.3 斐波那契数列 . . . 182
5.3.1 斐波那契最小循环节（Pisano 周期） . . . 182
5.4 卡特兰数 . . . 187
5.4.1 Catalan 与投票问题：把前缀限制变成反射双射 . . . 187
5.4.1.1 弱前缀条件：减去首次走到负一的坏路径 . . . 187
5.4.1.2 严格领先：先取走必需的第一步 . . . 187
5.4.1.3 初始有余额：反射起点，而非只改终点 . . . 188
5.4.1.4 取模时优先使用差式，除法条件不能省 . . . 188
5.5 斯特林数与斯特林反演 . . . 189
5.5.1 定义与性质 . . . 189
5.5.1.1 行列算法的选择与推导 . . . 189
5.5.2 无符号第一类斯特林数：同一行 . . . 190
5.5.3 无符号第一类斯特林数：同一列 . . . 192
5.5.4 第二类斯特林数：同一行前缀 . . . 194
5.5.5 第二类斯特林数：同一列 . . . 195
5.5.6 斯特林反演与应用 . . . 197
5.5.6.1 手算检查与建模用法 . . . 197
5.5.6.2 题目协议与验证范围 . . . 197
5.6 分拆数 . . . 198
5.6.1 整数分拆与重数上限 . . . 198
5.6.2 整数分拆：重数限制的差一问题 . . . 200
5.6.2.1 三个容易混淆的限制 . . . 201
5.6.2.2 重数限制与禁止倍数部分 . . . 201
5.7 Pólya 计数 . . . 202
5.7.1 Burnside 平均：任意模数下的精确除法 . . . 202

5.7.2 置换的循环长度分解 . . . 205
5.7.3 Pólya 环染色：旋转与翻转 . . . 206
5.7.4 群作用计数：先确定什么算同一个方案 . . . 208
5.7.5 项链与手链：从置换循环推公式 . . . 208
5.7.6 指定颜色库存：给每个循环分配整份颜色 . . . 209
5.7.7 群阶不可逆：先提升模数，再作整数除法 . . . 209

第六章 线性代数 211
6.1 矩阵 . . . 211
6.1.1 模矩阵乘法与幂 . . . 211
6.1.2 素数模矩阵求逆 . . . 213
6.1.3 模 2 矩阵求逆（按位消元） . . . 216
6.1.4 Max-plus 矩阵幂 . . . 218
6.2 行列式 . . . 220
6.2.1 素数模行列式 . . . 220
6.2.2 任意正模数行列式：欧几里得消元 . . . 223
6.2.3 精确整数行列式（Bareiss） . . . 225
6.3 线性基 . . . 227
6.3.1 64 位异或线性基 . . . 227
6.3.2 带位置限制的异或线性基 . . . 229
6.3.3 异或线性空间求交 . . . 233
6.3.4 Zassenhaus：同时求和空间与交空间 . . . 235
6.3.5 无向图最大异或行走 . . . 237
6.4 特征多项式 . . . 240
6.4.1 有限状态计数到线性递推：先证明阶数，再用 BM . . . 240
6.4.1.1 先把状态、方向和输出写清楚 . . . 240
6.4.1.2 Cayley–Hamilton 给出阶数至多 d . . . 240
6.4.1.3 为什么前 2d 项足以恢复这个标量序列 . . . 240
6.4.1.4 模型：不含连续子串 101 的二进制串 . . . 241
6.4.1.5 按目标选算法，并把采样成本算进去 . . . 241

第七章 线性规划 242
7.1 单纯形法 . . . 242
7.1.1 线性规划：两阶段单纯形 . . . 242

第八章 概率论 247
8.1 随机变量的数字特征 . . . 247
8.1.1 期望选型：计数拆指标，等待拆尾和 . . . 247
8.1.2 条件期望：先定义状态，再解自环 . . . 247
8.1.3 模概率：可逆的不只是输入分母 . . . 248

第九章 博弈论 249
9.1 公平组合游戏 . . . 249
9.1.1 SG 的前提、递推与异或策略 . . . 249
9.1.2 减法游戏小例与三类失效边界 . . . 249

第十章 数值算法 **251**
10.1 插值 . . . 251
10.1.1 拉格朗日插值与连续点优化 . . . 251
10.2 数值积分 . . . 254
10.2.1 有工作上限的自适应 Simpson 积分 . . . 254
10.2.2 数值积分：局部估计、误差预算与采样盲区 . . . 257
10.3 高斯消元 . . . 258
10.3.1 压位模 2 高斯消元与全部解 . . . 258
10.3.2 素数模高斯消元与全部解 . . . 262
10.3.3 实数高斯消元与数值解空间 . . . 265

第十一章 Berlekamp–Massey 算法 **268**
11.1 Berlekamp–Massey：最短线性递推 . . . 268

第十二章 配套知识与验证范围 **270**
12.1 杜教筛：预筛与缓存都不可省略 . . . 270
12.2 64 位判素与质因数分解 . . . 270
12.3 单个数的欧拉函数 . . . 271
12.4 线性筛：三张表的约定 . . . 271
12.5 区间筛：只保存目标区间 . . . 271
12.6 矩形 GCD 计数：缩放后容斥 . . . 272
12.7 幂的约数和：不除以 $p-1$ . . . 272
12.8 任意数组的批量逆元 . . . 272
12.9 连续整数的线性逆元表 . . . 273
12.10 线性同余：最小解与最小周期 . . . 273
12.11 扩展 CRT：逐式合并 . . . 273
12.12 Garner：总周期可以超过整数范围 . . . 274
12.13 素数模平方根 . . . 274
12.14 乘法阶与原根 . . . 275
12.15 Carmichael：最小公共指数 . . . 275
12.16 Lucas：按素数进制逐位计算 . . . 275
12.17 合数模数组合数与插值 . . . 276
12.18 阶乘表：组合数与排列数 . . . 276
12.19 组合计数 . . . 276
12.20 生成函数与递推 . . . 276
12.21 线性代数与生成树 . . . 277
12.22 模线性方程组：工厂工期 . . . 277
12.23 线性递推：系数方向与外推条件 . . . 277
12.24 素数模高次剩余与单位根 . . . 279
12.25 素数幂模高次剩余与根的提升 . . . 280
12.26 一般合数模高次剩余：CRT 组合 . . . 281
12.27 自动准备模数分解与原根 . . . 282
12.28 任意正模数行列式 . . . 283
12.29 参数化 NTT 与模数适配 . . . 284
12.30 Bostan–Mori 与快速递推单项 . . . 285
12.31 集合变换与不相交子集卷积 . . . 286
12.32 Taylor 平移与等比点求值 . . . 287
12.33 CDQ 卷积递推：贡献只跨越一次 . . . 288

12.34 多项式带余除法：先反转再求逆 . . . 288
12.35 级数开根：前导零和自由尾项 . . . 288
12.36 ln 与 exp：只处理所需前缀 . . . 288
12.37 幂函数的大指数：保留三份指数信息 . . . 289
12.38 集合变换：逆变换对模数的要求 . . . 289
12.39 浮点 FFT：舍入前先约束最坏误差 . . . 289
12.40 模 2 压位消元与完整解空间 . . . 290
12.41 模 2 消元后的最少开关用法 . . . 290

第一部分　跨类依赖 **291**

附录 A　树状数组 **292**
A.1 树状数组第 k 小与组合依赖 . . . 292

附录 B　矩阵树定理 **294**
B.1 带权矩阵树：无向树与内外向树形图 . . . 294
B.2 任意正模数带权矩阵树 . . . 297

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　高精度计算

相关应用：约分有理数：signed64 值与 signed128 中间值，或 cpp_int 任意精度；不是实数浮点模板

1.1　精确分数四则运算

Rational<> 默认保存 long long 分子 num 与正分母 den，所有构造和运算后立即约分，零固定为 0/1。构造 Rational<>(n,d)，默认 0/1；支持四则、复合赋值、负号、六种比较和输出，整值输出整数，其余输出 num/den。x+=x、x-=x、x*=x、非零 x 的 x/=x 均有效。num/den 可读，不要直接修改以破坏约分和正分母不变量。

默认以 signed128 计算中间值，合法 64 位规范分数的交叉乘积及加减均可表示；要求每次构造及运算后的最简分子在 signed64 内、正分母 ≤LLONG_MAX，除数非零。不要求约分前的积能放入 64 位。比如 LLONG_MIN 除以 LLONG_MIN 可约为 1，而 1 除以 LLONG_MIN 的正分母无法表示，后者不属于接口域；LLONG_MIN 取负也超域。assert 仅辅助检查前置条件，NDEBUG 下仍必须遵守。

任意精度用 Rational<cpp_int,cpp_int>，额外包含 Boost 多精度整数头文件，详见完整用法；按位数承担大整数乘除及 gcd 成本，不能把它当常数时间。其他 T/W 组合须自行证明中间值与符号处理范围。assign/make 仅内部辅助，外部调用构造和运算符。double 显式转换只是近似；大整数分子分母各自转 double 可能溢出，不用于精确判定。比较始终采用整数。

```
template<class T = long long, class W = __int128_t>
struct Rational
{
    T num = 0, den = 1;

    void assign(W n, W d)
    {
        assert(d != 0);
        if (d < 0)
        {
            n = -n;
            d = -d;
        }
        W a = n < 0 ? -n : n, b = d;
        while (b != 0)
        {
            W r = a % b;
            a = b;
            b = r;
        }
        n /= a;
        d /= a;
        if constexpr (numeric_limits<T>::is_bounded)
        {
            assert(n >= W(numeric_limits<T>::lowest()));
            assert(n <= W(numeric_limits<T>::max()));
            assert(d <= W(numeric_limits<T>::max()));
        }
        num = T(n);
        den = T(d);
    }
```

精确分数（接上页同一结构体）：

```
    Rational(T n = 0, T d = 1)
    {
        assign(W(n), W(d));
    }

    static Rational make(W n, W d)
    {
        Rational r;
        r.assign(n, d);
        return r;
    }

    friend Rational operator+(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.den + W(b.num) * a.den, W(a.den) * b.den);
    }

    friend Rational operator-(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.den - W(b.num) * a.den, W(a.den) * b.den);
    }

    friend Rational operator*(const Rational &a, const Rational &b)
    {
        return make(W(a.num) * b.num, W(a.den) * b.den);
    }

    friend Rational operator/(const Rational &a, const Rational &b)
    {
        assert(b.num != 0);
        return make(W(a.num) * b.den, W(a.den) * b.num);
    }

    Rational operator-() const
    {
        return make(-W(num), W(den));
    }

    Rational &operator+=(const Rational &b)
    {
        return *this = *this + b;
    }

    Rational &operator-=(const Rational &b)
    {
        return *this = *this - b;
    }
```

精确分数（接上页同一结构体）：

```
    Rational &operator*=(const Rational &b)
    {
        return *this = *this * b;
    }

    Rational &operator/=(const Rational &b)
    {
        return *this = *this / b;
    }

    friend bool operator==(const Rational &a, const Rational &b)
    {
        return a.num == b.num && a.den == b.den;
    }

    friend bool operator!=(const Rational &a, const Rational &b)
    {
        return !(a == b);
    }

    friend bool operator<(const Rational &a, const Rational &b)
    {
        return W(a.num) * b.den < W(b.num) * a.den;
    }

    friend bool operator>(const Rational &a, const Rational &b)
    {
        return b < a;
    }

    friend bool operator<=(const Rational &a, const Rational &b)
    {
        return !(b < a);
    }

    friend bool operator>=(const Rational &a, const Rational &b)
    {
        return !(a < b);
    }

    explicit operator double() const
    {
        return double(num) / double(den);
    }

    friend ostream &operator<<(ostream &out, const Rational &a)
    {
        out << a.num;
        if (a.den != 1)
            out << '/' << a.den;
        return out;
    }
};
```

模板题使用：P1572 计算分数

题意：读取只含分数、加减号的表达式，输出最简分数或整数；用输入流直接读带符号分子。原题保证逐项约分后的中间结果在 int 范围内，故默认 64 位版足够。本地已验证，尚无本次线上 AC。

所需模板：Rational（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1572

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Rational<> sum;
    long long num, den;
    char slash;
    while (cin >> num >> slash >> den)
        sum += Rational<>(num, den);
    cout << sum << '\n';
}
```

接口用法演示：任意精度分数 API

演示约定：先读 q，随后每行 an ad op bn bd；分母非零，op 为 +、-、*、/或 <，除法要求 bn 非零。前四种输出最简结果，< 输出比较值-1/0/1。需要额外抄入 Boost 头文件，cpp_int 不依赖本书 BigInteger。整数长度不受 64 位限制，时间与内存随位数增长。

所需模板：Rational（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using boost::multiprecision::cpp_int;
    using F = Rational<cpp_int, cpp_int>;
    int q;
    cin >> q;
    while (q--)
    {
        cpp_int an, ad, bn, bd;
        char op;
        cin >> an >> ad >> op >> bn >> bd;
        F a(an, ad), b(bn, bd);
        if (op == '+')
            cout << a + b << '\n';
        else if (op == '-')
            cout << a - b << '\n';
        else if (op == '*')
            cout << a * b << '\n';
        else if (op == '/')
            cout << a / b << '\n';
        else
            cout << (a < b ? -1 : a == b ? 0 : 1) << '\n';
    }
}
```

1.2 高精度整数：有符号加减乘与小整数除法

BigInteger(x) 接收 long long，BigInteger(s) 接收可带正负号及前导零的非空十进制串；str() 返回规范化十进制，cmp(b) 返回 $-1/0/1$。支持一元负号、加减乘，divmod(int b) 返回商和余数，b≠0；商向零截断，余数与被除数同号或为 0。pow(unsigned e) 计算非负整数幂，约定 0 的 0 次方为 1。

d 是低位在前的 10^9 进制绝对值，sign 为 ±1；零为空 d 且 sign=1，勿直接修改。n,m 为分节数：加减比较 O(n+m)，乘法 O(nm+n+m)，小整数除法 O(n)，结果空间 O(n+m)；幂执行 O(log(e+1)) 次乘法，位数增长另计。字符串长度、所有中间分节数及两操作数长度之和须小于 INT_MAX 并留进位空间。不含大整数除大整数；不用浮点或 Boost。

```
struct BigInteger
{
    static constexpr int base = 1000000000;
    vector<int> d;
    int sign = 1;
    void trim()
    {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sign = 1;
    }
    BigInteger(long long x = 0)
    {
        sign = x < 0 ? -1 : 1;
        auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
        while (v)
        {
            d.push_back(v % base);
            v /= base;
        }
        trim();
    }
```

十进制构造、输出与比较（接上页结构体）：

```
    explicit BigInteger(const string &s)
    {
        assert(!s.empty());
        int first = s[0] == '-' || s[0] == '+';
        sign = s[0] == '-' ? -1 : 1;
        assert(first < (int)s.size());
        for (int r = int(s.size()); r > first; r -= 9)
        {
            int v = 0;
            for (int i = max(first, r - 9); i < r; i++)
            {
                assert('0' <= s[i] && s[i] <= '9');
                v = v * 10 + s[i] - '0';
            }
            d.push_back(v);
        }
        trim();
    }
    string str() const
    {
        if (d.empty())
            return "0";
        string s = sign < 0 ? "-" : "";
        s += to_string(d.back());
        for (int i = int(d.size()) - 2; i >= 0; i--)
        {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
    int abs_cmp(const BigInteger &b) const
    {
        if (d.size() != b.d.size())
            return d.size() < b.d.size() ? -1 : 1;
        for (int i = int(d.size()) - 1; i >= 0; i--)
            if (d[i] != b.d[i])
                return d[i] < b.d[i] ? -1 : 1;
        return 0;
    }
    int cmp(const BigInteger &b) const
    {
        if (sign != b.sign)
            return sign < b.sign ? -1 : 1;
        return sign * abs_cmp(b);
    }
```

取负与加减法（接上页结构体）：

```
    BigInteger operator-() const
    {
        BigInteger c = *this;
        if (!c.d.empty())
            c.sign = -c.sign;
        return c;
    }
    BigInteger operator+(const BigInteger &b) const
    {
        if (sign != b.sign)
        {
            if (abs_cmp(b) < 0)
                return b + *this;
            BigInteger c = *this;
            int borrow = 0;
            for (int i = 0; i < (int)d.size(); i++)
            {
                int v = d[i] - borrow;
                if (i < (int)b.d.size())
                    v -= b.d[i];
                borrow = v < 0;
                c.d[i] = v + borrow * base;
            }
            c.trim();
            return c;
        }
        BigInteger c;
        c.sign = sign;
        int carry = 0;
        int n = max(d.size(), b.d.size());
        for (int i = 0; i < n || carry; i++)
        {
            int v = carry;
            if (i < (int)d.size())
                v += d[i];
            if (i < (int)b.d.size())
                v += b.d[i];
            c.d.push_back(v % base);
            carry = v / base;
        }
        return c;
    }
    BigInteger operator-(const BigInteger &b) const
    {
        return *this + (-b);
    }
```

乘法、小整数除法与整数幂（接上页结构体）：

```
    BigInteger operator*(const BigInteger &b) const
    {
        BigInteger c;
        c.sign = sign * b.sign;
        c.d.resize(d.size() + b.d.size());
        for (int i = 0; i < (int)d.size(); i++)
        {
            long long carry = 0;
            for (int j = 0; j < (int)b.d.size(); j++)
            {
                long long v = 1LL * d[i] * b.d[j] + c.d[i + j] + carry;
                c.d[i + j] = v % base;
                carry = v / base;
            }
            c.d[i + b.d.size()] = carry;
        }
        c.trim();
        return c;
    }
    pair<BigInteger, int> divmod(int b) const
    {
        assert(b != 0);
        long long v = b < 0 ? -1LL * b : b;
        long long rem = 0;
        BigInteger q;
        q.sign = sign * (b < 0 ? -1 : 1);
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; i--)
        {
            rem = rem * base + d[i];
            q.d[i] = rem / v;
            rem %= v;
        }
        q.trim();
        return {q, int(sign * rem)};
    }
    BigInteger pow(unsigned int e) const
    {
        BigInteger a = *this;
        BigInteger r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            e >>= 1;
            if (e)
                a = a * a;
        }
        return r;
    }
};
```

模板题使用：P1480 高精度除法 I

题意：0≤a≤10^5000，1≤b≤10^9；输出整数商。divmod 同时返回余数，这里只打印 first。完整协议本地通过，在线状态见验证记录。

所需模板：BigInteger（第 6 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1480

```
int main()
{
    string s;
    int b;
    cin >> s >> b;
    BigInteger a(s);
    cout << a.divmod(b).first.str() << '\n';
}
```

模板题使用：P2142 高精度减法

题意：输入两个正整数，输出 a−b，结果可能为负；字符串构造后直接减法。完整协议本地通过，尚无本实现在线 AC。

所需模板：BigInteger（第 6 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2142

```
int main()
{
    string s, t;
    cin >> s >> t;
    BigInteger a(s), b(t);
    cout << (a - b).str() << '\n';
}
```

模板题使用：P1303 高精度乘法

题意：读取两个非负十进制整数，输出乘积。竖式压位乘法，不适用于要求 NTT 复杂度的超长输入。完整协议本地通过，尚无本实现在线 AC。

所需模板：BigInteger（第 6 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1303

```
int main()
{
    string s, t;
    cin >> s >> t;
    BigInteger a(s), b(t);
    cout << (a * b).str() << '\n';
}
```

接口用法演示：HDU1134 来源模型：卡特兰数

演示约定：按照 kuangbin 完整来源程序，预处理 C0..C100，反复读 n，−1 结束。递推 Ci=Ci−1(4i−2)/(i+1)，每步整除。官方题面暂未读取成功，当前仅来源模型，不计正式题目或 AC。

所需模板：BigInteger（第 6 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIG-INTEGER.md

```
int main()
{
    vector<BigInteger> f(101);
    f[0] = 1;
    for (int i = 1; i <= 100; i++)
        f[i] = (f[i - 1] * BigInteger(4 * i - 2)).divmod(i + 1).first;
    int n;
    while (cin >> n && n != -1)
        cout << f[n].str() << '\n';
}
```

接口用法演示：有符号高精度运算接口

演示约定：输入 a,b、非零 int 除数 d 和非负 unsigned 指数 e。依次输出和、差、积、比较结果、a/d 的商与余数、a 的 e 次方。负数除法向零截断；控制指数和位数以免结果过大。这是接口演示，不计 OJ。

所需模板：BigInteger（第 6 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIG-INTEGER.md

```
int main()
{
    string s, t;
    int d;
    unsigned int e;
    cin >> s >> t >> d >> e;
    BigInteger a(s), b(t);
    cout << (a + b).str() << '\n';
    cout << (a - b).str() << '\n';
    cout << (a * b).str() << '\n';
    cout << a.cmp(b) << '\n';
    auto [q, r] = a.divmod(d);
    cout << q.str() << ' ' << r << '\n';
    cout << a.pow(e).str() << '\n';
}
```

第二章　快速幂

2.1　无符号 64 位模乘与模幂

Mod64::mul(a,b,m) 与 power(a,e,m)，a、b、e 支持完整 unsigned long long，m 为非零无符号 64 位模数。乘积用 uint128 后取模，不通过浮点近似；m=1 返回 0，e=0 返回 1 mod m。mul 为 O(1)，power 为 O(log(e+1)) 次模乘、O(1) 空间。它也是 64 位判素的底层组件。

```
struct Mod64
{
    using ull = unsigned long long;
    using u128 = __uint128_t;

    static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }

    static ull power(ull a, ull b, ull m)
    {
        assert(m);
        ull r = 1 % m;
        a %= m;
        for (; b; b >>= 1, a = mul(a, a, m))
            if (b & 1) r = mul(r, a, m);
        return r;
    }
};
```

接口用法演示：完整无符号 64 位模运算（接口演示）

演示约定：自定义协议，非 OJ 题：0≤q≤200000，随后 q 行 a b m，无符号十进制 0≤a,b≤2^64−1、1≤m≤2^64−1；每行输出 a*b mod m 及 a^b mod m。用 unsigned long long 直接读写，不先转有符号类型；乘法前提升 uint128。模 1 两项均 0，零指数返回 1 mod m（含 0^0）；不要求素数或互质。读入失败、q 越界或模 0 返回 1，不作完整词法或 EOF 校验。须 64 位 unsigned long long 及 uint128 支持；不恢复已排除的 P1226。

所需模板：Mod64（第 15 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/mod64.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q) || q < 0 || q > 200000) return 1;
    while (q--)
    {
        unsigned long long a, b, m;
        if (!(cin >> a >> b >> m) || !m) return 1;
        cout << Mod64::mul(a, b, m) << ' ' << Mod64::power(a, b, m) << '\n';
    }
}
```

2.2 模幂与几何级数同时求值

power_sum(a,n,mod) 返回 {a^n mod mod, ∑(i=0..n-1) a^i mod mod}。a 为完整 unsigned long long，n 为 uint128 项数，mod 为正 unsigned long long。n=0 返回 {1%mod,0}；a=0 且 n>0 时和为 1%mod。无需逆元，合数模数也可用。时间与递归空间 O(log(n+1))，至多 128 次折半。只抄此函数需 utility、cassert。

```
pair<unsigned long long, unsigned long long>
power_sum(unsigned long long a, __uint128_t n, unsigned long long mod)
{
    assert(mod >= 1);
    if (!n) return {1 % mod, 0};
    auto [p, s] = power_sum(a, n / 2, mod);
    unsigned long long square = __uint128_t(p) * p % mod;
    unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
    if (n % 2)
        return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
    return {square, sum};
}
```

应用题补充：AtCoder ABC293 E Geometric Progression

题意：给定 1≤A,M≤10^9、1≤X≤10^12，求 ∑(i=0..X-1) A^i mod M。直接读取 power_sum(A,X,M).second；M 可能是合数，A-1 不一定可逆，不能无条件使用几何级数除法。M=1 输出 0，A=1 时结果 X mod M。第一分量为 A^X，本题不输出。此为比赛应用题，不计独立模板题覆盖。

所需模板：power_sum（第 16 页，2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/abc293/tasks/abc293_e

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, x, mod;
    cin >> a >> x >> mod;
    cout << power_sum(a, x, mod).second << '\n';
    return 0;
}
```

第三章　数论

3.1　数论基础

相关应用：约数和公式应用；需核对正文定位，导航无独立幂约数和页。

3.1.1　幂的约数和

代码依赖：PollardRho（第 45 页，3.6.1 节）、power_sum（第 16 页，2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

传入 a、b、mod 和可复用的 PollardRho 分解器 solver，返回 a^b 的正约数和取模。a、b、mod 均为 unsigned long long，a≥1、mod≥1。a=1 或 b=0 时结果为 1%mod。依赖 power_sum 与分解器；重数 e 对应 e*b+1 项，长度用 uint128。时间由分解及每个不同质因子的 O(log(e*b+2)) 求和组成。

```
unsigned long long divisor_sum_power(unsigned long long a,
                                     unsigned long long b,
                                     unsigned long long mod,
                                     PollardRho &solver)
{
    assert(a >= 1 && mod >= 1);
    if (!b) return 1 % mod;
    auto factors = solver.factor(a);
    unsigned long long answer = 1 % mod;
    for (int i = 0, j; i < (int)factors.size(); i = j)
    {
        j = i + 1;
        while (j < (int)factors.size() && factors[j] == factors[i]) j++;
        __uint128_t length = __uint128_t(j - i) * b + 1;
        auto sum = power_sum(factors[i], length, mod).second;
        answer = __uint128_t(answer) * sum % mod;
    }
    return answer;
}
```

模板题使用：Luogu P1593 因子和

题意：给定 1≤a≤50000000、0≤b≤50000000，求 a^b 的所有正约数之和模 9901。只分解 a，质因子重数 e 对应 e*b+1 项几何级数，无需构造 a^b 或求逆元。a=1 或 b=0 输出 1。PollardRho 可复用；本题模数固定，但核心允许任意正 uint64 模数，长度以 uint128 计算。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、power_sum（第 16 页，2.2 节）、divisor_sum_power（第 18 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1593

```
int main()
{
    unsigned long long a, b;
    scanf("%llu%llu", &a, &b);
    PollardRho solver;
    printf("%llu\n", divisor_sum_power(a, b, 9901, solver));
    return 0;
}
```

3.2 模算术简介

3.2.1 动态模整数与合数模逆元

代码依赖：Barrett32（第 35 页，3.2.5 节）。抄写时一并准备；标准库类型不列入算法依赖。

mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline Barrett32 reducer{998244353};
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        reducer = Barrett32(p);
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        r.v = reducer.mul(v, b.v);
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；传 move(a) 调用 batch_units，只求一次总积逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄；move 后不再读取 a。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：Barrett32（第 35 页，3.2.5 节）、mint（第 20 页，3.2.1 节）、batch_units（第 54 页，3.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_units(move(a));
    if (!inverse) return 1;
    mint<> answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

3.2.2 静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

联合使用示例见第 178 页（Luogu P1313 计算系数），其中直接调用本模板。

相关应用：完整 uint64 固定模整数与合数单位逆元，精确整数模乘替代依赖 long double 精度的实现

3.2.3 完整 64 位固定模整数

代码依赖：Mod64（第 15 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

StaticMod64<P> 支持 1≤P≤UINT64_MAX 的固定模数，可为合数。构造接受不超过 64 位的标准有符号/无符号整数，默认零；公开 v 为规范 uint64 余数，只读，不要转为 long long 输出大余数。加减用先比较后运算避免溢出；乘法依赖 Mod64 的 uint128 精确取余，不用浮点。加减乘 O(1)，支持一元负号、相等比较与复合赋值。

pow(k) 接受完整 uint64 非负指数，0^0=1 mod P，O(log(k+1))。try_inv() 用 int128 扩展欧几里得，gcd(v,P)≠1 时返回 nullopt；inv 及除法要求分母可逆，O(log P)。模 1 的唯一余数为 0，逆元也返回 0。构造用 int128 先求余再校正，支持 LLONG_MIN 与 UINT64_MAX；求逆系数用 int128，最终规范到 [0,P)。依赖 GNU/Clang 的 128 位整数；不支持把更宽整数直接传给构造函数。不提供字符串流重载：按题目读 signed64 或 unsigned64 后构造，输出 v。

```
template <uint64_t P> struct StaticMod64
{
    static_assert(P > 0);
    uint64_t v = 0;

    StaticMod64() = default;

    template <integral T> StaticMod64(T x)
    {
        static_assert(sizeof(T) <= 8);
        __int128_t r = __int128_t(x) % __int128_t(P);
        if (r < 0)
            r += P;
        v = r;
    }

    StaticMod64 operator+(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = v >= P - b.v ? v - (P - b.v) : v + b.v;
        return r;
    }

    StaticMod64 operator-(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = v >= b.v ? v - b.v : P - (b.v - v);
        return r;
    }
```

64 位模整数（接上页同一结构体）：

```
    StaticMod64 operator-() const
    {
        StaticMod64 r;
        r.v = v == 0 ? 0 : P - v;
        return r;
    }

    StaticMod64 operator*(StaticMod64 b) const
    {
        StaticMod64 r;
        r.v = Mod64::mul(v, b.v, P);
        return r;
    }

    StaticMod64 &operator+=(StaticMod64 b)
    {
        return *this = *this + b;
    }

    StaticMod64 &operator-=(StaticMod64 b)
    {
        return *this = *this - b;
    }

    StaticMod64 &operator*=(StaticMod64 b)
    {
        return *this = *this * b;
    }

    StaticMod64 pow(uint64_t k) const
    {
        StaticMod64 r;
        r.v = Mod64::power(v, k, P);
        return r;
    }
```

64 位模整数（接上页同一结构体）：

```
    optional<StaticMod64> try_inv() const
    {
        __int128_t a = v, b = P, x = 1, y = 0;
        while (b)
        {
            __int128_t q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        if (x < 0)
            x += P;
        return StaticMod64(uint64_t(x));
    }

    StaticMod64 inv() const
    {
        auto r = try_inv();
        assert(r.has_value());
        return *r;
    }

    StaticMod64 operator/(StaticMod64 b) const
    {
        return *this * b.inv();
    }

    StaticMod64 &operator/=(StaticMod64 b)
    {
        return *this = *this / b;
    }

    bool operator==(const StaticMod64 &) const = default;
};
```

接口用法演示：完整 64 位模整数与合数逆元 API

演示约定：固定合数模 UINT64_MAX。每组输入 signed64 整数 a、uint64 整数 b 和非负指数 k，输出 a、b 规范余数及 a+b、a-b、a*b、a^k、b 逆元；b 不可逆输出-1。大余数直接输出 uint64 成员 v，不能先转 signed64。

所需模板：Mod64（第 15 页，2.1 节）、StaticMod64（第 27 页，3.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/STATIC-MOD64.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = StaticMod64<UINT64_MAX>;
    int q;
    cin >> q;
    while (q--)
    {
        long long a;
        uint64_t b, k;
        cin >> a >> b >> k;
        Z x(a), y(b);
        auto inv = y.try_inv();
        cout << x.v << ' ' << y.v << ' ';
        cout << (x + y).v << ' ' << (x - y).v << ' ';
        cout << (x * y).v << ' ' << x.pow(k).v << ' ';
        if (inv)
            cout << inv->v << '\n';
        else
            cout << -1 << '\n';
    }
}
```

相关应用：奇数固定模数的 Montgomery 表示及整数约减；辅助原根/平方根另列

3.2.4 Montgomery 固定模整数

Montgomery<P> 为可选固定模整数，要求奇数 1<P<2^30，允许合数。构造接受完整 long long 并正规化；get() 返回普通余数。v 是 [0,P) 内的 Montgomery 表示，只读，不能作为普通余数使用或直接赋值。加减乘与一元负号 O(1)，乘法不用除法；比较直接比较规范内部表示。pow(k) 保留完整 uint64 非负指数，约定 0^0=1，不把指数对 P-1 取模。try_inv() 用扩展欧几里得，非单位返回 nullopt；inv 与除法要求除数可逆，复杂度 O(log P)。

内部取 R=2^32, neg_inv=-P^{-1} mod R, r2=R^2 mod P; reduce(x) 要求 x<P*R, 以 q=(x mod R)*neg_inv mod R 使 x+qP 能被 R 整除，再取高 32 位并最多减 P。x+qP<2PR<2^63；核心只用 uint64。乘积和构造均满足输入界。inverse32/reduce 为内部辅助；原根、模平方根另用已有模板。本类型不替换普通 ModInt 或动态 mint，不保证所有平台都比编译期常模取余快。

```
// Odd modulus, including composites. v is a Montgomery residue; use get().
template <uint32_t P> struct Montgomery
{
    static_assert(P > 1 && P < (1U << 30) && (P & 1));

    static constexpr uint32_t inverse32()
    {
        uint32_t x = P;
        for (int i = 0; i < 4; i++)
            x *= 2U - P * x;
        return -x;
    }

    static constexpr uint32_t neg_inv = inverse32();
    static constexpr uint32_t r2 = -uint64_t(P) % P;
    uint32_t v = 0;

    // Internal: x < P * 2^32. The returned residue lies in [0, P).
    static uint32_t reduce(uint64_t x)
    {
        uint32_t q = uint32_t(x) * neg_inv;
        uint32_t y = (x + uint64_t(q) * P) >> 32;
        return y >= P ? y - P : y;
    }

    Montgomery(long long x = 0)
    {
        long long y = x % P;
        if (y < 0)
            y += P;
        v = reduce(uint64_t(y) * r2);
    }

    uint32_t get() const
    {
        return reduce(v);
    }
```

模整数（接上页同一结构体）：

```
    Montgomery operator+(Montgomery b) const
    {
        Montgomery r;
        r.v = v + b.v;
        if (r.v >= P)
            r.v -= P;
        return r;
    }

    Montgomery operator-(Montgomery b) const
    {
        Montgomery r;
        r.v = v >= b.v ? v - b.v : v + P - b.v;
        return r;
    }

    Montgomery operator-() const
    {
        Montgomery r;
        r.v = v == 0 ? 0 : P - v;
        return r;
    }

    Montgomery operator*(Montgomery b) const
    {
        Montgomery r;
        r.v = reduce(uint64_t(v) * b.v);
        return r;
    }

    Montgomery &operator+=(Montgomery b)
    {
        return *this = *this + b;
    }

    Montgomery &operator-=(Montgomery b)
    {
        return *this = *this - b;
    }

    Montgomery &operator*=(Montgomery b)
    {
        return *this = *this * b;
    }
```

模整数（接上页同一结构体）：

```
    Montgomery pow(uint64_t k) const
    {
        Montgomery a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }

    optional<Montgomery> try_inv() const
    {
        long long a = get(), b = P, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return Montgomery(x);
    }

    Montgomery inv() const
    {
        auto r = try_inv();
        assert(r.has_value());
        return *r;
    }

    Montgomery operator/(Montgomery b) const
    {
        return *this * b.inv();
    }

    Montgomery &operator/=(Montgomery b)
    {
        return *this = *this / b;
    }

    bool operator==(const Montgomery &) const = default;
};
```

接口用法演示：Montgomery 固定模整数 API

演示约定：固定模 998244353。每组输入 signed64 整数 a、b 与 uint64 非负指数 k，依次输出 a+b、a-b、a*b、a^k、b 逆元，均正规化；b 不可逆输出-1。内部 v 不能直接输出，须 get()。

所需模板：Montgomery（第 31 页，3.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/MONTGOMERY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = Montgomery<998244353>;
    int q;
    cin >> q;
    while (q--)
    {
        long long a, b;
        uint64_t k;
        cin >> a >> b >> k;
        Z x(a), y(b);
        auto inv = y.try_inv();
        cout << (x + y).get() << ' ' << (x - y).get() << ' ';
        cout << (x * y).get() << ' ' << x.pow(k).get() << ' ';
        cout << (inv ? (long long)inv->get() : -1) << '\n';
    }
}
```

相关应用：预计算倒数的 32 位 Barrett 模乘；不代表 64 位浮点快速乘或 Montgomery

3.2.5 32 位 Barrett 快速模乘

Barrett32(m) 预计算运行时模数的 64 位倒数，1≤m≤UINT32_MAX；mul(a,b) 要求 0≤a,b<m，返回 a*b mod m 的规范 uint32 余数。构造 O(1) 且含一次 64 位除法，此后的每次乘法只用整数乘法、取高位和一次校正，不使用浮点数或整数除法。对象可复制，多个对象的模数互不影响；mod 和 reciprocal 只读，需要换模时重新构造。

令 R=2^64，m>1 时 reciprocal=ceil(R/m)，估计商只可能等于真实商或多 1。用 64 位 z<q*m 判断是否多减一次，再补回 m；不要先截成 32 位后比较，这在大于 INT_MAX 的模数下会错误。m=1 时规范输入只能为 0，倒数按 unsigned64 回绕为 0，返回 0。依赖 GNU/Clang 的 uint128 乘法；不适用于 64 位模数，后者仍用 Mod64。

```
struct Barrett32
{
    uint32_t mod;
    uint64_t reciprocal;

    Barrett32(uint32_t m) : mod(m)
    {
        assert(m != 0);
        reciprocal = UINT64_MAX / m + 1;
    }

    // Requires a < mod and b < mod.
    uint32_t mul(uint32_t a, uint32_t b) const
    {
        uint64_t z = uint64_t(a) * b;
        uint64_t q = (__uint128_t(z) * reciprocal) >> 64;
        uint64_t y = q * mod;
        return uint32_t(z - y + (z < y ? mod : 0));
    }
};
```

接口用法演示：Barrett 规范余数模乘 API

演示约定：输入非零 uint32 模数 m 与询问数，每组 a b 满足 0≤a,b<m，输出 a*b 模 m。支持 m=1 和大于 INT_MAX 的模数；输入本身不正规化。

所需模板：Barrett32（第 35 页，3.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BARRETT32.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    uint32_t mod;
    int tests;
    cin >> mod >> tests;
    Barrett32 reducer(mod);
    while (tests--)
    {
        uint32_t a, b;
        cin >> a >> b;
        cout << reducer.mul(a, b) << '\n';
    }
}
```

3.2.6 模运算的适用条件

模逆元存在当且仅当 $\gcd(a,m)=1$。费马逆元 a^{p-2} 仅适用于素数模数及 $a\not\equiv 0$。对正模数 m,n，同余式 $x\equiv a \pmod m$、$x\equiv b \pmod n$ 可合并当且仅当 $\gcd(m,n)\mid(b-a)$；新模数为 $\operatorname{lcm}(m,n)$。

$$\binom{n}{k}\equiv\prod_i\binom{n_i}{k_i}\pmod p \quad (p \text{ 为素数，} n_i,k_i \text{ 是}p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。素数模数下，阶乘与逆阶乘预处理要求上限小于模数；达到模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

3.2.6.1 除法与解同余式不是一回事

以下先设 $m\ge 2$。把除以 a 写成乘逆元，要求 a 是单位元，即 $\gcd(a,m)=1$。若 $g=\gcd(a,m)>1$，并非所有含 a 的同余式都无解：

$$ax+b\equiv 0 \pmod m \iff \frac{a}{g}x+\frac{b}{g}\equiv 0 \pmod{m/g} \qquad (g\mid b).$$

$g\nmid b$ 时无解；否则约掉 g 后系数与模数互质，在模 m/g 下恰有一个解 x_0。原模 m 的全部不同解为 $x_0+t(m/g)$，$0\le t<g$。约去公因子时模数也要除以 g，不能从 $2x\equiv 2 \pmod 6$ 错误推出 $x\equiv 1 \pmod 6$；实际是 $x\equiv 1 \pmod 3$，在 $[0,6)$ 内有 1、4 两个解。而 $2x\equiv 1 \pmod 6$ 无解，二者都不能用“除以 2”统一处理。

本库 `linear_congruence(a,b,m)` 的符号约定就是 $ax+b\equiv 0$，返回最小非负解和最小正周期；无解返回 $(-1,-1)$。例如 `linear_congruence(2,-2,6)` 返回 $(1,3)$。若题面是 $ax\equiv c$，转换常数时不能在 signed64 中直接对最小负数取负；应先把右端规范化到 $[0,m)$ 再变号传入；只在 128 位中取负后直接窄化回 signed64 仍不安全。实现见第 3.10.1 节（第 59 页）。

单逆元 `mod_inverse` 不可逆返回 -1，`ModInt::try_inv` 不可逆返回 `nullopt`，不是同一种失败表示。`inv` 和除法仍以可逆为前提；关闭断言不意味着非法除法会被安全拒绝。分别见第 3.9.2 节（第 55 页）和第 3.2.2 节（第 24 页）。$m=1$ 在这些整数模接口中另按唯一剩余类处理：代表为 0，逆元也返回 0，同余式周期为 1；不能因此把只适用于域的高斯、NTT、FPS 或组合数条件推广到模 1。

3.2.6.2 知道整数可整除时，先升模数再整除

若已知整数 X 被正整数 d 整除，目标是 $(X/d) \bmod m$（$m \geq 1$），即使 $\gcd(d, m) > 1$，仍可先计算规范剩余 $r = X \bmod (md)$，使 $0 \leq r < md$。写成 $X = qmd + r$，可知 $d \mid r$，且

$$\frac{X}{d} = qm + \frac{r}{d} \equiv \frac{r}{d} \pmod{m}.$$

这是整数整除，不是模逆元。负 X 也成立，但须先规范化剩余，不能直接依赖 C++ 负余数。例如 $X = 6, d = 2, m = 4$，正确答案是 3；先取 $X \bmod m = 2$ 再除以 2 会错误得到 1。只知道 $X \bmod m$ 不足以恢复商：$X = 2$ 与 6 同余模 4，其除以 2 后的余数却不同。

必须保证能计算 md 并能在这个新模数下求出 X；先扩宽乘法、检查范围，乘模的中间乘积同样要安全，必要时用多精度。提升后的模数可能是合数，不能简单给素数模 Binomial、NTT 或费马逆元更换参数。若所有生成 X 的运算本来只有加法与乘法，可以在新模数下直接进行；含除法时要重新验证。轨道计数中的群阶整除应用见第 5.7.7 节（第 209 页），不重复抄一套专门模板。

3.2.6.3 组合数的整数性不等于分母可逆

p 为素数且预处理 $N < p$ 时，$N!$ 才能保证非零可逆，适用现有 `Binomial` 契约。仅有 $N < m$ 在合数模下不够，例如 $2! \bmod 6 = 2$ 没有逆元。二项式系数作为整数始终存在，不代表阶乘比值可以逐项在任意模数下相除。大下标、素数模考虑 Lucas；合数模考虑素数幂分解、去除质因子后的单位部分和 CRT，使用现有 exLucas 的条件与资源限制，不把素数模版直接套过去。接口和边界见第 5.1.1 节（第 177 页）。同理，插值中的节点差、级数积分的分母以及模概率的分母，都须各自检查可逆性。

3.2.6.4 大指数要同时保留余数和阈值信息

对于整数 $a \geq 0$、非负整数指数 b 与正模数 m，令 $\phi = \varphi(m)$。一个足够但不一定最小的安全降幂规则是

$$a^b \equiv a^e \pmod{m}, \qquad e = \begin{cases} b, & b < \phi, \\ b \bmod \phi + \phi, & b \geq \phi. \end{cases}$$

约定零次幂为 1，结果再对 m 取模；$m = 1$ 返回 0。若 $\gcd(a, m) = 1$，直接对 ϕ 取模也可；不互质时则需阈值。例如 $2^4 \bmod 8 = 0$，只将 4 对 $\varphi(8) = 4$ 取模会得到 $2^0 \bmod 8 = 1$。反过来，对小指数无条件加 ϕ 也错：$2^1 \bmod 8 = 2$，而 $2^5 \bmod 8 = 0$。

为什么这个阈值足够。 分解 $m = \prod p^k$。若 $p \nmid a$，欧拉定理给出周期 $\varphi(p^k)$，它整除 ϕ。若 $p \mid a$，只要指数至少 k，幂就为 0 模 p^k；而 $\phi \geq \varphi(p^k) = p^{k-1}(p-1) \geq k$。因此 $b \geq \phi$ 时原指数和降幂后的指数在每个素数幂分量都一致，再由 CRT 合并。$b < \phi$ 时保留原指数，不需要讨论何时进入周期。

十进制字符串可逐位维护 $b \bmod \phi$ 和饱和值 $\min(b, \phi)$，只需知道是否达到阈值，不必保存完整大整数；前导零不改变它们。现有 `euler_power(a,b,m,phi)` 要求调用者传入*真实的* $\varphi(m)$，其断言不能证明传入值就是欧拉函数。模数分解/求 ϕ 的成本需另算。降幂后的指数可能需要 65 位，不能用 unsigned64 暂存相加结果；本实现内部用 128 位。接口见第 3.8.1 节（第 52 页）。本库基数/模数为 unsigned64，不能把负数直接窄化后当成数学上的规范取模。费马、欧拉与线性同余的基础背景可参见 MIT 18.781 第 4 讲；这里的非互质阈值和升模整除附有独立推导，不借用素数模 API 扩大适用范围。

3.3 素数

3.3.1 确定性 64 位 Miller–Rabin 判素

代码依赖：Mod64（第 15 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Prime64::prime(n) 接收完整 unsigned long long，n<2 返回 false。先试除小素数，再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数，结果在 64 位输入范围内确定。继承 Mod64，模乘用 uint128；O(log n) 次模乘，O(1) 空间，不包含因数分解。

```
struct Prime64 : Mod64
{
    static bool prime(ull n)
    {
        if (n < 2) return false;
        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
            if (n % p == 0) return n == p;
        ull d = n - 1;
        int s = 0;
        while (!(d & 1))
        {
            d >>= 1;
            ++s;
        }
        for (ull a :
             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
        {
            if (a % n == 0) continue;
            ull x = power(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool ok = false;
            for (int j = 1; j < s; j++)
            {
                x = mul(x, x, n);
                if (x == n - 1)
                {
                    ok = true;
                    break;
                }
            }
            if (!ok) return false;
        }
        return true;
    }
};
```

模板题使用：Library Checker primality_test

题意：逐个判断给定正整数是否为素数，输出 Yes 或 No。此题输入不超过 10^{18}；模板支持的完整范围另见接口约定。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/primality_test

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        unsigned long long n;
        cin >> n;
        cout << (Prime64::prime(n) ? "Yes" : "No") << '\n';
    }
}
```

3.4 欧拉函数

3.4.1 单个数的欧拉函数

代码依赖：PollardRho（第 45 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

euler_phi(n) 使用试除，$O(\sqrt{n})$ 时间、O(1) 额外空间；euler_phi(n,solver) 复用 PollardRho，时间由随机分解主导，分解结果与递归占 O(log n) 空间。$1 \le n \le$ ULLONG_MAX，phi(1)=1。大素数或平衡大半素数应选分解器入口，不要用试除穷扫平方根；批量求 1..N 用 LinearSieve。先除质因子再乘 p-1，结果及中间乘积均不超过原 n。只抄试除函数时不需要 Prime64/PollardRho。

```
unsigned long long euler_phi(unsigned long long n)
{
    assert(n >= 1);
    unsigned long long answer = n;
    for (unsigned long long p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            answer = answer / p * (p - 1);
            while (n % p == 0) n /= p;
        }
    if (n > 1) answer = answer / n * (n - 1);
    return answer;
}

unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
{
    assert(n >= 1);
    auto factors = solver.factor(n);
    unsigned long long answer = n;
```

```
    unsigned long long previous = 0;
    for (auto p : factors)
        if (p != previous)
        {
            answer = answer / p * (p - 1);
            previous = p;
        }
    return answer;
}
```

联合使用示例见第 52 页（Luogu P5091），其中直接调用本模板。

3.5 筛法

3.5.1 线性筛、欧拉函数与 Möbius 函数

构造参数 0≤n<INT_MAX，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为 0..n 数组。x≥2 时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。n≥1 时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时 O(log x)。n=0 不可访问下标 1。

```
struct LinearSieve
{
    vector<int> prime, lp, phi, mu;

    LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
    {
        if (n) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!lp[i])
            {
                lp[i] = i;
                prime.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prime)
            {
                if (p > n / i) break;
                int j = i * p;
                lp[j] = p;
                if (i % p == 0)
                {
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    break;
                }
                phi[j] = phi[i] * (p - 1);
                mu[j] = -mu[i];
            }
        }
    }
};
```

应用题补充：Luogu P2158 [SDOI2008] 仪仗队

题意：从方阵一角看可见格点，观察者所在原点不计；1≤N≤40000。坐标范围 0..N−1，可见当且仅当两坐标互素。N=1 输出 0；其余用 1+2 倍 phi[1..N−1] 之和，注意 phi(1)=1。先筛到 N−1，再累加 phi；时间、空间均 O(N)，答案用 long long。

所需模板：LinearSieve（第 42 页，3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2158

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    LinearSieve sieve(n - 1);
    long long ans = 1;
    for (int i = 1; i < n; i++) ans += 2LL * sieve.phi[i];
    cout << ans << '\n';
}
```

3.5.2 区间筛素数

segmented_primes(l,r) 返回闭区间内递增的全部素数，0≤l≤r≤INT_MAX；区间长度须能分配内存。独立函数，每次重建底筛；不依赖预存素数表。令 W=r-l+1，时间 O((√r+W)log log(r+3))，空间 O(√r+W)。倍数起点、递增变量均用 long long，避免 INT_MAX 端点溢出；0 和 1 不进入结果。

```
vector<int> segmented_primes(int l, int r)
{
    assert(0 <= l && l <= r);
    int limit = 0;
    while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
    vector<char> composite(limit + 1);
    vector<char> removed(1LL * r - l + 1);
    for (int p = 2; p <= limit; p++)
        if (!composite[p])
        {
            for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
            long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
            for (long long j = start; j <= r; j += p) removed[j - l] = true;
        }
    vector<int> primes;
    for (long long x = max(l, 2); x <= r; x++)
        if (!removed[x - l]) primes.push_back(x);
    return primes;
}
```

应用题补充：POJ 2689 / UVA 10140 Prime Distance

题意：读闭区间 [L,U] 至 EOF，筛出递增素数，再比较相邻素数差；最近/最远均取最先出现的并列对，更新必须用严格 < 和 >。不足两个素数输出 There are no adjacent primes.，否则保留驱动的逗号、空格和句点格式。无 0 0 结束标记，每次调用重建筛。UVA 官方同题限制 1≤L<U≤2147483647、U-L≤1000000，所以闭区间容量是 U-L+1≤1000001；核心另支持 0 和单点，0/1 不为素数。倍数及末端递增用 long long，避免 INT_MAX 溢出。POJ 原站本次不可达；限制和样例仅据 UVA 官方 PDF 复核，不声称 POJ 限制或线上 AC 已复核。

所需模板：segmented_primes（第 43 页，3.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/101/10140.pdf

```
int main()
{
    int l, r;
    while (scanf("%d%d", &l, &r) == 2)
    {
        auto primes = segmented_primes(l, r);
        if (primes.size() < 2)
        {
            puts("There are no adjacent primes.");
            continue;
        }
        int closest = 1;
        int distant = 1;
        for (int i = 2; i < (int)primes.size(); i++)
        {
            int gap = primes[i] - primes[i - 1];
            if (gap < primes[closest] - primes[closest - 1]) closest = i;
            if (gap > primes[distant] - primes[distant - 1]) distant = i;
        }
        printf("%d,%d are closest, %d,%d are most distant.\n",
               primes[closest - 1],
               primes[closest],
               primes[distant - 1],
               primes[distant]);
    }
    return 0;
}
```

3.6 分解质因数

3.6.1 Pollard–Rho 因数分解

代码依赖：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

factor(n) 返回升序质因子（含重数），1≤n≤ULLONG_MAX。n=1 返回空数组；n≥2 时 size()==1 等价于 n 为素数，否则 front()/back() 分别取最小/最大质因子。依赖 Mod64 与 Prime64。公开使用 factor；rho/split 为内部辅助，rho 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 rho 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 O(log n)。

```
struct PollardRho
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    mt19937_64 rng;

    PollardRho(ull seed = 712367821) : rng(seed) {}

    ull rho(ull n)
    {
        if (n % 2 == 0) return 2;
        for (;;)
        {
            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
            auto f = [&](ull v)
            {
                return (Mod64::mul(v, v, n) + u128(c)) % n;
            };
            // Retry bounded attempts; Las Vegas: only exact divisors returned.
            for (int i = 0; i < 200000 && d == 1; i++)
            {
                x = f(x);
                y = f(f(y));
                d = gcd(x > y ? x - y : y - x, n);
            }
            if (1 < d && d < n) return d;
        }
    }

    void split(ull n, vector<ull> &a)
    {
        if (n == 1) return;
        if (Prime64::prime(n))
        {
            a.push_back(n);
            return;
        }
        ull d = rho(n);
        split(d, a);
        split(n / d, a);
    }
```

```
42
43      vector<ull> factor(ull n)
44      {
45          assert(n >= 1);
46          vector<ull> a;
47          split(n, a);
48          sort(a.begin(), a.end());
49          return a;
50      }
51  };
```

模板题使用：Luogu P4718

题意：逐个判断 n 是否为素数，是则输出 Prime，否则输出最大质因子。题目保证 n≥2，factor 返回升序含重数的质因子，因此可取 back；若用于 n=1，须先处理空结果。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4718

```
1   int main()
2   {
3       int t;
4       scanf("%d", &t);
5       PollardRho solver;
6       while (t--)
7       {
8           unsigned long long n;
9           scanf("%llu", &n);
10          auto factors = solver.factor(n);
11          if (factors.size() == 1)
12              puts("Prime");
13          else
14              printf("%llu\n", factors.back());
15      }
16      return 0;
17  }
```

3.7 裴蜀定理 & 一次不定方程

3.7.1 扩展欧几里得与 Bézout 系数

extended_gcd(a,b,x,y) 约定 0≤a,b≤2^63，参数和输出系数为 int128，返回非负 gcd 并满足 ax+by=gcd(a,b)。a=b=0 返回 0、x=1、y=0；有符号线性方程用 linear_equation。递归实现，中间乘加为 int128；O(log(min(a,b)+1)) 时间与递归空间。

```
__int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    i128 d = extended_gcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

模板题使用：AOJ NTL_1_E Extended Euclid Algorithm

题意：给出正整数 1≤a,b≤10^9，求 ax+by=gcd(a,b)，先最小化 |x|+|y|，仅在并列最小时取 x≤y。直接调用 extended_gcd；该正整数范围内递归所得解已满足最小性，a=b 时返回 (0,1)。系数用 ___int128_t 接收，输出转 long long 安全；不应对所有答案强行要求 x≤y。时间、递归空间均 O(log(min(a,b)+1))。

所需模板：extended_gcd（第 47 页，3.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    __int128_t x, y;
    extended_gcd(a, b, x, y);
    printf("%lld %lld\n", (long long)x, (long long)y);
    return 0;
}
```

3.7.2 二元一次不定方程

代码依赖：extended_gcd（第 47 页，3.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

linear_equation(a,b,c,x,y) 求 ax+by=c 的任意整数特解。a、b、c 支持完整 long long，x、y 必须为 int128。无解返回 false 且 x=y=0；a=b=c=0 返回 true、x=y=0，此时所有整数对都是解。其他有解情况，令 g=gcd(|a|,|b|)，通解为 (x+(b/g)t,y-(a/g)t)。依赖 extended_gcd；只需抄递归 exgcd 与本函数。时间、递归空间 O(log M)，其中 M=max(|a|,|b|)+2；不承诺特解最小或为正。

```
bool linear_equation(
    long long a, long long b, long long c, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    x = 0;
    y = 0;
    if (!a && !b) return c == 0;
    i128 aa = a < 0 ? -i128(a) : i128(a);
    i128 bb = b < 0 ? -i128(b) : i128(b);
    i128 u, v;
    i128 g = extended_gcd(aa, bb, u, v);
    if (c % g) return false;
    x = u * (c / g);
    y = v * (c / g);
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    return true;
}
```

模板题使用：Luogu P5656

题意：解 ax+by=c，题目 a、b、c 均为正数。无整数解输出 -1；有正整数解则输出数量及 x、y 的最小/最大值，否则输出 x、y 各自的最小正值，它们不一定构成同一组解。正整数不含 0。

所需模板：extended_gcd（第 47 页，3.7.1 节）、linear_equation（第 48 页，3.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5656

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long a, b, c;
        scanf("%lld%lld%lld", &a, &b, &c);
        __int128_t x, y;
        if (!linear_equation(a, b, c, x, y))
        {
            puts("-1");
            continue;
        }
        long long g = gcd(a, b);
        long long dx = b / g;
        long long dy = a / g;
        long long xmin = (x % dx + dx - 1) % dx + 1;
        long long ymin = (y % dy + dy - 1) % dy + 1;
        long long ymax = (c - a * xmin) / b;
        if (ymax <= 0)
        {
            printf("%lld %lld\n", xmin, ymin);
            continue;
        }
        long long count = (ymax - 1) / dy + 1;
        long long xmax = xmin + (count - 1) * dx;
        printf("%lld %lld %lld %lld %lld\n", count, xmin, ymin, xmax, ymax);
    }
    return 0;
}
```

3.7.3 一次不定方程：特解与正整数解范围

实现见第 3.7.2 节（第 48 页）。对不全为零的 a, b，令 $g = \gcd(|a|, |b|) > 0$。递归扩展欧几里得先求 $|a|u+|b|v = g$；当且仅当 $g \mid c$ 时有整数解。把系数乘以 c/g，再恢复 a, b 的符号，就得到 $ax_0 + by_0 = c$。两个特解相减，再利用 $a/g, b/g$ 互素，可知所有解恰为

$$x = x_0 + \frac{b}{g}t, \qquad y = y_0 - \frac{a}{g}t, \qquad t \in \mathbb{Z}.$$

若 $a = b = 0$，则 $c = 0$ 时所有整数对都是解，否则无解；此时不能使用除以 g 的通解式。

底层 `exgcd` 的本库调用约定为非负系数，上限 2^{63}；返回非负的 g，$(0, 0)$ 约定返回 0。包装函数先转为 128 位再取相反数，因而可以处理 `LLONG_MIN`。缩放后的特解也用 128 位保存；不要提前转回 `long long`。即使特解可表示，直接验证 $ax + by$ 的两个乘积仍可能超过 128 位，本地验证使用任意精度整数。

对 P5656 的正系数 a, b, c，设 $d_x = b/g, d_y = a/g$，先将 x_0 调整到 $x_{\min} \in [1, d_x]$，并计算与它配对的 $y_{\max} = (c - ax_{\min})/b$。若 $y_{\max} > 0$，正整数解个数为

$$N = \left\lfloor \frac{y_{\max} - 1}{d_y} \right\rfloor + 1, \quad x_{\max} = x_{\min} + (N-1)d_x, \quad y_{\min} = y_{\max} - (N-1)d_y.$$

否则不存在同时为正的解，但 x 和 y 仍各自有最小正值；分别按周期归一化即可，它们通常不是同一组解。正值归一化使用 `(v % d + d - 1) % d + 1`，不要把余数 0 当作正整数。驱动中 $a, b, c \le 10^9$，归一化后的乘积不超过 10^{18}，故该部分可用 64 位。

双版本以小范围有符号输入穷举、任意精度等式证书、64 位端点和斐波那契递归链验证。完整 P5656 驱动另验证 200000 组输入，包含正解数量与四个极值、无解和无正解分支；普通与 ASan/UBSan 测试均通过，在线 AC 待补。

来源：kuangbin 2.3；WIDA 数论打印稿的 exgcd 与不定方程例题。

https://oi-wiki.org/math/number-theory/bezouts/。

底层接口见第 3.7.1 节（第 47 页）。

3.7.3.1 有符号矩形范围：把两维约束化成一个参数区间

现在允许 a, b, c 为负或零，要求

$$L_x \le x \le R_x, \qquad L_y \le y \le R_y.$$

先处理空区间；任意一维下界大于上界，答案为 0。若 $a = b = 0$，$c \neq 0$ 时无解；$c = 0$ 时答案为 $(R_x - L_x + 1)(R_y - L_y + 1)$，不能套用一维参数族。否则先检查 $g \mid c$，再取得上面的特解与步长 $d_x = b/g$、$d_y = -a/g$。不要把负系数或负步长直接取绝对值而保留旧特解。

对任一坐标 $z = z_0 + dt$，条件 $L \le z \le R$ 给出：

$$\begin{array}{r|l} d > 0 & \lceil (L - z_0)/d \rceil \le t \le \lfloor (R - z_0)/d \rfloor \\ d < 0,\ k = -d > 0 & \lceil (z_0 - R)/k \rceil \le t \le \lfloor (z_0 - L)/k \rfloor \\ d = 0 & L \le z_0 \le R \text{ 时不限制} t\text{，否则无解} \end{array}$$

例如 $6x - 9y = 3$，$x \in [-5, 4], y \in [-1, 5]$ 时，解恰为 $(2, 1)$ 和 $(-1, -1)$。分别计算 x, y 允许的区间，取交集 $[T_l, T_r]$；解数是 $\max(0, T_r - T_l + 1)$。不全为零的系数保证至少一个步长非零，有限矩形因此会给出有限参数区间；实现时用“尚无上下界”标记，不必猜一个足够大的哨兵。通解对整数 t 是单射，且已包含全部整数解，所以交集计数没有遗漏或重复。若要实际列举答案，再依次代入这些整数 t，输出成本为 $O(\text{解数})$。

这里的 floor/ceil 是数学取整，不能用 C++ 的负数向零截断替代。对正分母 D，令 C++ 商余为 $q = n/D, r = n\%D$，则

$$\lfloor n/D \rfloor = q - [r < 0], \qquad \lceil n/D \rceil = q + [r > 0].$$

例如 $-5/2$ 的 floor 是 -3、ceil 是 -2。不要使用只对非负数成立的 `(n+D-1)/D`，也不要在未证明安全时先计算 `-n`。参数平移 $x_0' = x_0 + d_x s, y_0' = y_0 + d_y s$ 只会把允许的 t 整体平移 $-s$，答案不变；这是检查实现符号的一个独立不变量。

范围运算比求一个特解要求更强的整数宽度：即使输入端点都是 64 位，差值和区间长度也未必能用 64 位保存。当两系数全零、矩形覆盖全部有符号 64 位坐标时，答案为 2^{128}，连无符号 128 位也放不下。本节通式按精确整数理解；实现前证明每一步范围，不能证明时使用任意精度整数。尤其不要用窄类型计算长度后才转宽，也不能在等式证书中先让 ax, by 溢出再相消。求特解之后只需常数次整数运算；大整数版本的位复杂度取决于操作数长度，不能据此承诺固定字长常数耗时。

本节的有界小整数矩形用逐点枚举核对，并检查方程同乘负一、交换变量、特解平移、零系数、无解与 64 位端点构造。这验证本地公式和建模，不新增模板、正式题覆盖或线上 AC。

3.8 费马小定理 & 欧拉定理

3.8.1 十进制大指数：扩展欧拉降幂

代码依赖：Mod64（第 15 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

euler_power(a,b,m,phi) 求 a^b mod m，a、m 为 unsigned long long，m>0；b 为非空十进制数字串，可为 0、可含前导零，约定 0^0=1。调用者必须传入正确的 phi=φ(m)，包括 $\varphi(1)=1$；函数只检查数值范围，不验证 phi 正确性。指数小于 phi 时保持原值，否则使用 b mod phi + phi，非互质底数也适用。相加和十进制更新都用 int128，支持完整 unsigned64 模数。O(|b|+log m) 时间、O(1) 额外空间，不含求 phi 的成本；依赖 Mod64::mul。

```
unsigned long long euler_power(unsigned long long a,
                               string_view b,
                               unsigned long long m,
                               unsigned long long phi)
{
    using U = unsigned long long;
    using I = __uint128_t;
    assert(m && phi && phi <= m && !b.empty());
    U rem = 0, cap = 0;
    for (char c : b)
    {
        assert('0' <= c && c <= '9');
        int d = c - '0';
        rem = (I(rem) * 10 + d) % phi;
        cap = min(I(phi), I(cap) * 10 + d);
    }
    I e = I(rem) + (cap == phi ? I(phi) : 0);
    U ans = 1 % m;
    a %= m;
    for (; e; e >>= 1, a = Mod64::mul(a, a, m))
        if (e & 1) ans = Mod64::mul(ans, a, m);
    return ans;
}
```

模板题使用：Luogu P5091

题意：输入顺序为底数 a、模数 m、指数 b；b 最多 20000001 位，以字符串读入。先用试除法计算 φ(m)，再做扩展欧拉降幂；题目 m≤10^8，不能把输入顺序写成 a,b,m。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、euler_phi（第 41 页，3.4.1 节）、euler_power（第 52 页，3.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, m;
    string b;
    cin >> a >> m >> b;
    auto phi = euler_phi(m);
    cout << euler_power(a, b, m, phi) << '\n';
}
```

相关应用：公共指数与欧拉定理相关；导航无独立 Carmichael 页，不伪造叶节点。

3.8.2 Carmichael：模乘法群的公共指数

carmichael(n) 返回 $\lambda(n)$，即使所有与 n 互素的 a 都满足 a 的 $\lambda(n)$ 次幂模 n 为 1 的最小正指数；约定 $\lambda(1)=1$。要求 $1\le n\le$INT_MAX。试除分解后将各素数幂的指数取 lcm，特别处理 2 的三次及以上幂。时间 $O(\sqrt{n})$，额外空间 O(1)；所有中间值不超过有符号 64 位范围，结果不超过 n。该指数只适用于可逆底数，不能据此直接对非互素底数的幂降指数。函数不依赖原根模块。

```
int carmichael(int n)
{
    assert(n >= 1);
    long long answer = 1;
    for (int p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            int power = 1;
            while (n % p == 0)
            {
                n /= p;
                power *= p;
            }
            int value = power / p * (p - 1);
            if (p == 2 && power >= 8) value /= 2;
            answer = answer / gcd(answer, (long long)value) * value;
        }
    if (n > 1) answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
    return answer;
}
```

应用题补充：UVa 10006 Carmichael Numbers

题意：多组 3≤n<65000，输入 0 结束，判定 n 是否为 Carmichael 数：必须为合数，并且对所有整数 a 满足 a^n≡a(mod n)。这里先排除素数，再判断 $\lambda(n)$ 是否整除 n-1；等价性包含非互素底数，不要直接拿 $\lambda(n)$ 给任意底数降幂。输出必须完全符合题面两种句式。该题只检查判定，不能替代 $\lambda(n)$ 数值及最小性的接口测试，亦不计独立模板题覆盖。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、carmichael（第 53 页，3.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/100/10006.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n && n)
    {
        bool special = !Prime64::prime(n) && (n - 1) % carmichael(n) == 0;
        if (special)
            cout << "The number " << n << " is a Carmichael number.\n";
        else
            cout << n << " is normal.\n";
    }
    return 0;
}
```

3.9 模逆元

3.9.1 模整数类型的批量逆元

batch_units(a) 返回 optional<vector<Z»，保持输入顺序；任一元素不可逆返回 nullopt，空输入返回空数组。Z 须可从 1 构造，支持乘法、复合乘法及 try_inv，且元素来自同一交换模整数环；目前可与 mint<tag> 直接组合。先累计前缀乘积，只对总乘积求一次逆，再倒序求各项逆元。总乘积可逆当且仅当所有因子可逆，因此合数模数也可使用。O(n) 次乘法加一次求逆，空间 O(n)，要求大小可表示为 int；建议传 move(a) 避免保留额外输入副本。返回前不能切换动态模数。现有 long long 版 batch_inverse 保留独立接口与历史证据，不由本函数替换。

```
template <class Z> optional<vector<Z>> batch_units(vector<Z> a)
{
    int n = a.size();
    vector<Z> pre(n + 1, Z(1));
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * a[i];
    auto x = pre[n].try_inv();
    if (!x) return nullopt;
    Z inv = *x;
    for (int i = n - 1; i >= 0; i--)
    {
        Z old = a[i];
        a[i] = inv * pre[i];
        inv *= old;
    }
    return a;
}
```

联合使用示例见第 23 页（Luogu P5431 【模板】模意义下的乘法逆元 2），其中直接调用本模板。

3.9.2 任意正模数的单个逆元

代码依赖：extended_gcd（第 47 页，3.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

mod_inverse(a,m) 支持完整 long long a 和正 long long m，将 a 规范化后用 extended_gcd 求逆。不可逆返回-1，可逆返回 0..m-1 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。O(log m) 时间与递归空间。

```
long long mod_inverse(long long a, long long m)
{
    using i128 = __int128_t;
    assert(m > 0);
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    if (extended_gcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

应用题补充：Luogu P1082 [NOIP 2012 提高组] 同余方程

题意：给出 2≤a,b≤2000000000，保证 ax≡1 (mod b) 有解，求最小正整数 x。直接调用 mod_inverse(a,b)；保证有解等价于 gcd(a,b)=1，且 b≥2，因此归一化结果位于 [1,b−1]，正好是最小正解。b 不必为素数，a 可以大于 b，不能改用费马小定理的幂公式。

所需模板：extended_gcd（第 47 页，3.7.1 节）、mod_inverse（第 55 页，3.9.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1082

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    printf("%lld\n", mod_inverse(a, b));
    return 0;
}
```

3.9.3 任意数组的批量逆元

代码依赖：mod_inverse（第 55 页，3.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

batch_inverse(a,mod) 返回与 a 同序的逆元向量。要求 mod 为正 long long，每个输入可为完整 long long；只要某一项与 mod 不互素，整个结果返回 nullopt，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 move(a) 可避免输入副本。依赖 mod_inverse 和 extended_gcd。仅求一次逆元，时间 O(n+log mod)，空间 O(n)；128 位中间乘法防止溢出。

```
optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
{
    assert(mod > 0);
    int n = a.size();
    vector<long long> prefix(n + 1, 1 % mod);
    for (int i = 0; i < n; i++)
    {
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
    }
    long long inverse = mod_inverse(prefix[n], mod);
    if (inverse == -1) return nullopt;
    for (int i = n - 1; i >= 0; i--)
    {
        long long value = a[i];
        a[i] = __int128_t(inverse) * prefix[i] % mod;
        inverse = __int128_t(inverse) * value % mod;
    }
    return a;
}
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：给定 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p，p 为素数，输出 ∑(i=1..n)k^i/a_i 模 p。先批量求全部逆元再累乘权重，注意首项权重是 k。完整抄写 read 与 main；read 仅适用于本题的非负十进制输入。move(a) 避免输入副本，此后不再读取 a。本题所有元素可逆；一般模数或零元素须检查 nullopt。乘积在 long long 范围内，批量核心内部用 int128 保护一般模数。

所需模板：extended_gcd（第 47 页，3.7.1 节）、mod_inverse（第 55 页，3.9.2 节）、batch_inverse（第 56 页，3.9.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    vector<long long> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_inverse(move(a), p);
    if (!inverse) return 1;
    long long answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight = weight * k % p;
        answer = (answer + weight * x) % p;
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.9.4 连续整数的线性逆元表

inverse_table(n,p) 返回长度 n+1 的 int 向量，位置 i 保存 i 模 p 的逆元，1≤i≤n。位置 0 固定为 0，仅为占位，不表示零有逆元；n=0 返回单个占位项。要求 p 为素数、2≤p≤INT_MAX 且 0≤n<p，不检测素性。时间和空间均 O(n)，中间乘法用 long long；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
vector<int> inverse_table(int n, int p)
{
    assert(p >= 2 && 0 <= n && n < p);
    vector<int> inverse(n + 1);
    if (n) inverse[1] = 1;
    for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
    return inverse;
}
```

模板题使用：Luogu P3811 【模板】模意义下的乘法逆元

题意：输入 1≤n≤3000000、n<p<20000528，p 保证为素数，输出 1..n 各自的逆元，每项一行。inverse_table 返回 n+1 项，0 是占位且不输出。该递推要求素数，不能照搬到合数；一般数组使用 batch_inverse。

所需模板：inverse_table（第 58 页，3.9.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3811

```
int main()
{
    int n, p;
    scanf("%d%d", &n, &p);
    auto inverse = inverse_table(n, p);
    for (int i = 1; i <= n; i++) printf("%d\n", inverse[i]);
    return 0;
}
```

3.10 线性同余方程

3.10.1 线性同余方程

代码依赖：extended_gcd（第 47 页，3.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

linear_congruence(a,b,m) 求 a*x+b≡0 (mod m)，返回 {最小非负解, 最小正周期}，无解返回 {-1,-1}。m 为正 long long，a、b 支持完整 long long。若 g=gcd(a,m) 整除 b，周期为 m/g；包括 a=0 和 m=1，所有整数均满足时返回 {0,1}。先归一化 a，并在 int128 中对 b 取相反数和乘法。依赖 extended_gcd；递归时间、空间 O(log(m+1))。注意本接口的常数项是 +b，若题目写 a*x≡c，应传 -c；c 为 LLONG_MIN 时须避免在 64 位中取负。

```
pair<long long, long long> linear_congruence(long long a, long long b, long long m)
{
    assert(m > 0);
    using i128 = __int128_t;
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    i128 g = extended_gcd(a, m, x, y);
    if (i128(b) % g) return {-1, -1};
    i128 period = m / g;
    x = x * (-i128(b) / g) % period;
    if (x < 0) x += period;
    return {(long long)x, (long long)period};
}
```

应用题补充：Luogu P1516 [ZJOI2002] 青蛙的约会

题意：两只青蛙从 x,y 同向跳，每次前进 m,n，圆周长 L。相遇式为 (m-n)*t+(x-y)≡0 (mod L)，故传 linear_congruence(m-n,x-y,L)，不能把常数项误写为 y-x。负速度差无需交换或另取负。返回 {最小非负次数, 最小正周期}；只输出第一项，无解输出 Impossible，不能把周期当答案。x,y,m,n 在 1..2×10^9 且 x≠y，L 在 1..2.1×10^9；差值用 long long。位置按 L 取模，初始重合可返回 0，等速不重合则无解。单组输入，无多测或结束标记。

所需模板：extended_gcd（第 47 页，3.7.1 节）、linear_congruence（第 59 页，3.10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1516

```
int main()
{
    long long x, y, m, n, length;
    scanf("%lld%lld%lld%lld%lld", &x, &y, &m, &n, &length);
    auto [time, period] = linear_congruence(m - n, x - y, length);
    if (time < 0)
        puts("Impossible");
    else
        printf("%lld\n", time);
    return 0;
}
```

3.11 中国剩余定理

3.11.1 广义 CRT：逐项合并同余

代码依赖：extended_gcd（第 47 页，3.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

crt_merge(r,m,b,n) 合并 x≡r (mod m) 与 x≡b (mod n)，m,n>0，r、m 须为不同变量。从 r=0、m=1 开始逐项合并；成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先规范化 r, m 不变，失败后不能视为完整解。依赖 extended_gcd，中间量用 int128；O(log min(m,n)) 时间与递归空间。

```
bool crt_merge(long long &r, long long &m, long long b, long long n)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(m > 0 && n > 0);
    r %= m;
    if (r < 0) r += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    ll g = (ll)extended_gcd(m, n, x, y);
    i128 diff = i128(b) - r;
    if (diff % g) return false;
    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
    r = (r + i128(m) * k) % mod;
    m = (ll)mod;
    return true;
}
```

模板题使用：Luogu P4777

题意：求同时满足多个同余式的最小非负整数，模数可以不互质。每行按模数、余数输入，逐式合并；合并后模数须在 long long 范围内。

所需模板：extended_gcd（第 47 页，3.7.1 节）、crt_merge（第 60 页，3.11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4777

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int count;
    cin >> count;
    long long r = 0, modulus = 1;
    while (count--)
    {
        long long m, b;
        cin >> m >> b;
        if (!crt_merge(r, modulus, b, m))
        {
            cout << -1 << '\n';
            return 0;
        }
    }
    cout << r << '\n';
    return 0;
}
```

3.11.2 Garner：大周期 CRT 与混合进制

代码依赖：mod_inverse（第 55 页，3.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

garner_digits(b,moduli) 返回按输入顺序排列的混合进制各位；garner(b,moduli,target) 返回最小非负 CRT 解对 target 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 long long 范围。target 为正 long long，与输入模数无需互素。输入不互素时抛 invalid_argument，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 int128。k 个模数耗时 $O(k^2+k \log M)$，空间 O(k)；M 为最大输入模数。依赖 mod_inverse 和 extended_gcd。

```
// Pairwise coprime positive moduli. Digits use the input order.
vector<long long> garner_digits(const vector<long long> &b,
                                const vector<long long> &moduli)
{
    using ll = long long;
    using i128 = __int128_t;
    int n = b.size();
    assert(moduli.size() == b.size());
    vector<ll> value(n), coefficient(n, 1), digit(n);
    for (ll m : moduli) assert(m > 0);
    for (int i = 0; i < n; i++)
    {
        ll m = moduli[i];
        ll inverse = mod_inverse(coefficient[i], m);
        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
        i128 delta = (i128(b[i]) - value[i]) % m;
        if (delta < 0) delta += m;
        digit[i] = delta * inverse % m;
        for (int j = i + 1; j < n; j++)
        {
            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
        }
    }
    return digit;
}
```

规范解对目标数取模

```
// Return the least nonnegative CRT solution modulo target, not all solutions.
long long
garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
{
    assert(target > 0);
    auto digit = garner_digits(b, moduli);
    long long answer = 0;
    for (int i = (int)digit.size() - 1; i >= 0; i--)
        answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
    return answer;
}
```

模板题使用：Library Checker Convolution (Mod 1,000,000,007)

题意：给出长度 n、m 的升幂系数，输出模 1000000007 的卷积，共 n+m−1 项。两数组长度均为 1 到 524288；输入系数非负且小于目标模数。分别在 167772161、469762049、1224736769 下做 NTT，再逐项调用既有 garner。真实非负系数不超过 524288 乘以 1000000006 的平方，严格小于三素数乘积。因此 CRT 最小非负代表就是原整数系数，最后对目标模数取余；仅有模数互素并不足够。输入通过 ModInt 构造器归一化。三模数公共变换容量为 2^24，本题至多需 2^20；无系数界时不能直接套用。

所需模板：extended_gcd（第 47 页，3.7.1 节）、mod_inverse（第 55 页，3.9.2 节）、garner（第 62 页，3.11.2 节）、ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod_1000000007

```
template <int p>
auto conv(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    return N::multiply(move(x), move(y));
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto x = conv<167772161>(a, b);
    auto y = conv<469762049>(a, b);
    auto z = conv<1224736769>(a, b);
    vector<long long> r(3), p = {167772161, 469762049, 1224736769};
    for (int i = 0; i < n + m - 1; i++)
    {
        r[0] = x[i].v;
        r[1] = y[i].v;
        r[2] = z[i].v;
        cout << garner(r, p, 1000000007) << (i + 1 == n + m - 1 ? '\n' : ' ');
    }
}
```

3.12 升幂引理

3.12.1 素因子估值与升幂：先约去公共因子

本节只处理整数的素因子重数。以下 p 为素数，a,b,n 为正整数；对非零整数 x，定义 $v_p(x)=\max\{e\ge 0: p^e \mid x\}$，负数按绝对值计算。**不定义有限的** $v_p(0)$：求差时先判断 $a=b$，此时 $a^n-b^n=0$，被每个正整数整除，却没有最大的有限素因子重数，不能进入下面的估值公式或反复除 p 的循环。

3.12.1.1 基本运算与公共因子

对非零整数 x,y，有

$$v_p(xy)=v_p(x)+v_p(y),\qquad v_p(x^n)=nv_p(x).$$

若 $y\mid x$，整数商满足 $v_p(x/y)=v_p(x)-v_p(y)$。当 $x+y\neq 0$ 时，$v_p(x+y)\ge \min(v_p(x),v_p(y))$；两者估值不同才必定取等。证明只需提出较小的 p 次幂：剩下两项中恰有一项不被 p 整除；估值相同时可能抵消，例如 $v_3(1)=v_3(2)=0$，但 $v_3(1+2)=1$。

计算 $a^n\pm b^n$ 时先记 $\alpha=v_p(a)$、$\beta=v_p(b)$。若 $\alpha\neq\beta$，则直接得到

$$v_p(a^n\pm b^n)=n\min(\alpha,\beta).$$

若 $\alpha=\beta=t$，写成 $a=p^tA, b=p^tB$，其中 $p\nmid AB$，则

$$v_p(a^n\pm b^n)=nt+v_p(A^n\pm B^n).$$

差式仍要求 $a\neq b$。约去公共因子后，才能对 A,B 检查下面的 LTE 条件；不能把 nt 误写为 t，也不能只凭 $p\mid a-b$ 就直接套公式。

3.12.1.2 奇素数：差式与和式的条件不同

以下两式均要求 p 为**奇素数**且 $p\nmid ab$：

$$\begin{aligned} p\mid a-b,\ a\neq b \quad &\Longrightarrow \quad v_p(a^n-b^n)=v_p(a-b)+v_p(n),\\ p\mid a+b,\ n\text{ 为奇数} \quad &\Longrightarrow \quad v_p(a^n+b^n)=v_p(a+b)+v_p(n).\end{aligned}$$

差式不要求 n 为奇数；和式不能漏掉这一条件。这里 $p\nmid ab$ 只要求两数都不含 p，并不要求 $\gcd(a,b)=1$。

为什么指数中每个 p 恰好贡献一次。设 $n=mp^s$，$p\nmid m$。当 $p\mid a-b$ 时，$(a^m-b^m)/(a-b)$ 模 p 等于 mb^{m-1}，故指数的 m 部分不增加估值。再设 $x=y+h$，$p\nmid y$、$v_p(h)=r\ge 1$。二项式展开 x^p-y^p：一次项 $py^{p-1}h$ 的估值为 $r+1$，其余项均至少为 $r+2$（末项用 $pr\ge r+2$）。所以每次把指数乘 p 都使估值增加 1，重复 s 次即得差式。和式可对 a^2,b^2 使用差式，再用 $a^{2n}-b^{2n}=(a^n-b^n)(a^n+b^n)$：当 n 奇且 $a\equiv -b\pmod p$ 时，$p\nmid a-b$、$p\nmid a^n-b^n$，两边相减只剩所求和式的估值。

3.12.1.3 素数 2：先分指数奇偶

若 a,b 均为奇数且 $a\neq b$，则

$$v_2(a^n-b^n)=\begin{cases} v_2(a-b), & n\text{ 为奇数},\\ v_2(a-b)+v_2(a+b)+v_2(n)-1, & n\text{ 为偶数}.\end{cases}$$

对奇数 a,b 的和式（允许 $a=b$），有

$$v_2(a^n+b^n)=\begin{cases} v_2(a+b), & n\text{ 为奇数},\\ 1, & n\text{ 为偶数}.\end{cases}$$

奇指数时，差式与和式各自除以 $a-b$、$a+b$ 后的多项式均是奇数个奇数的和或交错和，商为奇数。偶指数差式写 $n=2^sm$、m 奇：先消去奇指数 m，再反复平方差分解；首两因子是 $a-b, a+b$，以后每个 $a^{2^j}+b^{2^j}$（$j\ge 1$）均模 4 余 2，恰好再贡献 $s-1$。偶指数和式也因奇数的偶次幂模 4 余 1，估值恰为 1。特别地，若 $4\mid a-b$，则 $v_2(a+b)=1$，差式可统一写成 $v_2(a^n-b^n)=v_2(a-b)+v_2(n)$；没有这个附加条件不能照搬奇素数公式。若 a,b 奇偶不同，和差均为奇数；若都偶，先按上一小节约去公共 2 因子。

3.12.1.4 把大整数整除转成指数条件

例：求使 $2^6 \cdot 3^2 \mid 7^n - 1$ 的最小正整数 n。奇数 n 时 $v_2(7^n-1) = v_2(6) = 1$，必不满足。偶数 n 时两种素数分别给出

$$v_2(7^n-1) = 1 + 3 + v_2(n) - 1 = 3 + v_2(n), \qquad v_3(7^n-1) = 1 + v_3(n).$$

所以须且仅须 $v_2(n) \geq 3$、$v_3(n) \geq 1$，即 $24 \mid n$，最小值为 24。一般对非零整数 X 和 $M = \prod p_i^{e_i}$，$M \mid X$ 等价于逐项检查 $v_{p_i}(X) \geq e_i$；无需真的构造可能远超机器字长的幂。LTE 只能处理满足条件的局部因子，不能代替任意底数的乘法阶或离散对数问题。实现时非零整数反复除 p 即可求估值；$a+b$、差的绝对值及 nt 的计算仍须防溢出。

条件漏一项，答案就可能错。

- 不检查 $p \nmid ab$：$p = 3, a = 6, b = 3, n = 2$，实际 $v_3(27) = 3$，硬套差式只得 $v_3(3) + v_3(2) = 1$。
- 不检查 $p \mid a - b$：$p = 3, a = 2, b = 1, n = 2$，实际 $v_3(3) = 1$，右侧 $v_3(1) + v_3(2) = 0$。
- 将奇素数差式用于 2：$a = 3, b = 1, n = 2$，实际 $v_2(8) = 3$，错式只得 $v_2(2) + v_2(2) = 2$。
- 和式允许偶指数：$p = 3, a = 2, b = 1, n = 2$，实际 $v_3(5) = 0$，错式却给 $v_3(3) + v_3(2) = 1$。

定理条件参见 OI Wiki：升幂引理。本节限于整数估值、LTE 及整除建模，不代表完整的 p 进数理论，也不新增实现或在线评测记录。

与已有模板分工。扩展 Lucas 的阶乘赋值用 $v_p(N!) = \sum_{j \geq 1} \lfloor N/p^j \rfloor$，不是把整个阶乘硬套 LTE；见 ExLucas（第 69 页，3.14.2 节）。素数幂开根还要检查单位部分是否有根，单有估值条件并不充分；见 PrimePowerRoots（第 87 页，3.18.2 节）。

3.13 阶乘取模

3.13.1 Legendre 赋值与 Kummer 进位判据

设 p 为素数，$v_p(a)$ 表示正整数 a 中质因子 p 的指数。$n \geq 0$ 时，Legendre 公式为

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p-1},$$

其中 $s_p(n)$ 是 n 的 p 进制数位和，$0! = 1$ 给出赋值 0。第一式把每个数的质因子逐层数：p 的倍数贡献至少 1 个，p^2 的倍数再贡献 1 个，依此类推。若 $n = \sum_i a_i p^i$，则交换有限求和得到

$$\sum_{j \geq 1} \lfloor n/p^j \rfloor = \sum_{i \geq 1} a_i (1 + p + \cdots + p^{i-1}) = \sum_i a_i \frac{p^i - 1}{p-1}.$$

素数条件不可省略：若把 p 错设为 4，$6! = 720$ 能被 4^2 整除，但 $\lfloor 6/4 \rfloor + \lfloor 6/16 \rfloor = 1$，不是最高 4 次幂的指数。

实现时反复执行 $n \leftarrow \lfloor n/p \rfloor$ 并累加，直到 $n = 0$，时间 $O(1 + \log_p(n+1))$。避免逐次计算 p^j，后者可能在判断终止前就溢出。已有 `ExLucas::valuation(n,p)` 使用这种方式，见第 3.14.2 节（第 69 页）；n 是 unsigned64、p 是正的 int 素数。该内部辅助函数不验证素数，尤其不能传入 0 或 1。由于 $v_p(n!) \leq n$，其 unsigned64 累加不会超出合法 n 的范围。

3.13.1.1 组合数：逐位相加时进了几次位

对于 $0 \leq k \leq n$，令 $h = n - k$，由阶乘比值的赋值相减可得

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p(h!) = \frac{s_p(k) + s_p(h) - s_p(n)}{p-1}.$$

把 $k + h = n$ 按 p 进制相加，记第 i 位的输入进位为 c_i，$c_0 = 0$，则

$$k_i + h_i + c_i = n_i + p c_{i+1}.$$

两数相加的 c_i 只可能是 0 或 1。对所有位求和，包含最终进位并在更高位补 0，得到 $s_p(k) + s_p(h) - s_p(n) = (p-1)\sum_{i \geq 1} c_i$。所以赋值恰好是加法产生的进位次数，这就是 Kummer 定理。传播进位也要计数：$1 + 7 = 8$ 的二进制加法有 3 次进位，对应 $\binom{8}{1} = 8$ 中的 2^3；不能只数原始两位都为 1 的位置。$k = 0$ 或 $k = n$ 均无进位，对应组合数 1。

3.13.1.2 素数幂为零与 Lucas 的边界

对整数 $a \geq 1$，

$$\binom{n}{k} \equiv 0 \pmod{p^a} \iff v_p\binom{n}{k} \geq a.$$

这是“至少 a 次进位”，不是严格大于 a；例如 $\binom{4}{2} = 6$ 模 2 为 0，模 4 不为 0。若模数 $m = \prod_j p_j^{a_j} > 1$，模 m 为零须每个素数幂分量都满足阈值，不能只命中一个。模 1 的结果恒为 0，另行处理。

零进位等价于每个数位都满足 $k_i \leq n_i$：逐位相减无需借位，恰好能无进位相加回 n。这也解释了 Lucas 公式在素数模下何时非零，见第 3.14.1 节（第 68 页）。但知道赋值只够判定可整除性，不足以恢复非零余数。例如 $\binom{4}{1} = 4$ 与 $\binom{5}{1} = 5$ 的 v_3 都是 0，模 3 余数却分别为 1 和 2。`ExLucas::choose` 因此还需计算去除 p 因子的单位部分、乘回 p^v，再通过 CRT 合并。这里不扩大其模数、预处理空间或参数类型的现有契约，也不能把不可逆阶乘直接相除。

$k > n$ 时按组合数约定答案为 0，必须先处理；0 没有有限的上述赋值，不能做无符号 $n - k$ 或套用有限进位公式。负参数不在本节范围。阶乘赋值是逐项质因子计数，不是对指数差使用 LTE；相关选型见第 3.12.1 节（第 64 页）。证明背景见 Jorza 数论讲义中的 Legendre 与 Kummer 定理；已有赋值算法的导航见 固定版本 OI Wiki 阶乘取模。

3.14 卢卡斯定理

3.14.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 $p\geq 2$。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p\leq 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 68 页，3.14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

3.14.2 合数模数组合数（扩展 Lucas）

ExLucas(mod) 预处理固定模数，choose(n,k) 返回 C(n,k) 模 mod；n、k 为 unsigned long long，k>n 返回 0，mod=1 返回 0。适合 mod≤10^6 的合数或素数模数，不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 q=p^e，预处理每段内不含 p 因子的前缀积；unit(n,f) 去除 n! 中全部 p 因子，valuation 统计指数。分子分母指数之差不少于 e 时该项为零，否则只对互素单位部分求逆，最后 CRT 合并。预处理 O($\sqrt{\text{mod}}$+$\sum$q)，空间 O($\sum$(q+1))；每次查询 O($\sum$(log_p(n+1) · log(n+1)+log q))，求和遍历不同素因子。所有乘法用 long long；同一对象可重复查询，factors、fac 与辅助函数视为内部数据。

```
struct ExLucas
{
    using ull = unsigned long long;
    using ll = long long;

    struct Factor
    {
        int p, q, exponent, start, crt;
    };

    int mod;
    vector<int> fac;
    vector<Factor> factors;

    static ll power(ll a, ull b, int m)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % m;
            a = a * a % m;
```

```
            b >>= 1;
        }
        return result;
    }

    ExLucas(int m) : mod(m)
    {
        assert(m >= 1);
        int x = m;
        for (int p = 2; p <= x / p; p++)
        {
            if (x % p) continue;
            int q = 1, e = 0;
            do
            {
                x /= p;
                q *= p;
                e++;
            } while (x % p == 0);
            add_factor(p, q, e);
        }
        if (x > 1) add_factor(x, x, 1);
    }

    void add_factor(int p, int q, int e)
    {
        int start = fac.size();
        fac.push_back(1);
        for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
        fac.push_back(fac.back());
        int phi = q - q / p;
        int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
        factors.push_back({p, q, e, start, crt});
    }

    static ull valuation(ull n, int p)
    {
        ull result = 0;
        while (n)
        {
            n /= p;
            result += n;
        }
        return result;
    }

    ll unit(ull n, const Factor &f) const
    {
```

```
        ll result = 1;
        while (n)
        {
            result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
            result = result * fac[f.start + n % f.q] % f.q;
            n /= f.p;
        }
        return result;
    }
```

```
    int choose(ull n, ull k) const
    {
        if (k > n || mod == 1) return 0;
        ll answer = 0;
        for (const auto &f : factors)
        {
            ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
            if (e >= ull(f.exponent)) continue;
            ll denominator = unit(k, f) * unit(n - k, f) % f.q;
            ll residue =
                unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
            residue = residue * power(f.p, e, f.q) % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4720

题意：求 C(n,m) 模 p，p 可以为合数且不超过 10^6。构造 ExLucas(p) 后 choose(n,m)；预处理与模数规模有关，不能直接用于 10^9 级模数。

所需模板：ExLucas（第 69 页，3.14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4720

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    unsigned long long n, m;
    int p;
    std::cin >> n >> m >> p;
    ExLucas comb(p);
    std::cout << comb.choose(n, m) << '\n';
}
```

相关应用：按上标范围预处理的合数模数组合数，与小模数大上标 ExLucas 互补

3.14.3 有界上标的任意模数组合数

CompositeBinomial(N,mod) 预处理上标 0..N 的组合数，0≤N<INT_MAX、1≤mod≤INT_MAX；mod 可为合数。choose(n,k) 返回 C(n,k) 模 mod；负 n、负 k、k>n 返回 0，其余要求 n≤N。mod=1 时所有结果为 0。每个素数幂分别保存去除对应素因子的阶乘、逆阶乘和指数前缀，再用 CRT 合并。

设 s 为 mod 的不同素因子个数。预处理 O($\sqrt{\text{mod}}$+s(N+log mod)) 时间、O(s(N+1)) 空间，单次查询 O(s)。各素数幂的 p 次幂表很短；指数达到该素数幂的次数即直接贡献 0。表长取决于 N，不按 mod 或素数幂大小开数组，适合 N≤10^5、mod 为 10^9 级数的多次查询。n 为完整 unsigned64 且模数小的场景仍用 ExLucas。

limit、mod、factors 及辅助函数视为内部数据；不支持自动扩表或修改模数。所有模乘提升为 long long，CRT 每乘一次即取模。已通过独立 Pascal 递推、Python 精确组合数、原稿安全小域对照和 ASan/UBSan；CF896D 为已验证的本地应用驱动，尚无本次线上 AC。

```
struct CompositeBinomial
{
    struct Factor
    {
        int p, q, crt;
        vector<int> fac, ifac, exponent, powers;
    };

    int limit, mod;
    vector<Factor> factors;

    static long long power(long long a, int e, int m)
    {
        long long result = 1;
        while (e)
        {
            if (e & 1)
                result = result * a % m;
            a = a * a % m;
            e >>= 1;
        }
        return result;
    }
```

有界上标组合数（接上页同一结构体）：

```
    CompositeBinomial(int n, int m) : limit(n), mod(m)
    {
        assert(n >= 0 && n < INT_MAX && m >= 1);
        int rest = m;
        for (int p = 2; p <= rest / p; p++)
        {
            if (rest % p != 0)
                continue;
            int q = 1;
            do
            {
                rest /= p;
                q *= p;
            } while (rest % p == 0);
            add_factor(p, q);
        }
        if (rest > 1)
            add_factor(rest, rest);
    }

    void add_factor(int p, int q)
    {
        int phi = q - q / p;
        int crt = (mod / q) * power(mod / q % q, phi - 1, q) % mod;
        Factor f{p, q, crt, vector<int>(limit + 1, 1),
                 vector<int>(limit + 1, 1), vector<int>(limit + 1), {}};
        for (long long x = 1; x < q; x *= p)
            f.powers.push_back(x);
        for (int i = 1; i <= limit; i++)
        {
            int x = i;
            f.exponent[i] = f.exponent[i - 1];
            while (x % p == 0)
            {
                x /= p;
                f.exponent[i]++;
            }
            f.fac[i] = 1LL * f.fac[i - 1] * x % q;
        }
        f.ifac[limit] = power(f.fac[limit], phi - 1, q);
        for (int i = limit; i >= 1; i--)
        {
            int x = i;
            while (x % p == 0)
                x /= p;
            f.ifac[i - 1] = 1LL * f.ifac[i] * x % q;
        }
        factors.push_back(move(f));
    }
```

有界上标组合数（接上页同一结构体）：

```
    int choose(int n, int k) const
    {
        if (n < 0 || k < 0 || k > n)
            return 0;
        assert(n <= limit);
        long long answer = 0;
        for (const auto &f : factors)
        {
            int e = f.exponent[n] - f.exponent[k] - f.exponent[n - k];
            if (e >= int(f.powers.size()))
                continue;
            long long residue = 1LL * f.fac[n] * f.ifac[k] % f.q;
            residue = residue * f.ifac[n - k] % f.q;
            residue = residue * f.powers[e] % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

接口用法演示：有界上标合数模数组合数 API

演示约定：输入预处理上界 N、正 int 模数 mod 和询问数；每组 n k 输出 C(n,k) 模 mod。负 n、负 k 或 k>n 为 0，否则必须 n≤N。数组按 N 开，支持大合数或大素数模数。

所需模板：CompositeBinomial（第 72 页，3.14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/COMPOSITE-BINOMIAL.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, tests;
    cin >> n >> mod >> tests;
    CompositeBinomial comb(n, mod);
    while (tests--)
    {
        int a, b;
        cin >> a >> b;
        cout << comb.choose(a, b) << '\n';
    }
}
```

应用题补充：CF 896D Nephren Runs a Cinema

题意：输入 n、mod、l、r，统计长度 n、步长为-1/0/1、所有前缀和非负且末尾高度在 [l,r] 的序列数。n≤10^5，1≤mod≤2×10^9。设 S(h) 为不限制前缀的末尾高度 h 计数，反射原理给出合法数 S(h)-S(h+2)，区间和只需四项 S；按向下步数枚举两个组合数乘积求 S。总时间 O(√mod+s(n+log mod))、空间 O(s(n+1))，s 为不同素因子数。

所需模板：CompositeBinomial（第 72 页，3.14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/896/D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, l, r;
    cin >> n >> mod >> l >> r;
    CompositeBinomial comb(n, mod);
    auto coefficient = [&](int h)
    {
        if (h > n)
            return 0LL;
        long long answer = 0;
        for (int down = 0; down <= (n - h) / 2; down++)
        {
            long long ways = comb.choose(n, down);
            ways = ways * comb.choose(n - down, down + h) % mod;
            answer = (answer + ways) % mod;
        }
        return answer;
    };
    long long answer = coefficient(l) + coefficient(l + 1);
    answer -= coefficient(r + 1) + coefficient(r + 2);
    answer = (answer % mod + mod) % mod;
    cout << answer << '\n';
}
```

相关应用：Gaussian 二项式的 q-Lucas 取值；普通组合数 Lucas 只是其高位部分

3.14.4 素数模 Gaussian 二项式

q_binomial(n,k,q,p) 返回 Gaussian 二项式在 q 处的值模素数 p，规范为 [0,p)。n、k 为 unsigned long long；k>n 返回 0。2≤p≤INT_MAX 且必须为素数，函数不做素性检验；q 为 int，可负，先归一化。q=0 时合法 k 返回 1，q=1 退化为普通组合数。

非零 q 先由 p-1 的因子求乘法阶 r。q-Lucas 把答案分成 C(n/r,k/r) 与低位 Gaussian 二项式 [n%r,k%r]；低位 k>n 直接为 0，其余只对非零因子求逆。普通组合数再按 p 进制 Lucas 计算，避免在单位根处直接计算 0/0。

空间 O(1)。试除分解 p-1 花 $O(\sqrt{p})$，缩阶至多 O(log p) 次模幂，各 O(log p)；设 a=n%r、b=k%r，则低位乘积 O(min(b,a-b))，普通 Lucas 逐位成本为 $\sum$min(ki,ni-ki)，另有 O(log p+log_p(n+1)) 开销。因此大素数且某位 k 接近 p/2 仍可能很慢，不是任意大模数的快速组合数算法。所有乘积提升为 long long。与 WIDA13B 原稿及独立递推本地对照，QOJ4281 子空间计数已 AC。WIDA13A 原调用存在零因子约分错误，使用本函数替代其 Gaussian 二项式任务；不提供通用 p 进阶数/单位部分接口。

```
// Gaussian binomial [n choose k]_q modulo a prime p, 2 <= p <= INT_MAX.
int q_binomial(unsigned long long n, unsigned long long k, int q, int p)
{
    assert(p >= 2);
    if (k > n)
        return 0;
    q %= p;
    if (q < 0)
        q += p;
    if (q == 0)
        return 1;
    auto power = [&](long long a, unsigned long long e)
    {
        long long r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a % p;
            a = a * a % p;
            e >>= 1;
        }
        return r;
    };

    int order = p - 1, rest = p - 1;
    for (int d = 2; d <= rest / d; d++)
        if (rest % d == 0)
        {
            while (rest % d == 0)
                rest /= d;
            while (order % d == 0 && power(q, order / d) == 1)
                order /= d;
        }
    if (rest > 1)
        while (order % rest == 0 && power(q, order / rest) == 1)
            order /= rest;
```

Gaussian 二项式（接上页同一函数）：

```
    int a = n % order, b = k % order;
    if (b > a)
        return 0;
    b = min(b, a - b);
    long long num = 1, den = 1;
    long long upper = power(q, a), lower = q, invq = power(q, p - 2);
    for (int i = 0; i < b; i++)
    {
        num = num * (upper + p - 1) % p;
        den = den * (lower + p - 1) % p;
        upper = upper * invq % p;
        lower = lower * q % p;
    }

    n /= order;
    k /= order;
    while (n || k)
    {
        int x = n % p, y = k % p;
        if (y > x)
            return 0;
        y = min(y, x - y);
        for (int i = 1; i <= y; i++)
        {
            num = num * (x - i + 1) % p;
            den = den * i % p;
        }
        n /= p;
        k /= p;
    }
    return num * power(den, p - 2) % p;
}
```

接口用法演示：Gaussian 二项式 API

演示约定：读测试组数，每组 n k q p，n/k 为非负 64 位无符号整数，q 为 int，p 是 [2,INT_MAX] 内的素数；输出 Gaussian 二项式模 p。k>n 输出 0。注意乘法阶及 Lucas 各位的乘积循环成本，详见接口说明。

所需模板：q_binomial（第 76 页，3.14.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/Q-BINOMIAL.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        unsigned long long n, k;
        int q, p;
        cin >> n >> k >> q >> p;
        cout << q_binomial(n, k, q, p) << '\n';
    }
}
```

应用题补充：QOJ 4281 Subspace

题意：输入素数 q、维数 n、子空间维数 k、素数 p，求 F_q^n 中 k 维子空间个数模 p。原题 q,n≤10^9，1≤k≤n，p≤200000。按有序线性无关向量组数除以每个子空间的有序基数，得到 Gaussian 二项式；单位根处用 q-Lucas，不能直接除零。QOJ 提交 3166447 已 AC（1ms）。

所需模板：q_binomial（第 76 页，3.14.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/contest/1341/problem/4281

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q, p;
    unsigned long long n, k;
    cin >> q >> n >> k >> p;
    cout << q_binomial(n, k, q, p) << '\n';
}
```

3.15 二次剩余

3.15.1 素数模平方根（Cipolla）

mod_sqrt(n,p) 返回 vector<long long>，按升序列出全部不同根；无解返回空向量，零或模数 2 返回单根。要求 p 为素数，2≤p≤INT_MAX，n 支持完整 long long 范围并在入口归一化。先用 Euler 判别；p≡3 (mod 4) 时直接快速幂，其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器；随机性影响寻找非剩余的尝试次数，不影响结果，通常期望 O(log p) 时间、O(1) 辅助空间。乘法用 long long，扩域实部的三因子乘积中间先取模；指数中的 p+1 先转 long long，避免 INT_MAX 溢出。此版本不处理合数模数及 64 位大素数，也不用于浮点开方。

```
vector<long long> mod_sqrt(long long n, int p)
{
    using ll = long long;
    assert(p >= 2);
    n = (n % p + p) % p;
    if (n == 0 || p == 2) return {n};
    auto power = [p](ll a, ll b)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return result;
    };
    if (power(n, (p - 1) / 2) != 1) return {};
    ll answer;
    if (p % 4 == 3)
        answer = power(n, (ll(p) + 1) / 4);
    else
    {
        static mt19937_64 rng(712367);
        ll a, w;
        do
        {
            a = rng() % p;
            w = (a * a % p - n + p) % p;
        } while (power(w, (p - 1) / 2) != p - 1);
        using P = pair<ll, ll>;
        auto mul = [p, w](P x, P y)
        {
            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
            ll imag = (x.first * y.second + x.second * y.first) % p;
            return P{real, imag};
        };
        P result{1, 0}, base{a, 1};
        ll exponent = (ll(p) + 1) / 2;
        while (exponent)
        {
            if (exponent & 1) result = mul(result, base);
```

```
            base = mul(base, base);
            exponent >>= 1;
        }
        answer = result.first;
    }
    ll other = p - answer;
    if (answer > other) swap(answer, other);
    return {answer, other};
}
```

模板题使用：Luogu P5491

题意：多组求 x*x 与 N 模 p 同余的全部解，题目保证 p 为奇素数。mod_sqrt 返回升序且去重的根，空数组表示无解并输出 Hola!。

所需模板：mod_sqrt（第 80 页，3.15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5491

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        long long n;
        int p;
        std::cin >> n >> p;
        auto roots = mod_sqrt(n, p);
        if (roots.empty())
            std::cout << "Hola!\n";
        else
        {
            for (int i = 0; i < (int)roots.size(); i++)
                std::cout << roots[i] << (i + 1 == (int)roots.size() ? '\n' : ' ');
        }
    }
}
```

3.16 阶 & 原根

3.16.1 乘法阶与原根

代码依赖：Mod64（第 15 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 PrimitiveRoot(n)，要求 2≤n≤INT_MAX；重新赋值 PrimitiveRoot(n) 可重建。order(a) 返回 a 模 n 的乘法阶，a 不可逆时返回 0；is_root(a) 判断该剩余类是否为原根，a 可为完整 long long。minimum() 返回 1..n-1 内最小原根，无原根返回 -1；all() 返回升序的全部原根，无原根返回空向量。mod、phi、exists 只读；factors 为 phi 的不同素因子，辅助 factor 为试除分解。依赖 Mod64::power。初始化 $O(\sqrt{n})$，求阶 $O(\log^2 n)$，原根判定 $O((1+\omega(phi))\log n)$。若最小原根为 G，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 O(phi log phi+c log c) 时间、O(c) 空间，c 为原根数；大模数只求最小原根或阶时，不应调用 all。

```
struct PrimitiveRoot
```

```
{
    int mod, phi;
    bool exists;
    vector<int> factors;

    static vector<int> factor(int n)
    {
        vector<int> result;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0)
            {
                result.push_back(p);
                while (n % p == 0) n /= p;
            }
        if (n > 1) result.push_back(n);
        return result;
    }
```

```
    PrimitiveRoot(int n)
    {
        assert(n >= 2);
        mod = n;
        phi = n;
        for (int p : factor(n)) phi -= phi / p;
        int odd = n;
        if (odd % 2 == 0) odd /= 2;
        exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
        factors = factor(phi);
    }

    int order(long long a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return 0;
        int result = phi;
        for (int p : factors)
            while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
                result /= p;
        return result;
    }
```

```
    bool is_root(long long a) const
    {
        if (!exists) return false;
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return false;
        for (int p : factors)
            if (Mod64::power(a, phi / p, mod) == 1) return false;
        return true;
    }

    int minimum() const
    {
        if (!exists) return -1;
        for (int g = 1; g < mod; g++)
            if (is_root(g)) return g;
        return -1;
    }
```

```
    vector<int> all() const
    {
        int g = minimum();
        vector<int> roots;
        if (g == -1) return roots;
        long long value = 1;
        for (int k = 1; k <= phi; k++)
        {
            value = value * g % mod;
            if (gcd(k, phi) == 1) roots.push_back(value);
        }
        sort(roots.begin(), roots.end());
        return roots;
    }
};
```

模板题使用：Luogu P6091

题意：多组枚举模 n 的全部原根，先输出个数，再输出升序原根的第 d、2d、3d 等项。all 返回升序数组，下标从 d-1 开始；即使没有取到任何原根，第二行也必须输出空行。

所需模板：Mod64（第 15 页，2.1 节）、PrimitiveRoot（第 81 页，3.16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, d;
        cin >> n >> d;
        auto roots = PrimitiveRoot(n).all();
        cout << roots.size() << '\n';
        bool first = true;
        for (int i = d - 1; i < (int)roots.size(); i += d)
        {
            if (!first) cout << ' ';
            cout << roots[i];
            first = false;
        }
        cout << '\n';
    }
    return 0;
}
```

3.17 离散对数

3.17.1 扩展 BSGS

代码依赖：Mod64（第 15 页，2.1 节）、mod_inverse（第 55 页，3.9.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 x≥0 使 a^x≡b (mod m)，无解返回 -1；1≤m≤10^{12}，a、b 可为完整 long long 范围，入口归一化，约定 0^0=1。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 O($\sqrt{m}$+log m)，空间 O($\sqrt{m}$)，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```
struct DiscreteLog
{
    using ll = long long;

    // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
    static ll solve(ll a, ll b, ll m)
    {
        assert(m >= 1 && m <= 1000000000000LL);
        a = (a % m + m) % m;
        b = (b % m + m) % m;
        if (m == 1 || b == 1) return 0;
        if (a == 1) return -1;
        ll offset = 0, k = 1;
        for (ll g; (g = gcd(a, m)) > 1;)
        {
            if (b == k) return offset;
            if (b % g) return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            ++offset;
        }
        ll target = (__int128)b * mod_inverse(k, m) % m;
        ll step = sqrtl(m) + 1;
        unordered_map<ll, ll> baby;
        baby.reserve(step);
        ll cur = 1 % m;
        for (ll j = 0; j < step; j++)
        {
            if (!baby.count(cur)) baby[cur] = j;
            cur = (__int128)cur * a % m;
        }
        ll inv = mod_inverse(Mod64::power(a, step, m), m);
        cur = target;
        for (ll i = 0; i <= step; i++)
        {
            auto it = baby.find(cur);
            if (it != baby.end()) return offset + i * step + it->second;
            cur = (__int128)cur * inv % m;
        }
        return -1;
    }
};
```

模板题使用：Luogu P4195

题意：多组求 a^x 与 b 模 p 同余的最小非负整数 x，a 与 p 不必互质。输入顺序为 a、p、b，调用顺序为 solve(a,b,p)；返回 -1 时输出 No Solution，三个 0 结束输入。

所需模板：Mod64（第 15 页，2.1 节）、extended_gcd（第 47 页，3.7.1 节）、mod_inverse（第 55 页，3.9.2 节）、DiscreteLog（第 85 页，3.17.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4195

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    long long a, p, b;
    while (std::cin >> a >> p >> b)
    {
        if (a == 0 && p == 0 && b == 0) break;
        long long answer = DiscreteLog::solve(a, b, p);
        if (answer == -1)
            std::cout << "No Solution\n";
        else
            std::cout << answer << '\n';
    }
}
```

3.18 高次剩余 & 单位根

3.18.1 素数模高次剩余：全部根的紧凑表示

代码依赖：DiscreteLog（第 85 页，3.17.1 节）、Mod64（第 15 页，2.1 节）、linear_congruence（第 59 页，3.10.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,p,g) 求 x^k≡a (mod p)；p 必须是 2 到 1e12 的素数，g 是模 p 原根，k 为正 unsigned long long，a 支持完整 signed long long。无解返回 nullopt；否则得到 first、ratio、count，全部解为 first*ratio^i mod p，0≤i<count，无重复；first 不保证最小。a≡0 时仅有根 0，返回 {0,1,1}。枚举时每次乘 ratio，乘法先转 int128；无需展开全部根即可读取解数。给定原根后，期望 O(√p) 时间与空间，依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶；合数模数不在此接口范围内。

```
struct KthResidue
{
    using ll = long long;
    ll first, ratio, count;

    // Prime p <= 1e12; g is a primitive root of p; k > 0.
    // All roots: first * ratio^i mod p, 0 <= i < count.
    static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
    {
        assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
        a %= p;
        if (a < 0) a += p;
```

```
        if (a == 0) return KthResidue{0, 1, 1};
        ll logarithm = DiscreteLog::solve(g, a, p);
        if (logarithm < 0) return nullopt;
        auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
        if (exponent < 0) return nullopt;
        ll first = Mod64::power(g, exponent, p);
        ll ratio = Mod64::power(g, period, p);
        return KthResidue{first, ratio, (p - 1) / period};
    }
};
```

联合使用示例见第 93 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

3.18.2 素数幂模高次剩余：含非互素被开方数

代码依赖：DiscreteLog（第 85 页，3.17.1 节）、Mod64（第 15 页，2.1 节）、linear_congruence（第 59 页，3.10.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,p,e,g) 求 x^k≡a (mod p^e)。p 是素数，e≥1，p^e≤1e12，k 为正 unsigned long long；a 为 signed long long。奇素数须提供模 p^e 的原根 g；p=2 时不使用 g。无解返回 nullopt，否则 size() 返回解数，get(i,j,lift) 返回指定根，范围是 count[0]、count[1]、lifts，全部组合无重复且无遗漏。字段 first、ratio[2]、step 描述乘法根族与加法提升；step*lifts=p^e。原根与素性由调用者保证。期望 $O(\sqrt{p^e})$ 时间和空间；单次 get 为 $O(\log(p^e))$，不强制展开全部根。依赖离散对数、线性同余及模幂。

```
struct PrimePowerRoots
{
    using ll = long long;
    ll first = 0, step = 1, lifts = 1;
    array<ll, 2> ratio{1, 1}, count{1, 1};

    ll size() const { return count[0] * count[1] * lifts; }

    ll get(ll i, ll j, ll lift) const
    {
        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
        assert(lift >= 0 && lift < lifts);
        ll x = (__int128)first * Mod64::power(ratio[0], i, step) % step;
        x = (__int128)x * Mod64::power(ratio[1], j, step) % step;
        return x + lift * step;
    }

    // p prime, 1 <= e, p^e <= 1e12, k > 0.
    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.
```

接上页同一结构体（第三页继续同一函数）：

```
    static optional<PrimePowerRoots>
    solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
    {
        assert(p >= 2 && p <= 1000000000000LL && e >= 1 && e <= 39 && k > 0);
        array<ll, 40> power{1};
        for (int i = 1; i <= e; i++)
        {
            assert(power[i - 1] <= 1000000000000LL / p);
            power[i] = power[i - 1] * p;
        }
        ll mod = power[e];
        a %= mod;
        if (a < 0) a += mod;
        PrimePowerRoots answer;
        if (a == 0)
        {
            answer.step = power[(e - 1) / k + 1];
            answer.lifts = mod / answer.step;
            return answer;
        }
        int t = 0;
        while (a % p == 0)
        {
            a /= p;
            ++t;
        }
        if (t % k) return nullopt;
```

接上页同一结构体（第三页继续同一函数）：

```
        ll scale = power[t / k], q = power[e - t];
        answer.first = 1;
        answer.step = scale * q;
        answer.lifts = mod / answer.step;
        auto cyclic = [&](ll base, ll target, ll order, int slot)
        {
            ll logarithm = DiscreteLog::solve(base, target, q);
            if (logarithm < 0) return false;
            auto [x, period] = linear_congruence(k % order, -logarithm, order);
            if (x < 0) return false;
            answer.first = (__int128)answer.first * Mod64::power(base, x, q) % q;
            answer.ratio[slot] = Mod64::power(base, period, q);
            answer.count[slot] = order / period;
            return true;
        };
        if (p != 2)
        {
            assert(g >= 1 && g < mod);
            if (!cyclic(g % q, a, q / p * (p - 1), 0)) return nullopt;
        }
        else if (q >= 4)
        {
            int sign = a % 4 == 3;
            auto [x, period] = linear_congruence(k % 2, -sign, 2);
            if (x < 0) return nullopt;
            answer.first = x ? q - 1 : 1;
            answer.ratio[0] = period == 1 ? q - 1 : 1;
            answer.count[0] = 2 / period;
            if (q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1)) return nullopt;
        }
        answer.first *= scale;
        return answer;
    }
};
```

联合使用示例见第 93 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

3.18.3 合数求根准备：分解模数与素数幂原根

代码依赖：Mod64（第 15 页，2.1 节）、PollardRho（第 45 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

root_factors(M,rho) 为 1≤M≤1e12 生成完整的 (p,e,g) 列表，可直接传入 CompositeRoots。rho 是可复用的 PollardRho 对象。按素数递增返回；p=2 时 g=0，其余 g 为模 p^e 的最小正原根。先分解 M，再分解各个欧拉函数值，逐个检查原根候选的阶。M=1 返回空列表。分解使用随机重试；原根搜索耗时取决于候选数，不包含 BSGS。

```
vector<tuple<long long, int, long long>> root_factors(long long mod, PollardRho &rho)
{
    assert(mod >= 1 && mod <= 1000000000000LL);
    auto primes = rho.factor(mod);
    vector<tuple<long long, int, long long>> answer;
    for (int i = 0; i < (int)primes.size();)
    {
        long long p = primes[i], power = 1;
        int e = 0;
        while (i < (int)primes.size() && primes[i] == (unsigned long long)p)
        {
            power *= p;
            ++e;
            ++i;
        }
        long long g = 0;
        if (p != 2)
        {
            long long phi = power / p * (p - 1);
            auto factors = rho.factor(phi);
            factors.erase(unique(factors.begin(), factors.end()), factors.end());
            auto primitive = [&](long long candidate)
            {
                if (candidate % p == 0) return false;
                for (auto q : factors)
                    if (Mod64::power(candidate, phi / q, power) == 1) return false;
                return true;
            };
            g = 2;
            while (!primitive(g)) ++g;
        }
        answer.push_back({p, e, g});
    }
    return answer;
}
```

联合使用示例见第 93 页（Luogu P5668 【模板】N 次剩余），其中直接调用本模板。

3.18.4 一般合数模高次剩余：CRT 组合与按索引取根

代码依赖：PollardRho（第 45 页，3.6.1 节）、PrimePowerRoots（第 87 页，3.18.2 节）、mod_inverse（第 55 页，3.9.2 节）、root_factors（第 90 页，3.18.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,k,factors) 求 x^k≡a (mod M)，factors 是完整且素数互异的 (p,e,g) 列表，M 为所有 p^e 的乘积，M≤1e12，k 为正 unsigned long long，a 为 signed long long。奇素数的 g 是模 p^e 原根；p=2 时填 0。无解返回 nullopt，否则 mod=M、total 为解数，get(index) 在 0≤index<total 上取出每个根恰好一次，不按数值排序。空因子列表表示模 1，唯一标准代表为 0。依赖 PrimePowerRoots 和模逆元；列表接口不做分解；重载 solve(a,k,M,rho) 自动分解并寻找原根，rho 为 PollardRho。重复同模数宜复用因子列表。预处理的主要成本是各素数幂的 BSGS；get 为 O(F log M)，F 为不同素因子数。无需展开全部根，parts 和 weight 保存局部根族及 CRT 系数。

```
struct CompositeRoots
{
    using ll = long long;
    ll mod = 1, total = 1;
    vector<PrimePowerRoots> parts;
    vector<ll> weight;

    ll get(ll index) const
    {
        assert(index >= 0 && index < total);
        ll answer = 0;
        for (int t = 0; t < (int)parts.size(); t++)
        {
            const auto &p = parts[t];
            ll local = index % p.size();
            index /= p.size();
            ll lift = local % p.lifts;
            local /= p.lifts;
            ll j = local % p.count[1], i = local / p.count[1];
            ll root = p.get(i, j, lift);
            answer = (answer + (__int128)root * weight[t]) % mod;
        }
        return answer;
    }

    // Factors are distinct primes with positive exponents: (p, e, g).
    // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.
```

局部求根与 CRT 系数（接上页同一结构体）：

```
    static optional<CompositeRoots>
    solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
    {
        assert(k > 0);
        CompositeRoots answer;
        for (auto [p, e, g] : factors)
        {
            auto roots = PrimePowerRoots::solve(a, k, p, e, g);
            if (!roots) return nullopt;
            ll m = roots->step * roots->lifts;
            assert(gcd(answer.mod, m) == 1);
            assert(answer.mod <= 1000000000000LL / m);
            answer.mod *= m;
            answer.total *= roots->size();
            answer.parts.push_back(*roots);
        }
        for (const auto &p : answer.parts)
        {
            ll m = p.step * p.lifts, rest = answer.mod / m;
            ll inverse = mod_inverse(rest % m, m);
            answer.weight.push_back((__int128)rest * inverse % answer.mod);
        }
        return answer;
    }

    static optional<CompositeRoots>
    solve(ll a, unsigned long long k, ll mod, PollardRho &rho)
    {
        return solve(a, k, root_factors(mod, rho));
    }
};
```

模板题使用：Luogu P5668 【模板】N 次剩余

题意：T≤100，每组按 n,m,k 读入，求 x^n≡k(mod m) 在 0..m-1 的所有解，先输出解数，非零时下一行升序输出全部根。1≤n≤10^9、0≤k<m≤10^9；所有组总解数 ≤10^6，每个素数幂上的根数亦 ≤10^6。root_factors 准备分解和原根：素数走 KthResidue（p=2 用原根 1），素数幂走 PrimePowerRoots，一般合数走 CompositeRoots。m=1 对应空因子列表，唯一根 0；无解只输出 0 一行。各入口原始顺序不保证升序，最后统一排序，不去重来掩盖重复根。枚举会占 O(c) 空间；大根族应仅在题目输出界内展开。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、extended_gcd（第 47 页，3.7.1 节）、mod_inverse（第 55 页，3.9.2 节）、DiscreteLog（第 85 页，3.17.1 节）、linear_congruence（第 59 页，3.10.1 节）、KthResidue（第 86 页，3.18.1 节）、PrimePowerRoots（第 87 页，3.18.2 节）、root_factors（第 90 页，3.18.3 节）、CompositeRoots（第 91 页，3.18.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5668

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    PollardRho rho;
    while (t--)
    {
        unsigned long long n;
        long long m, k;
        cin >> n >> m >> k;
        auto f = root_factors(m, rho);
        vector<long long> a;
        if (f.size() == 1)
        {
            auto [p, e, g] = f[0];
            if (e == 1)
            {
                auto r = KthResidue::solve(k, n, p, p == 2 ? 1 : g);
                if (r)
                {
                    long long x = r->first;
                    for (long long i = 0; i < r->count; i++)
                    {
                        a.push_back(x);
                        x = (__int128)x * r->ratio % p;
                    }
                }
            }
            else
            {
                auto r = PrimePowerRoots::solve(k, n, p, e, g);
                if (r)
                    for (long long i = 0; i < r->count[0]; i++)
                        for (long long j = 0; j < r->count[1]; j++)
                            for (long long z = 0; z < r->lifts; z++)
                                a.push_back(r->get(i, j, z));
            }
        }
```

继续处理操作（接上页同一 main）：

```
        else
        {
            auto r = CompositeRoots::solve(k, n, f);
            if (r)
                for (long long i = 0; i < r->total; i++) a.push_back(r->get(i));
        }
        sort(a.begin(), a.end());
        cout << a.size() << '\n';
        if (!a.empty())
        {
            for (int i = 0; i < (int)a.size(); i++)
                cout << a[i] << (i + 1 == (int)a.size() ? '\n' : ' ');
        }
    }
    return 0;
}
```

3.19 数论分块

3.19.1 整除分块：前缀和来源决定算法

对 $H(t)=\sum_{i=1}^{t}h(i)$、$H(0)=0$，设 $q_A=\lfloor A/l\rfloor, q_B=\lfloor B/l\rfloor$。两个商同时不变的最大右端点为

$$r=\min(\lfloor A/q_A\rfloor, \lfloor B/q_B\rfloor), \quad \text{整块贡献}(H(r)-H(l-1))q_Aq_B.$$

循环只到 $l\le\min(A,B)$，所以进入循环时两个商均非零；零边长直接得到空和。例如 $A=10,B=8,l=3$，只按 B 得到 $r=4$，而 $\lfloor 10/3\rfloor=3,\lfloor 10/4\rfloor=2$，必须取 $r=3$。不应只按一边分块再假定另一边的商相同。

单个 n 的商至多 $O(\sqrt{n})$ 种；双商分块的边界来自两组边界的并集，段数为 $O(\sqrt{A}+\sqrt{B})$，同时不超过 $\min(A,B)$。**不能统一写成** $O(\sqrt{\min(A,B)})$**：**当 $A=m,B=m^2$ 时，B/d 在 $d=1,\dots,m$ 的整数商两两不同，产生 m 段。分块只减少求和段数；若每段都临时线性计算 $H(r)$，仍然可能很慢。

3.19.1.1 按数据与资源选前缀和

上界可预筛，询问多： 第 3.5.1 节（第 42 页）的 `LinearSieve s(L)` 提供 `s.mu`、`s.phi`；再建前缀和为 $O(L)$ 时间、空间，分块中每次查表 $O(1)$。互质矩形直接使用 `CoprimePairs`；GCD 总和则用 φ 的前缀和。筛上限需覆盖所查询的前缀下标，矩形只需短边。μ 的前缀可为负，不能用无符号类型；φ 前缀不能沿用单项的 `int`。

只求少量巨大前缀： 第 3.21.1 节（第 99 页）的 `DuJiao s(B)` 支持 `s.mertens(n)` 与 `s.totient_sum(n)`，分别返回有符号 64 位与 128 位精确整数，当前约定 $0\le n\le 10^{11}$。$1\le B<$ `INT_MAX` 且内存可用；构造参数是预筛上限，不是最大查询值。对单次最大参数 N，预筛加递归的常用期望时间界为 $O(B+N/\sqrt{B})$，选 B 约 $N^{2/3}$ 得 $O(N^{2/3})$；哈希按期望常数计。这不是随便取一个很小 B 就能保证快速。

内存先于理论最优值。当前 `DuJiao` 两张长期前缀表各用 64 位，仅这部分就约 $16(B+1)$ 字节；构造时还同时存在 `LinearSieve` 的三张 `int` 表与素数表，缓存另计（按常见 4 字节 `int`、8 字节 `long long`）。$N=10^{11}$ 时 $N^{2/3}$ 约为 2.15×10^7，两张前缀表已约 345 MB，不能不看内存限制就照此开表。多组不同查询会新增缓存，不能直接承诺单次查询的空间界。重复参数可复用缓存；不要每查一次前缀就重新构造一个筛对象。

优先利用问题本身的化简。例如巨大正方形的互质有序对可以直接取 `2 * s.totient_sum(n) - 1`（$n\ge1$），全程保持 128 位。一般矩形可把 H 换为 `mertens`，但各段前缀查询也会触发递归，不能把整个嵌套过程仅按外层 $O(\sqrt{A}+\sqrt{B})$ 计时；没有额外分析时也不能直接套一个单前缀的 $O(N^{2/3})$ 总界。当前 `CoprimePairs` 不接受杜教筛对象，需自行写相应分块调用。

递推从哪来。对 $n \ge 1$，再用 $\mu * \mathbf{1} = \varepsilon$ 与 $\varphi * \mathbf{1} = \mathrm{id}$ 对 $1, \dots, n$ 求和并交换次序，得到

$$M(n) = 1 - \sum_{i=2}^{n} M(\lfloor n/i \rfloor), \qquad \Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi(\lfloor n/i \rfloor).$$

其中 $M(n) = \sum_{i=1}^{n} \mu(i)$，两式均按等商段乘上段长，第二式没有额外的 i 权重。递归只作用于小于 n 的参数；前缀 0 为 0。这些结论及分块背景参见 OI Wiki：数论分块、OI Wiki：杜教筛。涉及乘法时应先提升类型再乘；模数下遇到负莫比乌斯和要归一化。这里只讲精确计数，若最后除以 2，先完成整数除法再取模，合数模数下不能擅自求 2 的逆元。

3.20 莫比乌斯反演

3.20.1 矩形 GCD 计数

代码依赖：LinearSieve（第 42 页，3.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 CoprimePairs(n) 预处理 mu 前缀和，0≤n<INT_MAX，O(n) 时间与空间；依赖 LinearSieve。count(a,b) 求 [1,a]×[1,b] 的互质数对，a,b 为非负 int，要求 min(a,b)≤n，任一为 0 返回 0。rectangle(a,b,c,d,k) 求 [a,b]×[c,d] 中 gcd(x,y)=k 的数对，要求 1≤a≤b、1≤c≤d、k≥1，min(b/k,d/k)≤n；闭区间容斥。返回 long long，中间分块累加用 int128。单次前缀查询 O($\sqrt{a}$+$\sqrt{b}$)，额外空间 O(1)。sum 为 mu 前缀和，只读。

```
struct CoprimePairs
{
    vector<int> sum;

    CoprimePairs(int n) : sum(LinearSieve(n).mu)
    {
        for (int i = 1; i <= n; i++) sum[i] += sum[i - 1];
    }

    long long count(int a, int b) const
    {
        assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
        __int128_t answer = 0;
        for (int l = 1, r; l <= min(a, b); l = r + 1)
        {
            int x = a / l;
            int y = b / l;
            r = min(a / x, b / y);
            answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
        }
        return (long long)answer;
    }

    long long rectangle(int a, int b, int c, int d, int k) const
    {
        assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
        return count(b / k, d / k) - count((a - 1) / k, d / k) -
               count(b / k, (c - 1) / k) + count((a - 1) / k, (c - 1) / k);
    }
};
```

应用题补充：Luogu P2522 [HAOI2011] Problem b

题意：给出最多 50000 组闭区间 [a,b]、[c,d] 和 k，统计 gcd(x,y)=k 的数对，所有端点及 k 在 1..50000。只初始化一次 CoprimePairs(50000)，每组调用 rectangle(a,b,c,d,k)，返回 long long。它先除以 k，再用四个互质前缀容斥；k 超过区间端点时零前缀自动处理。一般调用 count(A,B) 须预处理到 min(A,B)，本题固定 50000 即可。此为比赛应用题，不计独立模板题覆盖。

所需模板：LinearSieve（第 42 页，3.5.1 节）、CoprimePairs（第 96 页，3.20.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2522

```
int main()
{
    CoprimePairs solver(50000);
    int t;
    scanf("%d", &t);
    while (t--)
    {
        int a, b, c, d, k;
        scanf("%d%d%d%d%d", &a, &b, &c, &d, &k);
        printf("%lld\n", solver.rectangle(a, b, c, d, k));
    }
    return 0;
}
```

3.20.2 整除与反演：先辨认求和方向

以下数论函数定义在**正整数**上；数组下标 0 只是前缀和的占位。狄利克雷卷积、单位元与两条常用恒等式为

$$(f*g)(n)=\sum_{d\mid n} f(d)g(n/d),\qquad \varepsilon(n)=[n=1],\quad \mathbf{1}(n)=1,\quad \mathrm{id}(n)=n.$$

$$\mu*\mathbf{1}=\varepsilon,\qquad \varphi*\mathbf{1}=\mathrm{id}\,.$$

本节按“难以直接统计的量、容易统计的整除条件、换序后的权重”推导，不要看到 GCD 就直接在原式前乘一个 μ。

3.20.2.1 约数和与倍数和不能混用

约数方向。若 $F(n)=\sum_{d\mid n} f(d)$，则 $f(n)=\sum_{d\mid n}\mu(d)F(n/d)$。这不要求 f 积性；用卷积写成 $F=f*\mathbf{1}$，再卷积 μ 就消掉了 $\mathbf{1}$。积性只在后续选用筛法或拆质数幂时才需要。

倍数方向。若 $E(k)$ 仅在 $1\le k\le V$ 非零，且 $G(d)=\sum_{j\ge 1,\,dj\le V} E(dj)$，则

$$E(k)=\sum_{t=1}^{\lfloor V/k\rfloor}\mu(t)G(kt).$$

代入 G 后，每个 $E(ks)$ 的系数是 $\sum_{t\mid s}\mu(t)=[s=1]$。这里求的是 k 的倍数，不能改成 k 的约数；有限支持保证换序合法，无限级数还须另外证明收敛，不能把竞赛里的有限证明直接外推。

例一：数组中互质的不同下标对。正整数数组 $a_1,\dots,a_n\le V$，统计 $i<j$ 且 $\gcd(a_i,a_j)=1$。令 $c_d=\#\{i: d\mid a_i\}$，则 $G(d)=\binom{c_d}{2}$ 统计 GCD 被 d 整除的下标对，答案是 $E(1)=\sum_{d=1}^{V}\mu(d)\binom{c_d}{2}$。例如数组 $(2,3,4,6)$，非零相关项给出 $6-3-1=2$，恰为下标对应的 $(2,3),(3,4)$。重复值仍按下标区分：$(1,1,2)$ 的三个下标对都互质，答案为 3。既然已用 $\binom{c_d}{2}$ 排除了相同下标，就不能再减去一个“自配对数”。

从频率表求所有 c_d，枚举 d 的倍数，总时间 $\sum_{d\le V} O(V/d)=O(V\log V)$、空间 $O(V)$；只求互质对，再筛出 μ 并线性累加即可。若要全部 $E(k)$，也可从大到小做 $E(k)=G(k)-\sum_{j\ge 2,\,kj\le V}E(kj)$，同为 $O(V\log V)$。数值 V 太大时不能因为 n 小就开长度 V 的数组，须改为分解、枚举实际约数等其他方案。

3.20.2.2 GCD 权值先反演，再交换求和

设所求为 $S=\sum_{x=1}^{A}\sum_{y=1}^{B} w(\gcd(x,y))$。先定义 $h=\mu*w$，于是 $w=h*\mathbf{1}$；再将公共约数提到外层：

$$S=\sum_{x=1}^{A}\sum_{y=1}^{B}\sum_{d\mid x,\,d\mid y} h(d)=\sum_{d=1}^{\min(A,B)} h(d)\left\lfloor\frac{A}{d}\right\rfloor\left\lfloor\frac{B}{d}\right\rfloor.$$

这同时涵盖互质计数 $w(t)=[t=1]$、$h=\mu$，以及 GCD 总和 $w(t)=t$、$h=\varphi$。乘出来的是**卷积后的** $h(d)$，一般不是 $\mu(d)w(d)$。

例二：$[1,3]\times[1,4]$ 的 GCD 总和。按三行直接求得 $4+6+6=16$；按上式得到

$$\varphi(1)\cdot 3\cdot 4+\varphi(2)\cdot 1\cdot 2+\varphi(3)\cdot 1\cdot 1=12+2+2=16.$$

若误用 $\mu(d)d$ 作系数，结果变成 $12-4-3=5$。已知 h 的前缀和后可按第 3.19.1 节分块；不知道前缀和如何高效取得时，“反演完了”还不等于算法已经足够快。基本反演恒等式参见 OI Wiki：莫比乌斯反演。

3.20.3 GCD 计数：缩放、边界与对称性

记 $C(A,B)$ 为 $[1,A]\times[1,B]$ 中的**有序**互质数对数，任一边为 0 时为 0。第 3.20.1 节（第 96 页）的 `CoprimePairs c(L)` 预筛至 L，`c.count(A,B)` 就是 $C(A,B)$；要求 A,B 为非负 `int` 且 $\min(A,B)\le L$。它不包括坐标轴，不会自动去掉 $x=y$，也不会替题目把有序对变成无序对。

3.20.3.1 恰好等于 k：先缩放整除对象，再做矩形容斥

当 $k\ge 1$ 且 $x,y>0$，$\gcd(x,y)=k$ 等价于 $x=ku,y=kv$ 且 $\gcd(u,v)=1$。对闭区间 $a\le x\le b$、$c\le y\le d$，其中 $a,c\ge 1$，令

$$A=\lfloor b/k\rfloor,\quad A_0=\lfloor (a-1)/k\rfloor,\qquad B=\lfloor d/k\rfloor,\quad B_0=\lfloor (c-1)/k\rfloor.$$

答案为 $C(A,B)-C(A_0,B)-C(A,B_0)+C(A_0,B_0)$。$A_0$ 统计严格小于 a 的 k 的倍数；应**先减 1 再除**，一般不等于 $\lfloor a/k\rfloor-1$。当 $k>\min(b,d)$ 时答案自动为 0。

例三：$[3,8]\times[5,10]$ 内 GCD 恰为 2。缩放后为 $[2,4]\times[3,5]$，前缀四项为 $C(4,5)-C(1,5)-C(4,2)+C(1,2)=15-5-6+2=6$。原坐标中的六对是 $(4,6),(4,10),(6,8),(6,10),(8,6),(8,10)$。当前接口的完整调用可写为：

```
int a = 3, b = 8, c = 5, d = 10, k = 2;
CoprimePairs solver(min(b / k, d / k));
long long answer = solver.rectangle(a, b, c, d, k); // 6
```

多测可一次预筛至所有查询的 $\max\min(b/k,d/k)$ 后复用。这里的 a,b,c,d,k 是正 `int`，$a\le b,c\le d$；接口本身不是 64 位大端点版本，不能把大整数截断后传入。

3.20.3.2 去掉顺序之前，先检查交换是否仍在定义域内

同一正整数区间 I。设 T 为 $I\times I$ 中 GCD 为 k 的有序对数，对角线上只有 (k,k) 可能合法。令 $\delta=[k\in I]$，则

$$\#\{x<y\}=\frac{T-\delta}{2},\qquad \#\{x\le y\}=\frac{T+\delta}{2}.$$

只有定义域在交换 x,y 后不变，才能将非对角线两两配对。例如 $C(5,5)=19$，互质的 $x<y$ 有 9 对、$x\le y$ 有 10 对。而 $C(2,3)=5$：其中 $(1,3),(2,3)$ 的交换不在矩形内，不能把 5 直接除以 2。若题目要求不同下标，即使两个值相同也可能是合法对象，应回到整除反演建模（第 3.20.2 节，第 97 页）的频率计数。

正方形可进一步化简。对 $n\ge 1$，按较大的坐标 y 分类，$x<y$ 且互质的数量为 $\sum_{y=2}^{n}\varphi(y)=\Phi(n)-1$，故

$$C(n,n)=2\Phi(n)-1,\qquad \Phi(n)=\sum_{i=1}^{n}\varphi(i).$$

例如 $\Phi(5)=10$，立即得到 19、9、10 三种计数。$n=0$ 单独返回 0，不能得到 -1。这个化简还能避免为一个正方形答案逐块查询莫比乌斯前缀和。

含零或负数时另外建模。正整数反演式不能直接包含 $\mu(0)$。按通常 $\gcd(0,y)=|y|$，若非负矩形包含坐标轴，GCD 为 $k>0$ 的轴上候选只有 $(0,k),(k,0)$；分别判断是否落在范围内，再加到正整数部分。$(0,0)$ 的 GCD 通常定义为 0，题面另有约定时从其约定；负数需要处理绝对值与重数。

3.21 狄利克雷双曲线法 & 杜教筛

3.21.1 杜教筛

代码依赖：LinearSieve（第 42 页，3.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造预筛上限 1≤B<INT_MAX，内存须可用；mertens(n) 返回 μ 的前缀和，totient_sum(n) 返回 φ 的前缀和（int128）。约定 0≤n≤10^{11}，n=0 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 pmu/pphi/limit。单次最大参数 N 的常用均摊分析为 O(B+N/$\sqrt{B}$)（哈希按期望常数计算），B 约 N^(2/3) 时为 O(N^(2/3))；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 O(log N)。

```
struct DuJiao
{
    using ll = long long;
    using I = __int128_t;
    int limit;
    vector<ll> pmu, pphi;
    unordered_map<ll, ll> mmu;
    unordered_map<ll, I> mphi;

    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
    {
        assert(limit >= 1);
        LinearSieve s(limit);
        for (int i = 1; i <= limit; i++)
        {
            pmu[i] = pmu[i - 1] + s.mu[i];
            pphi[i] = pphi[i - 1] + s.phi[i];
        }
    }

    ll mertens(ll n)
    {
        if (n <= limit) return pmu[n];
        if (mmu.count(n)) return mmu[n];
        ll ans = 1;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= (r - l + 1) * mertens(n / l);
        }
        return mmu[n] = ans;
    }

    I totient_sum(ll n)
    {
        if (n <= limit) return pphi[n];
        if (mphi.count(n)) return mphi[n];
```

```
        I ans = I(n) * (n + 1) / 2;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= I(r - l + 1) * totient_sum(n / l);
        }
        return mphi[n] = ans;
    }
};
```

模板题使用：Luogu P4213

题意：多次查询 φ 与 μ 的前缀和，先输出 φ，再输出 μ。先读入全部 n，预筛到 min(最大 n,2000000)，同一结构复用缓存。题目 n<2^31，所以 φ 的总和可安全转为 long long；模板更大范围的 int128 返回值不能照搬此转换。

所需模板：LinearSieve（第 42 页，3.5.1 节）、DuJiao（第 99 页，3.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4213

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    vector<long long> a(t);
    for (auto &n : a) cin >> n;
    int b = min(2000000LL, *max_element(a.begin(), a.end()));
    DuJiao d(b);
    for (long long n : a)
        cout << (long long)d.totient_sum(n) << ' ' << d.mertens(n) << '\n';
}
```

模板题使用：Library Checker sum_of_totient_function

题意：求 φ 前缀和对 998244353 的余数，n≤10^10。先在 int128 中求和并取模，再转为 long long 输出；不能先把未取模的总和截成 64 位。预筛上限按输入截到 2000000。

所需模板：LinearSieve（第 42 页，3.5.1 节）、DuJiao（第 99 页，3.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_totient_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n;
    cin >> n;
    DuJiao d(min(n, 2000000LL));
    cout << (long long)(d.totient_sum(n) % 998244353) << '\n';
}
```

3.22 素数计数

3.22.1 素数计数（整除分块筛）

prime_count(n) 返回不大于 n 的素数个数，精确 long long，不取模。支持 0≤n≤10^12；n<2 返回 0。仅存不同的 floor(n/i)，由大到小筛去最小质因子为 p 的合数；平方根经整数修正。O(n^0.75 / log n) 时间、O($\sqrt{n}$) 空间，小 n 按常数处理；范围上界不表示所有平台时限保证。

```
long long prime_count(long long n)
{
    assert(0 <= n && n <= 1000000000000LL);
    if (n < 2) return 0;
    int s = sqrtl(n);
    while (1LL * (s + 1) * (s + 1) <= n) s++;
    while (1LL * s * s > n) s--;
    vector<long long> w, g;
    vector<int> small(s + 1), large(s + 1);
    for (long long l = 1, r; l <= n; l = r + 1)
    {
        long long x = n / l;
        r = n / x;
        int id = w.size();
        if (x <= s)
            small[x] = id;
        else
            large[n / x] = id;
        w.push_back(x);
        g.push_back(x - 1);
    }
    auto id = [&](long long x)
    {
        return x <= s ? small[x] : large[n / x];
    };
    for (int p = 2; p <= s; p++)
    {
        long long before = g[id(p - 1)];
        if (g[id(p)] == before) continue;
        for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            g[j] -= g[id(w[j] / p)] - before;
    }
    return g[0];
}
```

模板题使用：Library Checker counting_primes

题意：输入 n，输出不大于 n 的素数个数，不取模。题目 n≤10^11。

所需模板：prime_count（第 101 页，3.22.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_primes

```
int main()
{
    long long n;
    cin >> n;
    cout << prime_count(n) << '\n';
}
```

3.23 Min_25 筛

3.23.1 Min_25 积性函数求和

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 Min25<mod>(n)，1≤n≤10^12，mod 为大于 3 的素数。sum(c, power) 求 ∑(1≤i≤n) f(i)，要求 f 为积性函数、f(1)=1，f(p)=c[0]+c[1]p+c[2]p^2；power(p,e,pe) 返回 f(p^e)，其中 pe=p^e、e≥1，回调必须与一次幂系数一致。支持同一对象多次 sum；不改筛表。预处理 O(n^0.75 / log n)，回调 O(1) 时递归求和保守 O(n) 上界；O(√n) 空间、O(log n) 递归栈。w 降序存不同整除商，g[id(x)][d] 是模意义下 ∑(p≤x) p^d；id 仅允许查询这些整除商（包含 1..√n），各字段只读。依赖 ModInt；普通递归版，未承诺全部官方时限达标。

```
template <int mod> struct Min25
{
    using ll = long long;
    using Z = ModInt<mod>;
    ll n;
    int s;
    vector<ll> w;
    vector<int> small, large, primes;
    vector<array<Z, 3>> g;

    Min25(ll n) : n(n)
    {
        static_assert(mod > 3);
        assert(1 <= n && n <= 1000000000000LL);
        s = sqrtl(n);
        while (1LL * (s + 1) * (s + 1) <= n) s++;
        while (1LL * s * s > n) s--;
        small.resize(s + 1);
        large.resize(s + 1);
        Z half = Z(2).inv(), sixth = Z(6).inv();
        for (ll l = 1, r; l <= n; l = r + 1)
        {
            ll x = n / l;
            r = n / x;
            int k = w.size();
            if (x <= s)
                small[x] = k;
            else
                large[n / x] = k;
            w.push_back(x);
            Z v = x;
            g.push_back(
                {v - 1, v * (v + 1) * half - 1, v * (v + 1) * (v * 2 + 1) * sixth - 1});
        }
```

```
        array<Z, 3> prefix{};
        for (int p = 2; p <= s; p++)
        {
            if (g[id(p)][0].v == g[id(p - 1)][0].v) continue;
            primes.push_back(p);
            array<Z, 3> weight{1, p, Z(p) * p};
            for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            {
                int k = id(w[j] / p);
                for (int d = 0; d < 3; d++)
                    g[j][d] = g[j][d] - weight[d] * (g[k][d] - prefix[d]);
            }
            for (int d = 0; d < 3; d++) prefix[d] = prefix[d] + weight[d];
        }
    }

    int id(ll x) const { return x <= s ? small[x] : large[n / x]; }

    template <class F> Z sum(array<Z, 3> c, F power) const
    {
        vector<Z> prefix(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); i++)
        {
            Z p = primes[i];
            prefix[i + 1] = prefix[i] + c[0] + c[1] * p + c[2] * p * p;
        }
        auto dfs = [&](auto &&self, ll x, int k) -> Z
        {
            if (x < 2 || (k && x <= primes[k - 1])) return 0;
            auto a = g[id(x)];
            Z ans = c[0] * a[0] + c[1] * a[1] + c[2] * a[2] - prefix[k];
            for (int i = k; i < (int)primes.size() && primes[i] <= x / primes[i]; i++)
            {
                ll p = primes[i], pe = p;
                for (int e = 1; pe <= x / p; e++, pe *= p)
                    ans = ans + power(p, e, pe) * self(self, x / pe, i + 1) +
                          power(p, e + 1, pe * p);
            }
            return ans;
        };
        return dfs(dfs, n, 0) + 1;
    }
};
```

模板题使用：Luogu P5325

题意：求积性函数前缀和，f(p^e)=p^e(p^e-1)，模 1000000007。一次幂为 p^2-p，故系数传 {0,-1,1}；回调的 pe 已是整数 p^e，无需再快速幂。

所需模板：ModInt（第 24 页，3.2.2 节）、Min25（第 103 页，3.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5325

```
int main()
{
    using Z = ModInt<1000000007>;
    long long n;
    cin >> n;
    Min25<1000000007> sieve(n);
    auto power = [](long long p, int e, long long pe)
    {
        Z x = pe;
        return x * (x - 1);
    };
    cout << sieve.sum({0, -1, 1}, power).v << '\n';
}
```

模板题使用：Library Checker sum_of_multiplicative_function

题意：多组输入 n,a,b，求 f(p^e)=ae+bp 的积性函数前缀和，模 469762049。一次幂系数为 {a,b,0}；每组重新构造筛表。官方把普通 Min_25 参考解标为允许超时，本地正确性通过不代表在线时限通过。

所需模板：ModInt（第 24 页，3.2.2 节）、Min25（第 103 页，3.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_multiplicative_function

```
int main()
{
    using Z = ModInt<469762049>;
    int t;
    cin >> t;
    while (t--)
    {
        long long n;
        int a, b;
        cin >> n >> a >> b;
        Min25<469762049> sieve(n);
        auto power = [&](long long p, int e, long long pe)
        {
            return Z(a) * e + Z(b) * p;
        };
        cout << sieve.sum({a, b, 0}, power).v << '\n';
    }
}
```

3.24 类欧几里德算法

3.24.1 有符号整除和：类欧几里德

floor_sum(n,m,a,b) 返回 i=0..n-1 的 floor((ai+b)/m) 之和，结果为 int128。要求 0≤n≤10^9、1≤m≤10^9，a、b 支持完整 long long；n=0 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 floor。O(log m) 时间、O(1) 空间，不依赖完整数论模块。

```
__int128_t floor_sum(long long n, long long m, long long a, long long b)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(n >= 0 && m > 0);
    i128 ans = 0;
    auto norm = [&](ll &v)
    {
        ll q = v / m;
        if (v % m < 0) --q;
        v = (ll)(i128(v) - i128(q) * m);
        return q;
    };
    ans += i128(norm(a)) * n * (n - 1) / 2;
    ans += i128(norm(b)) * n;
    while (true)
    {
        ans += i128(n) * (n - 1) / 2 * (a / m);
        a %= m;
        ans += i128(n) * (b / m);
        b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) break;
        n = (ll)(y / m);
        b = (ll)(y % m);
        swap(a, m);
    }
    return ans;
}
```

模板题使用：Library Checker sum_of_floor_of_linear

题意：求 i=0..n-1 的 floor((a*i+b)/m) 之和。此题有 0≤a,b<m，因此结果可转为 long long 输出；更一般的有符号输入不能照搬该转换。

所需模板：floor_sum（第 106 页，3.24.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_floor_of_linear

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        long long n, m, a, b;
        cin >> n >> m >> a >> b;
        // This task has 0 <= a,b < m, so the sum is at most n*(n-1)/2.
        cout << (long long)floor_sum(n, m, a, b) << '\n';
    }
}
```

3.24.2 整除和、带权和与平方和

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

floor_moments(n,m,a,b) 对 i=0..n-1，令 y_i=floor((a*i+b)/m)，返回 {∑y_i, ∑i*y_i, ∑y_i^2}，元素为 ModInt<998244353>，取整数用.v。0≤n≤1000000001，1≤m≤10^9；a、b 支持完整 long long。n=0 返回全零。时间与递归空间 O(log(m+1))；模数固定，499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间，驱动传 n+1，输出索引顺序 0、2、1。

```
array<ModInt<998244353>, 3>
floor_moments(long long n, long long m, long long a, long long b)
{
    using Z = ModInt<998244353>;
    assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
    if (!n) return {0, 0, 0};
    auto normalize = [&](long long &v)
    {
        long long q = v / m;
        if (v % m < 0) q--;
        v = (__int128_t(v) - __int128_t(q) * m);
        return Z(q);
    };
    Z x = normalize(a);
    Z y = normalize(b);
    Z count = n;
    Z s1 = count * Z(n - 1) * Z(499122177);
    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
    Z f = 0, g = 0, h = 0;
    if (a)
    {
        long long height = (a * (n - 1) + b) / m;
        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
        Z t = height;
        f = count * t - u;
        g = t * s1 - (w - u) * Z(499122177);
        h = count * t * t - Z(2) * v - u;
    }
    return {f + x * s1 + y * count,
            g + x * s2 + y * s1,
            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
                Z(2) * y * f};
}
```

模板题使用：Luogu P5170

题意：求 i=0..n 的整除和、整除值平方和、i 乘整除值之和，模 998244353。t 至 10^5，n/a/b/c 至 10^9 且 c>0。核心参数顺序 (n+1,c,a,b)，返回顺序是和/带权和/平方和；本题输出索引 0、2、1，不能只核对第一项。

所需模板：ModInt（第 24 页，3.2.2 节）、floor_moments（第 108 页，3.24.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5170

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long n, a, b, c;
        scanf("%lld%lld%lld%lld", &n, &a, &b, &c);
        auto result = floor_moments(n + 1, c, a, b);
        printf("%d %d %d\n", result[0].v, result[2].v, result[1].v);
    }
    return 0;
}
```

3.24.3 类欧几里德：有符号整除求和

接口见第 3.24.1 节（第 106 页）。`floor_sum(n,m,a,b)` 返回

$$F(n,m,a,b)=\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor.$$

范围约定 $0\le n\le 10^9$、$1\le m\le 10^9$，a,b 支持完整有符号 64 位，返回 128 位。n 是项数；若题目下标为 $0\ldots N$，需传 $N+1$ 并检查仍在约定范围内。

先令 $a=q_am+r_a$、$b=q_bm+r_b$，其中余数属于 $[0,m)$，答案先加上 $q_an(n-1)/2+q_bn$。C++ 的负整数除法向零截断，因此余数为负时需要把商减 1，再用 128 位乘法计算规范余数。这一步不能简单用两次取模替代后丢弃商的贡献。

对规范化后的 $0\le a,b<m$，设 $y=an+b$。若 $y<m$，余下贡献为 0；否则交换格点计数的求和方向可得

$$F(n,m,a,b)=F(\lfloor y/m\rfloor,a,m,y \bmod m).$$

下一轮先提取新的整商，再继续交换。进入交换分支时 $a>0$，不会产生零分母。模数按欧几里得过程减小，时间 $O(\log(m+1))$，额外空间 $O(1)$。

双版本以任意精度向下取整逐项求和对拍，覆盖负系数、零项数和整除边界；十亿项的常数、连续整数、反向连续整数及整倍数斜率使用闭式公式独立核验。同时检查模数 1 下两个系数的 64 位极值。普通与 ASan/UBSan 均通过，暂无在线 AC。本节只验证基本整除和；下节单独给出带下标权与平方和的接口及证据。

3.24.4 类欧几里德的三个求和量

实现见第 3.24.2 节（第 108 页）。令 $y_i = \lfloor (ai+b)/m \rfloor$，维护 $f = \sum y_i$、$g = \sum i y_i$、$h = \sum y_i^2$，下标均为 $0 \le i < n$。范围为 $0 \le n \le 1000000001$、$1 \le m \le 10^9$，a, b 支持完整有符号 64 位。空和 $n = 0$ 直接返回 $(0, 0, 0)$；下述含 $n-1$ 的高度递推只在 $n \ge 1$ 时使用。结果固定模 998244353，不要更换模数却保留 2、6 的逆元常数。

先把 $a = xm + a'$、$b = ym + b'$ 归一化；设规范化问题返回 (f_0, g_0, h_0)，并记 $S_1 = \sum i$、$S_2 = \sum i^2$，则

$$\begin{aligned} f &= f_0 + xS_1 + yn, \\ g &= g_0 + xS_2 + yS_1, \\ h &= h_0 + x^2 S_2 + y^2 n + 2xyS_1 + 2xg_0 + 2yf_0. \end{aligned}$$

负系数的商仍须向下取整。规范化后的 $a' = 0$ 时三个剩余量均为 0。

对 $0 < a' < m$，令 $t = \lfloor (a'(n-1) + b')/m \rfloor$。高度 $j = 1 \dots t$ 首次出现的下标为 $r_j = \lceil (mj - b')/a' \rceil$。以 $q = j - 1$ 编号，递归参数为 $(t, a', m, m + a' - b' - 1)$；设返回 (u, v, w)，即 $u = \sum r_j$、$v = \sum (j-1) r_j$、$w = \sum r_j^2$，则

$$f_0 = nt - u, \qquad g_0 = tS_1 - (w-u)/2, \qquad h_0 = nt^2 - 2v - u.$$

这来自按高度计算每一层的贡献，其中 $y_i^2 = \sum_{j=1}^{y_i} (2j-1)$。分母每次严格减小，所以只有 $O(\log(m+1))$ 层递归。

P5170 的上界包含 n，驱动传入 $n+1$；题目输出顺序为 f, h, g，与本接口不同。P5170 完整输入输出见第 3.24.2 节（第 109 页）；本题多组数量至 10^5。核心函数支持十亿加一项，并对完整有符号 64 位系数做任意精度逐项对拍；大项数用一次多项式的和与平方和闭式检查。两套完整程序各验证十万组输入，检查最大闭区间端点与全部三个输出。普通与 ASan/UBSan 均通过，在线 AC 待补；任意更高次矩尚不在这个接口内。

补充的大参数测试包含 3000 组随机参数、64 位系数端点和斐波那契型欧几里得链。反转 $i \mapsto n-1-i$ 应保持 f, h 不变，并使 g 变为 $(n-1)f - g$。将每项替换为 $-y_i - 1$，三个结果应变为 $-f - n$、$-g - S_1$、$h + 2f + n$。相邻前缀还应满足增量 $(y_n, n y_n, y_n^2)$，其中新增项以任意精度计算。仅在两个接口的共同范围 $n \le 10^9$ 内，第一结果另与独立接口 `floor_sum` 交叉核对。这些恒等式测试是大参数一致性证据，与前述逐项独立答案对拍区分记录；普通与 ASan/UBSan 均通过。

来源与题目：https://www.luogu.com.cn/problem/P5170。

第四章　多项式与生成函数

4.1　快速傅里叶变换

4.1.1　复数 FFT（可复用单位根）

ComplexFFT(n) 预计算单位根，n 为 1..2^22 的二次幂。transform(a,inverse=false) 原地正变换，正向指数为正；inverse=true 用共轭根并除以 n。a 长度须等于 n；C=complex<long double>。根直接用角度计算，不沿整层连乘。单次变换 O(n log(n+1))，根表 O(n)，可复用。浮点环境须二进制至少 53 位、最近舍入、禁用 fast-math；整数卷积还要求单位根绝对误差 ≤8×2^-53，本机全根已用百位参考核验。变换本身是近似值。

```
struct ComplexFFT
{
    using Real = long double;
    using C = complex<Real>;
    static constexpr int max_size = 1 << 22;
    int n;
    vector<C> roots;

    explicit ComplexFFT(int size) : n(size)
    {
        static_assert(numeric_limits<Real>::radix == 2);
        static_assert(numeric_limits<Real>::digits >= 53);
        assert(n > 0 && n <= max_size && (n & (n - 1)) == 0);
        roots.resize(n / 2);
        Real pi = acosl(-1);
        for (int i = 0; i < n / 2; i++)
        {
            Real angle = pi * (2.0L * i / n);
            roots[i] = C(cosl(angle), sinl(angle));
        }
    }
```

正逆变换（接上页同一结构体）：

```
    void transform(vector<C> &a, bool inverse = false) const
    {
        assert(a.size() == (unsigned)n);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int half = 1; half < n; half *= 2)
        {
            int step = n / (2 * half);
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    C w = roots[j * step];
                    if (inverse) w = conj(w);
                    C u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (inverse)
            for (auto &x : a) x /= n;
    }
};
```

模板题使用：Luogu P3803 【模板】多项式乘法（FFT）—直接变换接口

题意：给出次数 0≤n,m≤1000000、系数均为 0..9 整数的两个多项式，按升幂读入并输出乘积的 n+m+1 项。补零到不小于 n+m+1 的最小二次幂 L（至多 2^21），显式构造 ComplexFFT，分别正变换、逐点相乘、逆变换后 llroundl 取整。保留本节的二进制至少 53 位、最近舍入、单位根误差界及禁用 fast-math 前提；本题范数乘积至多 81000081。时间 O(L log L)，空间 O(L)。这是底层变换接口的完整用法，卷积包装器和精确 CRT 用法另列。

所需模板：ComplexFFT（第 112 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int size = 1;
    while (size < n + m + 1) size *= 2;
    auto fft = ComplexFFT(size);
    vector<ComplexFFT::C> a(size), b(size);
    for (int i = 0, x; i <= n; i++) cin >> x, a[i] = x;
    for (int i = 0, x; i <= m; i++) cin >> x, b[i] = x;
    fft.transform(a);
    fft.transform(b);
    for (int i = 0; i < size; i++) a[i] *= b[i];
    fft.transform(a, true);
    for (int i = 0; i <= n + m; i++) cout << llroundl(a[i].real()) << ' ';
    cout << '\n';
}
```

应用题补充：HDU4609 随机三根组成三角形的概率

题意：T≤100，3≤n≤100000，长度 1..100000。频数平方卷积去掉自选再除 2；正长度使坏三元组有唯一最长根，减去所有“两根和 ≤ 第三根”的计数。范数平方 $\le n^2$，2^18 点下误差界小于 0.001。

所需模板：ComplexFFT（第 112 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4609

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<int> a(n);
        for (int &v : a) cin >> v;
        int m = *max_element(a.begin(), a.end()), size = 1;
        while (size <= 2 * m) size *= 2;
        ComplexFFT fft(size);
        vector<ComplexFFT::C> f(size);
        for (int v : a) f[v] += 1;
        fft.transform(f);
        for (auto &v : f) v *= v;
        fft.transform(f, true);
        vector<long long> count(size);
        for (int i = 0; i < size; i++) count[i] = llroundl(f[i].real());
        for (int v : a) count[2 * v]--;
        for (auto &v : count) v /= 2;
        for (int i = 1; i < size; i++) count[i] += count[i - 1];
        long long all = 1LL * n * (n - 1) * (n - 2) / 6;
        long long good = all;
        for (int v : a) good -= count[v];
        cout << fixed << setprecision(7) << (long double)good / all << '\n';
    }
}
```

4.1.2 浮点 FFT 整数卷积

代码依赖：ComplexFFT（第 112 页，4.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_fft(a,b) 接受 vector<int>，返回 vector<long long>，空输入返回空；非空结果长度 ≤2^22。令 A、B 为两输入最大绝对值，要求 A · B · sqrt(a.size()*b.size())≤2×10^12。依赖 ComplexFFT 及其浮点前提，最坏误差界小于 0.166，llroundl 对正负数正确舍入。超过此界须改用精确整数卷积，不能只看结果是否装入 long long。O(N log N) 时间、O(N) 空间；覆盖 P3803 百万次数/系数 0..9。

```
// IEEE nearest rounding, root error <= 8 * 2^-53; no fast-math.
vector<long long> convolution_fft(const vector<int> &a, const vector<int> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= ComplexFFT::max_size);
    long double ma = 0, mb = 0;
    for (int v : a) ma = max(ma, fabsl((long double)v));
    for (int v : b) mb = max(mb, fabsl((long double)v));
    assert(ma * mb * sqrtl((long double)a.size() * b.size()) <= 2e12L);
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    ComplexFFT fft(n);
    vector<ComplexFFT::C> x(n), y(n);
    for (int i = 0; i < (int)a.size(); i++) x[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) y[i] = b[i];
    fft.transform(x);
    fft.transform(y);
    for (int i = 0; i < n; i++) x[i] *= y[i];
    fft.transform(x, true);
    vector<long long> result(size);
    for (int i = 0; i < size; i++) result[i] = llroundl(x[i].real());
    return result;
}
```

模板题使用：Luogu P3803

题意：输入两个多项式的次数与升幂系数，输出整数乘积。次数最多一百万，系数 0..9，满足本浮点入口的范数限制；输出用 long long。此处实际调用复数 FFT，原精确整数卷积入口另行保留。

所需模板：ComplexFFT（第 112 页，4.1.1 节）、convolution_fft（第 116 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_fft(a, b);
    for (long long x : c) cout << x << ' ';
    cout << '\n';
}
```

应用题补充：HDU1402 十进制高精度乘法

题意：EOF 读两个至多 50000 位整数，支持可选正负号和前导零。逐位卷积后进位，零不带负号。范数乘积 ≤4050000；同 FFT 数值前提下舍入误差小于 0.000001。

所需模板：ComplexFFT（第 112 页，4.1.1 节）、convolution_fft（第 116 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=1402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    while (cin >> s >> t)
    {
        bool negative = (s[0] == '-') != (t[0] == '-');
        int x = s[0] == '+' || s[0] == '-';
        int y = t[0] == '+' || t[0] == '-';
        vector<int> a(s.size() - x), b(t.size() - y);
        for (int i = x; i < int(s.size()); i++) a[s.size() - 1 - i] = s[i] - '0';
        for (int i = y; i < int(t.size()); i++) b[t.size() - 1 - i] = t[i] - '0';
        auto c = convolution_fft(a, b);
        c.push_back(0);
        for (int i = 0; i + 1 < int(c.size()); i++)
        {
            c[i + 1] += c[i] / 10;
            c[i] %= 10;
        }
        while (c.size() > 1 && c.back() == 0) c.pop_back();
        if (negative && c.back() != 0) cout << '-';
        for (int i = int(c.size()) - 1; i >= 0; i--) cout << c[i];
        cout << '\n';
    }
}
```

4.1.3 拆系数 FFT 任意模卷积

代码依赖：ComplexFFT（第 112 页，4.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_mod_fft(a,b,mod) 接受 vector<int>，返回 [0,mod) 系数；1≤mod≤INT_MAX，允许合数与负输入，mod=1 返回零。空输入返回空，非空结果长度 ≤2^19。依赖 ComplexFFT 及其浮点前提；归一化后拆成三个 11 位块，六次正变换和九次逆变换，分别舍入后模合并。每路误差界小于 0.079；不依赖更宽的 long double。时间 O(N log N)，空间 O(N)，常数高于打包版本以保持数值界与代码清楚；覆盖 P4245。

```
// Three 11-bit limbs; same floating-point assumptions as convolution_fft.
vector<int> convolution_mod_fft(const vector<int> &a, const vector<int> &b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 19));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    vector<int> result(size);
    if (mod == 1) return result;
    ComplexFFT fft(n);
    using C = ComplexFFT::C;
    const int base = 1 << 11;
    vector<vector<C>> x(3, vector<C>(n)), y = x;
    for (int i = 0; i < (int)a.size(); i++)
    {
        unsigned v = (a[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) x[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        unsigned v = (b[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) y[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int j = 0; j < 3; j++)
    {
        fft.transform(x[j]);
        fft.transform(y[j]);
    }
    long long weight[5] = {1};
    for (int i = 1; i < 5; i++) weight[i] = weight[i - 1] * base % mod;
    vector<C> z(n);
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < n; k++) z[k] = x[i][k] * y[j][k];
            fft.transform(z, true);
            for (int k = 0; k < size; k++)
            {
                long long v = llroundl(z[k].real()) % mod;
                result[k] = (result[k] + v * weight[i + j]) % mod;
            }
        }
    }
    return result;
}
```

模板题使用：Luogu P4245

题意：输入两个多项式的次数、任意模数与升幂系数，输出模乘积。题目次数不超过十万、模数最多 1000000009；函数会先归一化再拆系数，不要求模数支持 NTT。这里使用三块浮点 FFT，并遵守其单位根与舍入前提。

所需模板：ComplexFFT（第 112 页，4.1.1 节）、convolution_mod_fft（第 118 页，4.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_mod_fft(a, b, mod);
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

相关应用：CDQ 分治配合 NTT 的卷积递推，附件标题为分治 FFT；算法不做浮点变换。

4.1.4 CDQ 分治卷积递推

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

cdq_convolution<mod=998244353,primitive=3>(kernel,initial) 求 f[i]=initial[i]+∑(j<i)f[j]kernel[i-j]，返回前 n 项。两数组为等长 vector<ModInt<mod»，kernel[0] 必须为 0；空输入返回空。模数须为 int 范围素数，primitive 是原根；依赖 NttConvolution。非空时要求 n+floor(n/2)-1≤NTT::max_size，保证每层完整卷积可变换。初始项可任意，并非只有 f[0]=1；左半区解定后，将其到右半区的卷积贡献加上，再递归右半区。O(n $\log^2$(n+1)) 时间、O(n) 额外空间，递归深度 O(log n)。本函数是静态线性递推，不提供流式追加或非线性叶子回调。P4721 使用 initial[0]=1、其他项 0。

```
// f[i] = initial[i] + sum_{j<i} f[j] * kernel[i-j]; kernel[0] must be zero.
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> cdq_convolution(const vector<ModInt<mod>> &kernel,
                                    vector<ModInt<mod>> initial)
{
    using Ntt = NttConvolution<mod, primitive>;
    using Poly = typename Ntt::Poly;
    assert(kernel.size() == initial.size());
    if (initial.empty()) return {};
    assert(kernel[0].v == 0);
    assert(initial.size() + initial.size() / 2 - 1 <= Ntt::max_size);
    int n = initial.size();
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l == 1) return;
        int m = (l + r) / 2;
        self(self, l, m);
        Poly left(initial.begin() + l, initial.begin() + m);
        Poly right(kernel.begin(), kernel.begin() + r - l);
        Poly product = Ntt::multiply(move(left), move(right));
        for (int i = m; i < r; i++) initial[i] += product[i - l];
        self(self, m, r);
    };
    solve(solve, 0, n);
    return initial;
}
```

模板题使用：Luogu P4721 分治 FFT

题意：输入 g[1..n-1]，计算 f[0]=1、f[i]=∑(j=1..i)f[i-j]g[j]，模 998244353。kernel[0] 留零，initial 仅第零项为 1；调用的是真正 CDQ 分治卷积，依赖 NTT 与模整数。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、cdq_convolution（第 120 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4721

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Z = ModInt<998244353>;
    vector<Z> g(n), b(n);
    for (int i = 1; i < n; i++)
    {
        int x;
        cin >> x;
        g[i] = x;
    }
    b[0] = 1;
    auto f = cdq_convolution(g, b);
    for (int i = 0; i < n; i++) cout << f[i].v << " \n"[i + 1 == n];
}
```

4.2 快速数论变换

4.2.1 可指定素数与原根的 NTT 卷积

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

声明 NttConvolution<mod, primitive>，primitive 默认 3。mod 必须为 int 范围内的素数，primitive 是模 mod 原根；不在接口内验证素性或阶。max_size 是 mod-1 的最大二次幂因子。ntt(a,invert) 原地变换，长度须为正二次幂且不超过 max_size；invert=true 为逆变换。multiply(a,b) 返回普通卷积，任一为空则为空；非空时长度和减一不超过 max_size。元素类型 Z，数组类型 Poly。O(n log n) 时间；每种 mod/primitive 实例缓存分层单位根，空间 O(历次最大变换长度)，同一实例调用时不并发扩容。输入输出均为自然顺序，逆变换复用正变换后翻转下标 1..n-1 并乘 n 的逆元。998244353 和 167772161 均可用原根 3。

```
template <int mod, int primitive = 3> struct NttConvolution
{
    static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
    static constexpr int max_size = (mod - 1) & -(mod - 1);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    // mod is prime, primitive is a primitive root modulo mod.
    static void ntt(Poly &a, bool invert = false)
    {
        int n = a.size();
        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
        static Poly roots{0, 1};
        for (int half = roots.size(); half < n; half *= 2)
        {
            Z step = Z(primitive).pow((mod - 1) / (2 * half));
            roots.resize(2 * half);
            for (int i = half / 2; i < half; i++)
            {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * step;
            }
        }
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
            if (i < j)
                swap(a[i], a[j]);
        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```
        for (int half = 1; half < n; half *= 2)
        {
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    Z u = a[i + j];
                    Z v = a[i + j + half] * roots[half + j];
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (invert)
        {
            reverse(a.begin() + 1, a.end());
            Z inverse = Z(n).inv();
            for (auto &x : a)
                x *= inverse;
        }
    }

    static Poly multiply(Poly a, Poly b)
    {
        if (a.empty() || b.empty())
            return {};
        assert(a.size() + b.size() - 1 <= max_size);
        int size = a.size() + b.size() - 1, n = 1;
        while (n < size)
            n *= 2;
        a.resize(n);
        b.resize(n);
        ntt(a);
        ntt(b);
        for (int i = 0; i < n; i++)
            a[i] *= b[i];
        ntt(a, true);
        a.resize(size);
        return a;
    }
};
```

模板题使用：Library Checker convolution_mod

题意：求两个多项式的卷积，模 998244353。输入为各自系数个数和升幂系数；输出 n+m-1 个系数。题目保证输入系数已经取模。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using C = NttConvolution<998244353>;
    int n, m;
    cin >> n >> m;
    C::Poly a(n), b(m);
    for (auto &x : a) cin >> x.v;
    for (auto &x : b) cin >> x.v;
    auto c = C::multiply(move(a), move(b));
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i].v << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.2 精确有符号整数卷积

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_i64(a,b) 返回 signed64 整数卷积，空输入返回空。非空结果长度 ≤2^24，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 cpp_int 参考验证；P3803 最大次数驱动本地通过，线上性能待验证。

```
template <int p>
vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> r(c.size());
    for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
    return r;
}

vector<long long> convolution_i64(const vector<long long> &a,
                                  const vector<long long> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    const __int128 mod = (__int128)pq * r;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_residue<p>(a, b);
    auto y = convolution_residue<q>(a, b);
    auto z = convolution_residue<r>(a, b);
    vector<long long> c(x.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        if (value > mod / 2) value -= mod;
        assert(LLONG_MIN <= value && value <= LLONG_MAX);
        c[i] = (long long)value;
    }
    return c;
}
```

模板题使用：Luogu P3803

题意：求两个整数多项式的乘积，系数按升幂输入。首行 n、m 是次数，系数个数分别为 n+1、m+1；输出精确整数，不能取模。本题系数在 0..9，满足 signed64 结果范围。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、convolution_i64（第 125 页，4.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), b(m + 1);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto c = convolution_i64(a, b);
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i] << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.3 二维 NTT 与二元多项式卷积

代码依赖：NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Ntt2D 的模板参数为 mod=998244353、primitive=3；Matrix 为 vector<vector<Z»，a[i][j] 表示 x^i y^j 系数。transform(a,invert=false) 先逐行再逐列原地变换，要求非空矩形、两维均为不超过 max_size 的正二次幂，模数与原根前提同 NttConvolution。multiply(a,b) 返回普通二元卷积；输入须为矩形，任一维为空返回空 Matrix，否则结果两维分别为行数和减一、列数和减一，均须 ≤max_size。两维独立补至二次幂 H、W，时间 O(HW(log H+log W))，工作空间 O(HW)，另有 NTT 历史单位根缓存；二维面积须能分配内存，不能只看单维上限。这是二维卷积，不是矩阵乘法。

```
template <int mod = 998244353, int primitive = 3> struct Ntt2D
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Matrix = vector<vector<Z>>;
    static constexpr int max_size = N::max_size;

    static void transform(Matrix &a, bool invert = false)
    {
        assert(!a.empty() && !a[0].empty());
        int n = a.size(), m = a[0].size();
        assert(n <= max_size && (n & (n - 1)) == 0);
        assert(m <= max_size && (m & (m - 1)) == 0);
        for (auto &row : a)
        {
            assert((int)row.size() == m);
            N::ntt(row, invert);
        }
        vector<Z> column(n);
        for (int j = 0; j < m; j++)
        {
            for (int i = 0; i < n; i++)
                column[i] = a[i][j];
            N::ntt(column, invert);
            for (int i = 0; i < n; i++)
                a[i][j] = column[i];
        }
    }
```

二维普通卷积（接上页同一结构体）：

```
    static Matrix multiply(Matrix a, Matrix b)
    {
        if (a.empty() || b.empty() || a[0].empty() || b[0].empty())
            return {};
        int h = a[0].size(), w = b[0].size();
        for (const auto &row : a)
            assert((int)row.size() == h);
        for (const auto &row : b)
            assert((int)row.size() == w);
        assert(a.size() + b.size() - 1 <= max_size);
        assert((long long)h + w - 1 <= max_size);
        int n = a.size() + b.size() - 1, m = h + w - 1;
        int x = 1, y = 1;
        while (x < n)
            x *= 2;
        while (y < m)
            y *= 2;
        a.resize(x);
        b.resize(x);
        for (auto &row : a)
            row.resize(y);
        for (auto &row : b)
            row.resize(y);
        transform(a);
        transform(b);
        for (int i = 0; i < x; i++)
            for (int j = 0; j < y; j++)
                a[i][j] *= b[i][j];
        transform(a, true);
        a.resize(n);
        for (auto &row : a)
            row.resize(m);
        return a;
    }
};
```

接口用法演示：二维卷积完整输入输出

演示约定：输入 n m h w 及 n×m、h×w 两个系数矩形，按行读取，允许有符号系数。输出结果行列数，再逐行输出普通二元卷积；空维度输出 0 0。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、Ntt2D（第 127 页，4.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIVARIATE.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, h, w;
    cin >> n >> m >> h >> w;
    using F = Ntt2D<>;
    F::Matrix a(n, vector<F::Z>(m)), b(h, vector<F::Z>(w));
    for (auto *matrix : {&a, &b})
        for (auto &row : *matrix)
            for (auto &x : row)
            {
                long long value;
                cin >> value;
                x = value;
            }
    auto c = F::multiply(a, b);
    cout << c.size() << ' ' << (c.empty() ? 0 : c[0].size()) << '\n';
    for (const auto &row : c)
    {
        for (auto x : row)
            cout << x.v << ' ';
        cout << '\n';
    }
}
```

4.2.4 三模 NTT：精确任意模数卷积

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

convolution_mod(a,b,mod) 接受 vector<int>，1≤mod≤INT_MAX，支持负输入、合数和模数 1。返回 [0,mod) 普通卷积；任一输入为空则返回空。非空结果长度 ≤2^24；先将输入归一化，再用三个固定 NTT 素数重构非负整数系数，int128 中间值后取目标模数，无浮点误差及 signed64 真实系数限制。直接取非负 CRT 代表，不经 signed64 窄化。O(L log L) 时间、O(L) 空间，L 为补齐二次幂；长度上限不是内存或时限承诺。参数按值传递，普通调用不改原数组，愿意移交数据时可 move。依赖 NttConvolution 及 ModInt，convolution_prime 为内部三模辅助。

```
template <int p>
vector<int> convolution_prime(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> result(c.size());
    for (int i = 0; i < (int)c.size(); i++)
        result[i] = c[i].v;
    return result;
}

vector<int> convolution_mod(vector<int> a, vector<int> b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty())
        return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    int n = a.size() + b.size() - 1;
    if (mod == 1)
        return vector<int>(n);
    for (int &v : a)
    {
        v %= mod;
        if (v < 0)
            v += mod;
    }
    for (int &v : b)
    {
        v %= mod;
        if (v < 0)
            v += mod;
    }
```

三模卷积与固定逆元 CRT 重构（接上页同一函数）：

```
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_prime<p>(a, b);
    auto y = convolution_prime<q>(a, b);
    auto z = convolution_prime<r>(a, b);
    for (int i = 0; i < n; i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        x[i] = value % mod;
    }
    return x;
}
```

模板题使用：Luogu P4245 任意模数多项式乘法（三模 NTT）

题意：输入两多项式次数 n,m≤100000 和模数 2≤p≤1000000009，系数 0..10^9，输出 n+m+1 个模 p 系数。独立精确三模版本，不继承浮点版的提交记录。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、convolution_mod（第 130 页，4.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    auto c = convolution_mod(a, b, mod);
    for (int x : c)
        cout << x << ' ';
    cout << '\n';
    return 0;
}
```

接口用法演示：精确模卷积：空数组、负数和合数模

演示约定：多组直到 EOF，输入两个数组长度 n,m 和 1≤mod≤INT_MAX，再输入 signed int 系数。每组输出标准模数代表；任一数组为空输出空行，mod=1 输出对应数量的 0。非空结果长度 ≤2^24，实际还须满足内存限制。API 扩展不属于 P4245 正式输入范围。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、convolution_mod（第 130 页，4.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    while (cin >> n >> m >> mod)
    {
        vector<int> a(n), b(m);
        for (int &x : a)
            cin >> x;
        for (int &x : b)
            cin >> x;
        auto c = convolution_mod(a, b, mod);
        for (int x : c)
            cout << x << ' ';
        cout << '\n';
    }
    return 0;
}
```

应用题补充：HDU4656 Evaluation：移位等比点求值

题意：官方仅一组，n 为 1..100000；b、c、d 为 1..1000000，n 个升幂系数为 0..1000000。按 k=0..n−1 输出 F(b * c^(2k) + d)，结果模 1000003。两次精确卷积，O(n log n) 时间、O(n) 空间；3 秒、65536K，当前最小抄写版已 AC。代码兼容 EOF 多组、signed int 参数及 c=0，扩展边界只由本地测试验证。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、convolution_mod（第 130 页，4.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4656

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    const int mod = 1000003;
    using Z = ModInt<mod>;
    int n, b, c, d;
    while (cin >> n >> b >> c >> d)
    {
        assert(1 <= n && n <= 100010);
        vector<Z> a(n);
        for (Z &v : a)
        {
            int x;
            cin >> x;
            v = x;
        }
        Z base = b, ratio = c, shift = d;
        if (ratio.v == 0)
        {
            Z first = 0, rest = 0;
            for (int i = n - 1; i >= 0; i--)
            {
                first = first * (base + shift) + a[i];
                rest = rest * shift + a[i];
            }
            cout << first.v << '\n';
            for (int i = 1; i < n; i++)
                cout << rest.v << '\n';
            continue;
        }
```

继续处理操作（接上页同一 main）：

```
        vector<Z> fact(n, 1), inv(n, 1);
        for (int i = 1; i < n; i++)
            fact[i] = fact[i - 1] * Z(i);
        inv[n - 1] = fact[n - 1].inv();
        for (int i = n - 1; i > 0; i--)
            inv[i - 1] = inv[i] * Z(i);
        vector<int> x(n), y(n);
        Z power = 1;
        for (int i = 0; i < n; i++)
        {
            x[n - 1 - i] = (a[i] * fact[i]).v;
            y[i] = (power * inv[i]).v;
            power = power * shift;
        }
        auto product = convolution_mod(x, y, mod);
        power = 1;
        for (int j = 0; j < n; j++)
        {
            a[j] = Z(product[n - 1 - j]) * inv[j] * power;
            power = power * base;
        }
        vector<Z> square(n, 1), inverse(n, 1);
        Z step = ratio, backward = ratio.inv();
        Z step2 = step * step, backward2 = backward * backward;
        for (int i = 1; i < n; i++)
        {
            square[i] = square[i - 1] * step;
            inverse[i] = inverse[i - 1] * backward;
            step = step * step2;
            backward = backward * backward2;
        }
        y.resize(2 * n - 1);
        for (int j = 0; j < n; j++)
        {
            x[j] = (a[j] * square[j]).v;
            y[n - 1 - j] = inverse[j].v;
            y[n - 1 + j] = inverse[j].v;
        }
        product = convolution_mod(x, y, mod);
        for (int k = 0; k < n; k++)
            cout << (square[k] * Z(product[n - 1 + k])).v << '\n';
    }
    return 0;
}
```

4.3 快速沃尔什变换

4.3.1 按位 OR、AND、XOR 卷积

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

SetConvolution<mod=998244353>；Z 为 ModInt<mod>，Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 ≤INT_MAX，表示固定二进制位数的全集，长度 1 合法。transform(a,op,invert=false) 原地变换，op 只能为 |、&、^；| 为子集和，& 为超集和，^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum$ a[i]b[j]（满足 i op j=s），a、b 长度必须相同。mod 可为 ≥2 的 int 范围合数；XOR 要求 mod 为奇数，逆 2 用 (mod+1)/2，不使用费马逆元。O(N log N) 时间，原地变换 O(1) 额外空间，卷积 O(N) 空间。

```
template <int mod = 998244353> struct SetConvolution
{
    static_assert(mod >= 2);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    static void transform(Poly &a, char op, bool invert = false)
    {
        assert(!a.empty() && a.size() <= INT_MAX);
        int n = a.size();
        assert((n & (n - 1)) == 0);
        assert(op == '|' || op == '&' || op == '^');
        assert(op != '^' || mod % 2 == 1);
        Z half = (mod + 1LL) / 2;
        for (int len = 1; len < n; len *= 2)
            for (int i = 0; i < n; i += len * 2)
                for (int j = 0; j < len; j++)
                {
                    Z &u = a[i + j], &v = a[i + j + len];
                    if (op == '|')
                        v = invert ? v - u : v + u;
                    else if (op == '&')
                        u = invert ? u - v : u + v;
                    else
                    {
                        Z x = u, y = v;
                        u = x + y;
                        v = x - y;
                        if (invert)
                        {
                            u = u * half;
                            v = v * half;
                        }
                    }
                }
    }
```

点乘并逆变换（接上页同一结构体）：

```
    static Poly multiply(Poly a, Poly b, char op)
    {
        assert(a.size() == b.size());
        transform(a, op);
        transform(b, op);
        for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
        transform(a, op, true);
        return a;
    }
};
```

模板题使用：Luogu P4717

题意：给两个长度为 2^m 的数组，分别求按下标 OR、AND、XOR 合并的卷积，模 998244353，按此顺序输出三行。multiply 的最后参数选择运算。

所需模板：ModInt（第 24 页，3.2.2 节）、SetConvolution（第 136 页，4.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4717

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<998244353>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (char op : string("|&^"))
    {
        auto c = F::multiply(a, b, op);
        for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
    }
}
```

4.3.2 不相交子集卷积

代码依赖：ModInt（第 24 页，3.2.2 节）、SetConvolution（第 136 页，4.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

subset_convolution<mod=998244353>(a,b) 返回 c[s]=$\sum$ a[t]b[s xor t]，求和枚举 t⊆s，等价于 i|j=s 且 i&j=0。依赖 SetConvolution，只使用 OR 变换；mod 为 ≥2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空，N=2^m≤INT_MAX；长度 1 的结果为 a[0]b[0]。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。O(m^{2}2^m) 时间、O(m2^m) 空间；两组分层数组约占 8(m+1)2^m 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```
// c[s] = sum_{t subset s} a[t] * b[s xor t].
template <int mod = 998244353>
vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
                                       const vector<ModInt<mod>> &b)
{
    using F = SetConvolution<mod>;
    using Poly = typename F::Poly;
    assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
    int n = a.size(), m = 0;
    assert((n & (n - 1)) == 0);
    while ((1 << m) < n) m++;
    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
    vector<int> rank(n);
    for (int s = 0; s < n; s++)
    {
        rank[s] = __builtin_popcount(unsigned(s));
        f[rank[s]][s] = a[s];
        g[rank[s]][s] = b[s];
    }
    for (int k = 0; k <= m; k++)
    {
        F::transform(f[k], '|');
        F::transform(g[k], '|');
    }
    Poly answer(n), h(n);
    for (int k = 0; k <= m; k++)
    {
        fill(h.begin(), h.end(), 0);
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
        F::transform(h, '|', true);
        for (int s = 0; s < n; s++)
            if (rank[s] == k) answer[s] = h[s];
    }
    return answer;
}
```

模板题使用：Luogu P6097

题意：把下标看作集合，求 c[s]=∑ a[t]*b[s⁀]，t 遍历 s 的子集，两部分不相交。输入首项为集合大小 m，数组长度 2^m；本题模数为 1000000009。

所需模板：ModInt（第 24 页，3.2.2 节）、SetConvolution（第 136 页，4.3.1 节）、subset_convolution（第 138 页，4.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6097

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<1000000009>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto c = subset_convolution<1000000009>(a, b);
    for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
}
```

4.4 Chirp Z 变换

4.4.1 Chirp Z：等比点求值

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

chirp_z(f,a,r,m) 返回 f(a*r^i)，i=0..m-1；f 为升幂系数数组，a、r 为 ModInt<mod>。m≥0、f.size()≤INT_MAX；空 f 表示零多项式，m=0 返回空。约定 r^0=1，因此 r=0 时首项为 f(a)、其余为 f(0)；a=0 或 r=1 也单独处理。普通分支依赖 NTT 素数及原根，非空长度 n 要求 n+m-1≤NTT::max_size。利用三角数指数与循环卷积，避免平方指数的整数溢出；O((n+m) log(n+m)) 时间、O(n+m) 空间，特殊分支 O(n+m)。默认模板参数 mod=998244353、primitive=3。输出顺序允许重复点。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>>
chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    assert(m >= 0 && f.size() <= INT_MAX);
    if (m == 0 || f.empty()) return Poly(m);
    if (a.v == 0 || r.v == 0 || r.v == 1)
    {
        Poly answer(m, f[0]);
        Z value = 0;
        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
        answer[0] = value;
        if (r.v == 1) fill(answer.begin(), answer.end(), value);
        return answer;
    }
    assert(f.size() + m - 1 <= N::max_size);
    int n = f.size(), size = 1;
    while (size < n + m - 1) size *= 2;
    Poly x(size), y(size), inverse(max(n, m));
    Z power = 1, triangle = 1, ri = r.inv();
    for (auto &v : inverse)
    {
        v = triangle;
        triangle = triangle * power;
        power = power * ri;
    }
```

构造循环卷积并取点值（接上页同一函数）：

```
    power = 1;
    for (int j = 0; j < n; j++)
    {
        x[n - 1 - j] = f[j] * power * inverse[j];
        power = power * a;
    }
    power = 1;
    triangle = 1;
    for (int j = 0; j < n + m - 1; j++)
    {
        y[j] = triangle;
        triangle = triangle * power;
        power = power * r;
    }
    N::ntt(x);
    N::ntt(y);
    for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
    N::ntt(x, true);
    Poly answer(m);
    for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
    return answer;
}
```

模板题使用：Library Checker multipoint_evaluation_on_geometric_sequence

题意：给多项式 f 的升幂系数及 a、r，依次输出 f(a*r^i)，i=0..m-1，模 998244353。r=0 时首项仍为 f(a)，后续各项为 f(0)。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、chirp_z（第 140 页，4.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation_on_geometric_sequence

```
int main()
{
    int n, m, a, r;
    scanf("%d%d%d%d", &n, &m, &a, &r);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = chirp_z(f, Z(a), Z(r), m);
    for (int i = 0; i < m; i++) printf("%d%c", answer[i].v, i + 1 == m ? '\n' : ' ');
}
```

4.5 多项式牛顿迭代

4.5.1 形式幂级数求逆

代码依赖：NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsInverse::inverse(a,n) 返回 a 的逆级数前 n 项，模数固定 998244353。要求 $1 \le n \le 2^{23}$、a 非空且 a[0]≠0；a.size() 在 int 范围内，不足项视为零，多余项只取需要前缀。继承 NttConvolution<998244353> 的 Poly、Z、multiply。Newton 公式 b←b(2-ab) 只更新新增高半段：长度 2k 循环卷积的回绕只污染低 k 项，清除低半段后再乘 b，高半段即修正项。每轮五次长度 2k 变换，复用 b 的频域值。O(n log n) 时间、O(n) 工作空间；单位根缓存另按 NTT 约定。

```
struct FpsInverse : NttConvolution<998244353>
{
    static constexpr int mod = 998244353;

    static Poly inverse(const Poly &a, int n)
    {
        assert(n > 0 && n <= max_size && !a.empty() && a[0].v);
        Poly b{a[0].inv()};
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 2 * k;
            int m = min(n, len);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            // Only the high half of a*b is free of cyclic wraparound.
            fill(f.begin(), f.begin() + k, Z(0));
            ntt(f);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i];
        }
        return b;
    }
};
```

模板题使用：Library Checker inv_of_formal_power_series

题意：给出常数项非零的形式幂级数，求其乘法逆的前 n 项，模 998244353。系数按升幂输入，题目保证已经取模。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsInverse;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::inverse(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.5.2 多项式带余除法

代码依赖：FpsInverse（第 142 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

PolynomialDivision::divide(a,b) 返回 pair<Poly,Poly>{商, 余式}，模数固定 998244353，系数从低次到高次。输入按值传入；先删除高次端零系数，零多项式统一返回空 vector；b 不可为零。支持零被除式、常数除数及 deg(a)<deg(b)。返回 a=q*b+r，deg(r)<deg(b)，商和余式都去掉高次零。依赖 FpsInverse 与 NttConvolution；反转后求逆得到商，再计算余式。输入各长度 $\le 2^{22}$；令 N=max(a.size(),b.size())，时间 O(N log(N+1))，额外空间 O(N)。trim(a) 是可直接使用的去高次零工具。P4512 的固定长度输出需在使用处 resize 补零。

```
struct PolynomialDivision : FpsInverse
{
    static void trim(Poly &a)
    {
        while (!a.empty() && a.back().v == 0) a.pop_back();
    }

    static pair<Poly, Poly> divide(Poly a, Poly b)
    {
        assert(a.size() <= max_size / 2 && b.size() <= max_size / 2);
        trim(a);
        trim(b);
        assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int k = a.size() - b.size() + 1;
        Poly ar(a.rbegin(), a.rbegin() + k);
        Poly br(b.rbegin(), b.rend());
        br.resize(min(k, (int)br.size()));
```

```
        auto q = multiply(ar, inverse(br, k));
        q.resize(k);
        reverse(q.begin(), q.end());
        auto product = multiply(q, b);
        for (int i = 0; i < (int)a.size(); i++) a[i] -= product[i];
        a.resize(b.size() - 1);
        trim(a);
        return {q, a};
    }
};
```

模板题使用：Luogu P4512 多项式除法

题意：输入两个多项式的次数与系数，输出商的 n-m+1 项和余式的 m 项；模板返回的商、余式均已去高次零，打印前按题目要求补零。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、PolynomialDivision（第 143 页，4.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4512

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n + 1), b(m + 1);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    q.resize(n - m + 1);
    r.resize(m);
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

模板题使用：Library Checker Division of Polynomials

题意：输入系数个数 N、M；输出去高次零后的商和余式长度，再输出两行系数。零商或零余式长度为 0，对应空行；允许被除式次数更小。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、PolynomialDivision（第 143 页，4.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/division_of_polynomials

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n), b(m);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    cout << q.size() << ' ' << r.size() << '\n';
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

4.5.3 二元形式幂级数求逆

代码依赖：FpsInverse（第 142 页，4.5.1 节）、Ntt2D（第 127 页，4.2.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

BivariateInverse::inverse(a,n,m) 返回模 x^n、y^m 的恰好 n×m 项逆级数，模数固定 998244353。n,m>0 且 a[0][0] 非零；输入可为不等长行，不足补零，超出目标矩形的系数忽略，各输入尺寸须在 int 范围内。n=1 时 $m \le 2^{23}$；n>1 时 $n \le 2^{23}$ 且 $2m-1 \le 2^{23}$。先用 FpsInverse 求第 0 行，再按 x 方向 Newton 倍增 b←b(2-ab)，每轮只更新 [k,min(n,2k)) 的新行。变换高 2k、宽为不小于 2m-1 的最小二次幂；a*b 的 x 方向回绕仅影响旧低半段，保留高半段并截断 y，再乘 b，取负得到新行。复用 b 的频域值，共五次二维变换。O(nm(log(n+1)+log(m+1))) 时间、O(nm) 工作空间，实际矩形内存远早于单维上限成为限制；第 0 行逆元同样唯一。依赖 Ntt2D、FpsInverse；不是普通方阵逆，不要求 n=m。

```
struct BivariateInverse : Ntt2D<998244353>
{
    static Matrix inverse(const Matrix &a, int n, int m)
    {
        assert(n > 0 && m > 0 && m <= max_size);
        assert(!a.empty() && !a[0].empty() && a[0][0].v != 0);
        auto row = a[0];
        if ((int)row.size() > m)
            row.resize(m);
        Matrix b{FpsInverse::inverse(row, m)};
        if (n == 1)
            return b;
        assert(n <= max_size && 2LL * m - 1 <= max_size);
        int width = 1;
        while (width < 2LL * m - 1)
            width *= 2;
        while ((int)b.size() < n)
        {
            int k = b.size(), height = 2 * k;
            int next = min(n, height);
            Matrix f(height, vector<Z>(width));
            for (int i = 0; i < min(next, (int)a.size()); i++)
                for (int j = 0; j < min(m, (int)a[i].size()); j++)
                    f[i][j] = a[i][j];
            Matrix g = b;
            g.resize(height);
            for (auto &v : g)
                v.resize(width);
            transform(f);
            transform(g);
            for (int i = 0; i < height; i++)
                for (int j = 0; j < width; j++)
                    f[i][j] *= g[i][j];
            transform(f, true);
```

截断误差并更新新行（接上页同一结构体、函数与循环）：

```
            // Keep only the high x-half, reduced modulo y^m.
            for (int i = 0; i < height; i++)
                fill(f[i].begin() + (i < k ? 0 : m), f[i].end(), Z(0));
            transform(f);
            for (int i = 0; i < height; i++)
                for (int j = 0; j < width; j++)
                    f[i][j] *= g[i][j];
            transform(f, true);
            b.resize(next);
            for (int i = k; i < next; i++)
            {
                b[i].resize(m);
                for (int j = 0; j < m; j++)
                    b[i][j] = Z(0) - f[i][j];
            }
        }
        return b;
    }
};
```

接口用法演示：二元逆级数完整输入输出

演示约定：输入目标精度 n m、输入尺寸 h w 及 h×w 系数，按行读取。常数项为 0 或输入为空输出-1，否则输出 n×m 项二元逆级数；尺寸须满足模板契约，超出目标的输入系数不参与。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、Ntt2D（第 127 页，4.2.3 节）、FpsInverse（第 142 页，4.5.1 节）、BivariateInverse（第 146 页，4.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/BIVARIATE.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, h, w;
    cin >> n >> m >> h >> w;
    BivariateInverse::Matrix a(h, vector<BivariateInverse::Z>(w));
    for (auto &row : a)
        for (auto &x : row)
        {
            long long value;
            cin >> value;
            x = value;
        }
    if (a.empty() || a[0].empty() || a[0][0].v == 0)
    {
        cout << -1 << '\n';
        return 0;
    }
    auto b = BivariateInverse::inverse(a, n, m);
    for (const auto &row : b)
    {
        for (auto x : row)
            cout << x.v << ' ';
        cout << '\n';
    }
}
```

模板题使用：Library Checker Inv of Bivariate Formal Power Series

题意：官方模板题：输入 N M 及 N×M 系数，NM≤500000、常数项非零，模 998244353。输出相同矩形精度的二元逆级数，逐行输出；输入形状既可极窄也可接近正方形。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、Ntt2D（第 127 页，4.2.3 节）、FpsInverse（第 142 页，4.5.1 节）、BivariateInverse（第 146 页，4.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series_2d

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BivariateInverse::Matrix a(n, vector<BivariateInverse::Z>(m));
    for (auto &row : a)
        for (auto &x : row)
        {
            int value;
            cin >> value;
            x = value;
        }
    auto b = BivariateInverse::inverse(a, n, m);
    for (const auto &row : b)
    {
        for (int j = 0; j < m; j++)
            cout << row[j].v << " \n"[j + 1 == m];
    }
}
```

4.6 多项式多点求值 | 快速插值

4.6.1 任意点多项式多点求值

代码依赖：PolynomialDivision（第 143 页，4.5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

固定模数 998244353，Poly 为升幂系数 vector<Z>。MultipointEvaluation(x) 建立点集乘积树，evaluate(f) 按输入点顺序返回 f(x[i])；允许重复点、零点、空点集、空或尾零多项式，返回长度始终等于点数。构造复制点值，查询按值接收 f，不改变树或调用者的左值，可多次复用；不再使用 f 时可 move 传入。系数个数 n≤2^22，点数 m<2^22，且内存可用；本地最大规模证据按报告列出，不表示已实测此绝对上限。每结点存首一多项式，查询将余式下传，叶值就是答案；仅求值，不包含快速插值。建树 O(m $\log^2$(m+1))，查询 O(n log(n+1)+m $\log^2$(m+1))，常驻空间 O(m log(m+1))、含临时数组 O(n+m log(m+1))，递归深度 O(log(m+1))。依赖 PolynomialDivision/FpsInverse/NTT 及模整数。

```
struct MultipointEvaluation : PolynomialDivision
{
    int n;
    vector<Poly> p;

    MultipointEvaluation(const Poly &x)
    {
        assert(x.size() < max_size / 2);
        n = x.size();
        p.resize(4 * max(1, n));
        if (n) build(1, 0, n, x);
    }

    void build(int u, int l, int r, const Poly &x)
    {
        if (r - l == 1)
        {
            p[u] = {Z(0) - x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(2 * u, l, m, x);
        build(2 * u + 1, m, r, x);
        p[u] = multiply(p[2 * u], p[2 * u + 1]);
    }
```

余式下传与求值入口（接上页同一结构体）：

```
    void eval(int u, int l, int r, Poly f, Poly &a) const
    {
        if (f.size() >= p[u].size()) f = divide(move(f), p[u]).second;
        if (f.empty()) return;
        if (f.size() == 1)
        {
            fill(a.begin() + l, a.begin() + r, f[0]);
            return;
        }
        int m = (l + r) / 2;
        eval(2 * u, l, m, f, a);
        eval(2 * u + 1, m, r, move(f), a);
    }

    Poly evaluate(Poly f) const
    {
        assert(f.size() <= max_size / 2);
        trim(f);
        Poly a(n);
        if (n) eval(1, 0, n, move(f), a);
        return a;
    }
};
```

模板题使用：Library Checker Multipoint Evaluation

题意：给出 n 项升幂系数和 m 个任意横坐标，按输入顺序输出全部 f(x[i])，模 998244353。1≤n,m≤131072，系数和点均已取模，最高次系数非零；横坐标可以重复。构造一次乘积树后调用 evaluate；同一树可以继续求其他多项式的点值。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、PolynomialDivision（第 143 页，4.5.2 节）、MultipointEvaluation（第 150 页，4.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = MultipointEvaluation;
    P::Poly f(n), x(m);
    for (auto &a : f)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto a = P(x).evaluate(f);
    for (int i = 0; i < m; i++) cout << a[i].v << (i + 1 == m ? '\n' : ' ');
}
```

4.6.2 任意点多项式快速插值

代码依赖：MultipointEvaluation（第 150 页，4.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

polynomial_interpolation(x,y) 返回模 998244353 下次数 <m 的唯一插值多项式，升幂 vector 恰好 m 项并保留高次零。optional 为空表示长度不符或归一化横坐标重复；两个空数组返回有值的空 vector，不改输入。复用 MultipointEvaluation 乘积树及导数求值，按 y/P'(x) 递归合并；重复点只在独立求值接口合法。$m \le 2^{22}-1$ 是既有除法/NTT 的统一充分界，官方最大实测 131072。时间 $O(m \log^2(m+1)+m \log p)$，p=998244353；单点逆元沿用模幂，不省略 m log p 项。总空间 O(m log(m+1)+1)，递归 O(log(m+1))。

```
optional<MultipointEvaluation::Poly>
polynomial_interpolation(const MultipointEvaluation::Poly &x,
                         const MultipointEvaluation::Poly &y)
{
    using P = MultipointEvaluation;
    using Poly = P::Poly;
    if (x.size() != y.size()) return nullopt;
    assert(x.size() < P::max_size / 2);
    int n = x.size();
    if (!n) return Poly{};
    const P tree(x);
    Poly d(n);
    for (int i = 1; i <= n; i++) d[i - 1] = tree.p[1][i] * i;
    auto w = tree.evaluate(move(d));
    for (auto v : w)
        if (!v.v) return nullopt;
    for (int i = 0; i < n; i++) w[i] = y[i] * w[i].inv();
    auto combine = [&](auto &&self, int u, int l, int r) -> Poly
    {
        if (r - l == 1) return {w[l]};
        int m = (l + r) / 2;
        auto a = self(self, 2 * u, l, m);
        auto b = self(self, 2 * u + 1, m, r);
        a = P::multiply(move(a), tree.p[2 * u + 1]);
        b = P::multiply(move(b), tree.p[2 * u]);
        for (int i = 0; i < r - l; i++) a[i] += b[i];
        return a;
    };
    return combine(combine, 1, 0, n);
}
```

模板题使用：Library Checker Polynomial Interpolation

题意：给出 n 个互异横坐标及对应点值，输出唯一的次数小于 n 的多项式的 n 项升幂系数，模 998244353。1≤n≤131072，坐标与点值均已取模；先以乘积根多项式的导数求权重，再递归合并。接口返回 optional，长度不符或模意义重复横坐标返回空 optional，空输入返回有值的空向量；本题保证输入合法。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、PolynomialDivision（第 143 页，4.5.2 节）、MultipointEvaluation（第 150 页，4.6.1 节）、polynomial_interpolation（第 153 页，4.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_interpolation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Poly = MultipointEvaluation::Poly;
    Poly x(n), y(n);
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : y)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto f = polynomial_interpolation(x, y);
    assert(f);
    for (int i = 0; i < n; i++) cout << (*f)[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7 多项式初等函数

4.7.1 形式幂级数开根（含前导零与无解）

代码依赖：FpsInverse（第 142 页，4.5.1 节）、mod_sqrt（第 80 页，3.15.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsSqrt::sqrt(a,n) 返回 optional<Poly>，模 998244353；无解为 nullopt，有解恰好 n 项。$0\le n\le 2^{22}$、a.size() 在 int 范围内，仅看前 n 项，不足补零；n=0 返回有值空 vector。全零返回全零，首个非零项次数 k 为奇数或系数非二次剩余时无解。否则提出 x^k，首项选较小模平方根，末尾未确定的 k/2 项取零。inverse_sqrt(a,n) 直接求所选根的逆：n>0 时要求常数项非零且为二次剩余，否则 nullopt；n=0 仍返回空。按 $b\leftarrow(3b-ab^3)/2$ 倍增，长度 4k 的循环卷积只回绕到低 k 项，保留旧低半段、更新高半段；sqrt 再乘原级数并移位。不枚举全部根。依赖 mod_sqrt、FpsInverse、NTT；O(n log(n+1)) 时间加一次标量开根的期望 O(log mod)，O(n) 工作空间，单位根缓存另按 NTT 约定。P5205 的 a[0]=1 得到首项 1。

```
struct FpsSqrt : FpsInverse
{
    static optional<Poly> inverse_sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        if (n == 0)
            return Poly{};
        if (a.empty() || a[0].v == 0)
            return nullopt;
        auto roots = mod_sqrt(a[0].v, mod);
        if (roots.empty())
            return nullopt;
        Poly b{Z(roots[0]).inv()};
        Z half = Z(2).inv();
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 4 * k;
            int m = min(n, 2 * k);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i] * g[i] * g[i];
            ntt(f, true);
            // b <- (3*b - a*b^3)/2; cyclic wrap only reaches below k.
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i] * half;
        }
        return b;
    }
```

平方根与前导零处理（接上页同一结构体）：

```
    static optional<Poly> sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        int k = 0;
        while (k < n && (k >= (int)a.size() || a[k].v == 0))
            k++;
        if (k == n)
            return Poly(n);
        if (k & 1)
            return nullopt;
        int target = n - k;
        Poly f(a.begin() + k, a.begin() + min(n, (int)a.size()));
        auto inverse = inverse_sqrt(f, target);
        if (!inverse)
            return nullopt;
        Poly b = multiply(f, *inverse);
        b.resize(target);
        b.insert(b.begin(), k / 2, Z(0));
        b.resize(n);
        return b;
    }
};
```

模板题使用：Luogu P5205 多项式开根

题意：输入 n 项、保证 a[0]=1，输出平方根的前 n 项，首项取较小的 1。通用接口也可用于无解输入，须先检查 optional。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、mod_sqrt（第 80 页，3.15.1 节）、FpsSqrt（第 155 页，4.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5205

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

模板题使用：Library Checker Sqrt of Formal Power Series

题意：输入任意常数项的 n 项级数；没有平方根输出 -1，否则输出 n 项。接口处理前导零、非二次剩余和全零级数，检查 optional 后才能读取结果。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、mod_sqrt（第 80 页，3.15.1 节）、FpsSqrt（第 155 页，4.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sqrt_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

接口用法演示：形式幂级数平方根倒数与重复调用

演示约定：输入多组 n 项级数，1≤n≤2^22，模 998244353；inverse_sqrt 直接求所选平方根的逆元模 x^n，平方根首项取较小根。无平方根或常数项为 0 时输出-1。同一进程支持多组调用，不必先开方再求逆。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、mod_sqrt（第 80 页，3.15.1 节）、FpsSqrt（第 155 页，4.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-MONTGOMERY-POLY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        FpsSqrt::Poly a(n);
        for (auto &x : a)
        {
            long long value;
            cin >> value;
            x = value;
        }
        auto inverse = FpsSqrt::inverse_sqrt(a, n);
        if (!inverse)
        {
            cout << -1 << '\n';
            continue;
        }
        for (int i = 0; i < n; i++)
            cout << (*inverse)[i].v << " \n"[i + 1 == n];
    }
}
```

4.7.2 形式幂级数微积分、对数与指数

代码依赖：FpsInverse（第 142 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 a.size()<998244353。log(a,n) 要求 n>0、a 非空、a[0]=1；exp(a,n) 要求 n>0、a 为空或 a[0]=0。返回前 n 项，不足输入项视为零，超过 n 的尾项不参与计算。内部每次卷积结果长度须 $\le 2^{23}$；$n \le 2^{22}$ 是统一充分界，输入数组长度须在 int 范围内。微积分 O(a.size())；log/exp 为 O(n log(n+1)) 时间和 O(n) 辅助空间。

```
struct FpsFunctions : FpsInverse
{
    static Poly derivative(const Poly &a)
    {
        Poly b;
        for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
        return b;
    }

    static Poly integral(const Poly &a)
    {
        assert(a.size() < mod);
        Poly b(a.size() + 1);
        vector<Z> inv(a.size() + 1);
        if (!a.empty()) inv[1] = 1;
        for (int i = 2; i <= (int)a.size(); i++)
            inv[i] = Z(mod - mod / i) * inv[mod % i];
        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
        return b;
    }

    static Poly log(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v == 1);
        Poly f(a.begin(), a.begin() + min(n, (int)a.size()));
        auto b = multiply(derivative(f), inverse(f, n));
        b.resize(n - 1);
        return integral(b);
    }
```

```
    static Poly exp(const Poly &a, int n)
    {
        assert(n > 0 && (a.empty() || a[0].v == 0));
        Poly b{1};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            auto t = log(b, m);
            for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
            t[0] = t[0] + 1;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker log_of_formal_power_series

题意：给定常数项为 1 的形式幂级数，求其形式对数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 0。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、FpsFunctions（第 160 页，4.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/log_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::log(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

模板题使用：Library Checker exp_of_formal_power_series

题意：给定常数项为 0 的形式幂级数，求其形式指数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 1。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、FpsFunctions（第 160 页，4.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/exp_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::exp(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7.3 形式幂级数快速幂（十进制大指数）

代码依赖：FpsFunctions（第 160 页，4.7.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

FpsPower::power(a,exponent,n) 返回 a 的非负整数次幂模 x^n 的前 n 项，固定模数 998244353。exponent 为非空十进制数字 string_view，允许前导零；整数指数可传 to_string(k)。$0 \le n \le 2^{22}$，a.size() 须在 int 范围内；短输入补零、长输入只看前 n 项。n=0 返回空，零次幂（含零多项式的零次幂）返回 1。对首个非零项次数 s，若真实 s*k≥n 则返回全零；否则提取常数 c，计算 x^(s*k)*c^k*exp(k*log(a/(c*x^s)))。指数分别保留模 p、模 p-1 和截到 n 的值；不可仅用 k mod p 判零或算位移。依赖 FpsFunctions、FpsInverse、NTT。指数位数 L，时间 O(L+n log(n+1))、额外空间 O(n)，不支持负指数或小数指数。

```
struct FpsPower : FpsFunctions
{
    static Poly power(const Poly &a, string_view exponent, int n)
    {
        assert(n >= 0 && n <= max_size / 2 && !exponent.empty());
        long long k = 0, phi = 0;
        int cap = 0;
        for (char c : exponent)
        {
            assert('0' <= c && c <= '9');
            int digit = c - '0';
            k = (k * 10 + digit) % mod;
            phi = (phi * 10 + digit) % (mod - 1);
            cap = min(n, cap * 10 + digit);
        }
        Poly result(n);
        if (n == 0) return result;
        if (cap == 0)
        {
            result[0] = 1;
            return result;
        }
        int first = 0;
        while (first < n && (first >= (int)a.size() || a[first].v == 0)) first++;
        if (first == n || (first && cap > (n - 1) / first)) return result;
        int shift = first * cap, m = n - shift;
        Poly b(a.begin() + first, a.begin() + min(first + m, (int)a.size()));
        Z leading = a[first], inverse_leading = leading.inv();
        for (auto &x : b) x *= inverse_leading;
        auto f = log(b, m);
        for (auto &x : f) x *= Z(k);
        auto t = exp(f, m);
        Z scale = leading.pow(phi);
        for (int i = 0; i < m; i++) result[i + shift] = t[i] * scale;
        return result;
    }
};
```

模板题使用：Luogu P5273 多项式幂函数（加强版）

题意：输入 n 和十进制大指数 k，输出 a^k 的前 n 项；k 必须用 string 读入，不能只读 k mod p。允许前导零和 k=0。本程序同样适用于 P5245（a[0]=1）及 Library Checker Pow of Formal Power Series，三个入口均已本地验证。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、FpsFunctions（第 160 页，4.7.2 节）、FpsPower（第 163 页，4.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string k;
    cin >> n >> k;
    FpsPower::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsPower::power(a, k, n);
    for (auto x : b) cout << x.v << ' ';
    cout << '\n';
}
```

4.8 常系数齐次线性递推

4.8.1 特征多项式求线性递推第 k 项

recurrence_nth(init,c,n,p) 使用 0-based 下标，n 为 unsigned long long。init 与 c 长度均为阶数 k；a[i]=∑c[j-1]a[i-j]。2≤p≤INT_MAX，数值均在 [0,p)，本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘，O(k^2 log(n+1)) 时间、O(k) 空间；不依赖 BM 或其他模板。较大阶数可使用本库 BostanMori::recurrence，须满足其 NTT 素数模数与容量条件；本函数允许合数模数。

```
// init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
int recurrence_nth(const vector<int> &init,
                   const vector<int> &c,
                   unsigned long long n,
                   int p = 998244353)
{
    assert(p >= 2 && init.size() == c.size());
    int k = c.size();
    if (!k) return 0;
    for (int x : init) assert(0 <= x && x < p);
    for (int x : c) assert(0 <= x && x < p);
    if (n < (unsigned)k) return init[n];
    auto multiply = [&](const vector<int> &a, const vector<int> &b)
    {
        vector<int> t(2 * k - 1);
        for (int i = 0; i < k; i++)
        {
            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
        }
        for (int i = 2 * k - 2; i >= k; i--)
        {
            for (int j = 1; j <= k; j++)
                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
        }
        t.resize(k);
        return t;
    };
    vector<int> answer(k), x(k);
    answer[0] = 1;
    if (k == 1)
        x[0] = c[0];
    else
        x[1] = 1;
    while (n)
    {
        if (n & 1) answer = multiply(answer, x);
        x = multiply(x, x);
        n >>= 1;
    }
    long long value = 0;
    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
    return value;
}
```

模板题使用：Luogu P5487

题意：给出 n 个初值，恢复唯一的最短递推并求第 m 项，n≤10000、n<m≤10^9、阶数 ≤5000，模 998244353。BM 返回 c[j-1] 乘 a[i-j]，首行仅输出系数、不带阶数；裁取恰好 c.size() 个初值后调用 recurrence_nth。全零序列空递推仍输出空首行和第二行 0。复杂度 O(nk+k^2log m)，O(n+k) 空间。

所需模板：berlekamp_massey（第 268 页，11.1 节）、recurrence_nth（第 165 页，4.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5487

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    vector<int> s(n);
    for (auto &x : s)
    {
        cin >> x;
        x %= 998244353;
        if (x < 0) x += 998244353;
    }
    auto c = berlekamp_massey(s);
    for (int i = 0; i < (int)c.size(); i++)
    {
        if (i) cout << ' ';
        cout << c[i];
    }
    cout << '\n';
    vector<int> init(s.begin(), s.begin() + c.size());
    cout << recurrence_nth(init, c, m) << '\n';
}
```

4.8.2 快速递推与有理生成函数单项

代码依赖：NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

BostanMori<mod=998244353,primitive=3>，依赖参数化 NTT；素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 [x^n]p(x)/q(x)，q 非空且 q[0]≠0，p 可空或次数不小于 q；令 d=max(p.size(),q.size())，要求 2d-1≤NTT::max_size。recurrence(init,c,n) 返回 a[n]，从下标 k 起 a[i]=c[0]a[i-1]+⋯+c[k-1]a[i-k]，init、c 长度均为 k；k=0 约定全零。非初值查询要求 2k+1≤NTT::max_size。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项，O(d log d log(n+1)) 时间、O(d) 空间；递推接口 d=k+1。

```
template <int mod = 998244353, int primitive = 3> struct BostanMori
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    using U = unsigned long long;

    // Coefficient of x^n in p(x) / q(x), with q[0] != 0.
    static Z coefficient(Poly p, Poly q, U n)
    {
        assert(!q.empty() && q[0].v != 0);
        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
        while (n && !p.empty())
        {
            Poly r = q;
            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
            Poly a = N::multiply(p, r);
            Poly b = N::multiply(q, r);
            p.clear();
            q.clear();
            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
            n >>= 1;
        }
        return p.empty() ? Z(0) : p[0] / q[0];
    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
    // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
    static Z recurrence(const Poly &init, const Poly &c, U n)
    {
        assert(init.size() == c.size());
        int k = c.size();
        if (k == 0) return 0;
        if (n < init.size()) return init[n];
        assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
        Poly q(k + 1);
        q[0] = 1;
        for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
        Poly p = N::multiply(init, q);
        p.resize(k);
        return coefficient(p, q, n);
    }
};
```

模板题使用：Luogu P4723

题意：给 k 个递推系数和 a[0..k-1]，求 a[n] 模 998244353。c[0] 乘前一项，c[k-1] 乘前 k 项；输入先系数后初值，recurrence 参数先初值后系数。通过 ModInt 赋值归一化负数。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、BostanMori（第 167 页，4.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4723

```
int main()
{
    unsigned long long n;
    int k;
    scanf("%llu%d", &n, &k);
    using B = BostanMori<>;
    B::Poly c(k), init(k);
    for (auto &x : c)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : init)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    printf("%d\n", B::recurrence(init, c, n).v);
}
```

4.9 多项式平移 | 连续点值平移

4.9.1 多项式系数平移

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

polynomial_shift<mod=998244353,primitive=3>(f,c) 返回 f(x+c) 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 ModInt<mod>。依赖参数化 NTT，mod 为素数且 primitive 为原根；非空长度 n 要求 2n-1≤NTT::max_size，因此阶乘均可逆。逆序的 f[i]i! 与 c^i/i! 卷积后提取对应项，O(n log n) 时间、O(n) 空间。c=0 合法。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    if (f.empty()) return {};
    assert(f.size() <= (N::max_size + 1ULL) / 2);
    int n = f.size();
    Poly fact(n), inv(n), a(n), b(n);
    fact[0] = 1;
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    inv[n - 1] = fact[n - 1].inv();
    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
    Z power = 1;
    for (int i = 0; i < n; i++)
    {
        a[n - 1 - i] = f[i] * fact[i];
        b[i] = power * inv[i];
        power = power * c;
    }
    auto product = N::multiply(a, b);
    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
    return a;
}
```

模板题使用：Library Checker polynomial_taylor_shift

题意：给多项式 f 的 n 个升幂系数和常数 c，输出 f(x+c) 的 n 个升幂系数，模 998244353。平移方向为加 c，c=0 时原样返回。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、polynomial_shift（第 169 页，4.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_taylor_shift

```
int main()
{
    int n, c;
    scanf("%d%d", &n, &c);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = polynomial_shift(f, Z(c));
    for (int i = 0; i < n; i++) printf("%d%c", answer[i].v, i + 1 == n ? '\n' : ' ');
}
```

4.10 符号化方法

4.10.1 生成函数建模到模板：三个可核验例子

下面按“约束、因子、取系数、截断、算法”走完建模。记 N 为所求最大次数、$L = N + 1$ 为数组长度，$M(L) = O(L \log L)$ 为 NTT 乘法复杂度。第 4.2.1 节（第 122 页）的 `NttConvolution<p,g>` 可指定素数与原根；`FpsInverse`（第 4.5.1 节，第 142 页）、`FpsFunctions`（第 4.7.2 节，第 160 页）、`FpsPower`（第 4.7.3 节，第 163 页）当前固定为 $p = 998244353$。这些 FPS 操作的长度参数 n 表示项数，求 x^N 必须传 $N+1$。

4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积）

盒子 i 要求 $l_i \le a_i \le u_i$，总数 $\sum a_i = N$。令 $b_i = a_i - l_i$、$c_i = u_i - l_i$、$R = N - \sum l_i$。若有 $u_i < l_i$ 或 $R < 0$，答案为 0；否则所求为

$$[x^R] \prod_{i=1}^{k} (1 + x + \cdots + x^{c_i}).$$

各因子选一个次数，唯一对应一个盒子的数量，所以系数不乘球的排列数。例如容量为 $(2, 3, 1)$、下界均为 0、总数为 4，方案恰为

$$(1, 3, 0),\ (2, 2, 0),\ (0, 3, 1),\ (1, 2, 1),\ (2, 1, 1),$$

答案是 5。零容量的因子为 1，不应当删除这个盒子所携带的其他条件。

截断。只保留 $0, \ldots, R$ 次，因为因子没有负次数，高次不会回流到低次。先把 c_i 截至 R，每次乘法后仅当长度超过 $R+1$ 才截短；不要把较短中间多项式补零至 $R+1$，否则会增加分治卷积的时间。最终若数组尚无第 R 项，该系数就是 0。小规模用有界背包的滑动窗口：

$$d_i(s) = d_i(s-1) + d_{i-1}(s) - [s > c_i] d_{i-1}(s - c_i - 1),$$

以 $d_0(0) = 1$、负下标为 0 初始化，时间 $O(k(R+1))$、空间 $O(R+1)$，只用加减，任意模数可用。不要为了出现乘积就立即写 NTT。

卷积与特殊结构。因子较多且总次数适中时，分治调用 `NttConvolution<998244353>::multiply`；设截短后的总次数为 $D=\sum\min(c_i,R)$，常用上界 $O(k+M(D+1)\log(k+1))$，每层截断通常还能减少工作。若所有 $c_i=c$，令 $P(x)=1+\cdots+x^{\min(c,R)}$，可用 `FpsPower::power(P,to_string(k),R+1)`，时间 $O(M(R+1)+\log(k+1))$。当 $k\geq 1$，另一条路线是容斥：

$$\sum_{j=0}^{\min(k,\lfloor R/(c+1)\rfloor)}(-1)^j\binom{k}{j}\binom{R-j(c+1)+k-1}{k-1}.$$

按组合数选型（第 5.1.3 节，第 179 页）选择组合数接口，再比较项数与卷积成本。$k=0$ 单独返回 $[N=0]$，不向公式塞入 $k-1=-1$。

4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆）

一次操作占用长度 1 或 2，序列总长度为 N，允许空序列。一个操作的 OGF 为 $B(x)=x+x^2$；因为操作有先后，

$$F(x)=\sum_{j\geq 0}B(x)^j=\frac{1}{1-x-x^2},\qquad f_N=[x^N]F(x).$$

$N=3$ 时有 $11,12,21$，$N=5$ 时为 8。求前 $N+1$ 项可调用 `FpsInverse::inverse({1,-1,-1},N+1)`（初始化为其 `Poly` 类型），常数项为 1，可求逆。若只处理这个二阶递推，$f_n=f_{n-1}+f_{n-2}$ 的 $O(N)$ 动态规划更快；这里只是演示可推广到一般步长计数的建模，FPS 不必优于特化算法。

一般每个长度 $s\geq 1$ 有 b_s 种操作，则 $B(x)=\sum b_sx^s$，仍取 $(1-B)^{-1}\bmod x^{N+1}$，时间 $O(M(N+1))$、空间 $O(N+1)$。若 N 巨大且允许步长有固定最大值，可证明有界阶递推后改用远项算法，不应先分配长达 $N+1$ 的数组。

反例。若每种长度只记录使用多少次而不区分先后，应改为 $(1-x)^{-1}(1-x^2)^{-1}$，长度 3 的答案为 2。若允许一种长度为 0 的操作，任意插入它便有无限多序列，$B(0)=0$ 的条件被破坏；不能把一个本来不有限的计数问题交给求逆模板。

4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数）

n 个不同元素，每块大小为 1 或 2，块内与块间均无序。固定的一个单点集合只有 1 种结构，一个二元集合也只有 1 种，故块的 EGF 为

$$B(x)=x+\frac{x^2}{2!},\qquad F(x)=\exp B(x),\qquad a_n=n![x^n]F(x).$$

指数展开的 $B(x)^k/k!$ 先将标签分到 k 个非空有序块，再去掉块的次序；这里各块不空且标签集不交，除以 $k!$ 有严格的计数依据。$n=4$ 时：全单点 1 种，一对加两单点有 $\binom{4}{2}=6$ 种，两对有 3 种，总计 10；不是系数 $[x^4]F=5/12$ 本身。

落到当前接口。在 $p=998244353$ 且 $N<p$ 下：

```
using F = FpsFunctions;
using Z = F::Z;
F::Poly b(N + 1);
if (N >= 1) b[1] = 1;
if (N >= 2) b[2] = Z(2).inv();
auto f = F::exp(b, N + 1);
Binomial<998244353> comb(N);
Z answer = f[N] * comb.fac[N];
```

数组存的是除过阶乘的系数，最后还原；本例建模与 FPS 共耗时 $O(M(N+1))$。若只求这一个模型，选出元素 1 的去向，还可独立得到 $a_0=a_1=1$、$a_n=a_{n-1}+(n-1)a_{n-2}$，给出 $O(N)$ 的简单算法与核验。

增加块数条件。恰好 k 块改为 $n![x^n]B(x)^k/k!$。可调用 `FpsPower::power(b,to_string(k),n+1)`，取第 n 项后乘 `comb.fac[n]` 与 `comb.ifac[k]`，表预处理至 $\max(n,k)<p$。先判 $k\leq n\leq 2k$，不满足便为 0；空对象 $(n,k)=(0,0)$ 为 1。例如 $(n,k)=(4,2)$ 只有三种两两配对。若盒子本身带标号，就没有除以 $k!$ 的步骤；若盒子允许空，单盒因子还应添 1。

4.10.1.4 抄代码前的四项检查与本地证据

1. **目标系数与长度：** 求 $[x^N]$ 使用长度 $N+1$；常数项是否代表空方案；EGF 输入是否已乘逆阶乘，输出是否已乘阶乘。
2. **代数前提：** 逆元分母非零；FPS 的 log 要求首项 1，exp 要求首项 0；涉及阶乘和积分时不能跨过特征。整数有意义的计数，不保证其 EGF 在任意模数下可直接计算。
3. **实际容量：** NTT 接口检查的是完整乘积长度 $a.size()+b.size()-1$，不是最后保留的 $N+1$；“乘后截断” 不会免掉中间数组。固定模数的最大 NTT 长度为 2^{23}，上述 FPS 取 $L \le 2^{22}$ 是安全的保守长度条件。`FpsPower` 还显式检查这个半容量限制；不同 FPS 的局部乘积不能只凭最终次数估算。
4. **先用极小样例数对象：** 至少检查空对象、一个元素、两个元素，以及能区分 “有序/无序” 的第一个例子。错误模型也能被正确的卷积模板稳定算错。

配套 `tests/math_knowledge_combinatorics.py` 用笛卡尔积枚举有界数量向量、显式枚举步长序列、枚举集合划分，独立核对三个例子的系数与边界；同时编译当前 C++ 接口并对照精确整数组合数。运行命令为 `python3 tests/math_knowledge_combinatorics.py`。这是本地建模与接口集成验证，不是新增在线评测 AC，也不取代各模板的完整回归。

4.11 Lagrange 反演

4.11.1 隐式生成函数与 Lagrange 反演

递归对象常给出 $T = x\Phi(T)$，而题目只问 T 或 $H(T)$ 的某一项。Lagrange 反演把这个未知级数的系数，改写为已知级数 Φ 的整数幂系数；不一定要先求出整个 T。本节的“反演”是**复合逆**，与第 10.1.1 节（第 251 页）的已知点值插值不同。

4.11.1.1 适用条件与取系数公式

先在 $\mathbb{Q}$ 或特征零域 K 上讨论形式幂级数。设 $\Phi(t), H(t) \in K[[t]]$，$\Phi(0) \neq 0$；若改用一般的 $\mathbb{Q}$-代数，要求 $\Phi(0)$ 是单位，即有乘法逆元。则

$$T(x) = x\Phi(T(x)), \qquad T(0) = 0$$

唯一确定 $T \in xK[[x]]$：右侧的 x^n 项只依赖 T 的前 $n-1$ 次项，可从常数项起递推；其一次项为 $\Phi(0)$。令 $f(t) = t/\Phi(t)$，则 $f(T(x)) = x$，且 $T(f(t)) = t$。对 $n \geq 1$，有

$$\boxed{[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n.}$$

特别地，取 $H(t) = t^k$，在 $n \geq k \geq 1$ 时得到

$$\boxed{[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n.}$$

边界先分支： $[x^0]H(T) = H(0)$；$T^0 = 1$，故 $[x^n]T^0 = [n=0]$；$k \geq 1$ 且 $0 \leq n < k$ 时为 0。不要把 $n = 0$ 代进带 $1/n$ 的公式。这里每个固定次数只涉及有限次代数运算，**不要求级数具有解析收敛半径**。

4.11.1.2 形式留数证明

对只有有限个负次项的形式 Laurent 级数，记 $\operatorname{Res}_t F = [t^{-1}]F$。形式导数没有 t^{-1} 项，故 $\operatorname{Res}_t F' = 0$。若 $g(0) = 0$ 且 $g'(0)$ 可逆，换元公式为

$$\operatorname{Res}_x F(x) = \operatorname{Res}_t F(g(t))g'(t).$$

只需检查单项式 $F(x) = x^j$：$j \neq -1$ 时右边是 $(g^{j+1})'/(j+1)$ 的留数，为 0；$j = -1$ 时，写 $g = tu$、$u(0) \neq 0$，便有 $g'/g = t^{-1} + u'/u$，留数为 1。取 $g = f = t/\Phi(t)$，再用乘积求导，得到

$$\begin{aligned}[x^n]H(T(x)) &= \operatorname{Res}_t \frac{H(t)f'(t)}{f(t)^{n+1}} \\ &= \frac{1}{n}\operatorname{Res}_t \frac{H'(t)}{f(t)^n} = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n.\end{aligned}$$

中间等号来自 $\operatorname{Res}_t(H(t)f(t)^{-n})' = 0$；它也清楚标出除以 n 的位置。

4.11.1.3 模型一按内部结点计数的满有序多叉树

固定整数 $d \geq 2$，每个结点要么是叶子，要么恰有 d 个**有序**孩子。用 x 标记内部结点，叶子本身是大小 0 的一种树，故 OGF 满足

$$A(x) = 1 + xA(x)^d.$$

令 $T = A - 1$，化为 $T = x(1+T)^d$。因此 $a_0 = 1$，而 $n \geq 1$ 时

$$a_n = [x^n]A = \frac{1}{n}\binom{dn}{n-1} = \frac{1}{(d-1)n+1}\binom{dn}{n}.$$

$d = 2$ 得到 $1, 1, 2, 5, 14, \ldots$；$d = 3$ 得到 $1, 1, 3, 12, 55, \ldots$。有 n 个内部结点时，共有 $dn+1$ 个结点、$(d-1)n+1$ 个叶子，所以这不是按所有结点计数。孩子位置有序；若把孩子改成无序集合，不能仍用 A^d。若先用 `Binomial<p> c(d*n)` 再求组合数，须确认 $dn < p$、表能放下，并在宽类型中计算 dn，最后检查选用的分母是否可逆。即使 $n < p$，也可能有 $dn \geq p$；此时回到第 5.1.3 节（第 179 页）的组合数选型，不能越界调用阶乘表。

4.11.1.4 模型二带标号有根树的 EGF

结点标签为 $1,\dots,n$，根也是结构的一部分，根的孩子**没有先后顺序**。删除根后，剩下的是一组标签互不相交的非空有根树，因此

$$T(x)=x\exp T(x),\qquad T(x)=\sum_{n\ge1}r_n\frac{x^n}{n!}.$$

取 $\Phi(t)=e^t$，对 $n\ge1$ 有

$$[x^n]T=\frac{1}{n}[t^{n-1}]e^{nt}=\frac{n^{n-1}}{n!},\qquad r_n=n!\,[x^n]T=n^{n-1}.$$

$n=0$ 没有有根树，$n=1$ 有 1 棵；$r_2=2,r_3=9,r_4=64$。数组里的 $[x^3]T=3/2$ 不是树的数目 9，必须还原阶乘。这里计算的是“选定根也计入方案”的数目；若指定某个标签为根，$n\ge2$ 时为 n^{n-2}。也不能把隐式的 $\exp T$ 替换成已知的 $\exp x$。

4.11.1.5 落到现有接口及模数边界

只求一个幂系数。第 4.7.3 节（第 163 页）的 `FpsPower::power` 当前固定模数 $p=998244353$，只支持非负整数指数。下面采用简单充分范围：$1\le k\le n<p$、$L=n-k+1\le 2^{22}$，且内存足够。把 Φ 的前 L 项存入 `FpsPower::Poly phi`，可写

```
using F = FpsPower;
using Z = F::Z;
int l = n - k + 1;
auto q = F::power(phi, to_string(n), l);
Z answer = Z(k) * Z(n).inv() * q[n - k];
```

长度参数是**项数**，故取 t^{n-k} 要传 $n-k+1$；不能只传 $n-k$。短输入缺项按零处理，若真有非零项则必须提供。指数是完整非负十进制字符串，不要自行只传 $n \bmod p$。单次耗时 $O(L\log(L+1)+\log(n+1))$、额外空间 $O(L)$；对 $n=1,\dots,N$ 分别取系数不能据此声称整体 $O(M(N))$，按此做法的通用上界为 $O(N^2\log(N+1))$，还未计输入构造。

辅助接口。第 4.7.2 节（第 160 页）提供 `derivative(a)`、`integral(a)`、`log(a,l)`、`exp(a,l)`；log 要求首项 1，exp 要求首项 0，积分分母须可逆。这些操作仍固定模数 998244353，$1\le l\le 2^{22}$ 是保守安全长度界。它们可用于构造 H' 或已知表达式的指数，**不是求解任意隐式方程的接口**。对一般 H，若长度 n 满足上述容量界，先算 $q=\Phi^n \bmod t^n$，再取 $n^{-1}\sum_{i=1}^{n} iH_iq_{n-i}$，其中 $H_i=[t^i]H$。已知 q 后只需 $O(n)$ 点积，不必计算 $H'q$ 的整段卷积。第 5.1.1 节（第 177 页）的 `Binomial<p> c(N)` 在素数 p、$N<p$ 下提供 `c.choose(n,k)`、`c.fac[n]` 和 `c.ifac[n]`；先显式建表至所有查询的最大下标，再读取或还原 EGF 阶乘。

不要混淆两种逆。$f^{\langle-1\rangle}$ 满足 $f(T)=x$，而第 4.5.1 节（第 142 页）的 `inverse` 求 $fB=1 \pmod{x^L}$，要求 $f(0)\ne0$。这里 $f=t/\Phi(t)$ 的常数项恰为 0，不能交给乘法求逆模板。本节不新增通用 FPS 复合、复合逆或整段隐式解的实现覆盖。

整数除法不能随意改成模逆元。在 $\mathbb{F}_p$ 中使用带 $1/n$ 的公式，至少要求 $p\nmid n$，且 Φ,H 的所需系数本来就在该域中有定义；EGF 的 $1/j!$ 也不能跨过 $j=p$。例如 $T=x(1+T)^2$、$n=p=3$，整数答案为

$$[x^3]T=\frac{1}{3}\binom{6}{2}=5\equiv2 \pmod 3,$$

但分子 15 与分母 3 先取模都为 0，不能用 0^{-1}；只保留分子模 3 也已经丢掉了除以 3 所需的信息。可改用无除法递推，或在能保证整除时保留足够整数/提升模数精度再除。这例也可改用等价式 $\binom{6}{3}/4$，但必须重新检查分母 4 在模 3 下可逆。

来源。定理、形式留数换元及两类树的公式核对 Ira M. Gessel, *Lagrange Inversion* 的定理 2.1.1、第 4.1 节、第 3.2–3.3 节；中文概念导航参见 OI Wiki：Lagrange 反演。以上模型说明、接口对应和模 3 反例为本手册重新组织的推导，不将知识补充记作在线评测通过。

4.12 指数生成函数

4.12.1 平移幂和多项式

代码依赖：FpsInverse（第 142 页，4.5.1 节）、ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

shifted_power_sum(n,m) 返回 ∑(k=1..m)(x+k)^n 的 n+1 个系数，按 x 次数从低到高，模 998244353。n≥0，2n+1≤2^23；m 为完整 unsigned long long，m=0 返回全零，n=0 返回常数 m。利用生成函数 (exp((m+1)t)-exp(t))/(exp(t)-1)，分子分母先除 t，再求逆与卷积；第 i 项为 n!/i! 乘该级数第 n-i 项。只在模域中计算 m+1，避免 uint64 溢出；n+1 小于模数，整周期幂和为零，故可先将 m 取模。O(n log(n+1)) 时间、O(n) 工作空间，另有 NTT 缓存；n=0 或 m 模素数为 0、1 时直接计算。依赖 FpsInverse。不计算普通的几何级数和。

```
vector<ModInt<998244353>> shifted_power_sum(int n, unsigned long long m)
{
    using F = FpsInverse;
    using Z = F::Z;
    assert(n >= 0 && 2LL * n + 1 <= F::max_size);
    Z count = m % F::mod;
    if (n == 0)
        return {count};
    if (count.v == 0)
        return vector<Z>(n + 1);
    vector<Z> fac(n + 2), invfac(n + 2);
    fac[0] = 1;
    for (int i = 1; i <= n + 1; i++)
        fac[i] = fac[i - 1] * i;
    invfac[n + 1] = fac[n + 1].inv();
    for (int i = n + 1; i > 0; i--)
        invfac[i - 1] = invfac[i] * i;
    if (count.v == 1)
    {
        vector<Z> result(n + 1);
        for (int i = 0; i <= n; i++)
            result[i] = fac[n] * invfac[i] * invfac[n - i];
        return result;
    }
    vector<Z> a(n + 1), b(n + 1);
    Z step = count + 1;
    Z power = 1;
    for (int i = 0; i <= n; i++)
    {
        power *= step;
        a[i] = (power - Z(1)) * invfac[i + 1];
        b[i] = invfac[i + 1];
    }
    auto sums = F::multiply(a, F::inverse(b, n + 1));
    vector<Z> result(n + 1);
    for (int i = 0; i <= n; i++)
        result[i] = fac[n] * invfac[i] * sums[n - i];
    return result;
}
```

接口用法演示：平移幂和多项式完整输入输出

演示约定：输入 n 和无符号 64 位 m，输出 ∑(k=1..m)(x+k)^n 的 n+1 个系数，从常数项开始，模 998244353。0≤n≤2^22-1；m=0 和 n=0 均可用。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、shifted_power_sum（第 175 页，4.12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-POLY-MULTIPLY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    auto f = shifted_power_sum(n, m);
    for (int i = 0; i <= n; i++)
        cout << f[i].v << "\n "[i != n];
}
```

第五章　组合数学

5.1　排列组合

5.1.1　阶乘组合数

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 24 页，3.2.2 节）、Binomial（第 177 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

5.1.2 计数建模：先确认哪些对象被区分

写公式前，先用一句话说明：**一个方案是什么，什么变化会产生新方案？**“把 n 个物品分成 k 组”不足以选公式；物品是否带标号、组是否带标号、组内是否有序、组间是否有序、空组是否允许，分别决定不同问题。

5.1.2.1 五个判别问题与两个最小反例

1. **物品可区分吗？**给定标签 $1,\dots,n$，交换两个物品可能改变方案，属于带标号计数。相同球的方案只记录各类或各盒数量，不能再乘一个 $n!$。

2. **容器可区分吗？**盒子 A、B 是不同位置，即使容量相同也不同。无标号盒子通常记录一个集合划分；空盒不形成新的块。

3. **次序在哪里？**一组元素组成集合、线性排列、圆排列，分别是不同结构。例如固定 s 个标签组成非空块时，集合只有 1 种，线性排列有 $s!$ 种，不区分起点的圆排列有 $(s-1)!$ 种。不能只看到“无序分组”就忽略组内次序。

4. **哪些数量固定？**“恰好 k 个非空组”与“至多 k 个组”不同；如果块数也要统计，可用额外变量 y 标记，每个块贡献一个 y。

5. **空对象是否是一种方案？**通常空序列、空集合各有 1 种，所以常数项为 1。“至少一个块”则要减去 1。无物品且无盒时通常计 1，非空物品放入零盒计 0；题目另有定义时以题面为准。

不能机械除以盒子数的阶乘。两个不同球放入两个有标号且允许空的盒子，有 $2^2=4$ 种；商去盒子标号后只有 $\{\{1,2\}\}$ 与 $\{\{1\},\{2\}\}$ 两种。这个例子恰巧等于 $4/2!$，却不是通用理由：三个盒子时有 $3^2=9$ 种，无标号且至多三块仍只有 2 种，而 $9/3!$ 甚至不是整数。原因是含空盒时不同方案的稳定子大小可能不同。若恰好 k 个非空块且只区分盒子标号，每个划分有恰好 $k!$ 个标号方式，才可除以 $k!$。

不能把“顺序不同”当成“类型数量不同”。用长度 1、2 的砖铺长度 3 的一行，方案是 $111, 12, 21$，共 3 种；若只问凑出总重量 3 的多重集合，则 12 与 21 是同一种，共 2 种。前者是序列，后者是每种类型独立选择重数。

5.1.2.2 OGF、EGF 的选择来自乘法，而非题目用词

当独立部分只需把大小相加，且大小为 i, j 的两个对象合并后恰有 $a_i b_j$ 种，使用 OGF 的普通卷积。若还须从 $i+j$ 个不同标签中选 i 个分给第一部分，多出 $\binom{i+j}{i}$，使用 EGF 更自然。

$$\text{OGF 数组：}\ A_i = a_i, \qquad \text{EGF 数组：}\ A_i = a_i/i!, \qquad a_n = n!A_n.$$

卷积模板始终计算普通数组卷积；它不会识别并补上 EGF 的阶乘。题目出现标签不代表只能用 EGF，出现无序也不代表只能用 OGF；判断依据是当前分解方式如何分配标签与次序。一般非空带标号块的 EGF 记为 $B(x)$，则有序块序列是 $1/(1-B(x))$，无序块集合是 $\exp B(x)$；二者都要求 $B(0)=0$，使每个固定次数只有有限项贡献。本段组合解释参见 OI Wiki：指数生成函数。

5.1.3 组合数选型：先看模数，再看下标与预算

组合恒等式负责推导；本节负责把 $\binom{n}{k}$ 落到可运行的接口。先用有符号或更宽类型处理负下标、$k>n$ 与 $n+k$ 溢出，再调用模板。特别是 `Lucas::choose` 与 `ExLucas::choose` 接收无符号参数，不要把容斥里的负数直接转换后传入。

素数 p，所有查询 $n \le N < p$： 用第 5.1.1 节（第 177 页）的 `Binomial<p> c(N)`，查询 `c.choose(n,k)`；排列数用 `c.permute(n,k)`。一次建表时间 $O(N+\log p)$、空间 $O(N)$，每次查询 $O(1)$。后续用 `c.init(M)` 显式扩表；每次真正扩表都做一次扩展欧几里得求逆，所以连续逐项扩表不能简单视作总共 $O(N)$。尽量一次扩至本批最大上界。此处 p 是编译期模数，调用者保证素数；模板的断言不等于判素。

素数 p，n 很大且 p 足够小： 用第 3.14.1 节（第 68 页）的 `Lucas c(p)`，再调用 `c.choose(n,k)`。完整预处理时间 $O(p+\log p)$、空间 $O(p)$，单次 $O(1+\log_p n)$。“下标大”不自动意味着适合此实现：$p=10^9+7$ 时，两张长度为 p 的 `int` 表约需 8 GB，不能靠 Lucas 定理消除这笔开销。

合数 m，所有素数幂因子的表可以放下： 用第 3.14.2 节（第 69 页）的 `ExLucas c(m)`，查询 `c.choose(n,k)`。它按 $m=\prod q_i$、$q_i = p_i^{e_i}$ 建表，不是只预处理至最大查询的 n。空间 $O(\sum q_i)$，建表时间可保守记为 $O(\sqrt{m}+\sum q_i+\sum \log q_i)$。当前 `unit` 的每一层还有一次快速幂，故单次可保守记为 $O(\sum_i (1+\log_{p_i}(n+1))\log(n+2)+\sum_i \log q_i)$，不能误写成纯粹 $O(\log n)$。模数 1 返回 0。若某个 q_i 很大，表就很大；ExLucas 不是任意 32 位合数的低内存黑盒。

三者都不合适时不要硬套。若 n 小、模数任意，Pascal 递推只用加法，算到第 n 行耗时 $O(n^2)$、滚动空间 $O(n)$；若只需 $k \le K$ 的列，可降至 $O(nK)$。若素数 p 很大、$k'=\min(k, n-k) < p$ 很小，可将连续 k' 个分子相乘，再乘 $(k'!)^{-1}$，耗时 $O(k'+\log p)$，即使 $n \ge p$ 也可用，因为分母仍是单位；这需要自行写短循环。合数模数下即便整数商存在，也不代表分母可逆，例如 $\binom{4}{2}=6\equiv 2 \pmod 4$，而 $2!$ 在模 4 下没有逆元。不能把整数公式中的“除法”直接换成模逆元。

5.2 容斥原理

5.2.1 容斥与二项式反演：恰好与至少

容斥不只求并集。题目若能算出若干性质的交集，往往可以先求交集和，再恢复“恰好满足几个性质”的分布。关键是**交集和带有重数**，不能把它直接当成“至少”的数量；性质之间也不要求独立。

5.2.1.1 从交集和恢复恰好数量

设有限全集为 Ω，性质对应集合为 $A_1,\dots,A_m$。记 $q(x)=|\{i:x\in A_i\}|$，以及

$$N_t=|\{x\in\Omega:q(x)=t\}|,\qquad S_j=\sum_{\substack{J\subseteq\{1,\dots,m\}\\|J|=j}}\left|\bigcap_{i\in J}A_i\right|,\quad S_0=|\Omega|.$$

空指标集的交集是全集。满足 t 个性质的对象在 S_j 中出现 $\binom{t}{j}$ 次，故

$$\boxed{S_j=\sum_{t=j}^{m}\binom{t}{j}N_t,\qquad N_r=\sum_{j=r}^{m}(-1)^{j-r}\binom{j}{r}S_j.}$$

第二式适用于 $0\le r\le m$；范围外的恰好数量为 0。**证明**。代入第一式，N_t 的系数为

$$\sum_{j=r}^{t}(-1)^{j-r}\binom{j}{r}\binom{t}{j}=\binom{t}{r}\sum_{h=0}^{t-r}(-1)^h\binom{t-r}{h}=\begin{cases}1,&t=r,\\0,&t>r.\end{cases}$$

因此其他重数全部抵消。取 $r=0$ 即没有任何性质的数量；普通并集容斥是 $|\bigcup_i A_i|=S_0-N_0=\sum_{j=1}^{m}(-1)^{j-1}S_j$。

最小反例。全集只有一个对象且它同时满足两个性质，则 $(S_0,S_1,S_2)=(1,2,1)$，但 $(N_0,N_1,N_2)=(0,0,1)$。$S_1=2$ 不是并集大小，$S_1-S_2=1$ 也不是恰好满足一个性质的数量；正确式为 $N_1=S_1-2S_2=0$。

5.2.1.2 至少数量与变换方向

对 $1\le r\le m$，将上述 N_t 从 $t=r$ 到 m 求和，得到

$$\boxed{|\{x:q(x)\ge r\}|=\sum_{j=r}^{m}(-1)^{j-r}\binom{j-1}{r-1}S_j.}$$

其中 S_j 的系数是 $\sum_{t=r}^{j}(-1)^{j-t}\binom{j}{t}=(-1)^{j-r}\binom{j-1}{r-1}$，可用 Pascal 恒等式逐项相消证明。注意系数不再是 $\binom{j}{r}$。$r=0$ 时直接返回 S_0，$r>m$ 时返回 0，不能把负下标交给组合数公式。

不要抄反方向。上面的 $S_j=\sum_{t\ge j}\binom{t}{j}N_t$ 是有限上三角变换。常见的另一种二项式反演是

$$b_n=\sum_{k=0}^{n}\binom{n}{k}a_k\quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k.$$

它是下三角变换：待求第 n 项只需 $b_0,\dots,b_n$；前者恢复 N_r 却需要 $S_r,\dots,S_m$。可用同一系数消去证明，但下标、符号和求和范围必须随对象含义重新确定。

适用范围。这些等式只有整数乘加减，没有除法，先在整数中成立，再对任意正模数取模都成立；不要求模数为素数。给每个对象一个权重也可用同样的逐对象线性证明。但模意义下的答案为 0 不表示实际对象不存在，且选用阶乘逆元实现组合数时另有条件。计算全部一般 S_j 可能需要枚举 2^m 个子集及各自交集，反演公式本身不会自动消除这个成本。只有证明交集大小仅依赖子集大小 j 后，才能用 $\binom{m}{j}$ 乘一个代表值。例如 $A_1=A_2=\{1,2\},A_3=\{3,4\}$ 的各集合都等大，但三个两两交集大小为 $2,0,0$，不能仅凭单集等大作此压缩。

5.2.1.3 例：有标号球放入有标号盒

n 个互异球各选择 m 个互异盒之一，计恰好 r 个非空盒的方案数；盒内不计顺序。不等同于无标号球的插板模型，也不额外乘 $n!$ 或除以 $m!$。先选出非空盒，再对其中的空盒做容斥，得到

$$\boxed{\binom{m}{r}\sum_{j=0}^{r}(-1)^j\binom{r}{j}(r-j)^n=\binom{m}{r}r!\left\{{n\atop r}\right\}.}$$

也可把原来 m 个盒的“为空”当作性质，此时 $S_j=\binom{m}{j}(m-j)^n$，恰好 r 个**非空**盒对应 N_{m-r}，不是 N_r。将它代入前式，并用 $\binom{j}{m-r}\binom{m}{j}=\binom{m}{r}\binom{r}{j-(m-r)}$ 即得到同一答案。右侧 Stirling 写法只是同一模型的交叉解释，不新增算法。

例如 $n=3,m=4$，恰好占用 1、2、3 个盒的数量分别为 $4,36,24$，合计 $4^3=64$。空盒性质的交集和为 $(64,108,48,4,0)$；恰好两个空盒是 $48-3\cdot4=36$，至少两个空盒却是 $48-2\cdot4=40$。这同时展示了“恰好”和“至少”的系数区别。

边界。$r<0$、$r>m$ 或 $r>n$ 时答案为 0。$n=0$ 时只有一个空映射，恰好 $r=0$ 才为 1，包括 $m=0$；$n>0,m=0$ 时无解。统一公式中的 $0^0=1$ 是空映射约定，不是浮点极限。

5.2.1.4 直接使用现有组合数与模整数

已给非负 `long long n` 和非负 `int m`、整数 `r`，固定 $p=998244353$，要求 $m<p$ 且能容纳长度 $m+1$ 的阶乘表。第 3.2.2 节（第 24 页）和第 5.1.1 节（第 177 页）可直接组合：

```
using Z = ModInt<998244353>;
Z answer = 0;
if (0 <= r && r <= m && r <= n)
{
    Binomial<998244353> c(m);
    for (int j = 0; j <= r; j++)
    {
        Z term = c.choose(r, j) * Z(r - j).pow(n);
        if (j & 1) answer -= term;
        else answer += term;
    }
    answer *= c.choose(m, r);
}
```

只需建到 m，不需要 $n!$；n 可以远大于 p，但必须在非负 signed-long-long 范围内，因为当前 `pow` 的指数类型不是 `unsigned long long`。单次建表与计算总耗时 $O(m+\log p+(r+1)\log(n+2))$、空间 $O(m)$；其中 $\log p$ 计入一次阶乘求逆，$\log(n+2)$ 包括 $n=0$ 的常数工作。不合法 r 的分支不建表。多询问可复用已建表，但要另行说明统一上界。

这里的素数限制来自 `Binomial` 用费马逆元，不来自容斥。合数模数不可直接替换该模板参数；$m\ge p$ 也不能继续使用此阶乘逆元表。应根据实际规模改用已有 Pascal、Lucas 或扩展 Lucas 路线，并重新检查各自条件。取模前后都应使用带符号的模运算处理交错和，不依赖无符号下溢。

来源与范围。恰好/至少性质数的两式核对 Richard P. Stanley, *Enumerative Combinatorics*, Volume 1 第 2 章习题 3，式 (2.39)–(2.40)；本节系数推导、反例、盒模型和接口衔接重新组织。基础容斥与概念导航参见 OI Wiki：容斥原理。本节不声称覆盖该来源页面的所有计数主题，也不计作新模板或在线评测通过。

5.3 斐波那契数列

5.3.1 斐波那契最小循环节（Pisano 周期）

代码依赖：Mod64（第 15 页，2.1 节）、PollardRho（第 45 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

period(m) 返回精确最小正周期，1≤m≤2^64−1，m=1 返回 1；结果为 unsigned int128，可能超过 64 位。fib(k,m) 递归倍增返回 (F_k,F_{k+1}) 模 m，k 为 unsigned int128；is_period(k,m) 检查 k>0 及完整状态 (0,1%m)。prime_period(p) 仅供素数 p≥2。复用 PollardRho 分解 m 和素数候选的 p±1；逐个除因子求素数最小周期，再逐层检查素数幂，不能无条件每层乘 p。互素分量周期取 lcm。总复杂度含随机整数分解，无确定的 O(log m) 保证；每次状态检查 O(log k) 次 128 位模运算，递归空间 O(log k)，k≤128 位。对象可重复使用，factorizer 仅控制分解随机过程；$\pi(m)\le 6m$，所有周期算术使用 128 位。

```
struct FibonacciPeriod
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    PollardRho factorizer;

    static pair<ull, ull> fib(u128 k, ull m)
    {
        assert(m >= 1);
        if (k == 0)
            return {0, 1 % m};
        auto [a, b] = fib(k / 2, m);
        ull c = Mod64::mul(a, (u128(2) * b + m - a) % m, m);
        ull d = (u128(Mod64::mul(a, a, m)) + Mod64::mul(b, b, m)) % m;
        if (k & 1)
            return {d, (u128(c) + d) % m};
        return {c, d};
    }

    static bool is_period(u128 k, ull m)
    {
        return k > 0 && fib(k, m) == make_pair(0ULL, 1 % m);
    }
```

素数候选缩减、素数幂逐层检查与互素 lcm（接上页同一结构体）：

```
    u128 prime_period(ull p)
    {
        if (p == 2)
            return 3;
        if (p == 5)
            return 20;
        bool square = p % 5 == 1 || p % 5 == 4;
        ull base = square ? p - 1 : p + 1;
        u128 t = u128(base) * (square ? 1 : 2);
        auto factors = factorizer.factor(base);
        if (!square)
            factors.push_back(2);
        for (ull q : factors)
            while (t % q == 0 && is_period(t / q, p))
                t /= q;
        return t;
    }

    static u128 gcd128(u128 a, u128 b)
    {
        while (b)
        {
            u128 r = a % b;
            a = b;
            b = r;
        }
        return a;
    }

    u128 period(ull m)
    {
        assert(m >= 1);
        auto factors = factorizer.factor(m);
        u128 answer = 1;
        for (int i = 0; i < (int)factors.size();)
        {
            ull p = factors[i];
            ull power = p;
            u128 t = prime_period(p);
            i++;
            while (i < (int)factors.size() && factors[i] == p)
            {
                power *= p;
                if (!is_period(t, power))
                    t *= p;
                i++;
            }
            answer = answer / gcd128(answer, t) * t;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4994 / 终于结束的起点

题意：输入单个模数 2≤M≤706150，输出最小 n>0 使 F_n=0、F_(n+1)=1（模 M）。π(M)≤6M，题目范围内可安全转 unsigned long long。独立模板问题；题面与样例已核对，无本项目线上 AC 或排名。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、FibonacciPeriod（第 182 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4994

```
int main()
{
    unsigned long long m;
    cin >> m;
    FibonacciPeriod solver;
    cout << (unsigned long long)solver.period(m) << '\n';
    return 0;
}
```

接口用法演示：完整 64 位模数与 128 位最小周期

演示约定：输入 T 与 T 个正 unsigned64 模数，逐行输出精确最小周期，使用 128 位十进制转换。包含 m=1 的约定；m=2*5^27 时答案 89406967163085937500 超过 64 位。API 演示，不是线上通过记录。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、FibonacciPeriod（第 182 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/fibonacci/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    FibonacciPeriod solver;
    while (tests--)
    {
        unsigned long long m;
        cin >> m;
        auto answer = solver.period(m);
        string output;
        do
        {
            output.push_back('0' + answer % 10);
            answer /= 10;
        }
        while (answer);
        reverse(output.begin(), output.end());
        cout << output << '\n';
    }
    return 0;
}
```

接口用法演示：kuangbin HDU3977 来源程序协议

演示约定：按完整来源程序输入 T 及正 int 模数，输出 Case #k: 周期。来源初值 (1,1) 与标准 (0,1) 仅相差一次可逆转移，周期相同。原 PDF 的注释吞掉左括号、旧 MSVC 输出格式须显式兼容修改；官方题面不可访问，故仅归为来源协议 API，不声称正式题目或 AC。

所需模板：Mod64（第 15 页，2.1 节）、Prime64（第 40 页，3.3.1 节）、PollardRho（第 45 页，3.6.1 节）、FibonacciPeriod（第 182 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=3977

```
int main()
{
    int tests;
    cin >> tests;
    FibonacciPeriod solver;
    for (int i = 1; i <= tests; i++)
    {
        int m;
        cin >> m;
        cout << "Case #" << i << ": ";
        cout << (unsigned long long)solver.period(m) << '\n';
    }
    return 0;
}
```

5.4 卡特兰数

5.4.1 Catalan 与投票问题：把前缀限制变成反射双射

先问限制的是最终数量，还是每个前缀。把一种操作记为 $+1$，另一种记为 -1，高度就是两种操作数量之差。固定 p 个正步、q 个负步，共有 $\binom{p+q}{q}$ 个次序；这只固定终点，未限制中途高度。合法括号串要求高度始终非负且最终为零；从空栈开始的入栈/出栈操作序列也有同样条件。这只计操作类型序列，不自动计带标签元素的任意排列、操作耗时或额外栈约束。

5.4.1.1 弱前缀条件：减去首次走到负一的坏路径

假设 $p \ge q \ge 0$，从高度 0 出发，每个非空前缀高度都要求 ≥ 0。坏路径必然有第一次到达 -1 的时刻；只把从起点到该时刻的所有正负步交换，后缀不变。该前缀原来负步比正步多 1，交换后总正步变成 $p+1$、负步变成 $q-1$。反过来，任意这样的新路径终点为 $p-q+2 \ge 2$，必然第一次到达 $+1$；把到此处的前缀再交换，就唯一恢复一条坏路径。因此这是双射，而不只是“看起来对称”，合法数为

$$W(p,q) = \binom{p+q}{q} - \binom{p+q}{q-1}.$$

约定 $\binom{N}{k} = 0$ 当 $k < 0$ 或 $k > N$。若 $p < q$，终点已非法，答案直接为 0，不能继续使用可能给负数的差式。$p = q = 0$ 的空序列计 1。反射必须截止于第一次越界；翻转整条路径或任意一个越界前缀都不构成上述逆映射。

取 $p = q = n$，得到括号串、Dyck 路径的 Catalan 数

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}, \qquad C_0 = 1.$$

这里 $\binom{2n}{n-1} = \binom{2n}{n+1}$，与前文差分公式相同。这些经典计数对象可参见 Stanley 的 Catalan 习题 19(h),(i)；使用时仍需先证明题目对象与路径一一对应。

5.4.1.2 严格领先：先取走必需的第一步

若要求每个非空前缀高度 > 0，且 $p > q$，第一步必须为正步。删去这一步，并将剩余路径的高度整体减 1，得到 $p-1$ 个正步、q 个负步的弱条件，故

$$S(p,q) = \binom{p+q-1}{q} - \binom{p+q-1}{q-1} = \frac{p-q}{p+q}\binom{p+q}{q}.$$

当 $p+q > 0$ 且 $p \le q$ 时答案为 0；空路径若按“所有非空前缀”定义，则真空成立、单独计 1，不能把它代入含 $p+q$ 分母的式子。例如 $p = 3, q = 2$：弱条件有 5 种，严格条件只有 $+++--$ 和 $++-+-$ 两种。“过程中可以打平”和“始终领先”不是同一条件，不能只检查最终 $p > q$。

5.4.1.3 初始有余额：反射起点，而非只改终点

从整数高度 $h \geq 0$ 出发，走 $L \geq 0$ 个 ± 1 步，到达 $e \geq 0$，全程不低于 0。先检查 $|e-h| \leq L$ 且 $L+e-h$ 为偶数，否则答案为 0。令 $k=(L+e-h)/2$ 为正步数，总路径数为 $\binom{L}{k}$。坏路径第一次到达 -1 之前，按高度变换 $y \mapsto -2-y$ 反射前缀，把起点 h 变成 $-h-2$，越界点 -1 不动，终点仍为 e。由于新起点在 -1 下方、终点非负，每条新路径都必经 -1，在第一次到达处反射回来就是逆映射。新路径正步数为 $k+h+1$，故

$$B(L,h,e)=\binom{L}{k}-\binom{L}{k+h+1}.$$

例：$L=4,h=2,e=0$，四种无约束次序中只有 $---+$ 越界，答案为 $4-1=3$。$L=0$ 时只有 $h=e$ 的空路径计 1。若下界为整数 b，先把起终点减去 b；若要求整数高度严格大于 b，则下界应取 $b+1$。

适用边界。单个常数下界和单位 ± 1 步使“第一次碰到边界”可逆。再加上界、禁止位置、加权路径、非单位步长或斜率边界时，不能照搬两项差式；可能需要状态 DP、重复反射或另一种专门计数。单步能跳过 -1 时，上述首次命中论证已经失效。

5.4.1.4 取模时优先使用差式，除法条件不能省

差式只要求能正确计算两个组合数，不要求 $n+1$、$p+q$ 可逆。例如 $C_2=2$，模 6 仍为 2；把 $\binom{4}{2}$ 先取模得到 0，再除以 3 无法恢复答案，因为 3 没有模 6 逆元。即使模数是素数，分母也可能被模数整除。

选择见第 179 页（5.1.3 节）：素数模数且所需最大上标小于模数时可用 Binomial（第 177 页，5.1.1 节）；跨过素数模数时考虑 Lucas（第 68 页，3.14.1 节），合数模数可按资源预算考虑 ExLucas（第 69 页，3.14.2 节）。后两者都有预处理空间/时间限制，不意味着任意大模数都可直接建立表。先以有符号或更宽类型检查负下标、不可达性及 $p+q, 2n, L+e-h, k+h+1$ 的溢出，再调用接收无符号参数的接口；最后将两个模值之差规范化。

5.5 斯特林数与斯特林反演

5.5.1 定义与性质

设 $n,k\ge 0$。无符号第一类 $c(n,k)$ 计数 n 个带标号元素的排列中，恰有 k 个不区分顺序的循环的方案数；循环内只忽略起点，方向不等同。第二类 $S(n,k)$ 计数把 n 个带标号元素划分为 k 个无标号非空集合的方案数。有符号第一类定义为 $s(n,k)=(-1)^{n-k}c(n,k)$。例如 $c(3,1)=2$、$c(3,2)=3$，而 $S(3,1)=1$、$S(3,2)=3$。

加入第 n 个元素：第一类可以自成一圈，或者插在已有 $n-1$ 个元素后；第二类可以自成一个块，或者加入已有 k 个块。因此

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\qquad S(n,k)=S(n-1,k-1)+kS(n-1,k),$$

$$s(n,k)=s(n-1,k-1)-(n-1)s(n-1,k).$$

三者的 $(0,0)$ 项均为 1；$n>0$ 的第零列为 0；$k>n$ 或任一下标为负时为 0。特别地 $c(n,n)=S(n,n)=1$，$n\ge 1$ 时 $c(n,1)=(n-1)!$、$S(n,1)=1$。

记 $x^{\underline{k}}=x(x-1)\cdots(x-k+1)$，$x^{\overline{k}}=x(x+1)\cdots(x+k-1)$，零次幂均为 1。三种基底转换是

$$x^{\overline{n}}=\sum_{k=0}^{n}c(n,k)x^k,\qquad x^{\underline{n}}=\sum_{k=0}^{n}s(n,k)x^k,\qquad x^n=\sum_{k=0}^{n}S(n,k)x^{\underline{k}}=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)x^{\overline{k}}.$$

固定列的指数生成函数为

$$\sum_{n\ge k}c(n,k)\frac{x^n}{n!}=\frac{(-\log(1-x))^k}{k!},\quad \sum_{n\ge k}s(n,k)\frac{x^n}{n!}=\frac{(\log(1+x))^k}{k!},\quad \sum_{n\ge k}S(n,k)\frac{x^n}{n!}=\frac{(e^x-1)^k}{k!}.$$

按循环/非空集合的标号集合构造可分别得到前后两式。第二类还有普通生成函数 $\sum_{n\ge k}S(n,k)x^n=x^k/\prod_{j=1}^{k}(1-jx)$。双变量指数生成函数分别为 $\sum c(n,k)y^kx^n/n!=(1-x)^{-y}$、$\sum S(n,k)y^kx^n/n!=\exp(y(e^x-1))$。这些等式先在有理数形式幂级数中成立；模素数 p 计算到 N 次时需 $N<p$，确保出现的整数和阶乘可逆。以下快速实现还须满足 NTT 长度限制。

5.5.1.1 行列算法的选择与推导

第一类整行用 $F_n(x)=x^{\overline{n}}$，递归一次求 F_m，再用 $F_{2m}(x)=F_m(x)F_m(x+m)$；奇数长度额外乘 $x+2m$。泰勒平移用逆阶乘卷积，递推 $T(n)=T(\lfloor n/2\rfloor)+O(n\log n)$，所以时间 $O(n\log n)$、空间 $O(n)$。第二类整行使用容斥 $S(n,k)=\sum_{i=0}^{k}(i^n/i!)((-1)^{k-i}/(k-i)!)$，一次卷积获得前缀。$0^0=1$ 单独处理，正底数的指数才对 $p-1$ 取模。

整列先取出最低次 x^k，只算 $m=N-k+1$ 项：第一类的单位常数项级数 $A(x)=\sum_{j\ge 0}x^j/(j+1)$，第二类为 $A(x)=\sum_{j\ge 0}x^j/(j+1)!$。用 $A^k=\exp(k\log A)$ 求幂，再令 $c(n,k)$ 或 $S(n,k)=[x^{n-k}]A^k\cdot n!/k!$。从 $n=k$ 的倍率 1 逐次乘 $n+1$，无需另求 $k!$。时间 $O(N+m\log m)$、空间 $O(N)$；$k=0$ 和 $k>N$ 直接按边界返回。

5.5.2 无符号第一类斯特林数：同一行

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_first_row<mod,primitive>(n) 返回无符号 c(n,0..n)，默认模 998244353、原根 3。c(n,k) 为 n 元排列恰有 k 个循环的个数。n≥0，n+1≤NTT 的 max_size；n=0 返回 {1}。倍增配合多项式平移计算 x(x+1)⋯(x+n-1)，O(n log n) 时间、O(n) 空间。带符号版本再乘 (-1)^(n-k)。依赖 NttConvolution；模 167772161 可用 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_first_row(int n)
{
    using N = NttConvolution<mod, primitive>;
    assert(n >= 0 && n < N::max_size);
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    Poly fact(n + 1), inv(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
    inv[n] = fact[n].inv();
    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
    function<Poly(int)> solve = [&](int len) -> Poly
    {
        if (len == 0) return {1};
        int m = len / 2;
        Poly f = solve(m);
        Poly a(m + 1), b(m + 1);
        Z power = 1;
        for (int i = 0; i <= m; i++)
        {
            a[m - i] = f[i] * fact[i];
            b[i] = power * inv[i];
            power = power * m;
        }
        auto c = N::multiply(a, b);
        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
        f = N::multiply(f, a);
        if (len % 2)
        {
            f.push_back(0);
            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
            f[0] = f[0] * (len - 1);
        }
        return f;
    };
    return solve(n);
}
```

模板题使用：Luogu P5408

题意：输出无符号第一类斯特林数 c(n,0..n)，即 n 元排列按循环个数计数。本题模数为 167772161，须显式传模板参数；无需乘交替符号。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、stirling_first_row（第 190 页，5.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5408

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_first_row<167772161>(n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

5.5.3 无符号第一类斯特林数：同一列

代码依赖：FpsPower（第 163 页，4.7.3 节）、ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_first_column(N,k) 返回无符号 c(0..N,k)，固定模 998244353。N>=0、N<2^22、k>=0；k>N 返回全零，k=0 仅第零项为 1。取生成函数的最低 x^k 后，用 FpsPower 求剩余单位级数的 k 次幂，逐项乘 n!/k!；第一类若需有符号数，另乘 (-1)^(n-k)。令 m=N-k+1，合法非零列时间 O(N+m log m)，空间 O(N)。所有分母小于模数；NTT 工作长度由 N 限制保证。依赖 FpsPower 及其逆、对数、指数和 NTT 链，不能直接改用 167772161。

```
vector<ModInt<998244353>> stirling_first_column(int n, int k)
{
    using F = FpsPower;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n)
        return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    vector<Z> a(n - k + 1);
    vector<Z> inv(n - k + 2);
    inv[1] = 1;
    for (int i = 2; i < (int)inv.size(); i++)
        inv[i] = Z(F::mod - F::mod / i) * inv[F::mod % i];
    for (int i = 0; i < (int)a.size(); i++)
        a[i] = inv[i + 1];
    auto power = F::power(a, to_string(k), a.size());
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = power[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Signed First Stirling Fixed K

题意：输入 N K，0<=K<=N<=500000，模 998244353；只输出 n=K..N 的有符号第一类 s(n,K)。库函数返回无符号 c(n,K)，逐项乘 (-1)^(n-K)，勿遗漏符号。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、FpsFunctions（第 160 页，4.7.2 节）、FpsPower（第 163 页，4.7.3 节）、stirling_first_column（第 192 页，5.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_first_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_first_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        if ((i - k) % 2)
            value = decltype(value)(0) - value;
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

5.5.4 第二类斯特林数：同一行前缀

代码依赖：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_second_row<mod,primitive>(n,m) 返回 S(n,0..m)，默认 mod=998244353、primitive=3。模数必须是素数，primitive 是原根。n 支持完整 unsigned long long；2m+1 不能超过 NTT 的 max_size。S(n,k) 为无标号非空集合划分数；n=0 时仅第零项为 1，k>n 为零。一次卷积配合逆阶乘，O(m log m + m log mod) 时间、O(m) 空间。依赖 NttConvolution；模 167772161 可直接使用模板参数 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
{
    using N = NttConvolution<mod, primitive>;
    assert(m >= 0 && m <= (N::max_size - 1) / 2);
    using Z = typename N::Z;
    typename N::Poly a(m + 1), b(m + 1);
    b[0] = 1;
    for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
    b[m] = b[m].inv();
    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
    for (int i = 0; i <= m; i++)
    {
        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
        a[i] = value * b[i];
        if (i % 2) b[i] = Z(0) - b[i];
    }
    auto answer = N::multiply(a, b);
    answer.resize(m + 1);
    return answer;
}
```

模板题使用：Luogu P5395

题意：输出第二类斯特林数 S(n,0..n)，即把 n 个不同元素分成各个数目的非空无标号集合的方案数。本题模数为 167772161，须显式传模板参数。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、stirling_second_row（第 194 页，5.5.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5395

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_second_row<167772161>(n, n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

5.5.5 第二类斯特林数：同一列

代码依赖：FpsPower（第 163 页，4.7.3 节）、ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

stirling_second_column(N,k) 返回 S(0..N,k)，固定模 998244353。N>=0、N<2^22、k>=0；k>N 返回全零，k=0 仅第零项为 1。取生成函数的最低 x^k 后，用 FpsPower 求剩余单位级数的 k 次幂，逐项乘 n!/k!；令 m=N-k+1，合法非零列时间 O(N+m log m)，空间 O(N)。所有分母小于模数；NTT 工作长度由 N 限制保证。依赖 FpsPower 及其逆、对数、指数和 NTT 链，不能直接改用 167772161。

```
vector<ModInt<998244353>> stirling_second_column(int n, int k)
{
    using F = FpsPower;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n)
        return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    int m = n - k + 1;
    vector<Z> inverse_fact(m + 1, 1), a(m);
    for (int i = 1; i <= m; i++)
        inverse_fact[i] = inverse_fact[i - 1] * i;
    inverse_fact[m] = inverse_fact[m].inv();
    for (int i = m; i > 0; i--)
        inverse_fact[i - 1] = inverse_fact[i] * i;
    for (int i = 0; i < m; i++)
        a[i] = inverse_fact[i + 1];
    auto power = F::power(a, to_string(k), m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = power[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Second Stirling Fixed K

题意：输入 N K，0<=K<=N<=500000，模 998244353；只输出 n=K..N 的第二类 S(n,K)。K=0 时输出首项 1、其余 0；不输出 n<K 的前导零。

所需模板：ModInt（第 24 页，3.2.2 节）、NttConvolution（第 122 页，4.2.1 节）、FpsInverse（第 142 页，4.5.1 节）、FpsFunctions（第 160 页，4.7.2 节）、FpsPower（第 163 页，4.7.3 节）、stirling_second_column（第 195 页，5.5.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_second_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_second_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

5.5.6 斯特林反演与应用

对下标从 0 开始的任意序列，

$$b_n=\sum_{k=0}^{n}S(n,k)a_k \quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}s(n,k)b_k=\sum_{k=0}^{n}(-1)^{n-k}c(n,k)b_k.$$

若正变换使用无符号第一类，则相应逆变换为

$$b_n=\sum_{k=0}^{n}c(n,k)a_k \quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)b_k.$$

符号不能省略，也不能把正逆公式中的 n,k 对调。将 $x^n=\sum_k S(n,k)x^{\underline{k}}$ 再展开下降阶乘并比较 x^j 系数，得到 $\sum_{k=j}^{n}S(n,k)s(k,j)=\delta_{n,j}$。两矩阵均为单位下三角矩阵，故这个右逆也是左逆，即 $\sum_{k=j}^{n}s(n,k)S(k,j)=\delta_{n,j}$，证明上述第一对公式。用对角矩阵 $D_{n,n}=(-1)^n$，有 $s=DcD$，得到第二对公式。这些逆关系在整数上成立，取任意模数仍成立；只有快速行列算法的实现要求素数和逆元。

5.5.6.1 手算检查与建模用法

取 $a_0,a_1,a_2,a_3=(1,2,3,4)$，第二类变换为 $b=(1,2,5,15)$。逆变换检查 $a_2=b_2-b_1=3$，$a_3=b_3-3b_2+2b_1=4$。以 k 为块数的无标号集合划分，若给一个含 k 块的划分赋权 a_k，总权即 $b_n=\sum_k S(n,k)a_k$；已知这些总权时，可以用带符号第一类恢复 a_n。例如 $a_k=1$ 得到 Bell 数 b_n，逆变换应恢复全 1 序列。若每个块还要独立涂一种颜色，则取 $a_k=q^k$。

在指数生成函数层面，令 $A(x)=\sum a_nx^n/n!$、$B(x)=\sum b_nx^n/n!$，第一对公式等价于 $B(x)=A(e^x-1)$ 与 $A(x)=B(\log(1+x))$。这是形式复合，不能把它误写成逐项乘积或一次普通卷积。四种快速行列模板不自动给出任意整个序列的快速斯特林变换；逐行点积适合少量行或小规模示例，不能冒称 $O(N\log N)$ 的完整变换。

5.5.6.2 题目协议与验证范围

整行的洛谷 P5408/P5395 使用素数 167772161；本节整列函数固定 998244353，不能只改调用处的类型就用于 P5409/P5396。Library Checker 第一类 fixed-k 题要求**有符号** $s(n,k)$，第二类要求 $S(n,k)$；两题只输出 $n=k,\dots,N$。紧随各函数的完整用法分别展示这些差异。原整行实现的洛谷历史 AC：第一类 P5408与第二类 P5395。新增整列已在 Library Checker 以 C++20 实际 AC：第一类 409999（4457ms）、第二类 410000（4407ms），均 14 点全过，时限 10 秒。递推对拍 69158 项、反演矩阵恒等式与 50 万规模边界通过，头文件及精确抄写版本均通过普通与 ASan/UBSan 检查。本轮整行也以当前源码通过 Library Checker：第一类 410002（589ms）、第二类 410001（392ms）。性能提醒：本地同编译器 C++17/O2、最大规模小 k 输入交替各 3 次，整列中位耗时约为官方参考的 6.06 倍（第一类）与 4.92 倍（第二类），超过 80% 慢速提醒线。复用通用幂级数便于抄写，但常数较大；此比较不是 OJ 排名。算法及公式参考OI Wiki 斯特林数，本库完整状态见 issue 18 与 verification/stirling-columns*.json。

5.6 分拆数

5.6.1 整数分拆与重数上限

Partitions(N,mod) 求无序正整数分拆，0≤N<INT_MAX、1≤mod≤INT_MAX，p[n] 为不限制重数的分拆数；p[0]=1%mod。预处理 O(N$\sqrt{N}$) 时间、O(N) 空间；模数不要求素数。limited(n,k) 限制每个正整数最多出现 k 次，0≤n≤N、0≤k≤INT_MAX；k=0 时仅空分拆存在，k=1 为互异分拆。单次 O(1+$\sqrt{(n/(k+1))}$)，额外空间 O(1)。p/mod 只读；中间五边形数与累加采用 long long。

```
struct Partitions
{
    int mod;
    vector<int> p;

    Partitions(int n, int mod) : mod(mod), p(n + 1)
    {
        assert(n >= 0 && mod >= 1);
        p[0] = 1 % mod;
        for (int i = 1; i <= n; i++)
        {
            long long value = 0;
            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
            {
                long long a = j * (3 * j - 1) / 2;
                long long b = j * (3 * j + 1) / 2;
                int sign = j % 2 ? 1 : -1;
                value += 1LL * sign * p[i - a];
                if (b <= i) value += 1LL * sign * p[i - b];
            }
            p[i] = (value % mod + mod) % mod;
        }
    }

    int limited(int n, int k) const
    {
        assert(0 <= n && n < (int)p.size() && k >= 0);
        long long step = 1LL + k;
        long long value = p[n];
        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
        {
            long long a = step * (j * (3 * j - 1) / 2);
            long long b = step * (j * (3 * j + 1) / 2);
            int sign = j % 2 ? -1 : 1;
            value += 1LL * sign * p[n - a];
            if (b <= n) value += 1LL * sign * p[n - b];
        }
        return (value % mod + mod) % mod;
    }
};
```

模板题使用：Library Checker partition_function

题意：输出 0..N 的全部整数分拆数，模 998244353，N≤500000。构造 Partitions 后直接读 p；p[0]=1 表示空分拆。五边形数递推 O(N$\sqrt{N}$) 时间、O(N) 空间，不是有序拆分。此用法不调用 limited。

所需模板：Partitions（第 198 页，5.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/partition_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Partitions part(n, 998244353);
    for (int i = 0; i <= n; i++) cout << part.p[i] << (i == n ? '\n' : ' ');
}
```

应用题补充：Luogu P6189 [NOI Online #1 入门组] 跑步

题意：正整数非增序列的总和为 n，等价于 n 的无序分拆。n≤10^5，1≤p<2^{30} 且不保证素数；直接构造 Partitions(n,p)，输出 p[n]。这是比赛应用，单独记录，不替代正式模板题。

所需模板：Partitions（第 198 页，5.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p;
    cin >> n >> p;
    Partitions part(n, p);
    cout << part.p[n] << '\n';
}
```

5.6.2 整数分拆：重数限制的差一问题

实现见第 5.6.1 节（第 198 页）。分拆不区分加数顺序，部分均为正整数。无约束生成函数 $P(z)=\prod_{i\ge1}(1-z^i)^{-1}$，空分拆给出 $p(0)=1$。五边形数定理为

$$\prod_{i\ge1}(1-z^i)=1+\sum_{j\ge1}(-1)^j\left(z^{j(3j-1)/2}+z^{j(3j+1)/2}\right).$$

与 $P(z)$ 相乘后比较系数，得到 $p(n)$ 的递推，符号按 $+,+,-,-,+,+,\dots$ 成对出现。每项只依赖更小的下标，每个 n 有 $O(\sqrt{n})$ 个非零偏移。实现先用 64 位累加，再归一化到模数范围；无需逆元，合数模数与模数 1 均可使用。

每个正整数最多出现 k 次的生成函数是

$$\prod_{i\ge1}(1+z^i+\cdots+z^{ki})=P(z)\prod_{i\ge1}(1-z^{(k+1)i}).$$

因此用已有 p 表与放大 $k+1$ 倍的五边形偏移相乘，符号为 $-,-,+,+,\dots$。本库 `limited(n,k)` 的上限包含 k；$k=0$ 时非零 n 无解，$k=1$ 为互异分拆。kuangbin 的注释写“不超过 k 个”，但其 `solve(n,k)` 使用放大 k 倍的公式，实际表示每种数少于 k 次，对应本库 `limited(n,k-1)`（$k\ge1$）。HDU 原题页面本次无法读取，不能由这条注释推断原题措辞。

两套实现以任意精度完全背包验证到 1000，以逐种部分、逐个重数枚举验证有界情况到 100，覆盖素数与合数模数、0/1 重数限制和大于总和的限制；另检查 $N=100000$ 的边界。普通与 ASan/UBSan 均通过，暂无在线 AC。

Library Checker 的分拆函数要求输出 $p(0),\dots,p(N)$，$N\le500000$，完整用法见第 5.6.1 节（第 199 页）；固定官方 11 组数据的本地 checker 已通过。P6189 将非增跑步距离序列对应到无序分拆，模数不保证素数，用法见第 5.6.1 节（第 199 页）。它是比赛应用，不替代正式模板题。额外独立参考将部分分成不超过 B 与大于 B 两组：前者完全背包，后者按恰好 j 个部分维护 $g_j(s)=g_j(s-j)+g_{j-1}(s-B-1)$，再卷积。取 $B=\lfloor\sqrt{N}\rfloor$，时间 $O(N\sqrt{N})$、滚动数组空间 $O(N)$，不复用被测的五边形数递推；覆盖 $N=10^5$ 的合数模数与模数 1。这两份用法只调用构造和 p 表，不代表 limited 接口已有正式题的线上验证。

来源：kuangbin 2.11；https://oi-wiki.org/math/combinatorics/partition/。

5.6.2.1 三个容易混淆的限制

令 $A_k(n)$ 为 n 的分拆中每个部分不超过 k 的个数。Ferrers 图每行长度为一个部分；交换行列是自身的逆变换，交换最大部分与部分总数。因此 $A_k(n)$ 也等于至多 k 个部分的分拆数，但两类分拆集合通常不同。其生成函数为 $\prod_{i=1}^{k}(1-z^i)^{-1}$，可用部分大小 $1,\dots,k$ 的完全背包计算。

恰好 k 个正部分时，从每个部分减去 1，删除零，得到 $n-k$ 的至多 k 部分分拆；反向用零补足到 k 项，再每项加 1，故答案是 $A_k(n-k)$。若还要求这 k 个部分互异，按非增序写为 $\lambda_1 > \cdots > \lambda_k$，减去阶梯 $(k, k-1, \dots, 1)$，得到弱降非负序列，答案为

$$A_k\left(n - \frac{k(k+1)}{2}\right).$$

以上默认 $n, k \geq 0$，负目标和的答案为零；$A_0(0) = 1$，$A_0(n) = 0$ $(n > 0)$。计算阶梯和时须先扩宽乘法，int 范围的 k 可写 `1LL*k*(k+1LL)/2`。空分拆是恰好零个部分的唯一对象。

这不同于本库 `limited(n,k)`：它限制每个值的出现次数，不限制部分总数。例如 $n = 5, k = 2$：至多两部分有 $(5), (4,1), (3,2)$ 共 3 种；每部分不超过 2 也有 3 种，恰好两部分只有 2 种；重数限制的整数计数为 5，`limited(5,2)` 返回它对构造模数的余数。对于 $n = 6, k = 2$，恰好两部分有 3 种，其中互异的只有 2 种。不能只看题面出现“不超过 k”就调用同一个接口。

5.6.2.2 重数限制与禁止倍数部分

设 $d = k + 1 \geq 1$，则

$$\prod_{i\geq 1} \frac{1-z^{di}}{1-z^i} = \prod_{\substack{i\geq 1 \\ d\nmid i}} \frac{1}{1-z^i}.$$

左边限制每个值至多出现 k 次，右边只允许不被 d 整除的部分，次数不限。等式来自消去分母中下标为 d 倍数的因子。取 z^n 系数时只涉及有限多个因子，故这是形式级数恒等式，不要求数值收敛。因此可用排除 d 倍数面值的完全背包独立核对 `limited`。$k = 1$ 给出互异分拆数等于奇数部分分拆数；$k = 0$ 则所有正部分被禁止，只剩空分拆。若 $k \geq n$，重数限制不再生效。以上整数计数等式可对任意正模数取模，不需模逆元。生成函数与奇数/互异特例可参考 MIT 18.212 第 20 讲；这里将消因子推导写成一般 d。原五边形数定理保留为引用定理，本节不展开 Franklin 对合证明。

本节的 A_k、恰好部分数及阶梯变换是建模公式，不是新增的 `Partitions` 成员函数。

5.7 Pólya 计数

5.7.1 Burnside 平均：任意模数下的精确除法

BurnsideAverage(g,p)：群阶 g>0，目标模数 p>0，要求 g*p<=LLONG_MAX/2（先用 int128 检查）。mod=g*p 供不动点计数使用；add(fixed,multiplicity=1) 累加非负 long long 计数与重数，内部 int128 乘法。power(a,k) 求 a^k 模 mod，a,k 非负，0^0=1。result() 返回 sum/g，支持 p=1 与 g 不可逆；sum 必须是对全部群元素的不动点计数之和模 mod，不是先模 p 后的计数。调用方保证确为群作用并完整枚举群元素或共轭类型，重数合计 g；不会生成闭包或验证群公理。字段只读，add 后可继续累计，只有完整合法总和才能 result。每次 add/result 为 O(1)，power 为 O(log(k+1))，空间 O(1)。

```
struct BurnsideAverage
{
    using ll = long long;
    ll order, mod, sum = 0;

    BurnsideAverage(ll g, ll p) : order(g)
    {
        assert(g > 0 && p > 0 && __int128(g) * p <= LLONG_MAX / 2);
        mod = g * p;
    }

    ll power(ll a, long long k) const
    {
        assert(a >= 0 && k >= 0);
        a %= mod;
        ll ans = 1 % mod;
        while (k)
        {
            if (k & 1) ans = __int128(ans) * a % mod;
            a = __int128(a) * a % mod;
            k >>= 1;
        }
        return ans;
    }

    void add(ll fixed, ll multiplicity = 1)
    {
        assert(fixed >= 0 && multiplicity >= 0);
        sum = (sum + __int128(fixed) * multiplicity) % mod;
    }

    ll result() const
    {
        assert(sum % order == 0);
        return sum / order;
    }
};
```

应用题补充：P1446 [HNOI2008] Cards

题意：三色各用 r/b/g 张，均 <=20；m<=60，m+1<p<100 且 p 为素数。输入置换与恒等构成完整群；补恒等并去重，绝不能把任意生成元直接平均。每循环整体染一种颜色，用二维背包计红蓝数量，绿色由已处理点数推得；中间模群阶 *p。

所需模板：BurnsideAverage（第 202 页，5.7.1 节）、permutation_cycles（第 205 页，5.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1446

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int r, b, g, m, mod;
    cin >> r >> b >> g >> m >> mod;
    int n = r + b + g;
    vector<vector<int>> group(m, vector<int>(n));
    for (auto &p : group)
    {
        for (int &x : p)
        {
            cin >> x;
            x--;
        }
    }
    vector<int> identity(n);
    iota(identity.begin(), identity.end(), 0);
    group.push_back(identity);
    sort(group.begin(), group.end());
    group.erase(unique(group.begin(), group.end()), group.end());
```

继续处理操作（接上页同一 main）：

```
    BurnsideAverage avg(group.size(), mod);
    for (auto &p : group)
    {
        vector<vector<long long>> dp(r + 1, vector<long long>(b + 1));
        dp[0][0] = 1;
        int used = 0;
        for (int len : permutation_cycles(p))
        {
            vector<vector<long long>> next(r + 1, vector<long long>(b + 1));
            for (int i = 0; i <= r; i++)
            {
                for (int j = 0; j <= b; j++)
                {
                    if (i + len <= r) next[i + len][j] += dp[i][j];
                    if (j + len <= b) next[i][j + len] += dp[i][j];
                    if (used - i - j + len <= g) next[i][j] += dp[i][j];
                }
            }
            for (auto &row : next)
            {
                for (auto &value : row) value %= avg.mod;
            }
            dp.swap(next);
            used += len;
        }
        avg.add(dp[r][b]);
    }
    cout << avg.result() << '\n';
}
```

5.7.2 置换的循环长度分解

permutation_cycles(p) 返回各不交循环长度, 按循环最小位置升序。p 是 0-based 的合法置换, n<INT_MAX; 空置换返回空 vector。本函数不校验输入，非置换不允许。O(n) 时间与空间；环上循环迭代不是 DFS。固定染色在每个循环内同色，命名颜色不限数量时为 colors^ 循环数；限定各色用量时以循环长度做背包。

```
vector<int> permutation_cycles(const vector<int> &p)
{
    int n = p.size();
    vector<int> seen(n), lengths;
    for (int i = 0; i < n; i++)
    {
        if (seen[i]) continue;
        int u = i, len = 0;
        do
        {
            seen[u] = 1;
            len++;
            u = p[u];
        } while (u != i);
        lengths.push_back(len);
    }
    return lengths;
}
```

接口用法演示：API 显式置换群染色

演示约定：输入 n size colors mod，然后 size 个 0-based 置换，必须完整列出群作用（包括恒等），不是生成元。统计循环数，累加 colors^ 循环数并平均；允许 n=0，空集合染色数为 1。

所需模板：BurnsideAverage（第 202 页，5.7.1 节）、permutation_cycles（第 205 页，5.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/polya/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, size;
    long long colors, mod;
    cin >> n >> size >> colors >> mod;
    BurnsideAverage avg(size, mod);
    for (int i = 0; i < size; i++)
    {
        vector<int> p(n);
        for (int &x : p) cin >> x;
        avg.add(avg.power(colors, permutation_cycles(p).size()));
    }
    cout << avg.result() << '\n';
}
```

5.7.3 Pólya 环染色：旋转与翻转

代码依赖：BurnsideAverage（第 202 页，5.7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

necklace_colorings(n,colors,mod,reflection=false)：n>=1 的环，每位置可选 colors>=0 种命名颜色，返回本质不同染色数模 mod。n 为 int；mod>0，群阶 g 为 n 或 2n，须 g*mod<=LLONG_MAX/2。默认只视旋转相同；reflection=true 也视翻转相同。允许颜色不用完，不能用于相邻异色、固定颜色用量或颜色重命名模型。n=1/2 仍适用，二面体抽象群作用可不忠实。用 BurnsideAverage 模 g*mod 后整除，无需模逆元；因子分解与枚举约数 O(sqrt(n))，每约数 O(omega(n)+log(n))，额外 O(omega(n))。

```
long long necklace_colorings(int n, long long colors, long long mod,
                             bool reflection = false)
{
    assert(n >= 1 && colors >= 0);
    BurnsideAverage avg((reflection ? 2LL : 1LL) * n, mod);
    vector<int> primes;
    int x = n;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p != 0) continue;
        primes.push_back(p);
        while (x % p == 0) x /= p;
    }
    if (x > 1) primes.push_back(x);
    auto add = [&](int d)
    {
        long long phi = d;
        for (int p : primes)
        {
            if (d % p == 0) phi = phi / p * (p - 1);
        }
        avg.add(avg.power(colors, n / d), phi);
    };
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d != 0) continue;
        add(d);
        if (d != n / d) add(n / d);
    }
    if (reflection)
    {
        if (n & 1) avg.add(avg.power(colors, n / 2 + 1), n);
        else
        {
            avg.add(avg.power(colors, n / 2), n / 2);
            avg.add(avg.power(colors, n / 2 + 1), n / 2);
        }
    }
    return avg.result();
}
```

模板题使用：P4980 【模板】Pólya 定理

题意：t<=1000，1<=n<=10^9；n 个点用 n 种命名颜色，仅旋转等价，模 10^9+7。直接调用旋转环计数。

所需模板：BurnsideAverage（第 202 页，5.7.1 节）、necklace_colorings（第 206 页，5.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4980

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        cout << necklace_colorings(n, n, 1000000007) << '\n';
    }
}
```

接口用法演示：API 环与手链命名颜色计数

演示约定：输入 n colors mod，输出仅旋转、旋转加翻转两种计数。遵守核心乘积界；特别可测试 mod 与群阶不互质和 n=1/2。

所需模板：BurnsideAverage（第 202 页，5.7.1 节）、necklace_colorings（第 206 页，5.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/math/combinatorics/polya/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long colors, mod;
    cin >> n >> colors >> mod;
    cout << necklace_colorings(n, colors, mod) << '\n';
    cout << necklace_colorings(n, colors, mod, true) << '\n';
}
```

5.7.4 群作用计数：先确定什么算同一个方案

把有标号的合法对象组成有限集合 X，把题目允许的等价变换组成有限群 G。作用必须把 X 映回 X，恒等变换不改变对象，复合变换与逐次操作一致。一个轨道 $Gx = \{gx : g \in G\}$ 就是一个最终方案。先确定对象与变换，再计算；不能因为题目出现“圆环”就除以长度。

- **只允许改变起点：**长度 $n \geq 1$ 的项链商去 n 个旋转，群为 C_n。
- **还允许翻面：**手链商去旋转与反射；$n \geq 3$ 时是阶 $2n$ 的二面体群。反射是否等价必须由题意决定，不能把项链答案再机械除以 2。
- **颜色仍有名字：**红、蓝互换通常是不同方案；若颜色也可重命名，作用已经改变，不能沿用本节的 k 种具名颜色公式。
- **标记与约束也要保留：**指定位置、位置权值或障碍可能破坏旋转对称。只有保持整个合法对象集合的变换，才能直接在这个集合上使用 Burnside。

记稳定子 $G_x = \{g : gx = x\}$，不动点集 $\mathrm{Fix}_X(g) = \{x : gx = x\}$。轨道–稳定子公式与 Burnside 引理为

$$|Gx| = \frac{|G|}{|G_x|}, \qquad |X/G| = \frac{1}{|G|} \sum_{g \in G} |\mathrm{Fix}_X(g)|.$$

证明只需双计数 (g, x) 且 $gx = x$ 的配对：一个轨道贡献 $\sum_{x \in Gx} |G_x| = |G|$，于是所有不动点数之和等于轨道数乘群阶。只有每个对象的稳定子都平凡时，才可直接用 $|X|/|G|$。例如 4 位二进制项链有 6 类，代表为 0000、1111、0001、0111、0011、0101；轨道大小依次为 $1, 1, 4, 4, 4, 2$，所以 $2^4/4 = 4$ 并非答案。理论说明可参见 Ben Lynn：Burnside's Lemma。

5.7.5 项链与手链：从置换循环推公式

位置暂用 $0, \ldots, n-1$，所有下标模 n，$n \geq 1$、$k \geq 1$。若位置置换 g 有 $c(g)$ 个循环，不动染色在每个循环上必须同色，不同循环独立选色，故不动染色数为 $k^{c(g)}$。

旋转。旋转 t 步为 $i \mapsto i + t$，有 $\gcd(n, t)$ 个循环，每个循环长度为 $n/\gcd(n, t)$；约定 $\gcd(n, 0) = n$。按循环长度 $d \mid n$ 归类，有 $\varphi(d)$ 个旋转，故旋转不动点总和与项链数为

$$R(n, k) = \sum_{t=0}^{n-1} k^{\gcd(n,t)} = \sum_{d \mid n} \varphi(d) k^{n/d}, \qquad N(n, k) = R(n, k)/n.$$

规模不大时逐个枚举 t 最容易抄；规模大时分解 n，枚举约数并计算各自的 $\varphi(d)$，减少求幂次数，别把 $O(n)$ 循环误当成约数枚举。

反射。写成 $i \mapsto t - i$；单点循环满足 $2i \equiv t \pmod{n}$，其余位置组成二元循环。这直接给出奇偶分类：

长度与反射类型	个数	单点、二元循环数	每个反射的不动染色数
n 奇数	n	$1,\ (n-1)/2$	$k^{(n+1)/2}$
n 偶数，穿过对顶点	$n/2$	$2,\ (n-2)/2$	$k^{n/2+1}$
n 偶数，穿过对边中点	$n/2$	$0,\ n/2$	$k^{n/2}$

于是手链数为

$$B(n, k) = \frac{R(n, k) + F(n, k)}{2n}, \qquad F(n, k) = \begin{cases} nk^{(n+1)/2}, & n \text{ 奇数}, \\ \dfrac{n}{2}\left(k^{n/2+1} + k^{n/2}\right), & n \text{ 偶数}. \end{cases}$$

循环分类亦可参见 Ben Lynn：The Cycle Index Polynomial。

偶数例：6 个位置、2 色。 $R = 2^6 + 2^3 + 2 \cdot 2^2 + 2 \cdot 2 = 84$，项链数为 14。反射贡献为 $3 \cdot 2^4 + 3 \cdot 2^3 = 72$，手链数为 $(84 + 72)/12 = 13$。**奇数例：5 个位置、3 色。** $R = 3^5 + 4 \cdot 3 = 255$，项链数为 51；5 个反射各贡献 3^3，手链数为 $(255 + 135)/10 = 39$。

退化长度不能偷换分母。 $n=1$ 时答案均为 k；$n=2$ 时均为 $k(k+1)/2$，即选一个允许重复的无序颜色对。上述公式仍成立：把旋转与反射保留为 $2n$ 个形式群元素时，不同元素可能诱导同一位置置换，这是非忠实作用，重复置换必须连同权重保留。也可以把置换去重后对像群求平均，但分母必须同步改为不同置换的个数；不能分子去重而仍除以 $2n$。本节不定义空环 $n=0$；若 $n>0, k=0$，没有染色，答案为 0。

5.7.6 指定颜色库存：给每个循环分配整份颜色

设颜色 $1,\dots,k$ 的数量为 $a_1,\dots,a_k\ge 0$，总和为 n。固定库存对位置置换不变，可以在这个合法子集上取轨道。若 g 的循环长度依次为 $\ell_1,\dots,\ell_c$，则

$$|\operatorname{Fix}_{\boldsymbol{a}}(g)| = [x_1^{a_1}\cdots x_k^{a_k}]\prod_{j=1}^{c}\left(x_1^{\ell_j}+\cdots+x_k^{\ell_j}\right).$$

每个因子给一个完整循环选颜色；各循环仍是不同的位置集合，即使长度相同也不能再除阶乘。把各个 g 的系数相加，最后除以 $|G|$。这也是循环指标代换 $z_\ell \leftarrow \sum_i x_i^\ell$ 的含义，不必为了使用 Pólya 再写通用群类。加权推导参见 Ben Lynn：Pólya's Inventory Theorem。

等长旋转循环可直接化成多项式系数。循环长度为 d 时，若存在 $d\nmid a_i$，不动点数为 0；否则为

$$\frac{(n/d)!}{\prod_i (a_i/d)!}.$$

实际计算可依次选每种颜色所占的循环：从 $r=n/d$ 开始，累乘 $\binom{r}{a_i/d}$ 后令 $r\leftarrow r-a_i/d$。小于素数模数的预处理范围内可用 `Binomial<p> c(n)` 与 `c.choose(r,a[i]/d)`，见第 5.1.1 节（第 177 页）。若随后需要在 $M|G|$ 下算总和，这个模数通常是合数，不能继续用素数阶乘逆元；按第 5.1.3 节（第 179 页）选组合数算法，或使用以下只加法的 DP。

二色恰有 r 个红色：取 $[x^r]\prod_j(1+x^{\ell_j})$。初始化 $f_0(0)=1$，其他项为 0；加入一个长度 ℓ 的循环时

$$f_{j+1}(s) = f_j(s) + [s\ge \ell] f_j(s-\ell), \qquad 0\le s\le r.$$

可用一维数组从 r 降序更新到 ℓ，时间 $O(c(r+1))$、空间 $O(r+1)$；加法后取模，对任意正模数都适用。$r\notin[0,n]$ 时答案为 0。多色可维护前 $k-1$ 色的库存，最后一色由已处理总长度确定；任何一色超出库存就丢弃状态；状态数至多 $\prod_{i<k}(a_i+1)$，每个循环最多 k 个转移，先估内存再选它。

例：6 个位置，3 红 3 蓝。恒等旋转贡献 $\binom{6}{3}=20$；旋转 2、4 步各有两个长度 3 的循环，各贡献 2；其他非恒等旋转贡献 0。项链数为 $(20+2+2)/6=4$。穿顶点反射的多项式为 $(1+x)^2(1+x^2)^2$，x^3 系数为 4；穿边反射为 $(1+x^2)^3$，x^3 系数为 0。手链数为 $(24+3\cdot 4)/12=3$。

额外约束须重新数不动点。"相邻位置异色"对旋转反射不变，Burnside 仍成立，但循环不能独立选色：应把每个循环缩成一个点，原相邻关系变成约束边；出现自环就无解，再数该约束图的染色。"位置 0 必须红色"则通常不对旋转封闭，不能在该子集上直接平均。若问题要求统计哪些完整轨道存在满足条件的代表，须另建模，不能把存在性条件当成不变库存。

5.7.7 群阶不可逆：先提升模数，再作整数除法

设不动点数总和为整数 S，群阶为 h，已知 $h\mid S$，目标是 $(S/h)\bmod M$，$M\ge 1$。只有 $\gcd(h,M)=1$ 时，才能乘模逆元；仅检查 $M\nmid h$ 不够。通用办法是在模 $L=Mh$ 下算出标准余数 $s=S\bmod L$，然后做普通整数除法：

$$S=qMh+s \quad\Longrightarrow\quad h\mid s, \qquad S/h\equiv s/h \pmod M, \qquad 0\le s/h<M.$$

必须先合计所有项，再除以 h；每个不动点数通常不单独被 h 整除。例如 4 位二进制项链，$S=24, h=4, M=8$：先模 8 得到 0 已丢失信息；先模 32 得到 24，再除以 4 才得到正确的 6。

可直接编译的无库存限制示例。输入每行 `n k M flip`，`flip=0` 只计旋转，`flip=1` 还计反射，输出轨道数模 M。使用第 2.1 节（第 15 页）的 `Mod64::mul` 与 `Mod64::power`；赛时把下面的仓库 include 换为所抄的 `Mod64`。

```cpp
#include "src/compact/mod64.hpp"

using ull = unsigned long long;
using u128 = __uint128_t;

ull color_orbits(int n, ull k, ull m, bool flip)
{
    assert(n >= 1 && m >= 1);
    ull h = (flip ? 2ULL : 1ULL) * n;
    u128 wide = u128(m) * h;
    assert(wide <= numeric_limits<ull>::max());
    ull mod = (ull)wide, sum = 0;
    auto add = [&](ull count, ull cycles)
    {
        ull term = Mod64::mul(count, Mod64::power(k, cycles, mod), mod);
        sum = (ull)((u128(sum) + term) % mod);
    };
    for (int t = 0; t < n; t++) add(1, gcd(n, t));
    if (flip)
    {
        if (n & 1) add(n, n / 2 + 1);
        else
        {
            add(n / 2, n / 2 + 1);
            add(n / 2, n / 2);
        }
    }
    assert(sum % h == 0);
    return sum / h;
}

int main()
{
    int n, flip;
    ull k, m;
    while (cin >> n >> k >> m >> flip)
        cout << color_orbits(n, k, m, flip != 0) << '\n';
}
```

该示例要求 $1 \le n \le \mathrm{INT_MAX}, k$ 为无符号 64 位整数，$1 \le M$ 且 $Mh \le 2^{64}-1$；运行预算还须容纳 $O(n \log(n+1))$ 时间，额外空间 $O(1)$。没有跨测试状态，支持多测与 $M = 1$。先转宽类型再算 Mh，加法也在 128 位中进行；单有安全模乘并不能防止模数乘积或求和溢出。若 $Mh > 2^{64} - 1$，本函数不适用；不可截断后再传给 `Mod64`。可改用多精度整数在 Mh 下计算，或在预算允许时精确算出 S 后整除；仅把模数类型改成 128 位，仍不能保证两个 128 位数的乘积不溢出。

配套 `tests/math_knowledge_orbits.py` 从本节直接抽取 C++ 示例编译，对照小染色的规范轨道枚举；另用循环库存 DP 对照按库存分类的轨道，检查奇偶反射、$n = 1, 2$、零色、复合模数与 64 位边界。运行 `python3 tests/math_knowledge_orbits.py`，加 `--sanitize` 可在 AddressSanitizer/UndefinedBehaviorSanitizer 下运行同一批 C++ 案例。这是本地知识示例验证，不是新增 OJ AC，也不替代底层模板的完整回归。

第六章　线性代数

6.1　矩阵

6.1.1　模矩阵乘法与幂

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

ModMatrix<p>::multiply(a,b) 计算 n×k 与 k×m 的矩阵积，要求非空且每行尺寸一致。power(a,e) 接收非空 n×n 方阵和 unsigned long long 指数；e=0 返回单位阵。只使用加乘，p 为 ≥2 的 int 模数，允许合数。矩阵乘法 O(nkm)，矩阵幂 O(n^3 log(e+1))，额外空间 O(n^2)；复用 ModInt。

```
template <int mod> struct ModMatrix
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    static Matrix multiply(const Matrix &a, const Matrix &b)
    {
        assert(!a.empty() && !b.empty());
        int n = a.size(), m = b[0].size(), k = b.size();
        assert((int)a[0].size() == k);
        Matrix c(n, vector<Z>(m));
        for (int i = 0; i < n; i++)
            for (int t = 0; t < k; t++)
                for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
        return c;
    }

    static Matrix power(Matrix a, unsigned long long e)
    {
        int n = a.size();
        assert(n > 0 && (int)a[0].size() == n);
        Matrix r(n, vector<Z>(n));
        for (int i = 0; i < n; i++) r[i][i] = 1;
        for (; e; e >>= 1, a = multiply(a, a))
            if (e & 1) r = multiply(r, a);
        return r;
    }
};
```

模板题使用：Library Checker matrix_product

题意：计算 n×m 矩阵与 m×k 矩阵的乘积，模 998244353。输入元素已取模，按行输出 n×k 个结果。

所需模板：ModInt（第 24 页，3.2.2 节）、ModMatrix（第 211 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_product

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using M = ModMatrix<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    M::Matrix a(n, vector<M::Z>(m)), b(m, vector<M::Z>(k));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    for (auto &row : b)
        for (auto &x : row) cin >> x.v;
    auto c = M::multiply(a, b);
    for (auto &row : c)
        for (int j = 0; j < k; j++) cout << row[j].v << (j + 1 == k ? '\n' : ' ');
}
```

6.1.2 素数模矩阵求逆

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

matrix_inverse<mod>(a) 返回 optional 方阵；a 为 n×n 的 ModInt<mod> 矩阵，mod 必须是素数。可逆时返回逆矩阵，奇异时返回 nullopt；按值传入不改原矩阵，空方阵返回有值的空矩阵。每次同时对原矩阵和单位阵进行换行、倍乘、消元；必须同步交换两边的行。依赖 ModInt；$O(n^3 + n \log mod)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod>
optional<vector<vector<ModInt<mod>>>> matrix_inverse(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = a.size();
    vector<vector<Z>> b(n, vector<Z>(n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        b[i][i] = 1;
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        while (p < n && !a[p][i].v) p++;
        if (p == n) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        Z inv = a[i][i].inv();
        for (int k = i; k < n; k++) a[i][k] = a[i][k] * inv;
        for (int k = 0; k < n; k++) b[i][k] = b[i][k] * inv;
        for (int j = 0; j < n; j++)
            if (j != i && a[j][i].v)
            {
                Z f = a[j][i];
                for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
                for (int k = 0; k < n; k++) b[j][k] = b[j][k] - f * b[i][k];
            }
    }
    return b;
}
```

模板题使用：Luogu P4783

题意：给定 n 阶方阵，求模 1000000007 的逆矩阵；不可逆输出 No Solution。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 24 页，3.2.2 节）、matrix_inverse（第 213 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4783

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<1000000007>(a);
    if (!b)
        puts("No Solution");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

模板题使用：Library Checker inverse_matrix

题意：给定 n 阶方阵，求模 998244353 的逆矩阵；不可逆输出 -1。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 24 页，3.2.2 节）、matrix_inverse（第 213 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inverse_matrix

```
int main()
{
    using Z = ModInt<998244353>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<998244353>(a);
    if (!b)
        puts("-1");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

6.1.3 模 2 矩阵求逆（按位消元）

matrix_inverse_mod2(a) 接收 n 行长度 n 的 01 字符串，返回 optional<vector<string»。可逆时返回同样格式的逆矩阵，奇异返回 nullopt；空矩阵返回有值的空矩阵，不改输入。将原矩阵与单位阵拼成 2n 位一行，按 uint64_t 存储，换行后用异或消元；无需固定 bitset 上限，移位量始终在 0..63。O(n^2 ceil(n/64)) 时间，工作数组 O(n ceil(n/64)) 个机器字，输入输出 O(n^2) 字节。仅用于 F2，不处理一般模数。

```
optional<vector<string>> matrix_inverse_mod2(const vector<string> &a)
{
    int n = a.size(), words = (2LL * n + 63) / 64;
    vector<vector<uint64_t>> b(n, vector<uint64_t>(words));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        for (int j = 0; j < n; j++)
        {
            assert(a[i][j] == '0' || a[i][j] == '1');
            if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
        }
        b[i][(n + i) / 64] |= 1ULL << ((n + i) % 64);
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        uint64_t mask = 1ULL << (i % 64);
        while (p < n && !(b[p][i / 64] & mask)) p++;
        if (p == n) return nullopt;
        swap(b[p], b[i]);
        for (int j = 0; j < n; j++)
            if (j != i && (b[j][i / 64] & mask))
                for (int k = i / 64; k < words; k++) b[j][k] ^= b[i][k];
    }
    vector<string> ans(n, string(n, '0'));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if ((b[i][(n + j) / 64] >> ((n + j) % 64)) & 1) ans[i][j] = '1';
    return ans;
}
```

模板题使用：Library Checker inverse_matrix_mod_2

题意：输入 n 阶模 2 方阵，每行是无空格的 01 字符串。求逆矩阵，按同样形式逐行输出；奇异则输出 -1。

所需模板：matrix_inverse_mod2（第 216 页，6.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inverse_matrix_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    auto b = matrix_inverse_mod2(a);
    if (!b)
        cout << -1 << '\n';
    else
        for (auto &row : *b) cout << row << '\n';
}
```

6.1.4 Max-plus 矩阵幂

a[i][j] 为转移收益，不存在的边为 neg；power(k) 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^119。O(n^3 log k)；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
struct MaxPlusMatrix
{
    using I = __int128_t;
    static constexpr I neg = -(I(1) << 120);
    int n;
    vector<vector<I>> a;

    MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
    {
        if (identity)
            for (int i = 0; i < n; i++) a[i][i] = 0;
    }

    MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
    {
        assert(n == b.n);
        MaxPlusMatrix c(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                if (a[i][k] != neg)
                    for (int j = 0; j < n; j++)
                        if (b.a[k][j] != neg)
                            c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
        return c;
    }

    MaxPlusMatrix power(unsigned long long e) const
    {
        MaxPlusMatrix b = *this, r(n, true);
        for (; e; e >>= 1, b = b * b)
            if (e & 1) r = r * b;
        return r;
    }
};
```

应用题补充：CSES 1724 Graph Paths II

题意：有向图求从 1 到 n 恰好经过 k 条边的最小费用，不可达输出-1。1≤n≤100，1≤m≤n(n−1)，1≤k,c≤10^9。邻接矩阵初始全为不可达，不能设成单位阵；边 u→v 存-c，重边取最大值，再调用 power(k)。先判不可达哨兵，再把结果取负输出；有限答案至多 10^18。中间值含最后一次平方也满足本模板 128 位范围。时间 O(n^3 log(k+1))、空间 O(n^2)；这是负权转换应用，不是 max-plus 正式模板题。

所需模板：MaxPlusMatrix（第 218 页，6.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    unsigned long long k;
    if (!(cin >> n >> m >> k)) return 0;
    MaxPlusMatrix a(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        --u, --v;
        a.a[u][v] = max(a.a[u][v], -MaxPlusMatrix::I(c));
    }
    auto r = a.power(k);
    auto x = r.a[0][n - 1];
    // Official bounds imply each finite answer is in [1, 10^18].
    if (x == MaxPlusMatrix::neg) cout << -1 << '\n';
    else cout << (long long)(-x) << '\n';
}
```

6.2 行列式

6.2.1 素数模行列式

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

det_prime<p>(a) 输入 n×n 方阵，返回行列式模 p；p 必须是素数。允许奇异矩阵和零维矩阵，det(empty)=1。按行交换改变符号，主元求逆消元。$O(n^3)$ 时间、传值副本 $O(n^2)$ 空间，可传 move(a)。合数模数改用 determinant_mod。

```
template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = (int)a.size();
    Z ans = 1;
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        int p = i;
        while (p < n && !a[p][i].v) ++p;
        if (p == n) return 0;
        if (p != i)
        {
            swap(a[p], a[i]);
            ans = Z(0) - ans;
        }
        ans = ans * a[i][i];
        Z inv = a[i][i].inv();
        for (int j = i + 1; j < n; j++)
        {
            Z f = a[j][i] * inv;
            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
        }
    }
    return ans;
}
```

模板题使用：Library Checker matrix_det

题意：求方阵行列式，模 998244353。输入元素已取模；函数按值接收矩阵，示例用 move 交出矩阵避免额外复制。

所需模板：ModInt（第 24 页，3.2.2 节）、det_prime（第 220 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = ModInt<998244353>;
    int n;
    cin >> n;
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    cout << det_prime<998244353>(move(a)).v << '\n';
}
```

应用题补充：Luogu P4336 黑暗前的幻想乡：公司集合容斥

题意：n 点、n-1 家公司，每家恰选一边成树；不同公司的同端点边分别计数。公司集合容斥配合矩阵树；Gray 码每步增删一家。2<=n<=17，模 1000000007，时间 O(2^(n-1)n^3)，空间 O(n^3)。删去末点；lap 后续仍需使用，勿 move。空集贡献为 0。本地验证通过，线上待验证。

所需模板：ModInt（第 24 页，3.2.2 节）、det_prime（第 220 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4336

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    cin >> n;
    int k = n - 1;
    vector<vector<pair<int, int>>> edges(k);
    for (auto &group : edges)
    {
        int m;
        cin >> m;
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            group.push_back({u - 1, v - 1});
        }
    }
    vector<vector<Z>> lap(k, vector<Z>(k));
    Z ans = 0;
    int prev = 0;
    for (int mask = 1; mask < (1 << k); mask++)
    {
        int gray = mask ^ (mask >> 1);
        int bit = __builtin_ctz(unsigned(gray ^ prev));
        Z delta = (gray >> bit & 1) ? 1 : -1;
        for (auto [u, v] : edges[bit])
        {
            if (u < k)
                lap[u][u] = lap[u][u] + delta;
            if (v < k)
                lap[v][v] = lap[v][v] + delta;
            if (u < k && v < k)
            {
                lap[u][v] = lap[u][v] - delta;
                lap[v][u] = lap[v][u] - delta;
            }
        }
        Z ways = det_prime<1000000007>(lap);
        if ((k - __builtin_popcount(unsigned(gray))) & 1)
            ans = ans - ways;
        else
            ans = ans + ways;
        prev = gray;
    }
    cout << ans.v << '\n';
}
```

6.2.2 任意正模数行列式：欧几里得消元

determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。O(n^3 log mod) 时间、O(n^2) 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 223 页，6.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

6.2.3 精确整数行列式（Bareiss）

determinant_exact(a) 按值接收 n 阶方阵，0≤n<INT_MAX，返回带符号行列式，空矩阵为 1，不修改调用者矩阵。Z 必须支持精确有符号整数四则运算，推荐 Boost cpp_int；不是模整数或浮点类型。固定宽度仅在所有中间乘积、差和商均不溢出时可用。每轮选择非零主元并换行，分子必须先完整计算，再除以上一主元；由 Sylvester 恒等式保证整除。当前列无主元则返回 0。$O(n^3)$ 次大整数算术操作、$O(n^2)$ 个整数对象；实际时间和位存储取决于中间数位数，不能视为机器字常数成本。

```
template<class Z>
Z determinant_exact(vector<vector<Z>> a)
{
    int n = a.size();
    for (const auto &row : a)
        assert((int)row.size() == n);
    if (!n)
        return Z(1);
    Z prev = 1;
    int sign = 1;
    for (int k = 0; k + 1 < n; k++)
    {
        int p = k;
        while (p < n && a[p][k] == 0)
            p++;
        if (p == n)
            return Z(0);
        if (p != k)
        {
            swap(a[p], a[k]);
            sign = -sign;
        }
        Z pivot = a[k][k];
        for (int i = k + 1; i < n; i++)
        {
            for (int j = k + 1; j < n; j++)
            {
                Z value = a[i][j] * pivot - a[i][k] * a[k][j];
                a[i][j] = value / prev;
            }
            a[i][k] = 0;
        }
        prev = pivot;
    }
    return sign * a[n - 1][n - 1];
}
```

接口用法演示：精确整数行列式

演示约定：T 组；每组 n 及 n^2 个有符号十进制整数，输出精确行列式。n=0 输出 1。cpp_int 为 Boost 任意精度有符号整数，输入与中间结果受可用内存和时间限制；此处为 API 用法，无线上 AC 声明。

所需模板：determinant_exact（第 225 页，6.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = boost::multiprecision::cpp_int;
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        vector<vector<Z>> a(n, vector<Z>(n));
        for (auto &row : a)
            for (auto &x : row)
                cin >> x;
        cout << determinant_exact(move(a)) << '\n';
    }
    return 0;
}
```

6.3 线性基

6.3.1 64 位异或线性基

值类型 U 为 unsigned long long，完整支持 64 位。默认空基；重新赋值 XorBasis() 清空。insert/Insert 返回是否增秩；contains/Contains 判断可表示性（允许空子集，所以 0 总可表示）。query/Query(x=0) 返回 x 与所张成空间异或后的最大无符号值。kth/Kth(k,nonempty=true) 返回第 k 小的不同异或值，1-based，参数 k 为 __uint128_t，非法序号返回 nullopt。默认只允许非空输入子集，0 仅在 dependent 为真时出现；传 false 则包括空子集所得的 0。minimum/Minimum 同此约定，空输入的非空最小值不存在。merge/Merge 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。rank 是维数，dependent 表示存在非空零异或子集。插入、成员判断、最大值为 O(64)；合并 $O(64^2)$。kth 会在需要时 rebuild/Rebuild 规范化缓存，首次有效查询 $O(64^2)$，后续 O(64)；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```
struct XorBasis
{
    using U = unsigned long long;
    using Wide = __uint128_t;
    U a[64]{};
    vector<U> p;
    int rank = 0;
    bool dependent = false, dirty = false;

    bool insert(U x)
    {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rank++;
                    dirty = true;
                    return true;
                }
                x ^= a[i];
            }
        dependent = true;
        return false;
    }
```

```
    bool contains(U x) const
    {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1) x ^= a[i];
        return x == 0;
    }

    U query(U x = 0) const
    {
        for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
        return x;
    }

    void rebuild()
    {
        vector<U> b(a, a + 64);
        for (int i = 0; i < 64; i++)
            if (b[i])
                for (int j = i + 1; j < 64; j++)
                    if (b[j] >> i & 1) b[j] ^= b[i];
        p.clear();
        for (U x : b)
            if (x) p.push_back(x);
        dirty = false;
    }
```

```
    // k is 1-based; nonempty excludes the empty input subset, not value zero.
    optional<U> kth(Wide k, bool nonempty = true)
    {
        if (!k) return nullopt;
        Wide index = k;
        if (!nonempty || dependent) index--;
        if (index >= (Wide(1) << rank)) return nullopt;
        if (dirty) rebuild();
        U answer = 0;
        for (int i = 0; i < rank; i++)
            if (index >> i & 1) answer ^= p[i];
        return answer;
    }

    optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }

    void merge(const XorBasis &other)
    {
        dependent |= other.dependent;
        for (U x : other.a)
            if (x) insert(x);
    }
};
```

模板题使用：Luogu P3812

题意：给定一组可能重复的非负整数，选任意子集使异或和最大。逐个 insert 后 query()；使用 unsigned long long 读取和输出。

所需模板：XorBasis（第 227 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3812

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    XorBasis basis;
    for (int i = 0; i < n; i++)
    {
        unsigned long long x;
        std::cin >> x;
        basis.insert(x);
    }
    std::cout << basis.query() << '\n';
}
```

6.3.2 带位置限制的异或线性基

默认构造为空，重新赋值 PositionBasis() 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对

应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 O(64)，空间 O(64)。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 O(64n)。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
struct PositionBasis
{
    using U = unsigned long long;
    U a[64]{};
    int pos[64];

    PositionBasis() { fill(pos, pos + 64, -1); }
```

```
    void insert(U x, int at)
    {
        assert(at >= 0);
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    pos[i] = at;
                    return;
                }
                if (at > pos[i])
                {
                    swap(a[i], x);
                    swap(pos[i], at);
                }
                x ^= a[i];
            }
    }

    bool contains(U x, long long lower = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
        return x == 0;
    }

    U query(long long lower = 0, U x = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if (pos[i] >= lower) x = max(x, x ^ a[i]);
        return x;
    }

    int rank(long long lower = 0) const
    {
        assert(lower >= 0);
        int answer = 0;
        for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
        return answer;
    }
};
```

应用题补充：Luogu CF1100F

题意：给数组及多个闭区间，求区间内任意子集的最大异或和。把查询按右端点排序，依次加入前缀元素及其 1-based 位置；query(l) 限制只使用位置不小于 l 的元素，最后按原查询顺序输出。

所需模板：PositionBasis（第 229 页，6.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/CF1100F

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q;
    std::cin >> n;
    std::vector<unsigned long long> a(n + 1);
    for (int i = 1; i <= n; i++) std::cin >> a[i];
    std::cin >> q;
    std::vector<std::array<int, 3>> queries(q);
    for (int i = 0; i < q; i++)
    {
        int l, r;
        std::cin >> l >> r;
        queries[i] = {r, l, i};
    }
    std::sort(queries.begin(), queries.end());
    std::vector<unsigned long long> answer(q);
    PositionBasis basis;
    int right = 0;
    for (auto [r, l, id] : queries)
    {
        while (right < r)
        {
            right++;
            basis.insert(a[right], right);
        }
        answer[id] = basis.query(l);
    }
    for (auto x : answer) std::cout << x << '\n';
}
```

6.3.3 异或线性空间求交

代码依赖：XorBasis（第 227 页，6.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

basis_intersection(a,b) 返回 span(a) 与 span(b) 的交空间的一组独立生成元，结果类型 XorBasis。支持完整 64 位无符号值，不修改输入，也不要求先 rebuild。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$，额外空间 O(64)。输出按独立生成元解释：dependent 为 false，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 result.kth(k,false)，包含零；零空间返回空基，rank=0、contains(0)=true。

```
XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
{
    using U = unsigned long long;
    U pivot[64], from_b[64]{};
    copy(a.a, a.a + 64, pivot);
    XorBasis result;
    for (U value : b.a)
    {
        U x = value, part = value;
        for (int i = 63; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    from_b[i] = part;
                    break;
                }
                x ^= pivot[i];
                part ^= from_b[i];
            }
        if (!x && part) result.insert(part);
    }
    return result;
}
```

模板题使用：LC Intersection of F2 Vector Spaces

题意：多组给两个 F2 向量空间的基，输出交空间的一组基。T≤100000，n,m≤30，输入独立且小于 2^30。把输入插入两个 XorBasis，调用求交，输出 rank 和 a 中的非零主元；答案不唯一，不能逐字比较不同实现输出。模板本身支持 64 位和相关生成元，属于题外扩展。零维输出 0。

所需模板：XorBasis（第 227 页，6.3.1 节）、basis_intersection（第 233 页，6.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_intersection(a, b);
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

6.3.4 Zassenhaus：同时求和空间与交空间

代码依赖：XorBasis（第 227 页，6.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

basis_sum_intersection(a,b) 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 dependent 均为 false。将 a 的向量 x 编成 (x,x)，b 的向量 y 编成 (y,0)，用 ___uint128_t 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$，额外空间 O(128)。结果满足 dim(A+B)+dim(A 交 B)=dim(A)+dim(B)，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```
// Return independent generators of (sum space, intersection space).
pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a, const XorBasis &b)
{
    using W = __uint128_t;
    W pivot[128]{};
    auto insert = [&](W x)
    {
        for (int i = 127; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    return;
                }
                x ^= pivot[i];
            }
    };
    for (auto x : a.a) insert((W(x) << 64) | x);
    for (auto x : b.a) insert(W(x) << 64);
    XorBasis sum, intersection;
    for (int i = 0; i < 128; i++)
        if (pivot[i])
        {
            if (i < 64)
                intersection.insert((unsigned long long)pivot[i]);
            else
                sum.insert((unsigned long long)(pivot[i] >> 64));
        }
    return {sum, intersection};
}
```

模板题使用：LC Intersection / Zassenhaus

题意：同一线性空间求交题，约束与前例相同。Zassenhaus 返回 {和空间, 交空间}，本题仅输出 second 的 rank 和非零主元，不能据此声称 first 也获线上验证。和空间由独立本地测试检查。两部分均为独立生成元；无需对输入枚举全部异或值。

所需模板：XorBasis（第 227 页，6.3.1 节）、basis_sum_intersection（第 235 页，6.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_sum_intersection(a, b).second;
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

6.3.5 无向图最大异或行走

代码依赖：XorBasis（第 227 页，6.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

XorWalk(n) 建图，点号 1..n，边权为 unsigned long long，支持完整 64 位。add(u,v,w) 加无向边，允许重边、自环和零权；重新赋值 XorWalk(n) 清空图。build(root=1) 用递归 DFS 预处理 root 的连通块，query(u,v) 返回该块内 u 到 v 的最大边权异或和；任一点未被本次 build 覆盖则返回 nullopt，即使这两点在另一连通块相连。加入新边后必须重新 build；可更换根重复构建。空图可初始化，但不能 build。这里的行走允许重复点、重复边；不是最大异或简单路径。u=v 也可能通过绕环得到正值。DFS 距离 dist 给出一条树路径的异或值，每条已访问边形成 dist[u] xor dist[v] xor w 的闭合行走值，加入线性基；两点答案为 cycles.query(dist[u] xor dist[v])。build 后 vis、dist 可只读；cycles 表示环异或空间，不保留具体环或行走方案，不能将其非空子集标志解释为行走非空性。建图 O(n+m)，build 为 O(n+64m)，每次 query 为 O(64)，动态空间 O(n+m+64)。

```
struct XorWalk
{
    using U = unsigned long long;
    int n;
    bool built = false;
    vector<vector<pair<int, U>>> g;
    vector<int> vis;
    vector<U> dist;
    XorBasis cycles;

    XorWalk(int size)
    {
        assert(size >= 0);
        n = size;
        g.resize(size_t(n) + 1);
        vis.resize(size_t(n) + 1);
        dist.resize(size_t(n) + 1);
    }
```

```
    void add(int u, int v, U w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        built = false;
    }

    void dfs(int u)
    {
        vis[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (!vis[v])
            {
                dist[v] = dist[u] ^ w;
                dfs(v);
            }
            else
                cycles.insert(dist[u] ^ dist[v] ^ w);
        }
    }

    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        fill(vis.begin(), vis.end(), 0);
        fill(dist.begin(), dist.end(), 0);
        cycles = XorBasis();
        dfs(root);
        built = true;
    }

    optional<U> query(int u, int v) const
    {
        assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
        if (!vis[u] || !vis[v]) return nullopt;
        return cycles.query(dist[u] ^ dist[v]);
    }
};
```

应用题补充：Luogu P4151

题意：无向连通带权图，求 1 到 n 可重复经过点边的最大异或和；n≤50000，m≤100000，权值 ≤10^18，允许重边、自环。点号保留 1-based，build(1) 后 query(1,n)。题目保证连通才可直接解引用；通用图先检查 optional。此题验证行走，不能改成简单路径。O((n+m) · 64)，DFS 递归。

所需模板：XorBasis（第 227 页，6.3.1 节）、XorWalk（第 237 页，6.3.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4151

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    XorWalk graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        unsigned long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.build(1);
    std::cout << *graph.query(1, n) << '\n';
}
```

6.4 特征多项式

6.4.1 有限状态计数到线性递推：先证明阶数，再用 BM

当题目只问极大长度 N 的一个计数，有限状态 DP 可以先生成少量项，再用标量递推求远项。关键不是“样本看起来有规律”，而是先证明无限序列的递推阶数上界。本节给出从固定转移矩阵到该上界、再到可靠外推的完整链条。

6.4.1.1 先把状态、方向和输出写清楚

设状态数 $d \ge 1$ 固定，$v_n \in R^d$ 是长度 n 时各状态的计数**列向量**。令 A_{ij} 为一步从状态 j 到状态 i 的方案数，重边或不同字符走向同一状态时须累加，不是只填一个 1。若每步规则相同，就有

$$v_{n+1} = Av_n, \qquad v_n = A^n v_0, \qquad a_n = u^{\mathsf{T}} v_n.$$

u 可以是终态的 0/1 指示向量，也可以是固定权重；$a_0 = u^{\mathsf{T}} v_0$ 必须保留。这里 R 可取整数环或模 m 的交换环。若按行向量建模，矩阵方向须整体转置。

三个前提缺一不可：状态维数有限且不随 n 增长，转移矩阵固定，更新与输出均为线性。最短路的 min-plus 更新不是这里的环上线性运算；随位置任意变化的 A_n 也不能直接套一个 A。固定周期转移可加“相位”状态，固定仿射项可加一个恒为 1 的坐标，但阶数上界须按扩充后的维数重算。

6.4.1.2 Cayley–Hamilton 给出阶数至多 d

写特征多项式为

$$\chi_A(t) = \det(tI - A) = t^d + q_{d-1}t^{d-1} + \cdots + q_0.$$

Cayley–Hamilton 定理说 $\chi_A(A) = 0$。给出一个不依赖特征值或可对角化性的证明：由伴随矩阵恒等式，令 $B(t) = \operatorname{adj}(tI - A) = \sum_{i=0}^{d-1} B_i t^i$，有 $(tI - A)B(t) = \chi_A(t)I$。取 $B_{-1} = B_d = 0$、$q_d = 1$，比较 t^i 的系数得 $B_{i-1} - AB_i = q_i I$ $(0 \le i \le d)$。分别左乘 A^i 后求和，得到

$$\sum_{i=0}^{d} q_i A^i = \sum_{i=0}^{d} \left(A^i B_{i-1} - A^{i+1} B_i\right) = 0.$$

中间项逐个抵消，两端为零。这才是矩阵多项式求值的证明；不能把标量行列式里的 t 直接换成矩阵，写成“$\det(AI - A) = 0$”。整个证明没有除法，因此对交换环同样成立。

把 $\chi_A(A) = 0$ 左乘 $u^{\mathsf{T}} A^n$、右乘 v_0，便有

$$\boxed{a_{n+d} = -\sum_{i=0}^{d-1} q_i a_{n+i} \quad (n \ge 0).}$$

故从下标 d 起就有一个至多 d 阶的递推，不必等到状态分布“稳定”。若接口约定 $a_n = \sum_{j=1}^{k} c_{j-1} a_{n-j}$，直接用特征多项式时取 $k = d$、$c_j = -q_{d-1-j}$。这里只需存在性和上界，**不必先算出整个特征多项式**才能调用 BM。

6.4.1.3 为什么前 2d 项足以恢复这个标量序列

现在限定在素数域 $\mathbb{F}_p$ 中。取精确的前 $2d$ 项 $a_0, \ldots, a_{2d-1}$，BM 返回该前缀的最短递推，阶数记为 k。已知 d 阶递推存在，故 $k \le d$。按返回系数及前 k 个初值延伸出无限序列 b，它与 a 的前 $2d$ 项相同。

令前移算子 E 满足 $(Ea)_n = a_{n+1}$，并令 $g(t) = t^k - c_0 t^{k-1} - \cdots - c_{k-1}$。则 $\chi_A(E)a = 0$、$g(E)b = 0$ 都从 $n = 0$ 成立。多项式算子可交换，因而差序列 $h = a - b$ 满足

$$\chi_A(E)g(E)h = 0, \qquad \deg(\chi_A g) = d + k \le 2d.$$

这是一个**首**一递推；h 的前 $2d$ 项全零，依次递推便知所有项全零。所以外推正确。不能无依据地说“两个至多 d 阶序列之差仍至多 d 阶”；此处用乘积得到的上界是 $2d$。$k = 0$ 时约定 b 为全零序列、$g = 1$，同一论证仍成立。

恢复的是指定输入与输出的标量递推。例如 $A = \operatorname{diag}(1, 2)$、$u = v_0 = (1, 0)^{\mathsf{T}}$，则 $a_n = 1$，BM 只需返回 [1]；但 $A - I \ne 0$，矩阵最小多项式仍是 $(t-1)(t-2)$。在域上，标量最小消去多项式整除矩阵最小多项式，后者整除 χ_A；不能凭一次投影把标量递推当成对所有向量都有效的矩阵恒等式。

6.4.1.4 模型：不含连续子串 101 的二进制串

状态保存末尾与禁串前缀的最长匹配，依次为 $\varepsilon, 1, 10$。三者都是合法终态；到达完整 101 的转移丢弃。追加字符的转移为

当前状态	0	1
ε	ε	1
1	10	1
10	ε	丢弃

$$A=\begin{pmatrix}1&0&1\\1&1&0\\0&1&0\end{pmatrix},\quad v_0=\begin{pmatrix}1\\0\\0\end{pmatrix},\quad u=\begin{pmatrix}1\\1\\1\end{pmatrix}.$$

这里 10 后接 0 变为状态 ε，后接 1 才违法；1 后接 1 仍在状态 1。直接算得

$$\chi_A(t)=t(t-1)^2-1=t^3-2t^2+t-1,\qquad (a_0,\dots,a_5)=(1,2,4,7,12,21),$$

于是 $a_n=2a_{n-1}-a_{n-2}+a_{n-3}$ $(n\ge 3)$。模 $p=998244353$ 的现有接口可按下列方向衔接；调用者用 `unsigned long long` 声明目标零基下标 N：

```
constexpr int p = 998244353;
vector<int> s{1, 2, 4, 7, 12, 21}; // a[0..2*d-1], d = 3
vector<int> c = berlekamp_massey(s, p); // {2, p-1, 1}
vector<int> init(s.begin(), s.begin() + c.size());
int answer = recurrence_nth(init, c, N, p);
```

若模数改变，应先对样本及系数正规化到 $[0,p)$，并在**目标素数域**内恢复递推；模一个素数得到的短递推，不保证在整数或另一个模数下有效。

6.4.1.5 按目标选算法，并把采样成本算进去

设 e 是存储的非零转移数，采样用稀疏边表更新、清零长度 d 的向量并做一次点积。前 $2d$ 项需要 $O(d(e+d))$ 时间、$O(e+d)$ 空间；稠密矩阵采样为 $O(d^3)$。本库 BM 在每个非零偏差处用二进制幂求逆，因此这段采样的 BM 后处理为 $O(d^2+d\log p)$ 时间、$O(d)$ 额外空间，不能把求逆无条件写成常数时间。

直接 DP。N 不大或要整个前缀时，稀疏实现为 $O(N(e+d))$。

矩阵幂。要完整 v_N，或复用多种初值/输出时，考虑第 6.1.1 节（第 211 页）的 `ModMatrix<p>::power(A,N)`。当前稠密实现为 $O(d^3\log(N+1))$ 时间、$O(d^2)$ 空间；求出矩阵幂后，可复用于不同初值。标量 BM 的系数一般不能如此复用。

小阶标量递推。只要固定 u, v_0 的远项时，先采样与 BM，再调用第 4.8.1 节（第 165 页）的 `recurrence_nth(init,c,N,p)`，单次 $O(k^2\log(N+1))$ 时间、$O(k)$ 额外空间。

大阶快速求项。k 较大时，第 4.8.2 节（第 167 页）提供 `BostanMori<p,g>` 的 `recurrence(init,c,N)`，在固定模数下为 $O(k\log(k+1)\log(N+1))$ 时间、$O(k)$ 空间。参数须转换为其 `Poly`，p 为素数、g 为原根；非初值查询要求 $2k+1\le$ `NttConvolution<p,g>::max_size`，不能对任意素数不加检查地使用。

BM 接口见第 11.1 节（第 268 页）。两种远项接口都要求初值**恰好**有 k 项，使用 `unsigned long long` 下标；$k=0$ 返回 0，$N<k$ 返回初值。Bostan–Mori 的快速求项不会消掉前面的采样和 BM 成本。

合数模数分清存在性与恢复算法。模 m 时，Cayley–Hamilton 仍保证递推存在，但本库 BM 用 x^{p-2} 求逆，只支持素数模数。若已独立得知正确递推，`recurrence_nth` 只用加乘及首一多项式约减，支持 $2\le m\le$ `INT_MAX` 的合数模数；初值和系数仍须在 $[0,m)$。`ModMatrix` 的乘法和非负次幂也不需要逆元。不要把这个结论扩展到求逆矩阵或当前 NTT 版 Bostan–Mori。

少取一项与删除末尾零的反例。取 $A_\lambda=\begin{pmatrix}0&\lambda\\1&0\end{pmatrix}$、$v_0=(1,0)^{\mathsf T}$、$u=(0,1)^{\mathsf T}$。$\lambda=0,1$ 的两种二状态模型都有前三项 $0,1,0$，第四项却分别为 0、1；即使知道 $d=2$，$2d-1$ 项也不能普遍保证唯一外推。$\lambda=0$ 时矩阵幂零，序列为 $0,1,0,0,\dots$，BM 的长度 2 系数 $[0,0]$ 不能删成空数组：它保留了从下标 2 才开始适用的递推，空数组会错误抹掉初始的 1。额外保留若干未参与 BM 的项做交叉检查很有用，但**检查通过不代替阶数证明**。

来源。定理与中文导航核对 OI Wiki：特征多项式；伴随矩阵系数证明参照 IIT Kanpur：Eigenvalues and Eigenvectors 的 Cayley–Hamilton 第二证明。算法概念另核对 OI Wiki 的 BM、常系数齐次线性递推及 矩阵。模型、外推论证与接口成本按本库重新整理；本节不新增矩阵最小多项式实现或在线 AC 覆盖。

第七章　线性规划

7.1　单纯形法

7.1.1　线性规划：两阶段单纯形

Simplex(A,b,c) 求 max c · x，约束 Ax≤b、x≥0；A 为 m×n，b 长 m，c 长 n，变量/约束均 0 基。构造后只调用一次 solve()，返回 status、value、x；optimal 时 value 与 x 有效，infeasible/unbounded 时只读 status。允许负 b、冗余约束及零维；内部表与 pivot/improve 仅供实现使用。

long double、绝对 eps=1e-12。两阶段人工变量法；按变量编号选入基、最小比值并按基变量编号解平局。精确运算下 Bland 规则避免循环，浮点容差下不保证任意病态输入的正确分类或终止；勿把 AC 视为精确 LP 保证。先缩放数量级，求解后复核原约束和目标值。每次换基 O((m+1)(n+1))，空间同阶；总换基次数最坏指数级。

```
struct Simplex
{
    using Real = long double;
    static constexpr Real eps = 1e-12L;
    enum Status { optimal, infeasible, unbounded };
    struct Result
    {
        Status status;
        Real value;
        vector<Real> x;
    };
    int n, m;
    vector<int> basic, nonbasic;
    vector<vector<Real>> d;

    Simplex(const vector<vector<Real>> &a, const vector<Real> &b,
            const vector<Real> &c) : n(c.size()), m(b.size()),
            basic(m), nonbasic(n + 1), d(m + 2, vector<Real>(n + 2))
    {
        assert((int)a.size() == m);
        for (int i = 0; i < m; i++)
        {
            assert((int)a[i].size() == n);
            for (int j = 0; j < n; j++) d[i][j] = a[i][j];
            basic[i] = n + i;
            d[i][n] = -1;
            d[i][n + 1] = b[i];
        }
        iota(nonbasic.begin(), nonbasic.end(), 0);
        nonbasic[n] = n + m;
        for (int j = 0; j < n; j++) d[m][j] = -c[j];
        d[m + 1][n] = 1;
    }
```

换基（接上页结构体）：

```
    void pivot(int r, int s)
    {
        Real inv = 1 / d[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i == r) continue;
            Real scale = d[i][s] * inv;
            for (int j = 0; j < n + 2; j++)
                if (j != s) d[i][j] -= scale * d[r][j];
        }
        for (int j = 0; j < n + 2; j++)
            if (j != s) d[r][j] *= inv;
        for (int i = 0; i < m + 2; i++)
            if (i != r) d[i][s] *= -inv;
        d[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }
```

表的含义 第 i 行表示基本变量等于右端值减去非基本变量的线性组合。目标行同样表示当前目标常数减去约化系数乘变量。负约化系数表示增加该变量可改善目标；仅系数为正的约束限制增加量，故用右端除系数的最小值选择离基。若没有限制行则沿该方向无界。

人工变量与两阶段 对每条约束引入同一个人工变量 t，系数为-1。从最负的右端行让 t 入基即得到可行起点，第一阶段最大化-t。最优值小于零说明原问题无解；为零时把仍在基内的人工变量换出，零冗余行可以保留。原目标行始终参加换基，第二阶段禁止人工变量入基，直接优化原目标。

选择入基与出基变量（接上页结构体）：

```
bool improve(int objective)
{
    while (true)
    {
        int s = -1;
        for (int j = 0; j <= n; j++)
        {
            if (nonbasic[j] == n + m) continue;
            if (d[objective][j] >= -eps) continue;
            if (s == -1 || nonbasic[j] < nonbasic[s]) s = j;
        }
        if (s == -1) return true;
        int r = -1;
        for (int i = 0; i < m; i++)
        {
            if (d[i][s] <= eps) continue;
            if (r == -1)
            {
                r = i;
                continue;
            }
            Real x = d[i][n + 1] / d[i][s];
            Real y = d[r][n + 1] / d[r][s];
            if (x < y - eps || (abs(x - y) <= eps && basic[i] < basic[r]))
                r = i;
        }
        if (r == -1) return false;
        pivot(r, s);
    }
}
```

两阶段求解与方案恢复（接上页结构体）：

```
    Result solve()
    {
        int r = -1;
        for (int i = 0; i < m; i++)
            if (r == -1 || d[i][n + 1] < d[r][n + 1]) r = i;
        if (r != -1 && d[r][n + 1] < -eps)
        {
            pivot(r, n);
            improve(m + 1);
            if (d[m + 1][n + 1] < -eps) return {infeasible, 0, {}};
            for (int i = 0; i < m; i++)
            {
                if (basic[i] != n + m) continue;
                int s = -1;
                for (int j = 0; j <= n; j++)
                    if (abs(d[i][j]) > eps &&
                        (s == -1 || nonbasic[j] < nonbasic[s])) s = j;
                if (s != -1) pivot(i, s);
            }
        }
        if (!improve(m)) return {unbounded, 0, {}};
        vector<Real> x(n);
        for (int i = 0; i < m; i++)
            if (basic[i] < n) x[basic[i]] = d[i][n + 1];
        return {optimal, d[m][n + 1], x};
    }
};
```

退化与数值 在精确运算下，入基选最小变量编号，最小比值并列时选最小基本变量编号，是 Bland 防循环规则。这里按 eps 判断约化系数和比值并列，所以不承诺浮点下严格防循环。极小系数、严重消去或尺度悬殊都可能改变分类；先对模型缩放，并用原输入复核返回方案。需要精确判定时应使用有理数算法，不能仅缩小 eps。

模板题使用：P13337 线性规划

题意：n,m≤100，整数系数绝对值 ≤100，题目保证随机数据。最大化 c · x，Ax≤b、x≥0。输出最优值及任一最优解，或 Infeasible/Unbounded；允许绝对或相对误差 1e-6。

所需模板：Simplex（第 242 页，7.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13337

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long double> c(n), b(m);
    vector<vector<long double>> a(m, vector<long double>(n));
    for (auto &v : c) cin >> v;
    for (int i = 0; i < m; i++)
    {
        for (auto &v : a[i]) cin >> v;
        cin >> b[i];
    }
    Simplex lp(a, b, c);
    auto result = lp.solve();
    if (result.status == Simplex::infeasible) cout << "Infeasible\n";
    else if (result.status == Simplex::unbounded) cout << "Unbounded\n";
    else
    {
        cout << fixed << setprecision(12) << result.value << '\n';
        for (auto x : result.x) cout << x << ' ';
        cout << '\n';
    }
}
```

第八章　概率论

8.1　随机变量的数字特征

8.1.1　期望选型：计数拆指标，等待拆尾和

先确定随机对象。均匀随机排列、逐步等概率转移和等概率选择最终方案是三种不同模型。例如从起点等概率走向 A 或 B，A 只有一个终点，B 有两个等概率终点：三个终点概率为 $1/2, 1/4, 1/4$，不是各 $1/3$。不能直接用路径条数代替路径概率。

计数型答案：给每个对象一个指示变量。若 $X = \sum_{i=1}^{m} I_i$，其中 $I_i = [\text{事件} A_i \text{ 发生}]$，则

$$\mathbb{E}X = \sum_{i=1}^{m} \Pr(A_i).$$

有限和的期望线性性不要求独立；一般实值变量应保证期望存在，避免相减两个无穷大。随机排列的不动点数为 $X = \sum_i [\pi(i) = i]$，每个位置固定的概率为 $1/n$，所以 $n \geq 1$ 时 $\mathbb{E}X = 1$。这比先求全部不动点个数的分布容易。$n = 2$ 时两个指示变量完全相同，恰好说明它们并不独立。

线性性不能推广成 $\mathbb{E}(XY) = \mathbb{E}X\mathbb{E}Y$。令 $X = Y$ 为一次公平硬币正面的指示变量，则左边为 $1/2$、右边为 $1/4$。方差应写成

$$\operatorname{Var}\left(\sum_i X_i\right) = \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j).$$

两两独立足以消去协方差；仅仅知道各自成功概率不足以计算方差。

等待时间或最大值：先问是否超过阈值。非负整数随机变量逐点满足 $X = \sum_{k\geq 1} [X \geq k]$，因此

$$\mathbb{E}X = \sum_{k\geq 1} \Pr(X \geq k) = \sum_{k\geq 0} \Pr(X > k).$$

非负项允许交换求和，结果也可能是 $+\infty$。两种写法的起点相差一。例如两个独立公平六面骰子的最大值 M 满足 $\Pr(M < k) = ((k-1)/6)^2$，故

$$\mathbb{E}M = \sum_{k=1}^{6} \left(1 - \frac{(k-1)^2}{36}\right) = 6 - \frac{55}{36} = \frac{161}{36}.$$

这里乘出平方用到了两个骰子独立；若两个骰子总是同点，期望就变成 $7/2$。

覆盖过程：按完成一件新事的阶段拆分。每次独立均匀抽取 n 种券，已有 i 种时，下一张为新种类的概率为 $(n-i)/n$。含成功那次在内的等待时间期望是 $n/(n-i)$，收齐的期望为 $n\sum_{j=1}^{n} 1/j$。每次抽样方式必须保持不变；非均匀抽券时，“已见种类数”通常不是充分状态，因为尚缺哪几种会改变转移概率，不能直接套调和数。

8.1.2　条件期望：先定义状态，再解自环

状态必须保存决定未来分布所需的信息。若到达终点后不再花费，设终点 t 的 $E_t = 0$。一次转移从 i 到 j 的概率为 p_{ij}，本次费用为 c_{ij}，则第一步分析给出

$$E_i = \sum_j p_{ij}(c_{ij} + E_j) = b_i + \sum_j p_{ij}E_j, \qquad b_i = \sum_j p_{ij}c_{ij}.$$

每步统一花费 1 时才有 $b_i = 1$；已经是终点时不能再多加一次。若目标是命中某个好终点的概率，则好终点边界为 1、坏终点为 0，非终点方程为 $h_i = \sum_j p_{ij}h_j$，没有常数 1。若过程可能永不终止，命中概率是边界问题的最小非负解，不能把任意代数解当成概率。

只有自环时可以局部移项。若 $p_{ii} < 1$，则

$$E_i = \frac{b_i + \sum_{j \neq i} p_{ij} E_j}{1 - p_{ii}}.$$

其余边若指向已算出的状态，就按依赖顺序做 DP。不能在存在 $i \to j \to i$ 时只去掉自环后仍当 DAG。每次独立以概率 $q > 0$ 成功，失败回到原状，每次花费 1，有 $E = 1 + (1-q)E$，故 $E = 1/q$。当 $q = 0$ 时永不成功，不能继续做除法。若统计“成功前失败次数”，当前成功不计费用，结果才是 $(1-q)/q$。

小例：等到连续两个正面。公平硬币独立抛掷，状态 0 表示当前没有末尾正面，状态 1 表示末尾恰有一个正面，状态 2 表示已完成。第一步方程为

$$E_0 = 1 + \tfrac{1}{2}E_0 + \tfrac{1}{2}E_1, \qquad E_1 = 1 + \tfrac{1}{2}E_0, \qquad E_2 = 0.$$

解得 $E_0 = 6, E_1 = 4$。错误地把每两次抛掷作为成功率 $1/4$ 的独立尝试会得到 8，它只检查不重叠的分组，漏掉跨组的连续正面。反过来把所有相邻两次都当独立试验也不成立。

有环时检查吸收，再选择消元。只保留起点可达、尚未吸收的状态，记它们之间的转移矩阵为 Q。有限期望向量满足 $(I-Q)E = b$；稠密直接消元的时间为 $O(s^3)$、空间为 $O(s^2)$，其中 s 是非终点状态数。DAG、链、带状矩阵或可按强连通分量分块的问题应先利用结构，不必一律构建全局稠密矩阵。概率极小时实数消元可能病态；残差小也不保证答案相对误差小。实数接口见GaussReal（第 265 页，10.3.3 节）；其秩与一致性按显式阈值判定，不能代替精确可解性或下述吸收条件的证明。

对有限状态、固定转移概率的链，若每个可达非终点都有一条正概率路径通向吸收态，则存在统一的 L 和 $\varepsilon > 0$，使任何非终点在未来 L 步内吸收的概率至少为 ε。由此 $\Pr(T > kL) \le (1-\varepsilon)^k$，所以几乎必然吸收且 $\mathbb{E}T \le L/\varepsilon < \infty$。等价的图判据是：可达区域中没有封闭的非吸收强连通分量。这里的边只取正概率转移，且每步费用有界时累计费用也具有有限绝对期望。

两类反例不能混淆。以概率 $1/2$ 立即终止、以概率 $1/2$ 进入永久自环，有正概率永不吸收，停止时间期望为无穷大。另取正整数随机变量 T，令 $\Pr(T = k) = 1/(k(k+1))$。概率和为 1，所以 T 几乎必然有限；但 $\Pr(T \ge k) = 1/k$，故 $\mathbb{E}T = \infty$。第二例说明无限状态或无限支撑下，“几乎必然终止”不足以推出有限期望，不能丢掉上述有限状态条件。

8.1.3 模概率：可逆的不只是输入分母

竞赛中的“概率对素数 p 取模”不是一个介于 0 与 1 的数，而是有理数的代数表示。既约分数 a/b 只有在 $p \nmid b$ 时，才能映射为 $a\,b^{-1} \pmod p$。模值不能比较大小、判定真实概率是否很小，也不能由模值为零断言真实概率为零。例如概率 $p/(p+1) > 0$，其模值为零。

直接转模方程的充分条件：每个转移概率和费用的分母均可逆，且 $I - Q$ 在模 p 下可逆。实数下可逆不保证模下可逆；自环移项中的 $1 - p_{ii}$ 也必须非零。例如每次以 $p/(p+1)$ 成功，期望为 $(p+1)/p$；原始转移概率的分母 $p+1$ 可逆，但期望的分母 p 不可逆。模方程变成 $0 \cdot E = 1$，无解不是概率模型矛盾，而是所求期望没有该模表示。

未约分表达式的分母不可逆，不代表最终答案一定不可表示。表达式 $p/p = 1$ 应先在整数或有理数层面约分，不能调用逆元求 $0/0$。类似地，模线性系统奇异时，可能存在有理数层面的消去或约分，但不能从任意模特解恢复唯一的真实期望。合数模数 m 下，分母须满足 $\gcd(b, m) = 1$，且不能用费马小定理求逆。

本库接口见第 3.2.2 节（第 24 页）的 `ModInt<p>` 与第 10.3.2 节（第 262 页）的 `GaussMod<p>::solve(a,n)`。前者的 `try_inv()` 用欧几里得，允许合数模数的单位元，不可逆时返回空值；`inv()` 与除法仍要求分母可逆。后者仍要求 p 为素数，不能随基础类型扩展而放宽；接收 s 行 $s+1$ 列增广矩阵，可填为 $[I - Q \mid b]$，显式传入变量数 s。先检查 `consistent`，再检查 `rank==s`，唯一解才读取 `particular`。这些接口不会替你证明吸收、期望有限或有理答案可取模。

第九章　博弈论

9.1　公平组合游戏

9.1.1　SG 的前提、递推与异或策略

先核对模型：双人轮流操作；双方在同一状态的合法操作完全相同（无偏游戏）；没有随机性与隐藏信息；不能行动者输（正常玩法）；状态图为有限 DAG。本节只在这些条件下使用普通 Sprague–Grundy 理论。“公平”在这里不表示胜率各半。

令 $g(u)$ 是状态的 SG 值，mex 表示集合中未出现的最小非负整数，则

$$g(u)=\operatorname{mex}\{g(v):u\to v\},\qquad \operatorname{mex}\varnothing=0.$$

按逆拓扑顺序计算；若递归记忆化，栈深度仍可能等于最长路径长度。若有 d 个后继，则 $g(u)\le d$，只需标记 $0,\dots,d$，超过 d 的后继值可以忽略。重复出现同一个后继值不影响 mex；用时间戳避免每个点都清空一张全局大表，可做到 $O(|V|+|E|)$ 时间和 $O(|V|+|E|)$ 空间（包含存图）。

$g(u)=0$ 时所有后继非零；$g(u)>0$ 时一定存在后继值为零。由 DAG 上归纳，零状态恰为先手必败态。不过 SG 数值不是胜率，也不是距离终点的步数。若题目只问一个不可分解游戏的胜负，布尔递推“存在后继必败则必胜”已足够；需要组合多个游戏时才必须保留比输赢更多的信息。

异或只针对不交和。一步恰好选择一个子游戏走一步，其他子游戏的状态及合法操作不变，且总游戏不能走时才输。此时组合值为 $G=g_1\oplus\cdots\oplus g_k$。$G=0$ 时任意一步都把一个 g_i 改为不同值，因此异或不再为零。$G\neq 0$ 时，取 G 最高为一位上也为一的某个 g_i，令 $h=g_i\oplus G<g_i$。mex 定义保证存在一步从该子状态走到 SG 值 h 的后继，操作后整体异或为零。实际输出方案时枚举这个子游戏的合法后继寻找 h，不能只输出一个目标数字。

如果一次允许同时改变两堆，或者一堆的操作会禁止另一堆的操作，就不是上述不交和。例如两堆各有一颗石子，允许取走任意一堆或同时取走两堆：把两堆当普通 Nim 会算出 $1\oplus 1=0$，但先手同时取完立刻获胜。

9.1.2　减法游戏小例与三类失效边界

一堆 n 颗石子，每次可取 $S=\{1,3,4\}$ 中一种数量且不能取成负数。空堆无合法操作，故 $g(0)=0$，对 $n>0$ 有

$$g(n)=\operatorname{mex}\{g(n-s):s\in S,\ s\le n\}.$$

从 $n=0$ 到 13 逐项得到

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$g(n)$	0	1	0	1	2	3	2	0	1	0	1	2	3	2

例如 $g(4)=\operatorname{mex}\{g(3),g(1),g(0)\}=\operatorname{mex}\{1,0\}=2$，不是合法操作数 3。两堆 $(4,5)$ 的异或为 $2\oplus 3=1$，从第二堆取一颗得到 $(4,4)$ 即可归零。两堆 $(1,1)$ 各自都是必胜态，但组合为必败态，不能把多个布尔“必胜”作逻辑或。

表中前两段相等本身不是周期证明。本例先验证 $g(n+7)=g(n)$ 对 $0\le n\le 3$ 成立，随后对 $n\ge 4$，两边的后继都完整包含减 $1,3,4$；由更小下标的归纳假设，mex 集合相同，故周期 7 对所有 $n\ge 0$ 成立。对一般有限减法集合，SG 值不超过 $|S|$，且从足够大的 n 起只依赖前 $\max S$ 项，因此整个长度为 $\max S$ 的窗口重复后才保证后续周期。它只保证最终周期，不能仅凭几项重复猜出从零开始的周期或短周期。

- **反常玩法 (misère)**。最后行动者输时，普通 SG 的终点规则与异或结论不可直接搬用。只有一颗石子的 Nim 在正常玩法下先手赢，在反常玩法下先手输。反常 Nim 有专门结论，不意味着所有减法游戏都能“把最终答案取反”。

- **有环游戏（loopy）**。单点自环将要求 $g = \mathrm{mex}\{g\}$，不存在这样的非负整数。允许无限行走时还可能有和局；必须按题目规定处理重复局面、步数上限和无限对局。若加入剩余步数后形成 DAG，可以在扩展状态上分析，不能把环当成已访问后直接返回零。
- **有偏游戏（partizan）**。双方在相同局面可走步不同，不满足无偏性。仅一条“左方可走、右方不可走”的边，就不能用不带行动方信息的 mex 描述。可按完整状态做胜负搜索，但一般不能将各部分的普通 SG 值异或。

本地知识例证。 `tests/math_knowledge_probability.py` 使用有理数精确运算检查指示变量、骰子尾和、连续正面方程及模分母反例，并以直接胜负递归核对小规模减法游戏不交和。它验证本节例子，不是通用概率求解器、SG 模板或在线评测 AC。

第十章　数值算法

10.1　插值

10.1.1　拉格朗日插值与连续点优化

Lagrange(xs,ys,p) 使用任意横坐标，xs、ys 为等长 0-based vector<long long>；Lagrange(ys,p) 专用于横坐标 0..n-1。重新构造赋值可更换数据。要求 p 为素数，2≤p≤INT_MAX，点数 n≤p，任意横坐标取模后两两不同；不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值，允许 k、xs、ys 为完整 long long 范围，内部先归一化；空数据约定零多项式。通用预处理 $O(n^2+n \log p)$，连续点预处理 O(n+log p)，每次查询 O(n)，空间 O(n)。查询利用前缀积与滚动后缀积，无需求逆，因此 k 与样本点模 p 相等时也正确。若求 1^d+…+k^d，要求 0≤d≤p-2，先计算 0..d+1 的前缀和作为 ys，再构造 Lagrange(ys,p)；用快速幂生成这些样本另花 O(d log(d+1))。仅处理素数域中的精确插值，不适用于浮点曲线拟合或重复节点的导数约束。

```
struct Lagrange
{
    using ll = long long;
    int n, mod;
    vector<ll> x, w;

    ll norm(ll a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        return a;
    }

    ll power(ll a, int b) const
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return result;
    }

    Lagrange(const vector<ll> &xs, const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p && xs.size() == ys.size());
        x.resize(n);
        w.resize(n);
        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
        for (int i = 0; i < n; i++)
        {
            ll denominator = 1;
            for (int j = 0; j < n; j++)
                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
```

```
            assert(denominator != 0);
            w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
        }
    }

    Lagrange(const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p);
        x.resize(n);
        w.resize(n);
        if (!n) return;
        vector<ll> invfac(n, 1);
        ll factorial = 1;
        for (int i = 1; i < n; i++) factorial = factorial * i % mod;
        invfac[n - 1] = power(factorial, mod - 2);
        for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
        for (int i = 0; i < n; i++)
        {
            x[i] = i;
            w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
            if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
        }
    }

    ll query(ll k) const
    {
        k = norm(k);
        vector<ll> pre(n + 1, 1);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
        ll answer = 0, suffix = 1;
        for (int i = n - 1; i >= 0; i--)
        {
            answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
            suffix = suffix * norm(k - x[i]) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4781

题意：给出 n 个横坐标模素数后互不相同的点，求次数小于 n 的插值多项式在 k 处的值，模 998244353。使用任意点构造函数。

所需模板：Lagrange（第 251 页，10.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4781

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    long long k;
    std::cin >> n >> k;
    std::vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) std::cin >> x[i] >> y[i];
    Lagrange poly(x, y, 998244353);
    std::cout << poly.query(k) << '\n';
}
```

10.2 数值积分

10.2.1 有工作上限的自适应 Simpson 积分

integrate(f,a,b,eps,depth=20,limit=100000)，long double 绝对容差 eps>0，有限端点，0≤depth≤60、limit≥0。返回 value、error 估计、evaluations 与 met；met 仅指局部估计条件满足，不是实际精度保证。反向区间变号，零长度不调用 f；预算/深度/中点停滞/非有限计算会停止且 met=false。正常有限样本下保留粗或修正估计；无初始估计时 value 为 NaN；不可把失败值当答案。f 须返回确定值，异常原样传播，不能启用 fast-math。复用样本，O(求值数) 次固定字长运算、O(depth+1) 栈；需 numeric 的 midpoint、cmath、limits、utility。

```
struct AdaptiveSimpson
{
    using ld = long double;
    struct Result
    {
        ld value, error;
        int evaluations;
        bool met;
    };

    template<class F>
    static Result integrate(F &&f, ld a, ld b, ld eps,
                            int depth = 20, int limit = 100000)
    {
        const ld inf = numeric_limits<ld>::infinity();
        const ld nan = numeric_limits<ld>::quiet_NaN();
        if (!isfinite(a) || !isfinite(b) || !isfinite(eps) || eps <= 0 ||
            depth < 0 || depth > 60 || limit < 0)
            return {nan, inf, 0, false};
        if (a == b) return {0, 0, 0, true};
        if (limit < 3) return {nan, inf, 0, false};
        bool reverse = a > b;
        if (reverse) swap(a, b);
        auto area = [](ld l, ld r, ld u, ld v, ld w)
        {
            return (r - l) / 6 * (u + 4 * v + w);
        };
        ld fa = f(a), fm = f(midpoint(a, b)), fb = f(b);
        int used = 3;
        ld whole = area(a, b, fa, fm, fb);
        if (!isfinite(fa) || !isfinite(fm) || !isfinite(fb) || !isfinite(whole))
            return {nan, inf, used, false};
```

采样细分与预算合并（接上页同一 integrate 函数）：

```
        auto rec = [&](auto &&self, ld l, ld r, ld u, ld v, ld w,
                       ld s, ld tol, int left) -> Result
        {
            ld m = midpoint(l, r), x = midpoint(l, m), y = midpoint(m, r);
            if (m == l || m == r || x == l || x == m || y == m || y == r ||
                used > limit - 2)
                return {s, inf, 0, false};
            ld fx = f(x), fy = f(y);
            used += 2;
            ld sl = area(l, m, u, fx, v), sr = area(m, r, v, fy, w);
            ld sum = sl + sr, delta = sum - s;
            ld value = sum + delta / 15, error = abs(delta) / 15;
            if (!isfinite(fx) || !isfinite(fy) || !isfinite(sl) ||
                !isfinite(sr) || !isfinite(value) || !isfinite(error))
                return {s, inf, 0, false};
            if (error <= tol) return {value, error, 0, true};
            if (!left || tol / 2 == 0) return {value, error, 0, false};
            auto p = self(self, l, m, u, fx, v, sl, tol / 2, left - 1);
            auto q = self(self, m, r, v, fy, w, sr, tol / 2, left - 1);
            value = p.value + q.value;
            error = p.error + q.error;
            if (!isfinite(value)) return {s, inf, 0, false};
            return {value, error, 0, p.met && q.met && error <= tol};
        };
        auto answer = rec(rec, a, b, fa, fm, fb, whole, eps, depth);
        if (reverse) answer.value = -answer.value;
        answer.evaluations = used;
        return answer;
    }
};
```

接口用法演示：有限区间积分（接口演示）

演示约定：自定义协议，非 OJ 题：输入 a b eps depth limit，对 f(x)=1/(1+x*x) 积分；a,b 有限且绝对值≤100、有限 eps>0、0≤depth≤60、0≤limit≤200000。估计条件满足时打印 value，否则 FAILED；这不是严格误差证书。反向区间变号；零长度返回 0 且不求值。depth=0 仍允许根的 5 点比较，但不递归细分。示例避免奇点，任意函数仍可能发生采样混叠，须审查正则性并按已知间断分段。

所需模板：AdaptiveSimpson（第 254 页，10.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/adaptive_simpson.hpp

```
int main()
{
    long double a, b, eps;
    int depth, limit;
    if (!(cin >> a >> b >> eps >> depth >> limit) ||
        !isfinite(a) || !isfinite(b) || abs(a) > 100 || abs(b) > 100 ||
        !isfinite(eps) || eps <= 0 || depth < 0 || depth > 60 ||
        limit < 0 || limit > 200000)
        return 1;
    auto f = [](long double x) { return 1 / (1 + x * x); };
    auto r = AdaptiveSimpson::integrate(f, a, b, eps, depth, limit);
    if (!r.met) cout << "FAILED\n";
    else cout << setprecision(20) << r.value << '\n';
}
```

10.2.2 数值积分：局部估计、误差预算与采样盲区

接口见第 10.2.1 节（第 254 页），用法见第 10.2.1 节（第 256 页）。先确认定积分存在，函数在闭区间采样点有定义，并分析间断、尖峰与振荡。端点奇异问题需要有依据的变量变换或其他适用算法，不能只靠不断减小 eps。

令 $m = (a+b)/2$，单段 Simpson 取

$$S(a,b) = \frac{b-a}{6}\big(f(a) + 4f(m) + f(b)\big).$$

它在精确算术下对至多三次多项式精确。若 $f \in C^4[a,b]$，有某个 $\xi \in (a,b)$ 满足

$$\int_a^b f(x)\,dx - S(a,b) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi).$$

这是单段误差；等距复合 Simpson 的小步长为 h、小区间数为偶数时，整体误差形式是 $-(b-a)h^4 f^{(4)}(\xi)/180$，不能混用两个长度参数。

设 $S_2 = S(a,m) + S(m,b)$、$\delta = S_2 - S(a,b)$。若已进入主导误差按四次步长缩放的渐近区间，则二分约把误差缩为原来的 $1/16$，由此得到指标 $E = |\delta|/15$ 与修正值 $S_2 + \delta/15$。递归时把当前绝对误差预算各分一半，端点与中点采样直接传给孩子，只增加两个四分点样本。返回的 error 是叶指标之和，`met` 仅说明计算出的指标满足预算；它不是实际误差证书。浮点预算二分和求和也会舍入，本实现会再检查父区间的汇总指标不超预算。

光滑也会漏采样 在 $[0,1]$ 上取

$$f(x) = \big[x(x-\tfrac14)(x-\tfrac12)(x-\tfrac34)(x-1)\big]^2.$$

初始五个采样点都得到 0，故本接口返回 0、指标 0、`met=true`，但该非负光滑多项式在区间内部并非恒零，真实积分严格为正。因此“足够光滑”配合一次小差值仍不能给任意函数保证。已知尖峰、间断或频率应先用于分段；强制若干层只能降低某些漏采样风险，不能消灭它。

有限资源与数值合同 eps 为正的绝对容差。depth=0 仍允许根的五点比较，只有继续细分被禁止；limit 是实际回调次数上限。需要继续细分时深度耗尽或容差下溢，以及所需求值预算不足、四分点与端点重合或非有限计算，都会令 `met=false`。失败 value 可能只是覆盖该区间的粗估计，初始估计不可用时为 NaN；不要直接打印失败值。零长度不调用函数，反向区间只反转积分值的符号。函数必须给出确定的采样值，异常直接传播；有限样本也可能在中间加法或乘法溢出时失败。

参考：`https://dlmf.nist.gov/3.5#ii`；OI Wiki 固定版本 `math/numerical/integral.md` 与 WIDA jiangly 的 Simpson 文件。只补本有界自适应接口；不把目录中的等距复合实现、原应用函数或未读到的 kuangbin PDF 页视为完成。

10.3 高斯消元

10.3.1 压位模 2 高斯消元与全部解

GaussXor::solve(a,n)，a 为 m 行 n+1 列的 01 字符串增广矩阵，0≤n<INT_MAX，最末字符为右端项。返回 consistent、系数秩 rank、01 字符串特解 particular 与零空间基 kernel；所有解为特解异或任意基子集。无解时解数组为空；rank 仍为系数秩。有解时 kernel.size()=n-rank，唯一解的 kernel 为空。允许空方程组或零变量，原输入不变。自由列按原列号递增，基在各自自由列为 1、其他自由列为 0；没有最少 1 个数保证。时间 O(mn+mn min(m,n)/64+n^2)，除输入外空间 O(m ceil((n+1)/64)+n^2)；输出本身可能有 n^2 字符。全程精确异或，不用 eps。

```
struct GaussXor
{
    struct Solution
    {
        bool consistent;
        int rank;
        string particular;
        vector<string> kernel;
    };

    // m rows of n+1 characters: coefficients followed by the right-hand side.
    static Solution solve(const vector<string> &a, int n)
    {
        assert(n >= 0);
        int m = a.size(), w = n / 64 + 1, row = 0;
        vector<vector<unsigned long long>> b(m, vector<unsigned long long>(w));
        for (int i = 0; i < m; i++)
        {
            assert(a[i].size() == (size_t)n + 1);
            for (int j = 0; j <= n; j++)
            {
                assert(a[i][j] == '0' || a[i][j] == '1');
                if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
            }
        }
        auto get = [&](int i, int j)
        {
            return b[i][j / 64] >> (j % 64) & 1;
        };
        vector<int> pos(n, -1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !get(p, col)) p++;
            if (p == m) continue;
            swap(b[p], b[row]);
            for (int i = 0; i < m; i++)
                if (i != row && get(i, col))
                    for (int j = col / 64; j < w; j++) b[i][j] ^= b[row][j];
            pos[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (get(i, n)) return {false, row, {}, {}};
```

构造特解与零空间基（接上页 solve）（接上页同一结构体）：

```
        Solution ans{true, row, string(n, '0'), {}};
        for (int j = 0; j < n; j++)
            if (pos[j] != -1) ans.particular[j] += get(pos[j], n);
        for (int j = 0; j < n; j++)
            if (pos[j] == -1)
            {
                string v(n, '0');
                v[j] = '1';
                for (int k = 0; k < n; k++)
                    if (pos[k] != -1) v[k] += get(pos[k], j);
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations_mod_2

题意：给定模 2 矩阵 A 和右端 b，输出无解-1，或解空间维数、一个特解和全部零空间基。输出维数是变量数减系数秩，不是 rank。输入每行是连续 01 字符，先把 b 追加到对应行；行列数各至 4096。

所需模板：GaussXor（第 258 页，10.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<string> a(m);
    for (auto &s : a) cin >> s;
    string b;
    cin >> b;
    for (int i = 0; i < m; i++) a[i] += b[i];
    auto sol = GaussXor::solve(a, n);
    if (!sol.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << sol.kernel.size() << '\n';
    cout << sol.particular << '\n';
    for (auto &v : sol.kernel) cout << v << '\n';
}
```

应用题补充：POJ 1681 Painter's Problem

题意：方格 w 为白、y 为黄，每次翻转自身及四邻，求全部变黄的最少操作数，无解输出 inf。按 kuangbin2.10.1 建模，枚举特解异或零空间基的所有子集；不是直接数特解的 1。本地验证到 15×15，原 OJ 题面当前不可达，线上尚待验证。

所需模板：GaussXor（第 258 页，10.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：http://poj.org/problem?id=1681

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        int m = n * n;
        vector<string> a(m, string(m + 1, '0'));
        vector<pair<int, int>> dir{{0, 0}, {0, 1}, {0, -1}, {1, 0}, {-1, 0}};
        for (int i = 0; i < n; i++)
        {
            string s;
            cin >> s;
            for (int j = 0; j < n; j++)
            {
                int id = i * n + j;
                a[id][m] = s[j] == 'w' ? '1' : '0';
                for (auto [di, dj] : dir)
                {
                    int u = i + di, v = j + dj;
                    if (0 <= u && u < n && 0 <= v && v < n) a[id][u * n + v] = '1';
                }
            }
        }
        auto sol = GaussXor::solve(a, m);
        if (!sol.consistent)
        {
            cout << "inf\n";
            continue;
        }
```

继续处理操作（接上页同一 main）：

```
        string x = sol.particular;
        int ans = m;
        function<void(int)> dfs = [&](int k)
        {
            if (k == (int)sol.kernel.size())
            {
                ans = min(ans, (int)count(x.begin(), x.end(), '1'));
                return;
            }
            dfs(k + 1);
            for (int j = 0; j < m; j++) x[j] ^= sol.kernel[k][j] - '0';
            dfs(k + 1);
            for (int j = 0; j < m; j++) x[j] ^= sol.kernel[k][j] - '0';
        };
        dfs(0);
        cout << ans << '\n';
    }
}
```

10.3.2 素数模高斯消元与全部解

代码依赖：ModInt（第 24 页，3.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

GaussMod<p>::solve(a,n) 接收 m 行 n+1 列增广矩阵，p 必须是素数，n≥0。返回 consistent、系数矩阵秩 rank、特解 particular 和零空间基 kernel；解为特解加基向量的任意线性组合。无解时解数组为空；空方程组需显式给出变量数。O(m n min(m,n)+n^2) 时间，矩阵及解空间输出 O(mn+n^2) 空间。只依赖 ModInt，不含行列式或矩阵树。

```
template <int mod> struct GaussMod
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<Z> particular;
        Matrix kernel;
    };

    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
    static Solution solve(Matrix a, int n)
    {
        int m = (int)a.size(), row = 0;
        vector<int> where(n, -1);
        for (const auto &v : a) assert((int)v.size() == n + 1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !a[p][col].v) ++p;
            if (p == m) continue;
            swap(a[p], a[row]);
            Z inv = a[row][col].inv();
            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col].v)
                {
                    Z f = a[i][col];
                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (a[i][n].v) return {false, row, {}, {}};
```

```
        Solution ans{true, row, vector<Z>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<Z> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations

题意：模 998244353 下解 n 个方程、m 个未知数的线性方程组。输入先给系数矩阵，再给常数列。无解输出 -1；否则输出解空间维数、一个特解和齐次解基。

所需模板：ModInt（第 24 页，3.2.2 节）、GaussMod（第 262 页，10.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = GaussMod<998244353>;
    int n, m;
    cin >> n >> m;
    G::Matrix a(n, vector<G::Z>(m + 1));
    for (auto &row : a)
        for (int j = 0; j < m; j++) cin >> row[j].v;
    for (auto &row : a) cin >> row[m].v;
    auto s = G::solve(move(a), m);
    if (!s.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << s.kernel.size() << '\n';
    auto print = [&](const vector<G::Z> &row)
    {
        for (int j = 0; j < m; j++) cout << row[j].v << (j + 1 == m ? '\n' : ' ');
    };
    print(s.particular);
    for (auto &row : s.kernel) print(row);
}
```

应用题补充：SP2883 Widget Factory（POJ2947 同题）

题意：EOF 读 n,m，0 0 结束；每条记录累加重复编号，星期差包含首尾两天。模 7 消元后区分无解、多解；唯一解的余数 0、1、2 加 7，映射到 3..9 天。n,m≤300，每条记录至多 10000 个编号。

所需模板：ModInt（第 24 页，3.2.2 节）、GaussMod（第 262 页，10.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP2883

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using A = GaussMod<7>;
    vector<string> days = {"MON", "TUE", "WED", "THU", "FRI", "SAT", "SUN"};
    int n, m;
    while (cin >> n >> m && n)
    {
        A::Matrix a(m, vector<A::Z>(n + 1));
        for (int i = 0; i < m; i++)
        {
            int k;
            string start, finish;
            cin >> k >> start >> finish;
            int l = find(days.begin(), days.end(), start) - days.begin();
            int r = find(days.begin(), days.end(), finish) - days.begin();
            a[i][n] = r - l + 1;
            while (k--)
            {
                int x;
                cin >> x;
                a[i][x - 1] = a[i][x - 1] + 1;
            }
        }
        auto answer = A::solve(a, n);
        if (!answer.consistent)
            cout << "Inconsistent data.\n";
        else if (answer.rank < n)
            cout << "Multiple solutions.\n";
        else
        {
            for (int i = 0; i < n; i++)
            {
                int value = answer.particular[i].v;
                if (value < 3) value += 7;
                if (i) cout << ' ';
                cout << value;
            }
            cout << '\n';
        }
    }
    return 0;
}
```

10.3.3 实数高斯消元与数值解空间

solve(a, n, eps) 接收 m×(n+1) 增广矩阵，0≤n<INT_MAX、m≤INT_MAX，输入/中间值须有限可表示，0<eps<1。long double 精度依平台；先按每行系数最大绝对值归一化（不含右端项），再部分选主元；绝对值 ≤eps 的候选列在活动行中明确置零。对所有非零消元因子更新，不以 eps 跳过。返回 consistent、rank、particular 和 kernel：均描述最终数值消元模型，不是原系统精确可解性/秩的证书；剩余右端项绝对值 ≤eps 按零处理。原全零系数行保留原始右端尺度，n=0 也用此约定。相容时自由变量取零给特解，kernel 列出自由变量方向；任意大线性组合的原方程误差不受 eps 统一保证。非相容时两个解数组为空；输入按值传入不变。主消元通常 O(mn min(m,n)) 时间，另有初始化；O(m(n+1)+n) 工作空间，另有 O(n(n−rank)) 解基输出。近阈值、病态或非有限过程不能当成数学无解；须结合原方程残差与问题条件判断。正常浮点语义，禁 fast-math；assert 仅调试诊断，非发布模式运行时校验。

```
struct GaussReal
{
    using R = long double;
    using Matrix = vector<vector<R>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<R> particular;
        Matrix kernel;
    };

    // Numerical classification after coefficient-row scaling, not exact rank.
    static Solution solve(Matrix a, int n, R eps)
    {
        assert(0 <= n && n < INT_MAX && a.size() <= (size_t)INT_MAX);
        assert(isfinite(eps) && 0 < eps && eps < 1);
        int m = a.size(), row = 0;
        vector<int> where(n, -1);
        for (auto &v : a)
        {
            assert(v.size() == (size_t)n + 1);
            R s = 0;
            for (int j = 0; j <= n; j++)
            {
                assert(isfinite(v[j]));
                if (j < n) s = max(s, abs(v[j]));
            }
            if (s > 0)
                for (R &x : v)
                {
                    x /= s;
                    assert(isfinite(x));
                }
        }
```

主元消元与数值解空间（接上页同一 solve 函数）：

```
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            for (int i = row + 1; i < m; i++)
                if (abs(a[i][col]) > abs(a[p][col])) p = i;
            if (abs(a[p][col]) <= eps)
            {
                for (int i = row; i < m; i++) a[i][col] = 0;
                continue;
            }
            swap(a[p], a[row]);
            R pivot = a[row][col];
            for (int j = col + 1; j <= n; j++)
            {
                a[row][j] /= pivot;
                assert(isfinite(a[row][j]));
            }
            a[row][col] = 1;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col] != 0)
                {
                    R f = a[i][col];
                    for (int j = col + 1; j <= n; j++)
                    {
                        a[i][j] -= f * a[row][j];
                        assert(isfinite(a[i][j]));
                    }
                    a[i][col] = 0;
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (abs(a[i][n]) > eps) return {false, row, {}, {}};
        Solution ans{true, row, vector<R>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<R> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = -a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

接口用法演示：实数线性系统（自定义数值协议）

演示约定：接口演示，非官方题目或线上 AC。输入 m n eps 及 m 行各 n+1 个实数；0≤m,n≤100 只是演示规模，输入和全部中间值有限，0<eps<1，需结合尺度/条件数选取。输出数值 consistent 与 rank；相容时再输出一个特解、自由方向个数和逐行方向向量。行系数最大值归一化后进行阈值消元，原全零系数行的右端项保持原尺度。不可把结果当作精确无解/唯一解证书。例中的 0.5 E0−0.5 E1=1、−0.5 E0+E1=1 来自连续两个正面等待时间，得到 E0=6,E1=4；概率模型有限性另行证明。

所需模板：GaussReal（第 265 页，10.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/gauss_real.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    long double eps;
    cin >> m >> n >> eps;
    GaussReal::Matrix a(m, vector<long double>(n + 1));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    auto r = GaussReal::solve(a, n, eps);
    cout << r.consistent << ' ' << r.rank << '\n';
    if (!r.consistent) return 0;
    cout << setprecision(20);
    for (auto x : r.particular) cout << x << ' ';
    cout << '\n' << r.kernel.size() << '\n';
    for (const auto &v : r.kernel)
    {
        for (auto x : v) cout << x << ' ';
        cout << '\n';
    }
}
```

第十一章　Berlekamp–Massey 算法

11.1　Berlekamp–Massey：最短线性递推

berlekamp_massey(s,p) 要求 p 为素数，2≤p≤INT_MAX，输入均在 [0,p)；返回 c，使 s[i]=∑ c[j-1]s[i-j] (j=1..k，i≥k)。空序列或全零序列返回空系数。有限前缀可能对应多种未来，不能无条件外推；若已知原序列满足阶数至多 d 的齐次线性递推，提供至少 2d 项才能恢复可外推的递推。模运算次数 O(Nk+N log p)，后项来自逐次求逆；空间 O(N)。不适用于合数模数或浮点拟合。

```
// Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
{
    assert(p >= 2);
    vector<int> c{1}, b{1};
    int len = 0, shift = 1, last = 1;
    for (int i = 0; i < (int)s.size(); i++)
    {
        assert(0 <= s[i] && s[i] < p);
        long long d = s[i];
        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
        if (d == 0)
        {
            shift++;
            continue;
        }
        long long inv = 1, x = last;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) inv = inv * x % p;
            x = x * x % p;
        }
        long long factor = d * inv % p;
        auto old = c;
        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
        for (int j = 0; j < (int)b.size(); j++)
            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
        if (2 * len <= i)
        {
            len = i + 1 - len;
            b = old;
            last = d;
            shift = 1;
        }
        else
            shift++;
    }
    c.resize(len + 1);
    c.erase(c.begin());
    for (int &x : c) x = x == 0 ? 0 : p - x;
    return c;
}
```

模板题使用：Library Checker find_linear_recurrence

题意：给模 998244353 下的有限序列，求最短线性递推。c[0] 乘前一项，输出阶数及系数；空序列或全零序列可以是 0 阶。递推只保证对已给序列成立，不能仅凭前缀推定未知后续。

所需模板：berlekamp_massey（第 268 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/find_linear_recurrence

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto c = berlekamp_massey(a);
    cout << c.size() << '\n';
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

联合使用示例见第 166 页（Luogu P5487），其中直接调用本模板。

第十二章　配套知识与验证范围

12.1　杜教筛：预筛与缓存都不可省略

整除反演建模（第 3.20.2 节，第 97 页）的两条卷积恒等式给出 `DuJiao` 的递推，见代码中的 `mertens` 与 `totient_sum`。把商相同的一段合并，系数为区间长度，递归参数严格变小；0 直接查表返回 0。超过预筛上限的结果在返回时写入缓存，后续相同参数直接复用。WIDA 收集的 `sumMu` 示例查询了 `fMu`，但未写入新结果；本库已有实现包含写回，不能照搬遗漏缓存的版本。

设最大查询为 N，预筛上限为 B。单次查询的常用上界为 $O(B+N/\sqrt{B})$，其中哈希访问按期望常数处理；取 B 约 $N^{2/3}$ 得到 $O(N^{2/3})$。多个不同查询可能继续增加缓存，不能把单次空间估计当成任意多询问的上界。保留缓存可复用，主动清空仅影响重算开销；预筛表与上限应视为只读。

接口范围为 $0 \le n \le 10^{11}$，返回精确整数；欧拉函数前缀和用 128 位。WIDA 的 `Z` 模数结果可在精确莫比乌斯前缀和返回后取模得到。负值先求余，仅在余数为负时加模数，避免无条件加模数的溢出风险。

两套实现以独立试除构建到 200000 的前缀参考，检查四种预筛上限、零输入、乱序与重复查询、缓存写回及清空重建。上限 10^{11} 使用 10^6 与 2×10^6 两种预筛规模核对双版本结果；这一项属于边界运行与一致性证据，不是独立答案对拍。普通与 ASan/UBSan 均通过，暂无该接口的在线 AC。

12.2　64 位判素与质因数分解

本库判素采用七个固定底数的 Miller–Rabin，覆盖无符号 64 位整数：

$$2,\ 325,\ 9375,\ 28178,\ 450775,\ 9780504,\ 1795265022.$$

将 $n-1=d2^s$、d 为奇数，检查 a^d 及后续平方链。小于 2 的输入不视为素数；底数模 n 为 0 时跳过该底数，不可误判为合数。模乘用无符号 128 位容纳完整的两个 64 位数之积。固定底数的适用范围依据公开 SPRP 记录，不外推到任意精度整数。

https://miller-rabin.appspot.com/

Pollard–Rho 用 $f(x)=x^2+c \pmod n$ 迭代，并求 $\gcd(|x-y|,n)$。只有 $1<d<n$ 的因子才返回；一次尝试超过轮数上限或得到 $d=n$，就换参数重试。其中模平方后的加法也在 128 位中执行，避免 n 接近 2^{64} 时加 c 溢出。该过程保证返回真因子，但随机尝试次数没有确定的最坏上界。递归分解两侧，直到判为素数，最后排序并保留重复因子。

调用 `factor(1)` 得到空数组；对 $n\ge2$，返回单元素说明 n 为素数。POJ 1811 所需的最小质因子可取排序结果首项；P4718 所需的最大质因子可取末项。不要对空数组取首尾，也不要绕过 `factor` 把素数传给内部 `rho`。

双版本对百万范围以独立埃氏筛核验判素，对 $1\ldots10000$ 以试除核验完整因子列表。另验证三个随机种子下的大质数平方、重复因子、强伪素数和 $2^{64}-1$；期望因子的素性均以试除独立验证。$2^{61}-1$ 使用完整分解 $n-1$ 的 Lucas 素性证书检查；模乘与模幂另以任意精度运算对拍。完整 POJ 1811 与 P4718 驱动分别验证 350 组输入，检查素数标签与最小、最大质因子。P4718 压力批次含 300 个接近 10^{18} 的平衡半素数，本机普通编译运行约 1.99、1.83 秒；机器差异与随机路径都会影响耗时，本地运行时间不直接代表线上性能。两套代码随后通过 P4718 全部 20 个测试点，最长单点显示约 2.11、2.12 秒，低于当前页面的 4 秒限制。该 AC 检查素数标签与最大质因子，范围为 $2\le n\le10^{18}$。完整因子重数、最小质因子、$n=1$ 与更大的无符号 64 位输入仍依据本地验证；普通与 ASan/UBSan 均通过，不构成随机运行时间保证。

来源：kuangbin 2.6；固定底数记录见上方链接。

12.3 单个数的欧拉函数

实现见第 41 页。对 $n \geq 1$，由容斥得到

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right),$$

其中只枚举不同质因子，故质数幂 p^k 的贡献为 $p^{k-1}(p-1)$。每次更新按先除后乘的顺序：$\text{ans} \leftarrow (\text{ans}/p)(p-1)$。当前尚未处理的质因子仍整除答案，而且答案只会减小，乘法结果不会超过原始 n。

无分解器参数的入口采用试除，循环上界用 $p \leq n/p$，剩下的 $n > 1$ 再处理一次。这能避免 p^2 溢出，但不消除 $O(\sqrt{n})$ 的耗时；不适合试除 64 位大素数。传入现有 `PollardRho` 对象时先分解，再在排序后的因子列表上去重更新；不重复构造分解器，后续调用沿用其随机状态。该入口没有确定的最坏运行时间。$\varphi(1) = 1$，两个入口均接受完整的正无符号 64 位输入；0 不在定义域内。

本地双版本验证包含 $1 \ldots 20000$ 的线性筛交叉检查、小范围直接计数互质整数，以及重复质因子、大质数平方、$2^{64} - 1$ 和已独立认证的大素数。普通与 ASan/UBSan 测试均通过；P4718 的 AC 只验证底层分解用途，不是此欧拉函数的新 AC。来源：kuangbin 2.7.1、2.7.3；WIDA 的单个数欧拉函数。

12.4 线性筛：三张表的约定

`LinearSieve` 同时构造最小质因子、欧拉函数和莫比乌斯函数。内层遇到第一个整除 i 的素数后停止，因此每个合数 j 只由 $p = \text{lp}(j)$ 与 $i = j/p$ 生成一次，时间为 $O(n)$。若 $p \mid i$，则 $\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$；否则二者分别为 $(p-1)\varphi(i)$ 和 $-\mu(i)$。

判素使用 $x \geq 2$ 且 $\text{lp}(x) = x$，不能漏掉前半条件。分解表内正整数时，反复取 $p = \text{lp}(x)$、令 $x \leftarrow x/p$，直到 $x = 1$；重复因子会重复输出。0 不可分解，1 的质因子列表为空。上限要求 $0 \leq n <$ `INT_MAX`，避免构造时 $n+1$ 溢出，并保证内存可用。筛乘积前先检查 $p \leq n/i$，不直接计算可能溢出的 ip。

双版本验证至 $n = 10^7$，包括小前缀、试除最小质因子、因子还原和逐项约数和证书：

$$\sum_{d \mid x} \varphi(d) = x, \qquad \sum_{d \mid x} \mu(d) = [x = 1].$$

这些恒等式在 $d = x$ 项系数为 1，可按 x 递增唯一确定两张表。普通与 ASan/UBSan 均通过；不据此宣称莫比乌斯反演的所有应用已验证。

来源：kuangbin 2.1.1、2.1.2、2.7.2、2.7.4、2.13.1；WIDA 的全部欧拉函数表。

WIDA 的“使用莫比乌斯反演求解欧拉函数”先存储全部约数，再使用

$$\varphi(n) = n - \sum_{d \mid n,\ d < n} \varphi(d).$$

这与 $\varphi * 1 = \text{id}$ 等价；再卷积 μ 可得 $\varphi(n) = \sum_{d \mid n} \mu(d)(n/d)$。本库用线性筛提供同一张表，无需额外保存约数列表。百万范围另分别运行直接卷积和真约数扣减，逐项核对两套筛表；两种方法都不复用线性筛的递推分支。

12.5 区间筛：只保存目标区间

实现见第 43 页。设 $0 \leq L \leq R \leq$ `INT_MAX`，先筛出不超过 $\lfloor \sqrt{R} \rfloor$ 的素数，再对每个素数 p 从

$$\max\bigl(p^2, \lceil L/p \rceil p\bigr)$$

开始标记区间内的倍数。任何合数都有不超过自身平方根的素因子，所以不会遗漏；从 p^2 开始也不会误删 p 自身。输出跳过 0 和 1，其余未标记数恰为素数。底筛与区间标记共用同一个外层素数循环，但使用两张独立的表。

令 $W = R - L + 1$，空间为 $O(\sqrt{R} + W)$，包含返回的素数数组；时间为 $O((\sqrt{R} + W)\log\log(R+3))$。函数每次调用都重建底筛，适合题目中的少量区间查询。端点虽然是 `int`，向上取整的加法、p^2、标记变量和输出遍历变量均用 `long long`；否则 $R =$ `INT_MAX` 时最后一次递增也会溢出。区间长度仍受可用内存约束，不适合一次保存整个 32 位值域。

POJ 2689 与 UVa 10140 的驱动读到 EOF，扫描相邻素数差。最近、最远都只在严格更优时更新，距离并列保留最先出现的一对；少于两个素数时输出题目指定句子。两套实现以试除和 Miller–Rabin 判素进行独立对拍，并检查平方边界、单点区间及右端点为 `INT_MAX` 的百万长度区间；完整驱动验证 208 组输入，含官方样例。普通测试与 ASan/UBSan 均通过；尚无本库在线 AC。

来源：kuangbin 2.1.3；https://onlinejudge.org/external/101/10140.pdf。

12.6 矩形 GCD 计数：缩放后容斥

实现见第 96 页。由 $[g=1]=\sum_{d\mid g}\mu(d)$，前缀矩形互质数对数为

$$F(A,B)=\sum_{i=1}^{\min(A,B)}\mu(i)\left\lfloor\frac{A}{i}\right\rfloor\left\lfloor\frac{B}{i}\right\rfloor.$$

两个商同时不变的右端点取 $r=\min(A/\lfloor A/l\rfloor,B/\lfloor B/l\rfloor)$，用莫比乌斯前缀和合并 $[l,r]$。不能只按一个商分块。只需筛到 $\min(A,B)$，不要求较长的一边也在预处理范围内。$A=0$ 或 $B=0$ 时循环为空，返回 0；正整数条件不包含坐标轴。

矩形内 $\gcd(x,y)=k$ 等价于 x,y 均为 k 的倍数，且缩放后互质。因此答案为四个前缀矩形的容斥：

$$\begin{aligned}&F(\lfloor b/k\rfloor,\lfloor d/k\rfloor)-F(\lfloor(a-1)/k\rfloor,\lfloor d/k\rfloor)\\&-F(\lfloor b/k\rfloor,\lfloor(c-1)/k\rfloor)+F(\lfloor(a-1)/k\rfloor,\lfloor(c-1)/k\rfloor).\end{aligned}$$

这里必须先减 1 再除以 k；零商无需特殊补偿。输入端点为正 `int`，最终答案不超过矩形面积，能用有符号 64 位保存。分块累加使用 128 位，避免依赖正负中间项的额外界。

双版本以小矩形逐对求 gcd 为独立参考，另验证百万正方形的 $F(N,N)=2\sum_{i=1}^{N}\varphi(i)-1$，以及按短边剩余类直接计数的 `INT_MAX` 长条。P2522 双版本完整程序验证五万次查询，包括缩放、窄边和无解；普通与 ASan/UBSan 均通过。尚无本库在线 AC。

来源：kuangbin 2.13.2（BZOJ 2301）；https://www.luogu.com.cn/problem/P2522。

12.7 幂的约数和：不除以 $p-1$

实现见第 18 页。若 $a=\prod p_i^{e_i}$，则 a^b 的约数和是 $\prod_i(1+p_i+\cdots+p_i^{e_ib})$。几何级数的除法公式在模意义下要求分母可逆；合数模数或 $p\equiv1$ 时不能直接套用。

令 $G(a,n)$ 同时返回 $P=a^n$ 与 $S=\sum_{i=0}^{n-1}a^i$。先递归求一半项数 $h=\lfloor n/2\rfloor$ 的 (P,S)；偶数项的结果为 $(P^2,S(1+P))$，奇数项再补最后的 $a^{2h}=P^2$，得到 $(P^2a,S(1+P)+P^2)$，全过程取模。$n=0$ 的结果为 $(1,0)$，因此底数为 0 时，非空级数的和仍为 1。本接口参数是项数，不是最高指数；约数和必须传 e_ib+1。

把幂与和一起返回，可使每层只做常数次运算，总时间 $O(\log(n+1))$。长度允许无符号 128 位，避免 e_ib 在 64 位中溢出；模数及底数允许完整无符号 64 位。所有模乘与必要的加法在 128 位中执行，任意正模数均可用。底数 $a\geq1$，$b=0$ 或 $a=1$ 都返回 $1 \bmod m$；0 的约数和不在接口定义域内。

双版本以直接枚举约数、逐项几何求和和任意精度矩阵幂独立核验，包括模数 1、合数模数、完整 128 位项数，以及指数乘重数超过 64 位的案例。P1593 完整程序各验证 126 组输入，包含 $p\equiv0,1 \pmod{9901}$ 和最大输入；普通与 ASan/UBSan 均通过，在线 AC 待补。

来源：kuangbin 2.12；https://www.luogu.com.cn/problem/P1593。

12.8 任意数组的批量逆元

设 $s_i=\prod_{j<i}a_j \pmod m$，$s_0=1$。仅对总乘积 s_n 求一次逆元，随后从右向左处理。若当前 $t=s_{i+1}^{-1}$，则

$$a_i^{-1}=ts_i \pmod m,\qquad s_i^{-1}=ta_i \pmod m.$$

先保存原 a_i 再覆盖输出，避免误用已写入的逆元更新 t。总乘积与 m 互素当且仅当每一项均与 m 互素，因此一次扩展欧几里得也能判断整个批次是否可行；模数无需为素数。实现见第 56 页。

接口返回可选向量：无解是空的可选值，空输入是有效的空向量，两者不同。模数为 1 时按接口约定返回全零。调用者可以移动不再使用的输入数组，避免额外复制。P5431 的驱动程序计算 $\sum_{i=1}^{n} k^i/a_i$，注意权重从 k^1 开始。五百万项规模采用缓冲的字符读取，并移动输入以控制峰值内存。双版本已做乘法证书和非互素拒绝的穷举检查，以及 64 位极值、空数组、模数 1 的检查；完整驱动另用逐项逆元和几何级数进行对拍。P5431 双版本均通过全部 5 个测试点；动态版记录 297561197，传统版记录 297561804。最慢单点分别为 386、384 毫秒，峰值内存均为 76.80 MB。合数模数、不可逆输入和完整 64 位范围以上述本地测试验证。

该方法可处理任意可逆数组；连续整数的专用递推见下一节。

来源：https://oi-wiki.org/math/number-theory/inverse/。

12.9 连续整数的线性逆元表

素数模数 p 下，对 $1 < i < p$，写 $p = qi + r$，其中 $q = \lfloor p/i \rfloor$，$r = p \bmod i$。r 非零，且 $r < i$。将等式模 p 后乘以 $i^{-1}r^{-1}$，得到

$$i^{-1} \equiv -\lfloor p/i \rfloor \, (p \bmod i)^{-1} \pmod{p}.$$

从 $1^{-1} = 1$ 开始递推，右侧总是已算过的较小下标，故总时间为 $O(N)$。实现见第 58 页；第 0 项只作占位，允许 $N = 0$。$N \geq p$ 时区间包含无逆元的 p，不能套用该表。

递推还依赖 r 可逆，不能仅凭 i 与合数模数互素就照搬。例如 $i = 3, m = 8$ 时 $r = 2$ 不可逆。kuangbin 的“简洁写法”递归式也有这一限制，本书以一般扩展欧几里得接口代替其不成立的合数推广。若 $\gcd(a, m) = 1$，扩展欧几里得给出 $ax + my = 1$，归一化 x 即为逆元；若不互素，则整数方程无解。素数下也可用 a^{p-2} 求逆元；kuangbin 标为“利用欧拉函数”的代码实际使用这一费马形式。

P3811 双版本完整程序逐行输出逆元，已在本地对三百万行结果逐项检验 $ix_i \equiv 1$。P3811 双版本均通过全部 6 个测试点，单点最长分别为 221 ms 与 219 ms。单个逆元另用小模数穷举与费马乘法证书交叉验证；这部分不由 P3811 的 AC 覆盖。

来源：https://oi-wiki.org/math/number-theory/inverse/；WIDA 数论逆元节与 kuangbin 2.4。

12.10 线性同余：最小解与最小周期

实现见第 59 页，接口约定为 $ax + b \equiv 0 \pmod{m}$。先把 a 归一化到 $[0, m)$，令 $g = \gcd(a, m)$。有解当且仅当 $g \mid b$；若 $au + mv = g$，则一个特解为 $u(-b/g)$，所有解为 $x = x_0 + (m/g)t$。归一化到 $[0, m/g)$ 后得到最小非负解。除以 g 后 a/g 与 m/g 互素，因而 m/g 是最小正周期。当 $a = 0$ 且 $m \mid b$ 时所有整数均为解，返回 $(0, 1)$；否则无解。模数 $m = 1$ 始终返回 $(0, 1)$。

输入常数项支持 `LLONG_MIN`，内部用 `-i128(b)`，不能先执行 64 位的 `-b` 再转换。若题目给出 $ax \equiv c \pmod{m}$，可先将 c 归一化到 $[0, m)$ 再取负传入，避免 $c =$ `LLONG_MIN` 时在调用点溢出。本接口依赖的 exgcd 见第 3.7.3 节（第 50 页），不必重复抄写。

P1516 中相遇条件为 $(m - n)t + (x - y) \equiv 0 \pmod{L}$。传入速度差与起点差，第一返回值就是最早非负相遇时间；失败时输出 `Impossible`。不能擅自把负的速度差取绝对值，也不能把两者同余的初始位置强制判为需要跳跃。

小模数测试逐个枚举全部剩余类，检查解集、最小解和最小周期；大数用任意精度等式证书验证。完整驱动以直接模拟、交换青蛙和构造大周期最早相遇时间验证 242 组输入，包括等速和初始位置同余。两套代码均通过普通与 ASan/UBSan 测试，在线 AC 待补。

来源：WIDA 收集的 jiangly exgcd 文件内 `sol`；https://www.luogu.com.cn/problem/P1516。

12.11 扩展 CRT：逐式合并

kuangbin 2.5 的 `solve` 对应单次合并，`MLES` 对应下面的逐式循环。实现见第 60 页的 `crt_merge`，初始取 $r = 0, m = 1$，依次合并 $x \equiv b \pmod{n}$。空方程组保持 $(0, 1)$，表示任意整数；一旦无解即停止，不能忽略失败继

续合并。先把余数归一化。设 $g=\gcd(m,n)$，由扩展欧几里得得到 $mu+nv=g$。写 $x=r+mt$，原问题化为 $mt\equiv b-r \pmod n$。若 $g\nmid(b-r)$ 则无解，否则

$$q=n/g,\qquad t\equiv\frac{b-r}{g}u \pmod q,\qquad 0\le t<q.$$

新周期为 mq，新余数为 $(r+mt)\bmod(mq)$。所有可行 t 相差 q，故新余数加新周期的任意整数倍恰好是全部解；逐式合并时这一不变量保持成立。模数不必互素，模数 1 也可直接参与。

参考实现使用 `int` 保存合并状态，本库扩展为 64 位并显式报告周期越界；避免可能溢出的 `abs(a-a0)`。

代码要求正模数且最终周期不超过有符号 64 位上限；中间乘法与扩展欧几里得系数使用 128 位整数。无解返回假；有解而周期越界抛出异常。失败时旧周期不变，旧余数已归一化。

k 条方程的时间为 $O(k\log M)$，其中 M 是输入及中间模数的上界；递归欧几里得使用 $O(\log M)$ 栈空间。

P4777 的完整程序按 “模数、余数” 读入，题目保证有解且周期不超过 10^{18}。本地验证包含双版本小模数穷举、任意精度整数对拍、有符号极值、无解与越界，以及十万条同余式的完整程序测试。另穷举至多三个小模数方程组成的所有有序系统，检查空系统与后续矛盾。双版本均已通过 P4777 全部 15 个测试点；动态版记录 297521102，传统版记录 297521173。

kuangbin 的“解同余方程组”是模 7 的线性方程组消元应用，见后面的工厂工期一节。来源：https://oi-wiki.org/math/number-theory/crt/；WIDA 数论模板的 excrt 节。

12.12 Garner：总周期可以超过整数范围

设正模数 m_i 两两互素，$P=\prod_i m_i$。经典构造为 $x=\sum_i b_iP_i(P_i^{-1}\bmod m_i)\bmod P$，其中 $P_i=P/m_i$。模 m_i 时其他项都为零，本项为 b_i；两解之差被所有 m_i 整除，因而被 P 整除。但直接写此式需要存储整个 P，并防止乘法溢出。

Garner 改用混合进制

$$x=d_0+d_1m_0+d_2m_0m_1+\cdots,\qquad 0\le d_i<m_i.$$

在确定第 i 位前，对每个后续模数维护已确定部分的余数 v_j，以及当前位权 $c_j=\prod_{t<i}m_t \bmod m_j$。于是

$$d_i=(b_i-v_i)c_i^{-1}\bmod m_i.$$

把 c_jd_i 加到 v_j，再将 c_j 乘 m_i 即可进入下一位。互素性保证逆元存在；模数 1 的位固定为 0，可直接保留。位数范围保证重建的 x 恰在 $[0,P)$，因此就是最小非负解。实现见第 62 页。

若只需 $x \bmod T$，从高位向低位做 Horner 计算：每步将当前值乘对应模数，加上该位，再对 T 取模。T 无需与 m_i 互素，因为这里不对 T 求逆。这返回的是规范解 x 的余数；一般解 $x+kP$ 对 T 的余数可能不同。例如模 3 余 2、模 5 余 3，规范解是 8；对 7 取模为 1，但另一个解 23 的余数是 2。若用多模 NTT 恢复整数系数，仍须证明真实系数位于所选规范区间，不能在乘积不够大时直接把 CRT 的代表元当成原整数。

双版本本地验证使用任意精度整数的经典 CRT 求和作为独立对拍，检查混合进制的完整重建、负余数、有符号极值、非互素输入拒绝和 1000 个模数。P1495 完整程序按题目保证的 $P\le10^{18}$ 输出规范解，双版本均通过全部 11 个测试点；动态版记录 297521594，传统版记录 297521658。超出该题范围的能力以上述本地测试验证。来源：https://oi-wiki.org/math/number-theory/crt/。

12.13 素数模平方根

奇素数 p 下，对 $n\not\equiv0 \pmod p$，Euler 判别式 $n^{(p-1)/2}\equiv1 \pmod p$ 当且仅当存在平方根，否则为 -1。当 $p\equiv3 \pmod 4$ 且有解时，$r=n^{(p+1)/4}$ 为一根，另一根是 $p-r$。

Cipolla 选择 a 使 $w=a^2-n$ 是非平方，令 $\omega^2=w$。在扩域中取 $\alpha=a+\omega$，有 $\alpha^p=a-\omega$，故 $\alpha^{p+1}=n$。令 $\beta=\alpha^{(p+1)/2}$，则 $\beta^2=n$ 且 $\beta^{p-1}=n^{(p-1)/2}=1$，所以 β 落在原素数域中。实现见第 80 页；零和模数 2 单独处理。模数为合数时不能照搬此判别和扩域构造。

12.14 乘法阶与原根

对 $\gcd(a,n)=1$，阶是最小的正整数 d 使 $a^d\equiv 1 \pmod n$，且 $d\mid\varphi(n)$。从 $d=\varphi(n)$ 开始，对每个素因子 q，只要 $a^{d/q}\equiv 1$ 就继续除去 q，所得即为实际阶。对于不可逆的 a，接口返回 0，表示此处没有定义乘法阶。

原根要求阶达到 $\varphi(n)$。只需对其每个不同素因子 q 检查 $a^{\varphi(n)/q}\not\equiv 1$：若实际阶是真因子，商中必含某个这样的 q。必须先检查 $\gcd(a,n)=1$；例如模 9 的候选 3 虽通过上述非 1 判定，却不是原根。$n\ge 2$ 时，原根恰存在于 $2,4,p^k,2p^k$，其中 p 是奇素数。原根存在且取一个原根 g 后，g^t 的阶是 $\varphi(n)/\gcd(t,\varphi(n))$，所以枚举与 $\varphi(n)$ 互素的指数即可得到全部原根，共 $\varphi(\varphi(n))$ 个；这些数按大小输出时还需排序。

实现见第 81 页。本地验证穷举小模数的幂循环，与双版本的阶、最小值和完整集合逐项比较；另验证大素数、奇素数幂、无原根合数和有符号输入边界。P6091 的完整程序输出排序后第 $d,2d,\dots$ 个原根，零个输出时仍保留空行。双版本均通过全部 11 个测试点；动态版记录 297558740，传统版记录 297559082。求阶、最小原根的最小性及更大输入范围仍以上述本地证据验证。若 $d=\operatorname{ord}_n(a)$，则 $\operatorname{ord}_n(a^k)=d/\gcd(d,k)$。两个可逆元素乘积的阶整除各自阶的最小公倍数；两阶互素时，乘积的阶等于两阶之积。来源：https://oi-wiki.org/math/number-theory/primitive-root/；kuangbin 2.17。

12.15 Carmichael：最小公共指数

$\lambda(n)$ 是模 n 的所有可逆元素之阶的最小公倍数，约定 $\lambda(1)=1$。它整除 $\varphi(n)$，但可以严格更小，例如 $\lambda(8)=2$ 而 $\varphi(8)=4$。按互素的素数幂分解 $n=\prod_i p_i^{e_i}$ 后，由 CRT，对每个分量同时成立的幂等式也对 n 成立，因此

$$\lambda(n)=\operatorname{lcm}_i\lambda(p_i^{e_i}),\qquad \lambda(p^e)=\begin{cases}2^{e-2}, & p=2,\ e\ge 3,\\ p^{e-1}(p-1), & \text{其他情形.}\end{cases}$$

奇素数幂的单位群有原根，公共指数即其大小。模 2^e 的奇数可表示为 $\pm 5^k$，当 $e\ge 3$ 时 5 的阶为 2^{e-2}，这给出二幂情形的上界和达到上界的元素。实现见第 53 页，试除版本适用于有符号 32 位正模数。

对可逆底数可把指数对 $\lambda(n)$ 取模；非互素底数仍须另行处理。例如模 8 下 $2^2=4$，但 $2^{2\bmod\lambda(8)}=1$，直接降幂会错误。另有 $\lambda(\operatorname{lcm}(a,b))=\operatorname{lcm}(\lambda(a),\lambda(b))$，这也提供了独立的组合检查条件。

合数 n 是 Carmichael 数当且仅当 $\lambda(n)\mid(n-1)$。等价的 Korselt 判别是：n 无平方因子，且每个素因子 p 都满足 $(p-1)\mid(n-1)$。素数也满足前一整除式，判定时必须排除素数。这些合数对所有互素底数都通过 Fermat 判定，不能用单纯的 Fermat 检验代替素性判定。无平方因子再配合 CRT，也使所有整数 a 满足 $a^n\equiv a \pmod n$。

双版本以小模数的全部实际阶之最小公倍数对拍，另检验大二幂、奇素数幂、32 位边界及公共指数证书。

UVa10006 完整程序对全部合法输入进行本地 Korselt 对拍；在线 AC 待补。

来源：https://oi-wiki.org/math/number-theory/primitive-root/；

应用题：https://onlinejudge.org/external/100/10006.pdf。

12.16 Lucas：按素数进制逐位计算

在特征为素数 p 的域中，$(1+x)^p=1+x^p$。若 $n=\sum_i n_i p^i$，则

$$(1+x)^n=\prod_i (1+x^{p^i})^{n_i}.$$

每个因子选择的次数在 $0,\dots,n_i$ 之间，严格小于 p，因此位数表示唯一。比较 x^k 的系数即得 $\binom{n}{k}=\prod_i\binom{n_i}{k_i}\pmod p$；若某位 $k_i>n_i$，答案立即为零。仅预处理 $0!,\dots,(p-1)!$ 及其逆元，这些数均非零。$p!$ 为零，不能把阶乘表延长后继续用除法公式。实现见第 68 页。

预处理后每位只做常数次乘法，适合固定小素数模数下的多次查询。接口接收 n,k，不接收“n,m 表示选 n 个”的隐含约定；P3807 的驱动程序显式调用 choose$(n+m,n)$。题目范围内求和安全，完整 64 位输入自行求和时仍要检查溢出。本地验证包含 Pascal 三角、精确大整数、小模数重建和最大无符号 64 位下标；P3807 双版本均已通过全部 25 个测试点；动态版记录 297556209，传统版记录 297556596。完整 64 位参数及额外边界以上述本地对拍验证。来源：https://oi-wiki.org/math/number-theory/lucas/。

12.17 合数模数组合数与插值

对素数幂 $q=p^e$，记 $v_p(n!)=\sum_{j\ge1}\lfloor n/p^j\rfloor$，$U_p(n)=n!/p^{v_p(n!)}$。设 $t=v_p(n!)-v_p(k!)-v_p((n-k)!)$。当 $t\ge e$ 时组合数模 q 为零；否则

$$\binom{n}{k}\equiv U_p(n)\,U_p(k)^{-1}\,U_p(n-k)^{-1}\,p^t \pmod q.$$

这里仅对与 p 互素的单位部分求逆。各素数幂上的结果通过 CRT 合并，对应第 69 页的实现。

在素数域中，n 个模 p 两两不同的横坐标确定唯一的次数小于 n 的多项式。定义 $w_i=y_i/\prod_{j\ne i}(x_i-x_j)$，则

$$f(k)=\sum_{i=0}^{n-1} w_i\prod_{j<i}(k-x_j)\prod_{j>i}(k-x_j).$$

该式不除以 $k-x_i$，查询点命中样本点时仍然成立。当 $x_i=i$ 时，分母为 $i!\,(n-1-i)!\,(-1)^{n-1-i}$，要求 $n\le p$。实现见第 251 页。对整数幂前缀和 $S_d(k)=\sum_{i=1}^{k} i^d$，当 $0\le d\le p-2$ 时，它在模 p 意义下可用次数至多 $d+1$ 的多项式表示；取 $0,1,\dots,d+1$ 的样本即可。不能只因某序列前几项能插值，就断言后续仍由该多项式给出。

12.18 阶乘表：组合数与排列数

当素数模数为 p 且 $0\le k\le n<p$ 时，阶乘均非零，因此

$$C(n,k)=n!\,(k!)^{-1}\,((n-k)!)^{-1},\qquad P(n,k)=n!\,((n-k)!)^{-1} \pmod p.$$

将表从旧上限扩展到 N，先正向补齐阶乘，只对 $N!$ 求一次逆元，再用 $(i-1)!^{-1}=i\,(i!)^{-1}$ 反向补齐新逆阶乘项即可。不需要每一项分别快速幂求逆。较小上限的初始化请求保留已有表。查询前显式扩表，之后查询为常数时间；内部两表重分配时旧元素引用不能继续使用。实现见第 177 页。

WIDA 的“组合数 (Lucas)”文件实际提供的是普通阶乘、逆阶乘、组合数和排列数，没有逐位 Lucas 递推。本书以此节覆盖该文件的数学功能，用显式扩表取代查询时自动扩表，适用范围仍须限制在模数以内。验证使用独立 Pascal 三角和下降阶乘，另检查百万项表及扩表前后的原查询结果。

12.19 组合计数

容斥：$|\bigcup A_i|=\sum_{\emptyset\ne S}(-1)^{|S|+1}|\bigcap_{i\in S}A_i|$。二项式反演：若 $b_n=\sum_{k=0}^{n}\binom{n}{k}a_k$，则 $a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k$。

$$\sum_k\binom{a}{k}\binom{b}{n-k}=\binom{a+b}{n},\qquad C_n=\binom{2n}{n}-\binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$，在模数下尤其有用，但组合数本身仍须正确求模。斯特林数的定义、快速行列求解与反演统一见第 189 页。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n,k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^{k}S(n,j)$	$S(n,k)$

上表默认 $k\ge1$；$n=0$、$k=0$ 等边界应按原问题约定单独处理。

12.20 生成函数与递推

普通生成函数 $A(x)=\sum a_nx^n$ 的乘法对应普通卷积。指数生成函数 $A(x)=\sum a_nx^n/n!$ 的乘法对应 $c_n=\sum_k\binom{n}{k}a_kb_{n-k}$。形式幂级数求逆要求常数项可逆；$\log A$ 取 $A(0)=1$，$\exp A$ 取 $A(0)=0$。有限域中积分涉及 $1/i$，必须避免 i 被特征整除。

$$B_{\text{new}}=B(2-AB) \pmod{x^{2m}},\qquad (\log A)'=A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用，必须另外证明序列确实具有不超过相应阶数的线性递推；少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

12.21 线性代数与生成树

域上的线性方程组一致时，解集为一个特解加零空间，零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值；模素数消元是精确域运算；合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数，非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树；重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向，不能直接复用无向封装。

12.22 模线性方程组：工厂工期

POJ2947（同题 UVa1564）给出每位工人制造的产品类型和起止星期。设第 j 种产品需 x_j 天，第 i 条记录中出现 a_{ij} 次，以星期一为 0、星期日为 6，则

$$\sum_j a_{ij}x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod 7.$$

起止两天均计入；同一种产品重复出现必须累加系数。跨星期无需特判，模数类型负责归一化负数。每个余数在 $[3,9]$ 中恰有一个代表，因此模方程的解数就是原问题的解数。

直接使用第 262 页的模高斯消元，模数取 7。增广矩阵有 m 行、$n+1$ 列，最后一列是右端项。先判断 `consistent`：假时输出无解；为真且秩小于 n 时输出多解；否则读取特解，余数小于 3 的分量加 7。不能先以秩不足判断多解，因为零系数行仍可能有非零右端项。本题所有非零系数均可逆；合数模数不具备这一性质，不能直接套用此消元。

完整双版本程序位于 `verify/poj/2947.*.cpp`。验证包含所有起止星期组合、产品重复 7 次导致系数归零、矛盾的零行、独立枚举所有 3～9 天赋值，以及用可逆行变换构造的 300 阶稠密系统。该应用验证覆盖分类及唯一解输出，不代替零空间基等其他接口的验证。来源：`https://onlinejudge.org/external/15/1564.pdf`；kuangbin 2.10.2。

12.23 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^{k} c_{j-1}a_{i-j}$，初值是 $a_0, \ldots, a_{k-1}$。对应特征多项式为

$$F(x) = x^k - c_0x^{k-1} - c_1x^{k-2} - \cdots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$。代码由高次向低次消去多项式项，第 j 个系数写入原次数减 j 的位置，避免倒置系数。例如 Fibonacci 初值为 $[0,1]$，递推系数为 $[1,1]$。

BM 在域上求有限前缀的最短递推。本书实现要求素数模数；一般的模合数环不能保证非零偏差存在逆元。仅凭若干样本无法证明真实序列以后一直遵循同一个规律。若事先知道序列满足阶数至多 d 的齐次线性递推，至少给出前 $2d$ 项，再用恢复出的系数求后续项。系数向量末尾的零不能擅自删除：它可能对应非平稳的初始段，改变阶数会改变递推开始适用的位置。

P5487 同时输出最短系数与远项，见第 166 页。第一行仅有 $c_0, \ldots, c_{k-1}$，没有阶数前缀；取恰好 k 个初值再求远项。全零前缀得到空系数时保留空首行，第二行输出 0。题目保证最短递推唯一；本程序用 BM 与特征多项式求余，时间 $O(nk + k^2 \log m)$，空间 $O(n + k)$。

小规模参考逐阶建立全部递推方程，用高斯消元验证最短性和唯一系数，用伴随矩阵幂求远项。最大 5000 阶使用独立构造 $a_n = \sum_{t=1}^{k} t^n$，特征多项式为 $\prod_{t=1}^{k}(x - t)$。前 $2k$ 项的 Hankel 矩阵是 VV^{T}，其中 $V_{i,t} = t^i$；不同非零底数给出可逆的 Vandermonde 矩阵，所以最短阶数确为 k。远项直接求 $\sum t^m$，无需调用被测递推算法。普通与 ASan/UBSan 均通过，本地时间与线上 AC、速度排名分开记录。

12.24 素数模高次剩余与单位根

实现见第 86 页；依赖本书离散对数、线性同余和快速幂。设 p 为素数，g 是原根，$k > 0$。$a = 0$ 时根只有零；否则写 $a = g^b$、$x = g^y$，方程转为

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$。当 $d \nmid b$ 时无解；否则指数解为 $y = y_0 + i(p-1)/d$，$0 \le i < d$。因此用三个数存全部根：

$$\texttt{first} = g^{y_0}, \qquad \texttt{ratio} = g^{(p-1)/d}, \qquad \texttt{count} = d.$$

依次乘 ratio 即可枚举；不保证数值递增，也不保证 first 是最小根。ratio 是阶为 d 的单位根，因而枚举恰有 d 个不同的根。模 $p = 2$ 同样有效；指数同余的模数是 1，唯一非零根为 1。

接口支持 $p \le 10^{12}$，需要调用者提供已验证的素数与原根；k 支持完整 unsigned long long，先对 $p-1$ 取模后解同余。a 支持完整 signed long long，先正规化。返回空 optional 表示无解，返回 `{0,1,1}` 表示唯一零根。给定原根后的期望时间、空间为 $O(\sqrt{p})$；枚举全部根另外需要 $O(d)$ 时间。模乘须先扩展到 128 位，不可用 64 位乘积直接取模。

测试穷举小于 100 的每个素数、全部原根及小指数，用任意精度模幂逐个枚举候选根并比较整个集合；覆盖负数、64 位边界、模 2、无解与零根。另检查 998244352 个根的紧凑表示，以及素数 999999999989 上的植根测试；后一个素数与原根的条件由试除及乘法阶证书独立确认。普通与 ASan/UBSan 均通过，尚无专门在线 AC。

这是 OI Wiki 高次剩余 中离散对数转线性同余方法的素数模实现。素数幂及非互素被开方数见下一节；一般合数模数见后文；改良 Tonelli–Shanks 尚需另补，不能把本节状态解读为该页面全部覆盖。

12.25 素数幂模高次剩余与根的提升

实现见第 87 页。令 $M = p^e$。当 $a = 0$ 时，全部根是 $p^{\lceil e/k \rceil}$ 的倍数。否则令 $t = v_p(a) < e$，必须有 $k \mid t$，否则无解。取 $s = t/k$，$q = p^{e-t}$，写 $x = p^s y$，只需求单位方程

$$y^k \equiv a/p^t \pmod{q}.$$

每个单位根 y 对应原模数下的根

$$x = p^s y + \ell p^s q, \qquad 0 \le \ell < p^{t-s}.$$

这些提升保持互不相同，也解释了非互素被开方数可能产生大量根。

奇素数时，提供模 p^e 的原根，约化后仍是模 q 的原根，沿用离散对数转线性同余的方法。$q = 2$ 时单位根只有 1；$q \ge 4$ 且为二的幂时，单位表示为 $(-1)^z 5^y$。符号指数模 2，5 的指数模 $q/4$；$q = 4$ 时只有符号分支。分别解两个指数同余即可，不可假设二的高次幂存在原根。

返回结构体保存 first、两个 ratio、两个 count，以及 step、lifts。全部根由下式给出：

$$\big(\texttt{first} \cdot \texttt{ratio}[0]^i \cdot \texttt{ratio}[1]^j \bmod \texttt{step}\big) + \ell \cdot \texttt{step}.$$

其中 $0 \le i < \texttt{count}[0]$、$0 \le j < \texttt{count}[1]$、$0 \le \ell < \texttt{lifts}$。`get` 已处理模乘溢出，`size` 返回三者乘积；不保证根按数值排序。零根情形复用同一表示。

模数上限 10^{12}，k 支持完整无符号 64 位。只检查范围，不检查 p 的素性和奇素数原根的阶，调用者须保证。默认参数 $g = 0$ 仅用于 $p = 2$；奇素数不能漏传原根。时间和空间由 BSGS 决定，期望 $O(\sqrt{q})$；单次按索引取根使用快速幂，需 $O(\log M)$。

已穷举多个素数的全部不超过 256 的素数幂、所有被开方数与小指数，用任意精度模幂直接枚举所有根，比较整个集合。另测有符号与无符号 64 位边界、2^{39} 的非平凡离散对数、3^{20} 的非互素方程，以及数千亿根的紧凑表示。普通与 ASan/UBSan 均通过；尚无专门在线 AC。一般合数的 CRT 组合见下一节；改良 Tonelli–Shanks 仍未完成，OI Wiki 页面继续标记部分覆盖。

12.26 一般合数模高次剩余：CRT 组合

实现见第 91 页。调用者提供完整分解 $M = \prod_{t=0}^{F-1} m_t$，其中 $m_t = p_t^{e_t}$ 两两互素，并提供每个奇素数幂的原根。先分别求局部根族；任一局部无解，整体即无解。若局部根数为 c_t，则整体根数为 $\prod_t c_t$。只保存局部根族，不创建这个大小的数组。

对每个因子预处理

$$E_t = \frac{M}{m_t}\left(\left(\frac{M}{m_t}\right)^{-1} \bmod m_t\right) \bmod M.$$

对一组局部根 r_t，唯一对应的标准代表是

$$x = \sum_t r_t E_t \bmod M.$$

逆元存在是因为因子两两互素。`get(index)` 将索引按各个 c_t 转成混合进制数字，再把局部索引拆成两个乘法根族索引和加法提升索引，最后用上述公式组合。CRT 双射保证不重复、不遗漏，输出顺序不保证递增。

$M \le 10^{12}$，$k > 0$；空因子列表表示 $M = 1$，返回唯一代表 0。`mod` 保存模数，`total` 保存总解数，`parts` 和 `weight` 保存局部根族与 E_t。模数乘积检查上界，CRT 的乘积先扩展至 128 位。单次按索引查询为 $O(F \log M)$；求解的主要时间和空间为局部 BSGS，因子列表接口不包含整数分解与原根搜索的成本；自动准备重载见下一节。

已对 $1 \le M \le 180$、所有被开方数与小指数直接枚举所有根，与两套代码的完整输出集合比较，并反转因子顺序重新检查。另覆盖有符号与无符号 64 位边界、混合素数幂上的非互素方程和大量零根，以及十一种不同素因子组成模数时的全部 1024 个平方根。普通与 ASan/UBSan 均通过；尚无专门在线 AC。至此已有已知分解及原根时的 BSGS 求根路线，更快的改良 Tonelli–Shanks 等内容仍未覆盖，OI Wiki 条目保留部分完成状态。

12.27 自动准备模数分解与原根

实现见第 90 页。`root_factors(M,rho)` 把 $M \le 10^{12}$ 分解为互异素数的幂，返回合数求根需要的 (p, e, g) 列表；模数 1 返回空列表。二的幂不需要原根，固定填零。奇素数幂 $m = p^e$ 的乘法群是循环群，设 $\varphi = m/p \cdot (p-1)$。分解 φ 后，候选 g 为原根当且仅当

$$p \nmid g, \qquad g^{\varphi/q} \not\equiv 1 \pmod{m} \quad \text{对每个不同素因子} q \mid \varphi.$$

从 2 起枚举候选，返回最小原根；所有奇素数幂均有原根。这里检查的是模 p^e 的阶，不能仅取一个模 p 的原根而不验证提升。

重载 `CompositeRoots::solve(a,k,M,rho)` 自动调用此准备过程。

随机分解工作区类型为 `PollardRho`。如果需要在同一个模数下处理多个方程，先保存因子列表，调用原有列表接口，避免重复分解和原根搜索。准备成本包括 M 与各 φ 的因数分解，以及逐个候选的模幂检验；不能把这部分直接算作常数时间。

独立试除验证 $1 \ldots 500$ 的全部模数与多组接近 10^{12} 的边界值，并用任意精度模幂验证返回原根的阶。小模数进一步检查更小候选不能生成全部单位。直接传模数的两套求根接口，在小模数上与全部候选根的枚举结果比较。普通与 ASan/UBSan 均通过；尚无自动准备入口的在线 AC。

12.28 任意正模数行列式

实现见第 223 页。素数模高斯消元可以除以任意非零主元，合数模数没有这个性质。本节只使用倍加行和交换行，不求逆元。先把元素正规化到 $[0, m)$，对第 i 列的两个元素 a, b，用整数商 $q = \lfloor a/b \rfloor$ 执行 $R_i \leftarrow R_i - qR_j$，再交换 R_i, R_j。该列的新数对为 $(b, a \bmod b)$，所以循环按欧几里得算法终止。其余元素每步对 m 取模，只改变整数代表，不改变行列式模 m 的值。

倍加行不改变行列式；交换行改变符号。消去所有下三角元素后，乘上对角元素及累计符号即得结果。剩余部分的该列全零时直接返回零，不需要判断某个非零元素是否与模数互素。例如 $\mathrm{diag}(2, 3)$ 在模 8 下的行列式为 6，不能因为第一个主元没有逆元就判定失败。

支持任意正 signed 64 位模数与完整 signed 64 位输入元素。正规化后单个乘积小于 2^{126}，行更新差值和结果乘积都放入有符号 128 位。空矩阵按约定返回 $1 \bmod m$；模 1 的所有返回值为零。时间上界 $O(n^3 \log m)$，矩阵副本占 $O(n^2)$ 空间。本节可以用于合数模的拉普拉斯余子式，但第 B.1 节（第 294 页）的 `MatrixTree<mod>` 接口仍使用素数模高斯，不能仅把其模板参数改成合数；合数模使用第 B.2 节（第 297 页）的独立接口。

测试用排列定义穷举所有元素属于 $\{-1, 0, 1\}$ 的三阶矩阵，并用任意精度 Bareiss 消元验证随机矩阵。覆盖不可逆主元、奇异矩阵、模 1、有符号 64 位边界、Fibonacci 长欧几里得过程，以及 300 阶对角矩阵和完全图余子式的 Cayley 公式。两套版本普通与 ASan/UBSan 均通过；尚无专门在线 AC。行变换依据可参考 OI Wiki 行列式。

12.29 参数化 NTT 与模数适配

实现见第 122 页。传入素数 p 和原根 g；长度 n 必须为二次幂且整除 $p-1$。代码自动计算 $L = 2^{v_2(p-1)}$ 作为长度上限，不代表分配这个长度的数组没有成本。两个非空数组的普通卷积需要补齐到不小于长度和减一的二次幂。模 998244353 的 $L = 2^{23}$，模 167772161 的 $L = 2^{25}$，两者均可用原根 3。不能只改模数而忽略原根和长度条件；接口只检查基本范围，不检查素性或原根的阶。

正变换按 g 的相应单位根计算，逆变换使用逆单位根并乘 n^{-1}。模数支持 int 范围内的素数，模整数的加减和乘积使用 64 位中间量。蝶形层以半长度递增，避免最后一层之后的有符号整数溢出。

独立参考用任意精度整数直接计算 DFT 和朴素卷积，同时验证正逆变换复原。已检查七组素数和原根，包括模 2、3、17、97，斯特林数的默认模数与洛谷题目模数，以及 2013265921、原根 31。这些测试中的素性及原根条件另用试除和乘法阶证书确认。大规模全一数组卷积逐项比较已知三角形系数，检查补齐长度的边界。两套实现普通与 ASan/UBSan 均通过。

12.30 Bostan–Mori 与快速递推单项

代码见第 167 页。对于 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，令 $Q(x) = 1 - \sum_{j=1}^{k} c_{j-1} x^j$，$P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$，则 $[x^n]P/Q = a_n$。注意系数按前一项、前两项的顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)}.$$

分母的奇数次项抵消，因此只保留分子与 n 同奇偶的项，分母保留偶数项，再将 n 右移一位。最终 $n = 0$ 时答案是 $P(0)/Q(0)$。实现不需要分子次数小于分母次数，空分子直接返回零；分母常数项必须非零。每轮系数长度不增加，初始最大长度 d 必须满足 $2d - 1$ 不超过 NTT 容量。总时间 $O(d \log d \log(n + 1))$，空间 $O(d)$；$n = 0$ 直接取常数项。

独立验证包括按定义做形式幂级数除法、友矩阵快速幂，以及 $(1 - x)^{-32000}$ 的二项式系数闭式；覆盖完整无符号 64 位下标、常数分母、不等于 1 的非零常数项、空分子、不真分式、零递推系数和多种 NTT 素数。P4723 驱动读取有符号系数并归一化；两套版本均获在线 AC：动态版与 传统版各 5 个测试点全部通过，最慢单点分别为 907 ms、905 ms。OJ 覆盖模 998244353、$n \le 10^9$、$k \le 32000$；其他模数、完整 64 位下标与通用有理函数接口的扩展范围由本地测试支持。OI Wiki 线性递推 中的连续区间取项与其他快速算法仍待补，不将单项算法视为整页覆盖。

12.31 集合变换与不相交子集卷积

代码见第 136 页与第 138 页。长度 $N = 2^m$ 确定全集的位数，数组第 s 项对应集合 s。OR 变换计算 $\widehat{a}_s = \sum_{t\subseteq s} a_t$，AND 变换计算所有超集的和；逆变换将逐位加法改成减法。XOR 变换的系数是 $(-1)^{\mathrm{popcount}(s\&t)}$，蝶形将 (u, v) 变为 $(u+v, u-v)$，逆变换每层再除以 2。因此 OR、AND 可用于任意模数，XOR 需要 2 可逆；本实现限制为奇数模数，使用 $(\bmod +1)/2$，不对合数模数使用费马小定理。三类卷积均是变换后逐项相乘，再逆变换。

$$c_s = \sum_{t\subseteq s} a_t b_{s\setminus t} \qquad \text{要求两个集合不相交。}$$

普通 OR 卷积允许重叠，不能直接用于上式。将输入按集合大小分层，$A_{i,s} = [|s| = i]a_s$，B 同理。每层 OR 变换后，对层数做卷积：$H_{k,s} = \sum_{i=0}^{k} \widehat{A}_{i,s}\widehat{B}_{k-i,s}$。对每个 k 逆 OR 变换，得到并集恰为 s、两集合大小之和为 k 的贡献。取 $k = |s|$，由 $|u| + |v| = |u \cup v| + |u \cap v|$ 得到交集为空。时间 $O(m^2 2^m)$，空间 $O(m2^m)$；本实现逐层复用结果缓冲区，不用第三组二维数组。$m = 20$ 时两组分层数组本身占 168 MiB，另计输入和线性缓冲。

验证独立枚举变换系数、所有下标对与子掩码，并检查复合模数与 `INT_MAX` 模数。大规模使用 $a_s = x^{|s|}$、$b_s = y^{|s|}$ 的逐位乘积恒等式：OR 为 $(x+y+xy)^{|s|}$，AND 为 $(xy)^{|s|}(1+x+y)^{m-|s|}$，XOR 为 $(x+y)^{|s|}(1+xy)^{m-|s|}$，不相交子集卷积为 $(x+y)^{|s|}$。P4717 两套版本各 10 点在线 AC：动态版和 传统版，最慢单点 84 ms、81 ms。

P6097 两套版本各 5 点在线 AC：动态版和 传统版，最大测试点显示 2.70 s、2.69 s，内存 188.83 MB、189.00 MB。两题 OJ 仅验证各自题面模数和长度范围，其他模数及长度 1 仍由本地独立测试覆盖。OI Wiki FWT 的其他运算及更广泛应用仍待覆盖。

12.32 Taylor 平移与等比点求值

代码见第 169 页和第 140 页。多项式使用升幂系数，输入长度 n 表示次数小于 n，允许末尾零。平移输出的第 i 项满足

$$[x^i]f(x+c)=\frac{1}{i!}\sum_{j=i}^{n-1}f_j j!\frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$，与 $c^j/j!$ 卷积，再取第 $n-1-i$ 项并乘 $1/i!$。保留原数组长度；零多项式可以用空数组表示，返回空数组。使用 NTT 素数及原根，要求 $2n-1$ 不超过变换容量；此时所需阶乘非零。时间 $O(n\log n)$、空间 $O(n)$。这与给出 $f(0),f(1),\dots$ 后平移点值的任务不同，后者尚待补充。

Chirp Z 返回 $f(ar^i)$，$0\le i<m$。设 $T_j=j(j-1)/2$，由 $ij=T_{i+j}-T_i-T_j$ 得

$$f(ar^i)=r^{-T_i}\sum_{j=0}^{n-1}f_j a^j r^{-T_j}r^{T_{i+j}}.$$

将第一项数组倒序，与 r^{T_j} 卷积，抽取第 $n-1+i$ 项。三角数幂通过累乘生成，不计算可能溢出的 $j(j-1)$。令 L 为不小于 $n+m-1$ 的最小二次幂，直接做模 x^L-1 的循环卷积。普通乘积最高次数为 $2n+m-3$，而所需区间任意下标 $t\ge n-1$ 满足 $t+L\ge 2n+m-2$，因此不会收到高次项回卷，所取系数仍然精确。要求 $n+m-1$ 不超过 NTT 容量，总时间 $O((n+m)\log(n+m))$、空间 $O(n+m)$。

约定 $r^0=1$，故 $r=0$ 时第一项为 $f(a)$，余项均为 $f(0)$；$a=0$ 时所有点为零，$r=1$ 时所有点相同。这些情况单独计算，避免对零求逆。$m=0$ 返回空数组，空输入返回 m 个零。同一份算法支持小阶公比、重复点与有限域中多次绕回，输出保持原顺序。

独立验证采用 Pascal 二项式展开和 Horner 求值，涵盖 7 种 NTT 素数、零公比与单位公比、小素数完整容量边界。大规模平移用 $(x+b)^d$ 检查每个系数；等比点求值用几何级数闭式逐点检查。普通与 ASan/UBSan 均通过。Library Checker 的 Polynomial Taylor Shift 和 等比点求值 已准备 vector 完整驱动；本地检查各题 524288 的最大规模，尚无在线 AC。参考OI Wiki 平移及 Chirp Z，更广泛的点值平移和应用另行覆盖。

12.33 CDQ 卷积递推：贡献只跨越一次

`cdq_convolution` 求

$$f_i = b_i + \sum_{j<i} f_j g_{i-j}, \qquad g_0 = 0.$$

在半开区间 $[l,r)$ 中，先递归求完左半 $[l,m)$，再将这段 f 与 $g_{0\ldots r-l-1}$ 卷积，把乘积的第 $i-l$ 项加入右半的 f_i，最后递归右半。任意 $j<i$ 的依赖只有一个最小分治节点将二者分到左右两边，所以每份贡献恰好加入一次。顺序不能改成先求两个孩子再卷积。

根层完整卷积结果长度为 $n+\lfloor n/2\rfloor-1$，须不超过 NTT 的最大二次幂容量；单项输入不做变换，空输入返回空。一般初始项 b_i 均可使用，P4721 只取 $b_0=1$。时间 $O(n\log^2(n+1))$，额外空间 $O(n)$，递归深度 $O(\log n)$。附件标题为“分治 FFT”，实现实际依赖 NTT；这里保留 CDQ 算法，不以形式幂级数求逆替代。独立二次递推、多素数域、二十万项闭式参考及完整主程序已本地验证，线上待补。

12.34 多项式带余除法：先反转再求逆

`PolynomialDivision::divide(a,b)` 返回规范化的商、余式，零多项式用空向量。去掉输入高次零后，除数必须非零；低次被除式直接返回零商和原被除式。设 $A=\deg a$、$B=\deg b$、$k=A-B+1$，反转后余式从 x^k 起，故

$$\operatorname{rev}(q) = \operatorname{rev}(a)\,\operatorname{rev}(b)^{-1} \pmod{x^k}.$$

除数最高项非零，反转后可求逆。取前 k 项再反转得到商，最后用 $r=a-qb$ 得余式。P4512 要求定长输出，使用程序负责补零；Library Checker 输出去高次零后的长度，零项对应空行。输入各长度不超过 2^{22} 是当前完整卷积容量的充分界，时间 $O(N\log(N+1))$、空间 $O(N)$。独立长除法、五十万项构造与官方本地 checker 已双模式验证；线上待补。

12.35 级数开根：前导零和自由尾项

`FpsSqrt::sqrt(a,n)` 仅使用前 n 项，不足补零。全零返回全零；首个非零项次数 k 为奇数，或其系数非二次剩余，则返回无解。否则设 $a=x^kh$，从较小的非零模平方根开始，迭代

$$b_{\text{new}} = \frac{b+h/b}{2}, \qquad b_{\text{new}}^2 - h = \frac{(b^2-h)^2}{4b^2}.$$

误差最低次数翻倍，故求出 h 的根前 $n-k$ 项后左移 $k/2$ 即可。所得根末尾还有 $k/2$ 个不受平方同余约束的系数，本接口统一取零。因此一般截断问题不应说成“只有两个解”。返回 optional；$n=0$ 返回有值的空向量。$n\le 2^{22}$；时间 $O(n\log(n+1))$ 加一次标量模开根，额外空间 $O(n)$。P5205 的 $a_0=1$ 返回首项 1，一般前导零与无解另由 Library Checker 使用程序展示。二次平方参考、最低次数边界、五十万项闭式及官方本地 checker 已双模式验证；线上待补。

12.36 ln 与 exp：只处理所需前缀

对数要求 $a_0=1$，由 $(\log a)'=a'/a$ 求导、求逆、积分得到；指数要求 $a_0=0$，从 $b=1$ 开始使用 $b\leftarrow b(1+a-\log b)$ 倍增。仅求前 n 项时，先把对数输入截到 n 项，避免输入的无关尾项造成变换容量超限。积分需 $1,\ldots,|a|$ 都可逆；固定素数域中要求 $|a|<998244353$。当前 $n\le 2^{22}$ 是变换容量充分界，log/exp 为 $O(n\log(n+1))$ 时间、$O(n)$ 空间。测试使用独立三角系数递推，并包含超过 NTT 容量的长输入只求一项、非二次幂截断与五十万项闭式。

12.37 幂函数的大指数：保留三份指数信息

`FpsPower::power(a,exponent,n)` 读入十进制数字串，计算模 x^n 的非负整数次幂。$n=0$ 返回空；零次幂统一返回单位多项式，包含 0^0。若 $k>0$，设 $a=x^s ch$、$h(0)=1$、$c\neq 0$；全零或 $sk\geq n$ 时结果全零，否则

$$a^k = x^{sk} c^k \exp(k\log h) \pmod{x^n}.$$

同时保留 $k \bmod p$、$k \bmod (p-1)$ 和 $\min(k,n)$：第一份用于对数系数，第二份用于 c^k，第三份用于真实零次幂及位移。当前 $n\leq 2^{22}<p$，故 $h(x)^p = h(x^p)\equiv 1 \pmod{x^n}$，单位首项部分只依赖 $k \bmod p$。指数余数为零并不表示原指数为零。判定 $sk\geq n$ 先用 $k>(n-1)/s$ 比较，通过后才乘，避免指数乘法溢出。指数位数 L，时间 $O(L+n\log(n+1))$、额外空间 $O(n)$。P5245 的常数项为 1，P5273 与 Library Checker 允许一般首项；共用主程序必须将指数读作字符串。独立截断乘法、十万位指数、五十万项二项式及官方本地 checker 已双模式验证；线上待补。

12.38 集合变换：逆变换对模数的要求

OR 变换是子集和，单层为 $(u,v)\mapsto(u,u+v)$；AND 是超集和，单层为 $(u+v,v)$。它们只需加减，模数可以为偶数或合数。XOR 变换使用字符 $(-1)^{\mathrm{popcount}(s\&i)}$；单层 $(u+v,u-v)$ 连做两次等于乘 2，故当前接口要求奇数模数，以 $(p+1)/2$ 为逆 2，不依赖费马小定理。长度为正的二次幂，长度 1 可用；三种卷积不能与不相交子集卷积混淆。独立字符定义式、下标对枚举、合数模数及最大规模完整驱动已双模式核验。

12.39 浮点 FFT：舍入前先约束最坏误差

复数变换采用正指数，逆变换使用共轭根并除以变换长度。长度为一时没有蝶形操作；位逆序从一开始，避免负移位。单位根按各自角度直接计算，避免长链旋转因子连乘。本节要求二进制至少 53 位有效精度、最近舍入、禁用 fast-math，且单位根复数绝对误差不超过 $8u$，其中 $u=2^{-53}$。三角函数精度是平台前提，不由 C++ 类型名称保证；本机已逐个核验最大网格的全部半圆根。

设变换长度 $N=2^\ell$，两输入欧氏范数乘积为 S。Percival 定理 5.1 对三次 radix-2 FFT 卷积给出误差界

$$S\left((1+u)^{3\ell}(1+\sqrt{5}u)^{3\ell+1}(1+8u)^{3\ell}-1\right).$$

整数入口限制 $N\leq 2^{22}$、$\max|a_i|\max|b_i|\sqrt{|a||b|}\leq 2\cdot 10^{12}$，从而每项误差小于 0.166；使用 llroundl 同时处理正负系数。超过此限制不能只因浮点结果接近整数就宣称正确。

任意模卷积先把有符号输入归一化，再拆成三个 11 位块。结果长度最多 2^{19}。每路 $S\leq 2047^2(2^{18}+1)$，误差小于 0.079。六次正变换复用，九次逆变换分别舍入后用整数模合并。此处以较大的常数换取明确的 53 位精度边界，不假定 long double 更宽。模数可为一或合数；空输入返回空，常数乘法合法。百万次数整数卷积、完整 int 输入、最大长度拆系数卷积已用闭式与精确整数参考核验；本地验证不等于在线 AC。

12.40 模 2 压位消元与完整解空间

`GaussXor::solve(a,n)` 的输入为 m 行 $n+1$ 列的 01 字符串，末列是右端项。每一主元列消去其余所有行；主元列成为单位列后，把自由变量全设零即可读出特解。每次只把一个自由变量设一，主元变量取对应自由列的系数，得到一个零空间基。这些基在自由列上的投影构成单位阵，所以独立且覆盖全部齐次解。全部解为特解异或任意基子集，有解时解空间维数为 $n - rank$；LC 的输出第一行就是该维数，不能输出系数秩。

每行按 64 位打包。异或从当前列所在字开始：新主元行在先前列均为零，所以同字中较早的位也不会被改坏。增广位编号为 n，始终分配 $\lfloor n/64 \rfloor + 1$ 个字，n 为 64 倍数时右端项在独立尾字。空方程组有 n 个自由变量；零变量系统若有任何右端项为一则无解。无解和零变量的有解情况都可能返回空特解，必须查看 `consistent`。

消元是精确模 2 运算，没有浮点误差；输出完整基最坏需要 n^2 个字符。特解不保证一的个数最少，开关最优化需要另外枚举自由基组合或使用题目结构。kuangbin 2.10.1 的 POJ1681 开关应用另以独立首行追赶和物理按键子集完成本地验证，见后文最少开关用法。独立全解集枚举和 4096 阶官方本地 checker 证据见 `docs/GAUSS-XOR.md`，不等于线上 AC。

12.41 模 2 消元后的最少开关用法

POJ1681 的源体将每个方格作为一个变量，行号为目标格，系数为自身及四邻的按键，原白格右端项为一、黄格为零。解集是特解异或零空间基的任意子集，须枚举后取一的最少个数，不能只数任意特解的一。递归枚举某个基时先不选、再异或选择，返回后再次异或恢复；无解输出 `inf`。

令自由维数为 d，枚举代价为 $O(n^2 2^d)$，递归深度 d。本方格问题第一行按法一旦确定，后续行由消除上一行的白格唯一确定，所以 $d \le n$。该界只针对这个四邻棋盘，不能套用到任意图或任意异或方程组。622 份棋盘采用独立首行追赶参考，其中三阶以内全部棋盘再与物理按键子集穷举核对；验证到十五阶、无解与多测重置。原 POJ 题面当前不可达，依据固定 kuangbin 源体重建应用；本地通过不冒充原站 AC。详见 `docs/SWITCH-MINIMUM.md`。

第一部分

跨类依赖

附录 A　树状数组

A.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。$n \le 100000$，$|x| \le 10000000$；总复杂度 $O(n \log n)$，空间 $O(n)$。

所需模板：Fenwick（第 292 页，A.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 B　矩阵树定理

B.1　带权矩阵树：无向树与内外向树形图

代码依赖：ModInt（第 24 页，3.2.2 节）、det_prime（第 220 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

声明 MatrixTree<mod>，调用 count(n, edges, root, kind) 返回所有合法树的边权乘积之和，mod 必须是素数。Edge=(u,v,Z)，点号 0..n-1，n≥1；Kind::undirected 为无向生成树，toward_root 为每点沿边能到根，away_from_root 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 root 决定答案。依赖 det_prime 的素数模行列式，$O(n^3+m)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod> struct MatrixTree
{
    using Z = ModInt<mod>;
    using Edge = tuple<int, int, Z>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
    {
        assert(n >= 1 && root >= 0 && root < n);
        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
        auto add = [&](int u, int v, Z w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = lap[i][i] + w;
            if (v != root) lap[i][j] = lap[i][j] - w;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return det_prime<mod>(move(lap));
    }
};
```

模板题使用：Luogu P6178

题意：求所有生成树边权乘积之和，模 1000000007。type=0 为无向图，type=1 为以输入点 1 为根的外向树；输入点号减 1 后根为 0。重边分别计数，自环不贡献。

所需模板：ModInt（第 24 页，3.2.2 节）、det_prime（第 220 页，6.2.1 节）、MatrixTree（第 294 页，B.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6178

```
int main()
{
    using M = MatrixTree<1000000007>;
    int n, m, type;
    scanf("%d%d%d", &n, &m, &type);
    vector<M::Edge> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u - 1, v - 1, w});
    }
    auto kind = type ? M::Kind::away_from_root : M::Kind::undirected;
    printf("%d\n", M::count(n, edges, 0, kind).v);
}
```

应用题补充：Luogu P4455 社交网络：根 1 内向树

题意：输入 a b 表示 a 能看见 b；选择 b 为 a 的消息来源，对应 a 指向 b 的父边，所有非根点沿父边最终到达 1。因此按原输入方向调用 toward_root，不能选 away_from_root；点号减 1，根为 0，模 10007。n 不超过 250，时间 O(n^3+m)，空间 O(n^2)。按朋友关系去重，自环忽略；重复关系不视为不同传播方案。不统计转发时间顺序。单点约定空树 1。官方题面已核对，完整协议本地验证；尚无本实现在线记录。

所需模板：ModInt（第 24 页，3.2.2 节）、det_prime（第 220 页，6.2.1 节）、MatrixTree（第 294 页，B.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4455

```
int main()
{
    using M = MatrixTree<10007>;
    int n, m;
    cin >> n >> m;
    vector<vector<int>> seen(n, vector<int>(n));
    vector<M::Edge> edges;
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        if (a != b && !seen[a][b])
        {
            seen[a][b] = 1;
            edges.push_back({a, b, 1});
        }
    }
    cout << M::count(n, edges, 0, M::Kind::toward_root).v << '\n';
}
```

B.2 任意正模数带权矩阵树

代码依赖：determinant_mod（第 223 页，6.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 MatrixTree<mod>。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 223 页，6.2.2 节）、MatrixTreeMod（第 297 页，B.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

模板题使用：Library Checker Counting Spanning Trees (Directed)

题意：有向多重图生成树计数，模 998244353，要求指定根 r 能到达所有点，使用 away_from_root。n 不超过 500、m 不超过 500000，点号及根为 0 起；每条原边权 1，自环忽略，重边分别计数。每个非根点恰有一个入边，根无入边；toward_root 表示所有点到达根，不能替代。n=1 计空树 1，无外向生成树计 0。

所需模板：determinant_mod（第 223 页，6.2.2 节）、MatrixTreeMod（第 297 页，B.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_directed

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(
                n, e, r, MatrixTreeMod::Kind::away_from_root, 998244353)
         << '\n';
}
```

算法索引

AdaptiveSimpson, 254

ballot theorem, 187
Barrett32, 35
basis_intersection, 233
basis_sum_intersection, 235
batch_inverse, 56
batch_units, 54
Berlekamp–Massey, 240
Berlekamp-Massey, 276
berlekamp_massey, 268
Bezout identity, 50
BigInteger, 6
Binomial, 177
BivariateInverse, 146
BostanMori, 167
Burnside, 208
BurnsideAverage, 202

Carmichael, 275
carmichael, 53
Catalan, 187, 276
Cayley–Hamilton, 240
cdq_convolution, 120
chirp_z, 140
Cipolla, 274
ComplexFFT, 112
CompositeBinomial, 72
CompositeRoots, 91
conditional expectation, 247
convolution_fft, 116
convolution_i64, 125
convolution_mod, 130
convolution_mod_fft, 118
coprime pairs, 272
CoprimePairs, 96
CRT, 37
crt_merge, 60

det_prime, 220
determinant_exact, 225
determinant_mod, 223
Dirichlet convolution, 97
DiscreteLog, 85

divisor sum, 272
divisor_sum_power, 18
Du Jiao sieve, 270
DuJiao, 99

EGF, 178
Euler theorem, 37
Euler totient, 271
euler_phi, 41
euler_power, 52
exCRT, 273
ExLucas, 69
exLucas, 276
extended_gcd, 47

Fenwick, 292
FibonacciPeriod, 182
floor moments, 111
floor quotient grouping, 94
floor sum, 110
floor_moments, 108
floor_sum, 106
FpsFunctions, 160
FpsInverse, 142
FpsPower, 163
FpsSqrt, 155

Garner, 274
garner, 62
GaussMod, 262
GaussReal, 265
GaussXor, 258
GCD 加权求和, 97
generating functions, 276
geometric series, 272

inclusion-exclusion, 276
indicator variable, 247
integer partitions, 200
inverse_table, 58

Korselt criterion, 275
KthResidue, 86
Kummer theorem, 66

Lagrange, 251
Lagrange interpolation, 276
Lagrange inversion, 173
Legendre formula, 66
lifting the exponent, 64

linear congruence, 273
linear sieve, 271
linear_congruence, 59
linear_equation, 48
LinearSieve, 42
LTE, 64
Lucas, 68, 275
Lucas theorem, 37

Matrix Tree theorem, 277
matrix_inverse, 213
matrix_inverse_mod2, 216
MatrixTree, 294
MatrixTreeMod, 297
MaxPlusMatrix, 218
mex, 249
Miller–Rabin, 270
Min25, 103
mint, 20
Mobius inversion, 97
Mod64, 15
mod_inverse, 55
mod_sqrt, 80
ModInt, 24
ModMatrix, 211
Montgomery, 31
multiplicative order, 275
MultipointEvaluation, 150

necklace_colorings, 206
Ntt2D, 127
NttConvolution, 122

OGF, 178

p-adic valuation, 64
Partitions, 198
pentagonal number theorem, 200
permutation_cycles, 205
Pollard–Rho, 270
PollardRho, 45
Polya, 208
polynomial_interpolation, 153
polynomial_shift, 169
PolynomialDivision, 143
PositionBasis, 229
power_sum, 16
Prime64, 40
prime_count, 101

PrimePowerRoots, 87
PrimitiveRoot, 81

q_binomial, 76
quadratic residue, 274

rank, 277
Rational, 1
recurrence_nth, 165
reflection principle, 187
Richardson extrapolation, 257
root_factors, 90

segmented sieve, 271
segmented_primes, 43
SetConvolution, 136
shifted_power_sum, 175
Simplex, 242
Simpson integration, 257
Sprague-Grundy, 249
StaticMod64, 27
Stirling numbers, 189, 276
stirling_first_column, 192
stirling_first_row, 190
stirling_second_column, 195
stirling_second_row, 194
subset_convolution, 138

tail sum, 247

Widget Factory, 277

XorBasis, 227
XorWalk, 237

上升阶乘, 189
下降阶乘, 189
二元一次不定方程, 50
二项式反演, 180
互质数对, 98

区间筛素数, 271
升幂引理, 64
原根, 275
复合逆, 173
容斥原理, 180
形式留数, 173

恰好计数, 180
扩展中国剩余定理, 273
批量逆元, 272

排列数, 276
斯特林反演, 197
斯特林数, 189
无序数对, 98
有限状态计数, 240
模高斯消元, 277
混合进制, 274
满射, 180

矩形 GCD 计数, 272
筛法选型, 94
类欧几里德, 110
线性同余方程, 273
线性逆元表, 273
线性递推, 240, 277
群作用, 208

计数建模, 178

阶乘表, 276
隐式生成函数, 173